

电子与嵌入式系统设计丛书

姿态传感器开发入门手册：基于 MPU9250

范旭阳 编著

下视界



前言

刚开始写这本书的时候，作者刚刚翻译完了 MPU9250 的开发手册，并把它公布在网上供大家参阅。为什么那么多的传感器非要翻译这个？因为它很重要。做机器人开发不管是在工作中还是在应用中，都会接触很多类型的传感器。自从乔布斯将姿态传感器带入智能手机，他在我们的生活中扮演着越来越重要的地位。传感器信号有模拟型和数字型。模拟型输出的是一个电压值，需要尽可能的采集到最高精度的电压并转换单位即可。而数字系传感器就有各种各样的接口，不管是 485 和还是 I2C 还是 CAN 总线，都可以直接获取数字信号，省掉了不少麻烦。他们往往都是通过寄存器来控制的。姿态传感器也不例外。如何调配这些寄存器，如何能挖掘他的功能，就成了姿态传感器的难点。刚好，前几年的时候我做一个姿态传感器项目，在翻遍了互联网中的很多相关资料后，却发现没有一个关于姿态传感器开发的完整版中文资料。我想这个东西的使用范围这么高，应该会有很多人开发才对，但相关的中文书籍和资料又太少，所以为了帮大家节约一点时间，于是我将 MPU9250 的官方 datasheet，都翻译成中文，共享出来。现在大家仍然可以在百度搜索到 MPU9250 的中文资料。我特意留了微博账号在手册上，效果还是很让我欣慰的，到现在每周都会有不少同学来找我问问题。后来出版社找到我，希望我能出一本开发姿态传感器方面的书籍。我觉得的确应该这么做，我愿意把我开发的经验总结出来，缩短大家的开发进程，遇到的问题大家可以一起讨论，一起解决，于是这本书就这么诞生了。

其实做机器人做久了就知道，调的不是机器人，而是传感器。传感器的精度越高，算法控制的越好，机器人就执行命令执行的越准。这也是工程师们一直努力的方向。

本人在写这本书的时候，对于不同知识层面的人其实还是有不同的考量的。我希望能面对所有知识水平的群众，不管是嵌入式大神还是 UI 设计师。你们都能从本书中学到一些你想要的东西。希望在我的精心讲解下，你能了解姿态传感器，并驾驭它，让你在开发的过程中少走弯路和死胡同，然后你们就可以给它加上各种花式玩法，比如自动排列的飞行器或者自动送货的飞行器。而且本书涉及

的大量算法知识，我都将以最浅显的方式给大家讲明。

本书主要内容

第 1 章，本章将从基础的物理知识和传感器的内部结构原理讲起让大家对姿态传感器有个概念。通过本章，您会对传感器的软硬件原理有个大致的了解。

第 2 章，本章主要以 MPU9250 为例，讲解姿态芯片的相关电路结构知识。如果不需要做电路开发的读者可以略过本章。

第 3 章，本章通过实例来配置传感器，并通过简单的计算出目前传感器的角度。详细讲解加速度，陀螺仪和磁力计的用法。

第 4 章，本章开始将着重讲解姿态的数学上基础知识。包括空间角度在数学上的三种表达方式。本章包含大量的数学计算以及计算原理的讨论。涉及线性代数。

第 5 章，本章讲解的是姿态融合算法，和卡尔曼滤波算法。得到较为稳定的角度数据。

第 6 章，本章则是通过传感器应用的例子来直接讲解传感器在现实项目重的一些开发思路和应用方法。帮助大家更好的开发姿态传感器。本章讲解的有，平衡车、智能手环、飞行器、VR 设备等。

附录 A，本章作为知识扩展章节。着重解说前面 7 个章节中一些需要额外需要补充的知识内容，不用劳烦大家再去查找相关的资料。

附录 B，MPU9250 的 DMP，通过开启此功能，MCU 可以直接获取到的四元数值。

附录 C，MPU9250 的内部寄存器及其使用方式。由于传感器的内部寄存器太多，大家可以不用每一个都研究得非常透彻，只需要研究自己需要用到的寄存器即可。可以把本章节当资料库来查阅。

如果你开发的是 MPU6050 或者其他版本甚至其他品牌的传感器，只需略过第 2 章和附录 BC 即可。其他功能和算法都是相通的，本书就讲姿态传感器的功能

算法和应用。

勘误和支持

本书提供了书中涉及的相关资料，包括底层驱动和芯片资料等。读者可通过以下方式与作者联系下载资料，若有新增或修改，恕不在本书告知：

- ❑ CSDN 博客/新浪微博：出迷者
- ❑ 提议邮箱：7@riddler.top
- ❑ B 站 ID：出迷者
- ❑ GitHub：<https://github.com/TomRiddler/MPU-9250>
- ❑ 微信公众号



著者：范旭阳
页数：306 页
字数：86 千字
版本：2018 年 4 月第 11 版
版权所有，侵权必究

目录

前言.....	ii
本书主要内容.....	iii
勘误和支持.....	iv
第 1 章 原理.....	1
1.1 史话.....	1
1.2 加速度计（惯性传感器）.....	10
1.2.1 硅微电容式加速度传感器.....	11
1.2.2 硅微压阻式加速度传感器.....	12
1.2.3 硅微热电偶式加速度传感器.....	14
1.2.4 谐振式加速度传感器.....	15
1.2.5 加速度传感器计算思路.....	15
1.3 陀螺仪.....	22
1.4 磁力计.....	29
第 2 章 电路设计.....	37
2.1 芯片引脚.....	42
2.2 I2C 和 SPI 通信协议.....	44
2.2.1 I2C（I2C）通信协议.....	44
2.2.2 SPI 通信协议.....	48
2.2.3 I2C 与 SPI.....	50
2.3 电路布局及注意事项.....	51
2.3.1 电路原理图设计.....	52

2.3.2 电路 PCB 设计.....	56
2.4 AD15 简单使用	58
第 3 章 加速度陀螺仪与磁力计.....	66
3.1 寄存器	75
3.1.1 电源管理寄存器 (0x6B 0x6C)	76
3.1.2 芯片配置寄存器 (0x1A).....	77
3.1.3 陀螺仪配置寄存器 (0x1B).....	79
3.1.4 加速度计寄存器 (0x1C 0x1D).....	79
3.2 配置.....	80
3.3 计算.....	85
加速度陀螺仪寄存器.....	85
3.3 磁力计校准和朝向角.....	96
3.4 朝向角.....	102
3.5 自检模式.....	103
3.5.1 加速度计与陀螺仪的自检.....	103
第 4 章 姿态.....	112
5.1 欧拉角.....	114
Yaw、Pitch 和 Roll.....	118
5.2 方向余弦.....	120
5.3 万向节死锁.....	124
5.4 四元数.....	127
5.5 四元数的角度转换.....	137

第 6 章 姿态融合.....	144
6.1 卡尔曼滤波.....	146
6.2 捷联惯导算法.....	159
第 7 章 应用.....	172
7.1 总程序.....	172
7.2 平衡车.....	176
7.3 智能手环.....	183
7.4 智能手机.....	186
7.5 飞行器.....	187
7.6 VR 设备.....	202
7.7 磁力控制器.....	206
附录 A 答疑.....	208
A.1 傅科是如何设计地球自转装置的.....	208
A.2 芯片读取数值的不同.....	209
A.3 交变科里奥利力在陀螺仪中的使用.....	209
A.4 关于特斯拉是否被“神”化.....	211
A.5 为什么磁力计这里的数据是一个圆.....	212
A.6 为什么苹果有 MPU6700 这种买不到型号芯片.....	212
A.7 关于 I2C 读取 10 位地址问题.....	213
A.8 关于资料的获取和使用的感悟.....	213
A.9 关于 QFN 封装.....	215
A.10 Arduino 的使用和编程.....	216

A. 11 马斯克和 NASA 的 XPACE 传感器精度.....	217
A. 12 关于 atan () 和 atan2 () 函数.....	217
A. 13 OFFSET 寄存器的使用	218
A. 14 关于复数计算.....	224
A. 15 四元数的计算公式.....	225
A. 16 欧拉角、余弦向量、四元数之间的转换.....	230
A. 17 卡尔曼滤波五个公式.....	232
A. 18 IMU 姿态融合算法	233
附录 B MPU9250-DMP	236
附录 C 寄存器	243
寄存器.....	251
3. 3 磁力计寄存器.....	287
3. 3. 1 磁力计寄存器详细说明.....	288
3. 3. 2 WIA: 设备 ID	290
3. 3. 3 INFO: 信息.....	290
3. 3. 4 ST1: 状态 1	291
3. 3. 5 HXL 到 HZH:测量数据.....	291
3. 3. 6 ST2: 状态 2	293
3. 3. 7 CNTL1: 控制 1	294
3. 3. 8 CNTL2: 控制 2	295
3. 3. 9 ASTC: 自测控制.....	295
3. 3. 10 TS1 TS2: 测试 1、2	296

3.3.11 I2CDIS: I2C 关闭 296

3.3.12 ASAX ASAY ASAZ: 灵敏值调节..... 296

下视界

第 1 章 原理

研究一种新知识，如果对相关知识的历史和发展有一些了解的话，会更加有利于我们对这种知识的掌握，也便于让我们对于整个知识体系结构有个规划。本章就我们就回溯过去，看看人类对于姿态认知的科学技术是如何演变至今的。

1.1 史话

在了解九轴姿态传感器之前，你可能或多或少听过三轴、六轴、九轴这样的词。注意这里的三轴和三自由度不一样，有很多人第一次接错总会把这个概念搞混。玩过机械手的同学对自由度这个概念应该比较了解。机械臂上每多一个转轴就会多一个自由度。拿人类的手臂来举例子，当我们只有上臂（肩膀到肘关节的部分）的时候，上臂便可以绕着肩膀前后旋转，这就是一自由度；此时装上下臂（肘关节到腕关节的部分），这时手臂就在之前绕肩关节转动的基础上又做到可以绕肘关节的精准旋转，这就是二自由度；如果再加上手掌，那么现在也能绕腕关节活动了，这就是我们人类的手。我们便可以用它轻易定位到我们想拿东西的位置，人类的手便是三自由度。但是本书中要着重讨论的三轴和这些却完全不一样。本书的三轴、六轴和九轴其实是指陀螺仪，加速度计和磁力计。这里每三个轴作为一次递增就是指每个轴数据的 X, Y, Z 三个坐标轴。现实生活中单方面所说的三轴其实是可以指加速度计也可以指陀螺仪的，而六轴则主要是指代加速度计和陀螺仪的合称。因为以前的姿态传感器并没有磁力计的传感器集成在其中，所以并没有把磁力计纳入在内。不过现在的磁力计的都集成在姿态传感器内部了，所以合称为九轴传感器，本书就是以 MPU9250 九轴传感器为例，给大家掰开揉碎的讲解九轴传感器的到底是怎么设计使用的，这样大家就会使用目前市面上所有的姿态传感器了。某些搞飞控的人还喜欢叫自己的飞控板为十轴，那是因为他们把气压传感器也算在了里面（气压计主要是测量飞行高度的，因为只有一项数据所以认为是单轴）。

古人对姿态这个的研究可谓是费尽心机，生活中处处都体现了其中的奥妙。从日晷，到沙漏甚至是榫卯，还有儿时玩的九连环等，或多或少的都用到了和姿

态设计有关的设计。这里我给大家介绍个最能体现古人对姿态设计之精妙的一个玩物——被中香炉。

被中香炉顾名思义就是一种香炉，被中的意思是说这种香炉是放在被子里的。那你可能会问，被子里放香炉难道不会把灰撒的到处都是吗？其实这种香炉中间用来点香的是一个特制的平台，它四周圆形包裹。无论你怎么转动这个容器，怎么滚动它，它内部燃香的平台永远是朝上的。古代的中国人经常用这个来驱蚊除臭，其实还可以拿来暖被窝。其最早的记载被发现在《西京杂记》中：“长安巧工丁缓者……又作卧褥香炉，一名被中香炉，本出房风，其法后绝，至缓始更为之。为机环转运四周，而炉体常平，可置之被褥，故以为名。”这段话是说，汉武帝时，京城长安有位叫丁缓(公元前 140~前 80)的能工巧匠，他制成了当时已经失传的被中香炉。在香炉中贮存着香料，点燃以后，放在被褥之中，随意滚动，香炉可以始终保持水平状态，不会倾翻，所以得名叫被中香炉。此处说的是已经失传的被中香炉，说明发明这个容器的人还要比这还要早的多。

如图 1-1 所示，这个就是被中香炉。



图 1-1 被中香炉

分析原理其实这个结构有没有大家可以参考如下图 1-2 结构的陀螺仪。

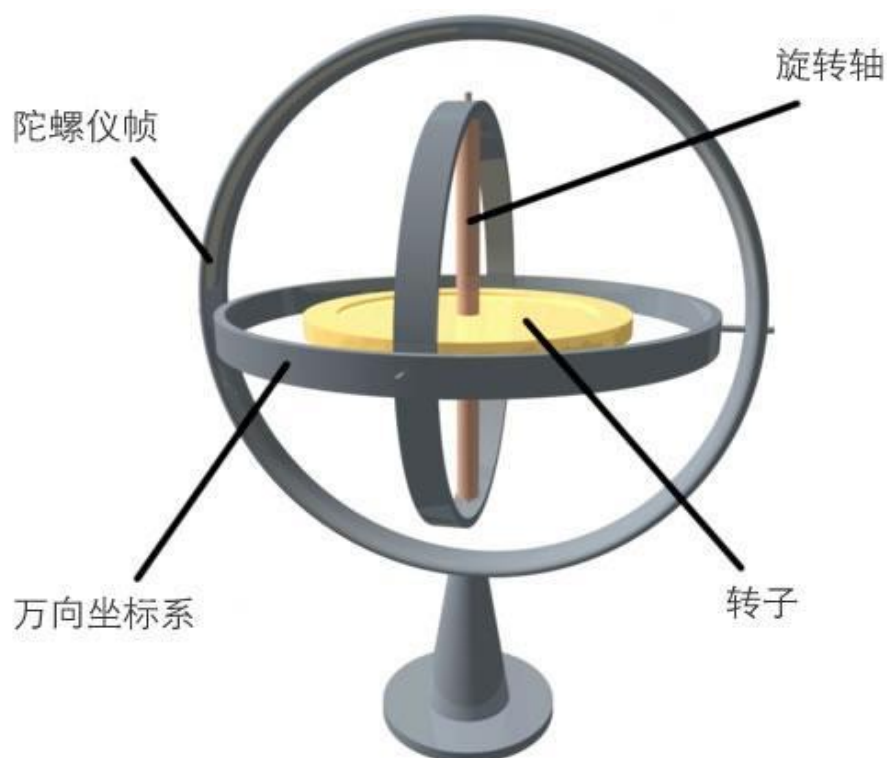


图 1-2 陀螺仪

如图 1-2 所示，每个个体圆环支架之间相互垂直，利用支架结构使其各控制一个方向（XYZ 轴），无论如何旋转都可以使内部的转子（相当于被中香炉放香料的地方）保持正面向上，再加上外面的球形的保护措施。所以无论这个香炉怎么转动都不会把最内部的燃香洒出。这种结构后来被推广到了欧洲，1500 年意大利科学家达·芬奇和卡丹(1501-1576)都做了类似万象支架的相关设计。万象支架就好比我们现在看书用的灯具，当你调节灯的方向的时候，就相当于对每个轴进行改变。类似的应用还有焊台的放大镜架子，唱歌歌手的立式话筒架子，工厂里面的机械手也是类似原理。万象支架如图 1-3 所示，如果在每个关节做个电机控制阀，那么这个支架就是四自由度，你能数出来吗？



图 1-3 现代支架

那为什么陀螺仪具有这样“不倒”的特性？我们可以参考一下陀螺的原理，陀螺具有三大特性：

第一，定轴性。陀螺在旋转时中心轴的距离是一定保持不变的，也就是上面图 1-2 所示的旋转轴；

第二，进动性。陀螺自身旋转的时候，其实还会沿着另一个轴做旋转运动。如图 1-4 所示，玩过陀螺的同学肯定知道，当长时间不抽打陀螺的时候就会出现这种绕两个旋转的现象。就像地球不但自转，而且还公转一样；

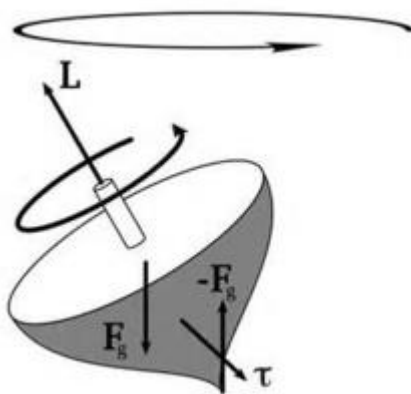


图 1-4 陀螺的进动

第三，章动性。当陀螺速度由快变慢的时候，我们可以注意到陀螺会做一上一下的点头运动，这就是章动性的体现。

1850 年的物理学家莱昂·傅科（J. Foucault）居然就根据陀螺的特性证明出了地球的自转。你知道是怎么做的吗？参阅附录 AA1 傅科是如何设计地球自转

装置的，获取更多细节。（在本书中有很多这种 AN 的设计，涉及的内容多半是杂谈等，是可以了解也可以不了解的，如果感兴趣的读者可以看看。但和九轴的开发没有什么太大的关系。）

这样，人们根据陀螺的特性才设计出了陀螺仪，如图 1-5 所示。

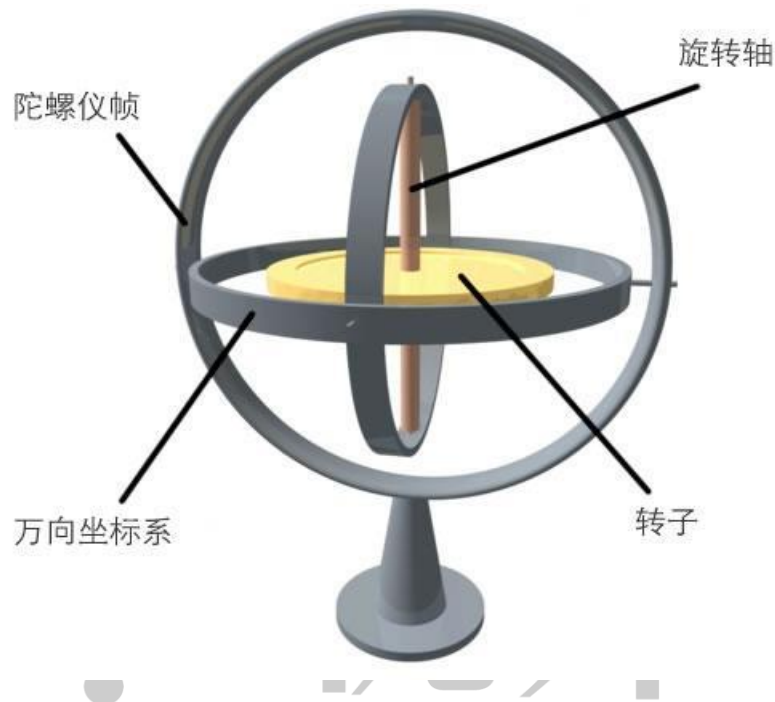


图 1-5 陀螺仪

来分析一下这个结构，陀螺仪的三个轴（陀螺仪帧、万向坐标系、旋转轴）刚好对应我们之前所说的三轴（X-Y-Z），中间那个转子和旋转轴对应的就是上面提到陀螺的特性中的两个轴。以转子和旋转轴的交点为原点，建立分别平行于三个坐标的三个轴，这就是高中所学的空间向量 XYZ 坐标轴了。没错，计算的时候要用到很多空间向量这方面的知识，所以曾经说学数学没用的同学可要打脸喽。

在大航海时代，陀螺仪首先被利用在航海技术上，船长根据陀螺仪来判定船只运行的倾斜度以便做出相应判断，比如浪高或者浪的强度和方向等。再后来有了飞机便开始在航空领域上广泛使用。因为飞机在空中，是无法像地面一样轻松靠肉眼来辨认方向和飞机是否水平飞行的，高空的云层和天际线混为一色很容易让人眼晕，特别是在海上开飞机的情况下。直接通过肉眼判断危险性太高，所以陀螺仪迅速得到了应用，成为飞行仪表的核心。没体验过的同学可以去玩玩模拟

飞行类型的游戏，然后把屏幕下方的水平仪表盘挡住，然后就知道没有陀螺仪辅助是什么感觉了。（此时你不容易判断是知道飞机是前倾了一点儿还是后仰了一点儿，左偏还是右斜，就没法保持绝对的水平。）

到了二战，各个国家的科学家开始拼命研究制造新式武器。其中德国研发了一种飞弹可以直接炸到英国本土，这就是今天导弹的雏形。



图 1-6 导弹战车

但是德国离英国那么远的距离如何才能做到精准打击而不至于打偏呢？于是德国人研究出来了惯性制导系统。惯性制导系统主要就是通过陀螺仪来确定方向和角速度，通过加速度计来矫正导弹的飞行姿态，并通过计算，算出飞行的姿态和方向，在飞行的时候可以控制导弹的姿态，让导弹能准确的飞到英国伦敦去。这个方法和现在的火箭飞行的原理也类似，在火箭飞向太空的时候，需要不停地做姿态计算，从而不停地调整航道，让火箭按照计划的航道飞行。不过那时的硬件技术没有那么好，内部的仪器占用了不少空间和重量，所以导弹显得笨重而粗大，不像现在都是微型导弹。

我们平时使用的智能手机里面其实也有陀螺仪传感器，注意了，可不是里面

真有个陀螺在那里在旋转跳跃闭着眼。随着科技的进步，现在的传感器早已经脱离了以前那种粗大笨重的结构。就拿陀螺仪为例子，现在都是激光陀螺仪，光纤陀螺仪和微机电陀螺仪等，虽说他们名字都叫陀螺仪，但是原理和以前的陀螺仪已经是两码事了，他们的体积和精密程度也不可同日而语了。

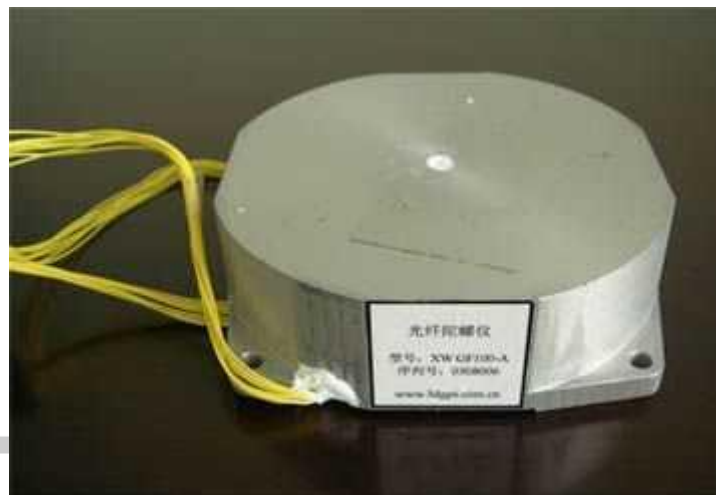


图 1-7 光纤陀螺仪

光纤陀螺仪利用的是萨格纳克(Sagnac)效应。(光传播时与光路的长度半径有关，而与介质和相移无关)利用光传播的特性，通过测量光程差计算出旋转的角速度，起到陀螺仪测量角速度的作用，从而达到陀螺仪的作用。



图 1-8 激光陀螺仪

激光陀螺仪其实也是通过算光程差来计算角速度，从而替代以前的陀螺仪测算方案。



图 1-9 微机电陀螺仪

微机电陀螺仪是利用物理学中的科里奥利力（对旋转体系中进行直线运动的质点由于惯性相对于旋转体系产生的直线运动的偏移的一种描述。科里奥利力来自于物体运动所具有的惯性。但这种力实际上是不存在的。），在内部产生微小的电容变化，然后通过电容测量，计算出角速度，以替代陀螺仪。几乎现在大部分的移动终端设备（智能手机、智能手环、VR、AR 设备等）里所用的陀螺仪，都是微机电陀螺仪（MEMS）。

而在传感器的开发上，分为模拟型和数字型。也就是说几乎所有的传感器都是把要测量的状态转化为电容，电压或者电感的值来进行测量，所以你只需要读取传感器相应的寄存器值，并做相应的算法计算，有些传感器甚至已经给你算好了值，直接读取就可以使用了。因而开发反而变得简单的多了呢。



图 1-10 虚拟现实设备

现随着移动互联网（或物联网）的崛起，导致了移动互联网设备的崛起，从而使得 MEMS 传感器的成本越来越低，其中涉及越来越多的行业和领域都要用到姿态传感器。手机就不用说了，当前炙手可热的飞行器。还有各种智能可穿戴设备，比如智能手环、智能手表等。以及现在的 VR（虚拟现实技术）、AR（现实增强技术）等。军用级应用在如在自动驾驶的汽车，自主航行的导弹，轮船，飞机。工厂里面的机械手等。恐怕没有姿态传感器这个小东西，生活远不如现在便利。但是对姿态传感器的应用的研究和应用上，人类还任重而道远。

1.2 加速度计（惯性传感器）

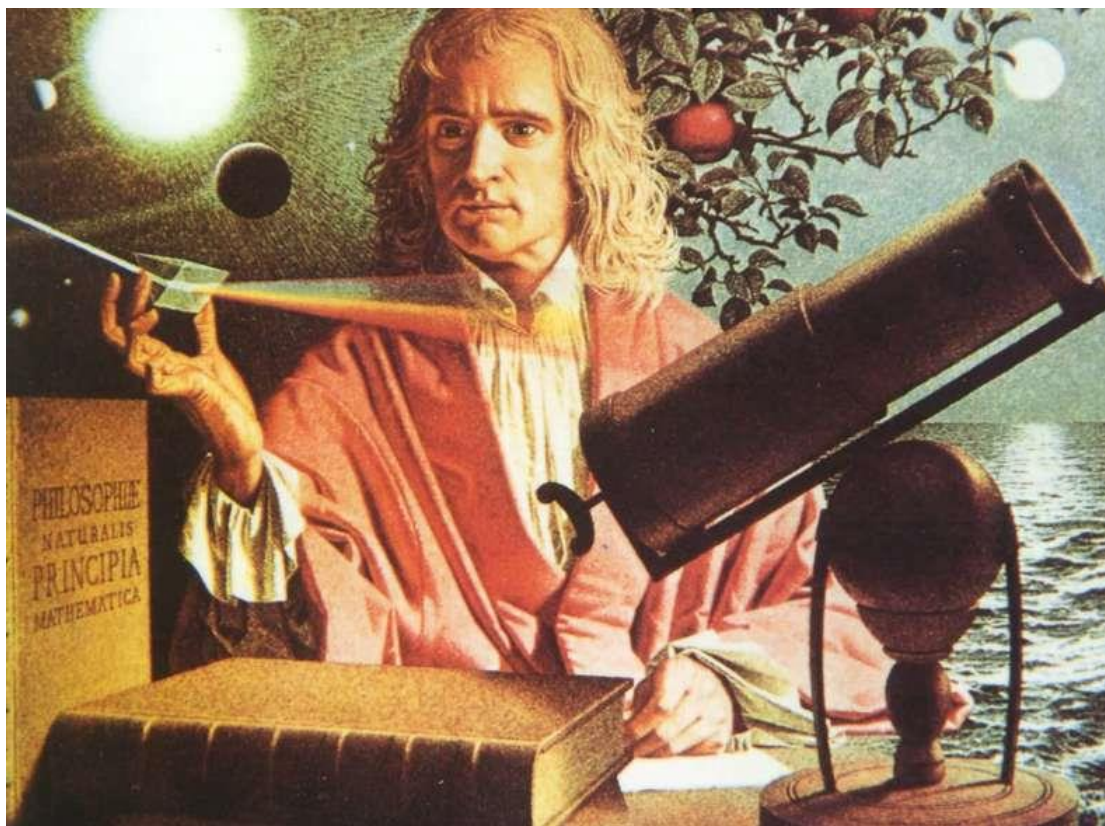


图 1-11 艾萨克·牛顿爵士

我们说过九轴传感器中分别有加速度传感器，陀螺仪和磁力计。首先从加速度计开始说起。

加速度计顾名思义是测量物体的加速度，也就是初中物理中所学的 a ($F=ma$, 牛顿第二定律)。我们如何测量的加速度呢，难道通过路程来反推？（我们在高中物理曾经学过 $s=vt+(1/2)at^2$ 通过它来反推路程？）如果此时车子走蛇形跑法（S 弯）的话，加速度我们又该怎么计算呢？况且目前不可能有绝对直线跑动的汽车。

思来想去还是力靠谱。没错，我们就是通过检测相互作用力来检测加速度的大小。根据生活中的经验不难知道加速度的大小和所受到力是成正比的。比如汽车起步时加速度变化很快，我们能感觉到很明显的推背感。这就是加速度和力关系的一个表现。那么这个力我们又该怎么去测呢？大家还记不记得以前去菜市场买菜用的挂钩秤，那个称重的弹簧就是来测力的。一个苹果的重量差不多 1 牛顿，

我们换算一下公式（ $1\text{kg}=9.8\text{N}$ ），大概就只到重量的大小。而秤的重量测量是通过弹簧的形变来判断的，因此我们要测量的力也可以通过弹簧的形变来判断，不过我们需要判断的可不止一条直线上的力。

如图 1-12 所示，我们把图中这样一个东西装在车内，通过检测弹簧的形变就能知道小球在 $X+$ 和 $X-$ 方向上加速度的变化了。当然光知道 x 轴方向的加速度还不够，我们还需要知道 Y 和 z 轴的加速度，才可以知道在整个空间中小球到底在做什么运动。

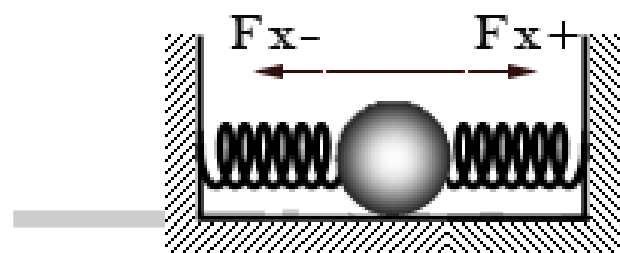


图 1-12 弹簧力（重新绘制一下）

我们判断出 x 轴方向上的力，同理也可以得到 Y 和 z 轴方向上的力，这样三个轴的力我们都得到了。当然现在的芯片内部结构也不可能是弹簧和小球的结构来探测加速度数值的。而是硅微电容式、压阻式、热电偶式等。

如果你对芯片内部原理感兴趣的话接下来我们将探讨一些芯片内部原理。

1.2.1 硅微电容式加速度传感器

硅微电容式加速度传感器内部通常为梳齿结构，如图 1-13 所示。敏感元件由活动部分 A、B 和固定电极三部分组成。其中的活动部分是由超静定梁、质量块和与质量块相连的活动电极组成的。整个梳齿结构分成 A 部分和 B 部分，固定电极 A 与上半部分活动电极组成电容 C_a ，固定电极 b 与下半部分活动电极组成电容 C_b 。将上、下极板不同区域用电极引线分开，这是为了实验的时候可以根据需要将其短接或分开。这样，当检测到 A 端有加速度产生时，产生的惯性力会使活动极板产生 1 个 α 偏角使电容器 C_a 的电容量增加，而使 C_b 的电容量减小，然后把电容器的电容量转换成电信号，经相敏放大后把输出电压反馈到电容静电力矩器，电容量力矩器产生的静电力矩与惯性力矩平衡。

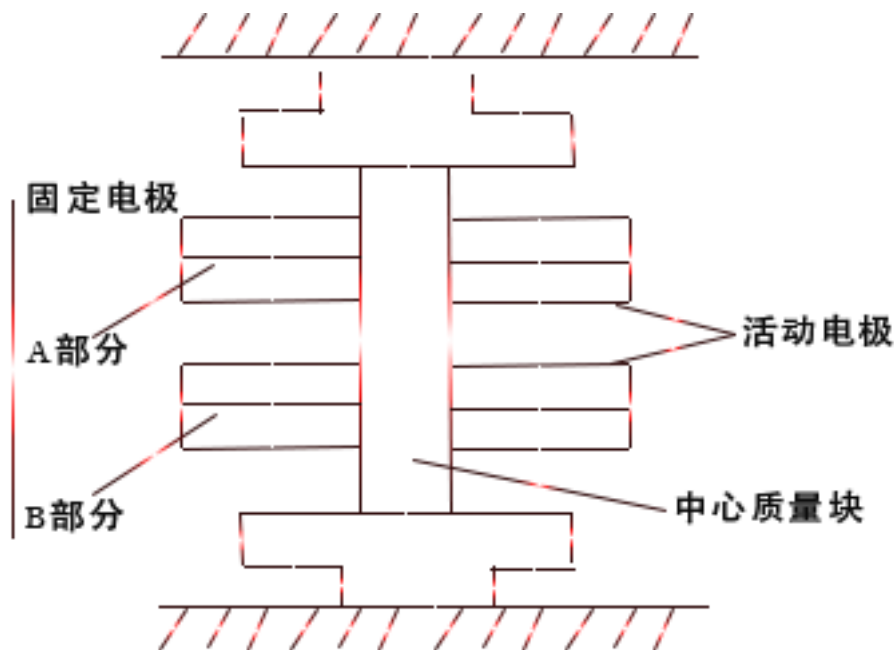


图 1-13 电容式加速度计内部原理

简单来说，就是传感器在受到加速度作用时，动极板会产生方向性运动。这使得动极板两侧的电容 C_a 、 C_b 发生变化。两侧电容的差值（2 倍于单侧的变化量）经激励正弦波信号调制后，会通过电容检测器检出。然后将该信号通过交流放大器放大、检波和适当的校补，反馈到中间的极板，由其在电容器极板间产生的静电力，此静电力的力矩使检测质量块保持在零位。它与加速度作用所引人的力矩大小相等，方向相反。这样，我们就得到反应加速度的电容值了。

采用 MEMS 有关工艺制成的微加速度计，其敏感芯片（也就是探测部分）的体积仅有 0.5 mm^2 ，和成年人的小指甲盖 $1/10$ 大小差不多，比采用精密机械加工成的加速度计还要小 $1 \sim 2$ 个数量级。由于其质量小，可承受高强度冲击，实验得出这种原理的微加速度计在不加电状态下三个方向至少可以承受数百乃至数千 G 以上的冲击。这就是 MPU9250 类型传感器的内部原理。

1.2.2 硅微压阻式加速度传感器

当半导体单晶硅材料在受到外力作用时候，会产生肉眼察觉不到的极微小应变，其原子结构内电子能级状态会发生变化，从而导致其电阻率发生剧烈的变化，由其材料制成的电阻也会出现极大变化，这种物理效应叫压阻效应。在双端固支梁结构中，质量块会像活塞一样做上下运动，该结构形式的传感器示如图 1-14

所示。

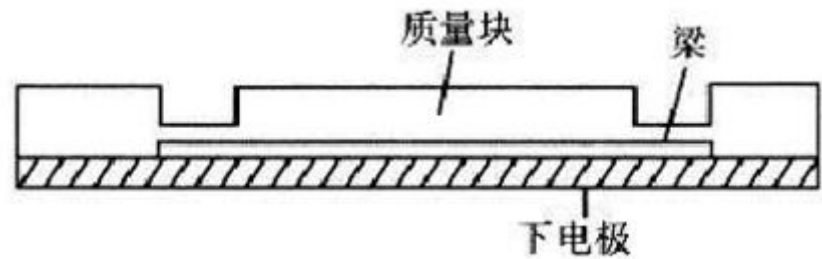


图 1-14 硅微压阻式内部原理图

压阻式加速度传感器的弹性元件一般采用硅梁加质量块的模式，质量块由悬臂梁支撑，并在悬臂梁上制作电阻，连接成测量电桥。在惯性力作用下质量块上下运动，悬臂梁上电阻的阻值随应力的作用而发生变化，引起测量电桥输出电压的变化，这样就能测量到了加速度的数据。压阻式硅微加速度传感器的典型结构形式有很多种，有悬臂梁、双臂梁、4 梁和双岛-5 梁等结构形式。弹性元件的结构形式及尺寸决定传感器的灵敏度、频响、量程等。质量块能够在较小的加速度作用下，使得悬臂梁上的应力较大，提高传感器的输出灵敏度。在大加速度下，质量块的作用会使悬臂梁上的应力超过屈服应力，变形过大，致使悬臂梁断裂。因为 g_n 值较高，所以加速度计是采用质量块和梁厚相等的单臂梁和双臂梁的结构形式来测量的，如图 1-15 所示。

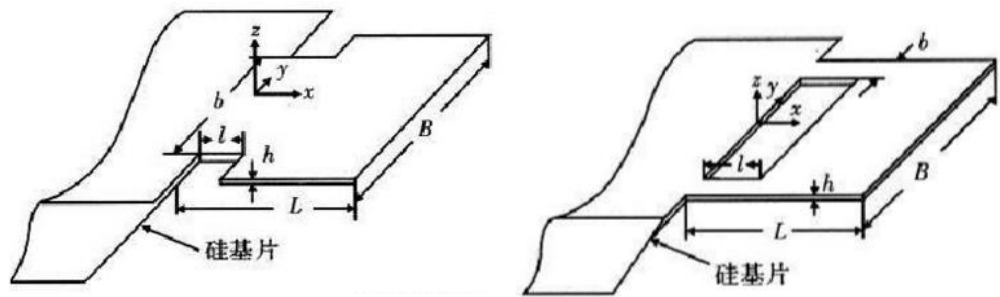


图 1-15 单臂梁结构

然后把测试得到的加速度和电压的数据记录下来，得到如图 1-16 所示的折线图，即可得到其中的比例关系。

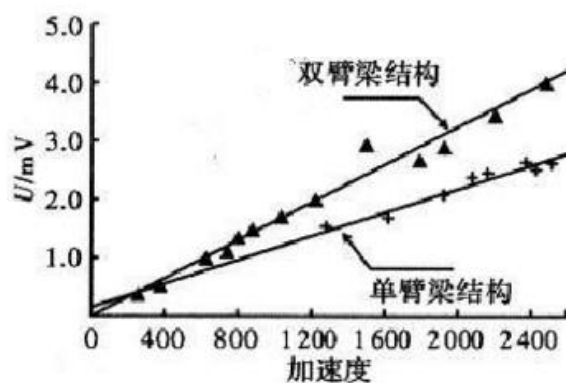


图 1-16 加速度与形变的关系

压阻式原理的加速度传感器虽然体积小、但是频率范围宽、测量加速度的范围宽，而且可以直接输出电压信号，并不需要复杂的电路接口，所以大批量生产时价格更低廉，可重复生产性更好，可直接测量连续的加速度数据，但其温度的漂移较大，对安装时的干扰也较敏感，所以在做微小加速度数据检测的情况下表现不是很好。

1.2.3 硅微热电偶式加速度传感器

硅微热电偶式的加速度传感器多应用于低成本的传感器，此类加速度传感器既可以测量动态加速度，也可以测量静态加速度。基于热交换原理，介质是气体。

如图 1-17 所示，热源处于硅片的中央，硅片悬在空穴中间。在热源的四周均匀分布有热电偶堆（铝/多晶硅）图中的加速度传感器上有两路信号，一路是测量 x 轴加速度的，另一路是测量三轴加速度的。在没有加速度的情况下，热源的温度梯度均匀分布，对四周的热电偶而言，温度是一样的，输出的电压也是一样的。当检测到加速度的时候，所有方向温度分布不再平衡。输出的电压也将随之改变。和上面一样可以得到一个测量的热度电压和加速度的线性图标，从而看出热电偶输出的电压差和加速度成正比。

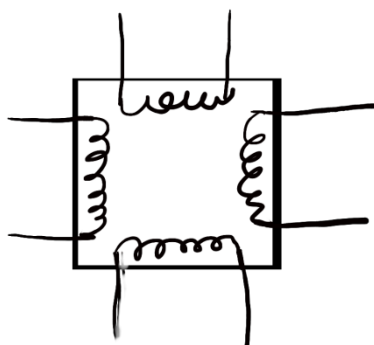


图 1-17 热电偶式传感器

1.2.4 谐振式加速度传感器

谐振式传感器的独特优点是它的准数字量输出可直接用于复杂的数字电路而不会产生其他类型加速度传感器在信号传递方面的弊端（比如产生温漂等）。谐振式传感器的敏感元件是谐振子，谐振子的固有谐振特性直接决定了该类型传感器具有很高的灵敏度和分辨率，但：

1. 硅谐振子的材料质量和制作质量一定要得到保证；
2. 要有足够高精度的数字信号处理电路来检测输出频率信号的微弱变化。

随着硅材料工艺、微机械加工工艺和集成电路的飞速发展，这些问题变得非常容易解决，也使得谐振式传感器成为低成本、高性能传感器的突出代表。

现在比较新颖的还有硅微光波导加速度传感器，原理也是通过加速度造成光的反射发生变化导致光的位移（当然这个位移和加速度又有一个关系式，这里不再给出了）来计算加速度的大小。

关于加速度计内部设计原理的介绍就先给大家讲到这里。更详细的内容大家可以自行去网上查阅相关资料，或关注中国微纳米技术俱乐部会长的微博账号——MEMSensor，会不定时的推送最新行业内的新内容，这里就不再过多赘述。

1.2.5 加速度传感器计算思路

我们现在再回到之前的弹簧模型。得到数值后怎么就能供我们计算使用呢？下面来讲一下加速度的相关计算思路。

为了方便大家思考和学习我们现在用一个四轴飞行器做假设，我们假设这里有一架四轴飞行器悬停在空中，而且在飞行器上固定了一个正方体的盒子。这个盒子中有一个无重量的小球（在计算时可以不受重力的影响），它的对应盒子的6个面都被弹簧正好抵着（此时弹簧形变为0），如图 1-18 所示。

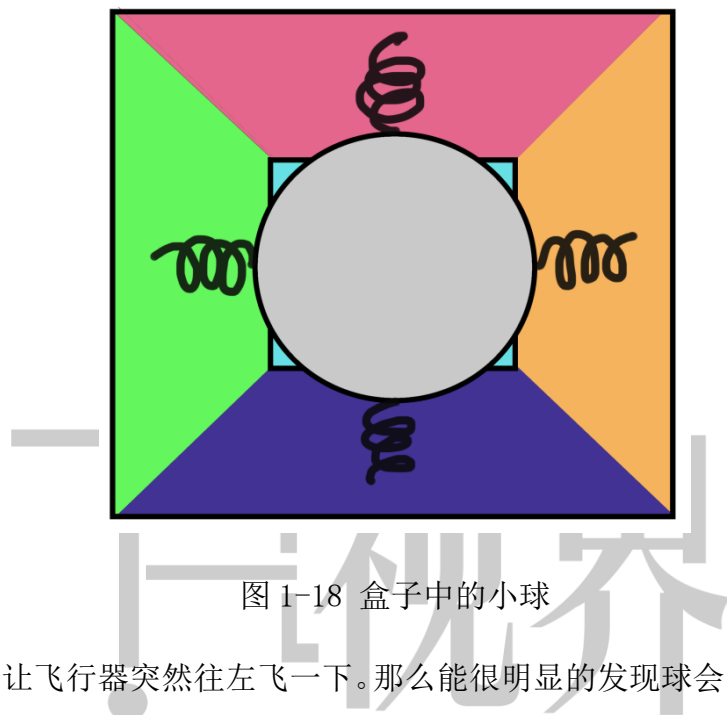


图 1-18 盒子中的小球

此时我们让飞行器突然往左飞一下。那么能很明显的发现球会向右边的墙压一点儿过去。如图 1-19 所示，此时我们的加速度计可能会感知到 $x=-1g$, $Y=0g$, $Z=0g$ ，我们可以明显的发现加速度和我们检测到的力的方向完全是相反的。

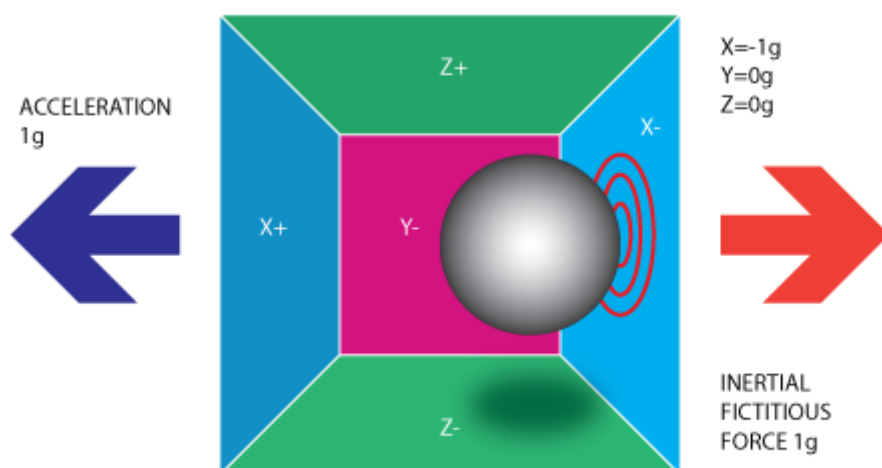


图 1-19 向右偏的小球

这种力我们称之为惯性力（其实是没有惯性力这种力的，我们也可以称之为假象力）。

只要是在地球上运动的物体，在计算的时候还有个力是不可忽略的，所以我们无论在哪都需要考虑进去，这个力就是重力。如图 1-20 所示，此时的小球有重量了。那么，我们的传感器也迅速测出了 $X=0g$ $Y=0g$ $Z=-1g$ 的数值。

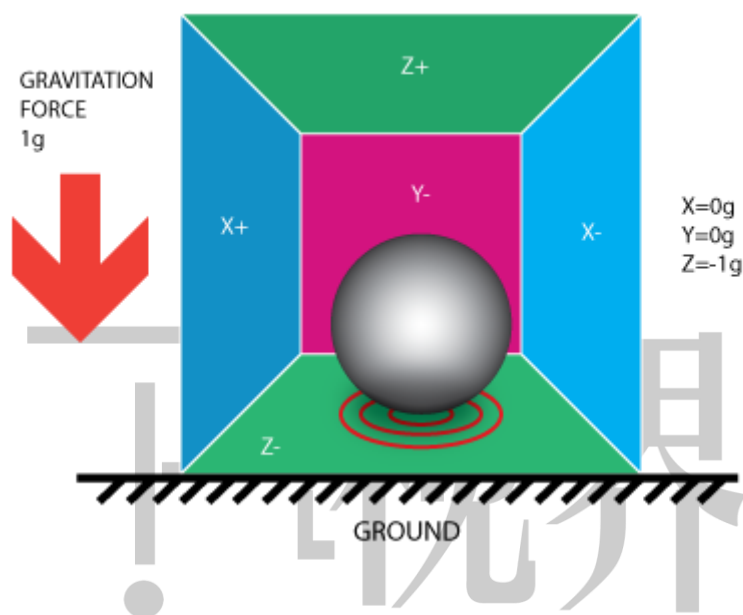


图 1-20 盒中小球受重力

如果以地球为参考系的话，就并不是有加速度就说明物体是在运动的。他此时相对地球静止。那么我们此时让飞行器顺时针倾斜 45° 看看会怎么样？如图 1-21 所示：

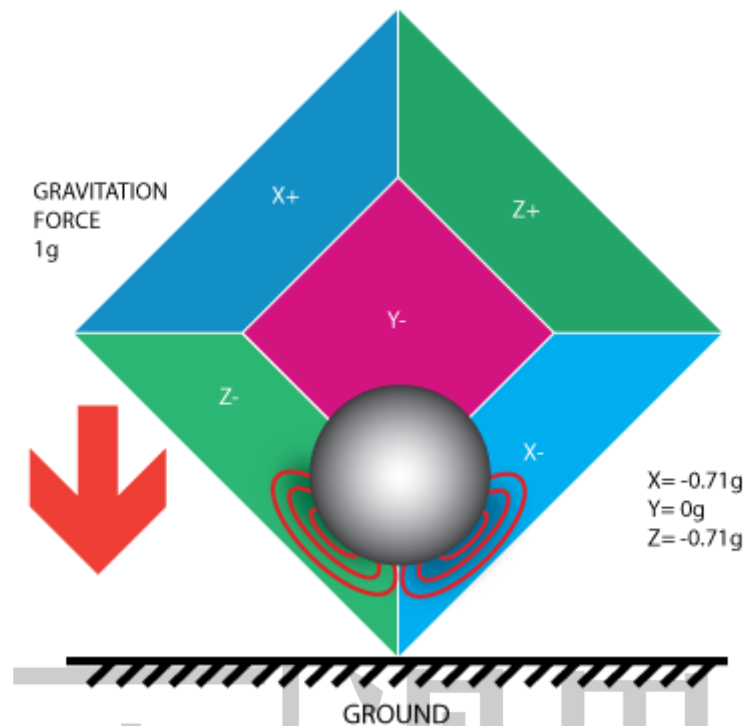


图 1-21 倾斜盒子受重力的小球

注意，此时小球并未与 Y 平面发生接触，我们可以从加速度计中读取到： $X = -0.71g$ $Y = 0g$ $Z = -0.71g$ ，此时飞行器是静止的，小球也是静止的，X 和 Z 方向上的力把重力加速度分担开来了。加速度既是瞬时力，得到的加速度的值又能干什么用呢？我们再来看。

在上面的基础上此时把倾斜的飞行器前后方向倾斜 45° 。得到如图 1-22 所示，此时小球对我们模型的三个面都有相同的压力了。

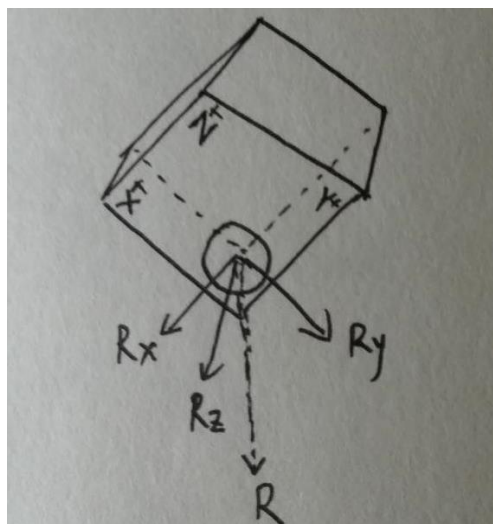


图 1-22 倾斜盒子中三面受重力的小球

我们把这个模型受力换成直观的空间坐标图 1-23，原点即为小球中心点。
 R_x ， R_y 与 R_z 分别对应上图中的 R_x ， R_y 与 R_z 。 R 为小球所受重力，也是 R_x ， R_y 和 R_z 的合力。

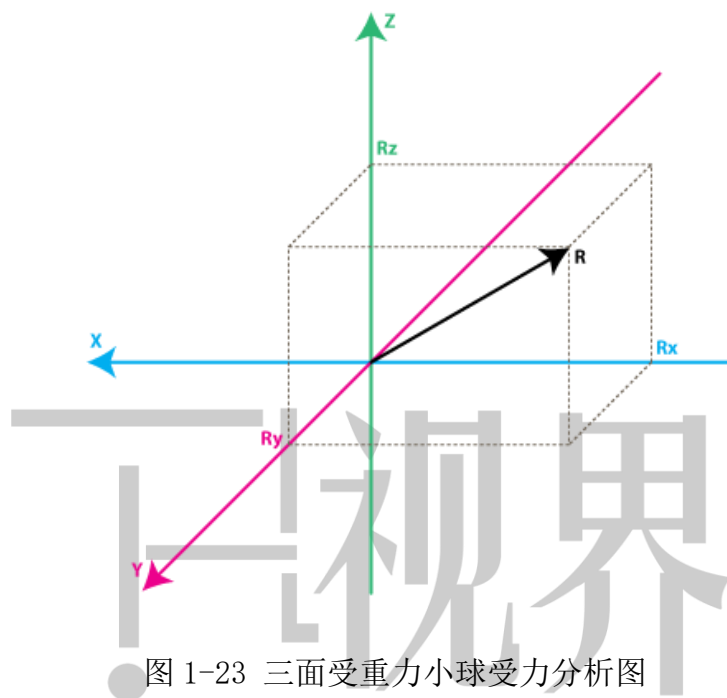


图 1-23 三面受重力小球受力分析图

现在应该知道获得的加速度的力是用来计算什么的吧。没错，就是角度！

下面通过一个例子来告诉大家角度具体是怎么计算的：

比如加速度计测得 $X=R_x$ $Y=R_y$ $Z=R_z$ 的这三个值，设矢量合力为 R （注意 R 也是我们芯片受到重力的方向）图 1-23，根据高中空间几何知识，那么不难得：

$$R^2=R_x^2+R_y^2+R_z^2$$

比如加速度计读到：（注意这里，为了计算出不同角度假设这里小球不再是均匀的与三面受力，这里我们用模拟型传感器来举例，MPU9250 是数字型传感器，但是原理相通）

$$R_x=2.00V$$

$$R_y=1.50V$$

$$R_z=0.75V$$

然后我们还需要减去加速度的偏移量才能得到实际的值，偏移量的大小是根据芯片所处不同情况（温度、气压、材料、位置等）而产生的不同误差来决定的。第三章和第四章以及第五章会慢慢讲解解决漂移的方法。为了方便理解，在这里假设加速度计三轴的偏移量都是 0.50V（这里只是假设，真实值正常情况下可能没有这么大偏移），此时可以得到：

$$R_x = 2.00V - 0.50V = 1.50V$$

$$R_y = 1.50V - 0.50V = 1.00V$$

$$R_z = 0.75V - 0.50V = 0.25V$$

我们最终要得到是加速度的单位所以还需要把加速度的电压值转换成加速度，这个值在芯片的加速度寄存器配置手册里面有写，配置的不同计算的数值不同，第三章会讲。那么此处为了方便计算假设传感器被设置的灵敏度为 0.5V/g，此时便可得到：

$$R_x = 1.50V / 0.5V/g = 3g$$

$$R_y = 1.00V / 0.5V/g = 2g$$

$$R_z = 0.25V / 0.5V/g = 0.5g$$

因此，就可以根据最初的公式 $R^2 = R_x^2 + R_y^2 + R_z^2$ 得到合力 R 的值为 3.64g。如图 1-24 所示：

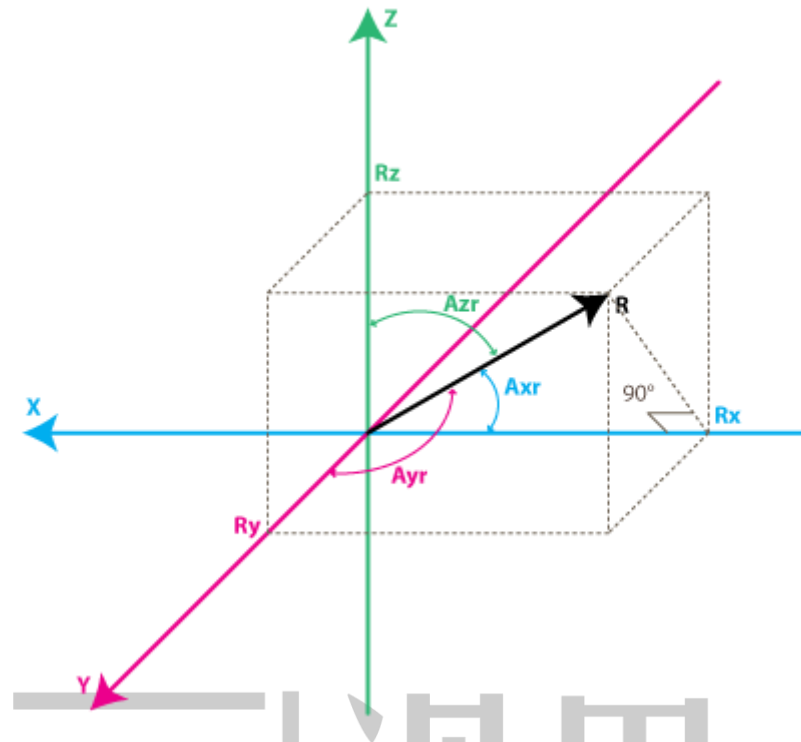


图 1-24 姿态角度分解

知道三角形两个边的数值了，那么此时姿态的角度的值根据三角函数也可以很容易计算出来：

$$\cos \angle xr = (R_x / R) = 0.824$$

$$\cos \angle yr = (R_y / R) = 0.549$$

$$\cos \angle zr = (R_z / R) = 0.137$$

然后计算反三角函数得到角度：

$$\angle xr = \arccos (R_x / R) = 34.5^\circ$$

$$\angle yr = \arccos (R_y / R) = 56.7^\circ$$

$$\angle zr = \arccos (R_z / R) = 82.12^\circ$$

此时，芯片此时姿态方向与每个轴的夹角度数就都知道了，这就是芯片的瞬时姿态。为什么说是瞬时姿态？因为加速度检测的是那一瞬间芯片所受到的瞬时加速度数值。现在就可以把得到的值传输到软件中去展示你现在传感器的实时姿态了。但你会发现，当你把芯片来回快速转动的时候，芯片却并不能连续正确的

反映出物体的姿态数据。这是为什么呢？我们还需要陀螺仪来帮忙。

1.3 陀螺仪



图 1-25 《盗梦空间》中的陀螺

其实我们在第一节引入概念的时候就讲过陀螺的原理。陀螺和陀螺仪当然是不一样的。上一章我们知道了加速度计是计算瞬时姿态的，那么既然姿态角度已经有了，我们还要陀螺仪干什么呢？我们先来分析分析陀螺仪的结构。如图 1-26 所示：



图 1-26 陀螺仪

仔细观察陀螺仪的结构就不难发现陀螺仪在三个相互垂直的方向各有一根

横梁。中间那根黄色的杆决定了黄色转子是围绕着它做高速旋转运动。这里面每一根杆都允许他所承载的载体在这个方向上做正反 360° 旋转。而这三根杆刚好反映出了中心转子在 x 轴和 y 轴 z 轴方向上分别旋转的角度。就好像我们的芯片分别围绕自身 x 轴，y 轴和 z 轴旋转的角度。如图 1-27 所示：

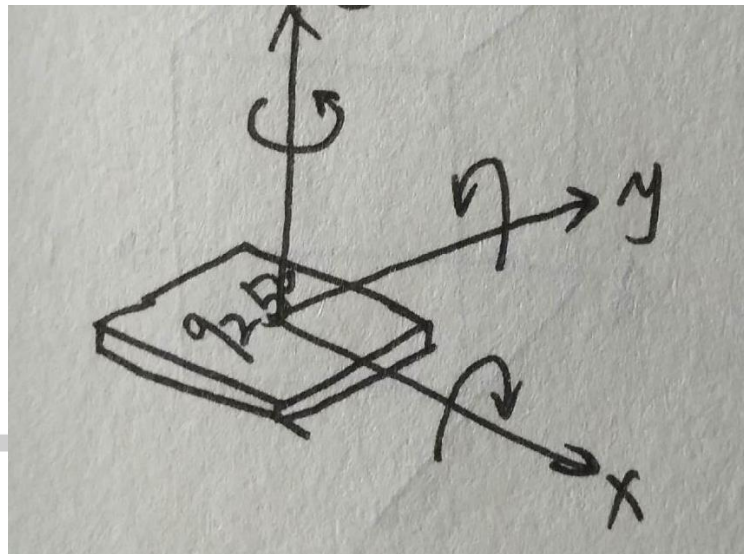


图 1-27 MPU9250 的陀螺仪三轴方向规定图

其实对于陀螺仪传感器我们还可以再分解成单轴，就像加速度分解成弹簧秤一样。

分解出来最相近的也就是图 1-28 所示的单轴角度传感器。



图 1-28 单轴角度传感器

单轴角度传感器是纪录传感器在一个单轴的角度倾斜的表现。也就是我们之前 XYZ 三轴，任取一轴做测量。比如图 1-29 所示，假如绕 x 轴旋转的话，就可以记录出传感器在绕 x 轴在 $-180^{\circ} \sim +180^{\circ}$ 的角度的单独变化。

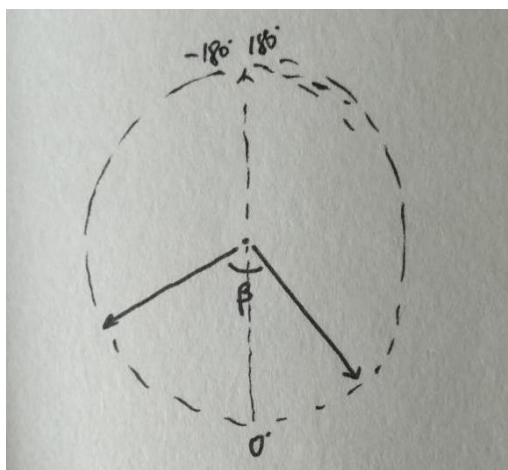


图 1-29 单轴角度传感器角度分析图

但是大家不要以为陀螺仪就是把三个单轴角度传感器绑在一起做成的。对 MEMS 陀螺仪内部芯片原理感兴趣的话，下面笔者简单介绍一下陀螺仪芯片中的原理。

之前讲的传统陀螺仪其实主要是利用角动量守恒的原理，它是一个不停转动的物体，它的转轴指向不随承载它的支架的旋转而变化。但是 MEMS 陀螺仪（gyroscope）的不可能也这么设计，因为要用微机械技术在硅片衬底上加工出一个可转动的结构可不是一件容易的事。所以工程师们在 MEMS 陀螺仪中利用的科里奥利力（旋转物体在有径向运动时所受到的切向力）。简单科普一下科里奥利力：

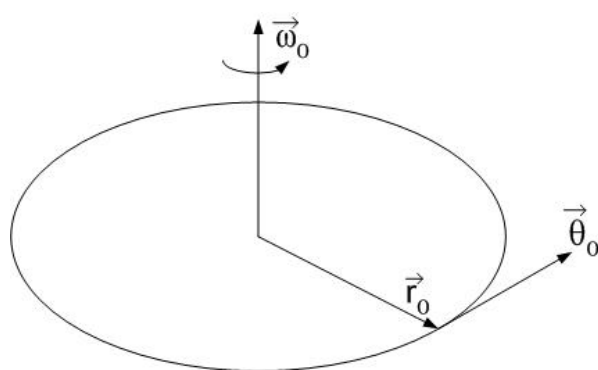


图 1-30 陀螺仪内部震动

在空间设立动态坐标系（见图 1-30）。通过方程计算加速度可以得到三项速度值，分别来自径向加速、科里奥利加速度和向心加速度。

假设我们此时在圆盘上放置一个小块，如图 1-31 所示。科里奥利力产生的条件就是物体在圆盘上做径向运动。因此，在 MEMS 陀螺仪的设计上，当这个物体被驱动，不停地来回做径向运动或者震荡，与此对应的科里奥利力就会不停地横向来回变化，并有可能使物体在横向作微小震荡，相位正好与驱动力差 90 度。（如图 1-31 所示）

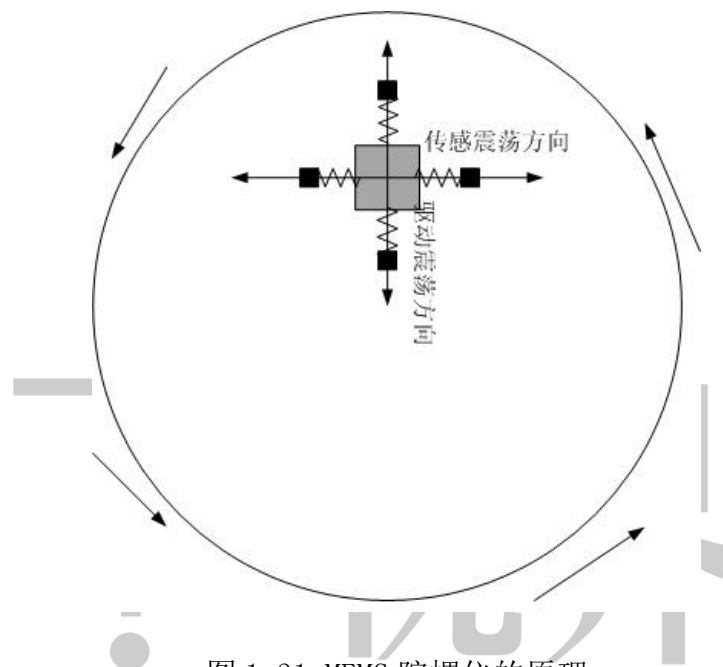


图 1-31 MEMS 陀螺仪的原理

由于 MEMS 陀螺仪内部有两个方向的可移动电容板。通过径向的电容板加震荡电压迫使物体作径向运动（有点像加速度计传感器中的自测试模式），横向的电容板用来测量由于横向科里奥利运动带来的电容变化。因为科里奥利力正比于角速度，所以我们可以由电容的变化来计算出角速度。**A3 交变科里奥利力在陀螺仪中的使用。**

以下便是陀螺仪芯片内部封装原理图。它采用了闭合回路、数字输出和传感器芯片跟 ASIC 芯片分开平放连线的封装方法。总体来说原理都和我们之前讲解的差不多，ADC 转换检测电压，再计算，叠加数据，反馈数据，增加温湿度传感器的补偿，以 SPI 信号模式输出。

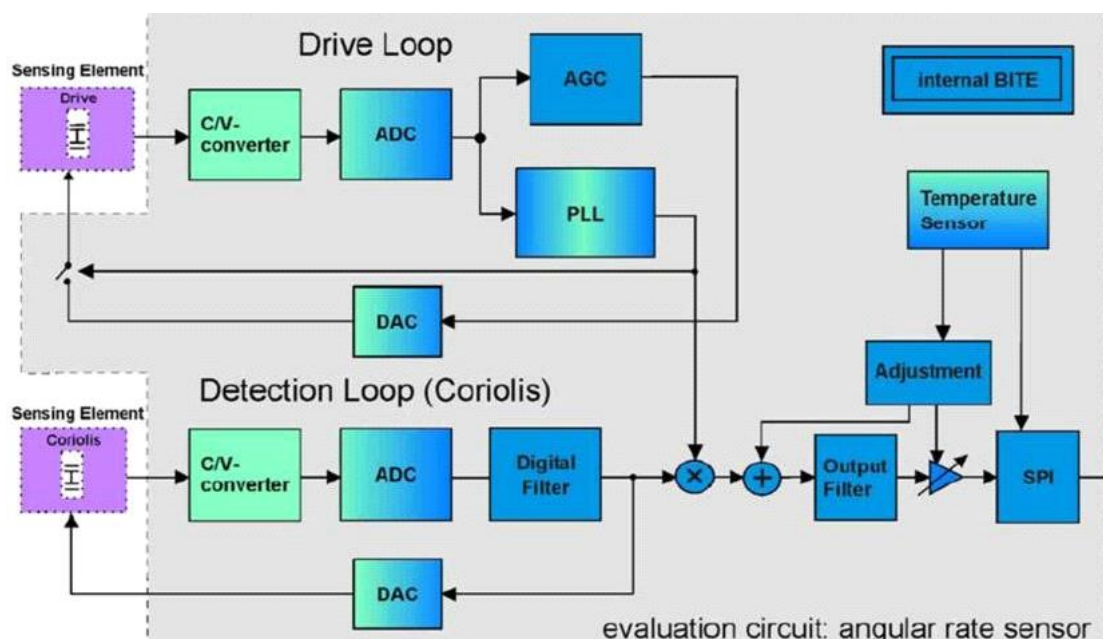


图 1-32 传感器内部原理

其实 MEMS 陀螺仪 (gyroscope) 的设计和工作原理不止这一种，但是公开的 MEMS 陀螺仪都是采用振动物体传感角速度的概念。利用振动来诱导和探测科里奥利力，这样的 MEMS 陀螺仪内部并没有旋转部件、且不需要轴承，可以用微机械加工技术大批量生产。

一般的 MEMS 陀螺仪由梳子结构的驱动部分和电容板形状的传感部分组成（见图 1-33）。

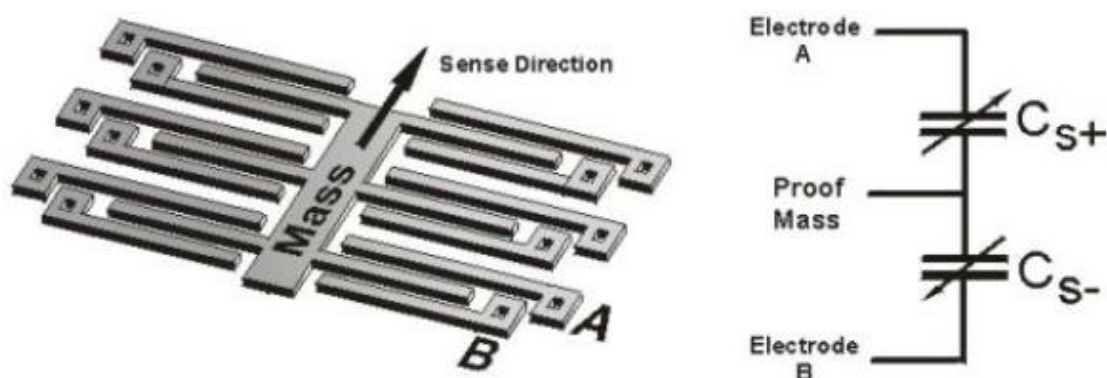


图 1-33 陀螺仪传感器内部结构

有的设计还带有去驱动和传感耦合的结构（见图 1-34）。

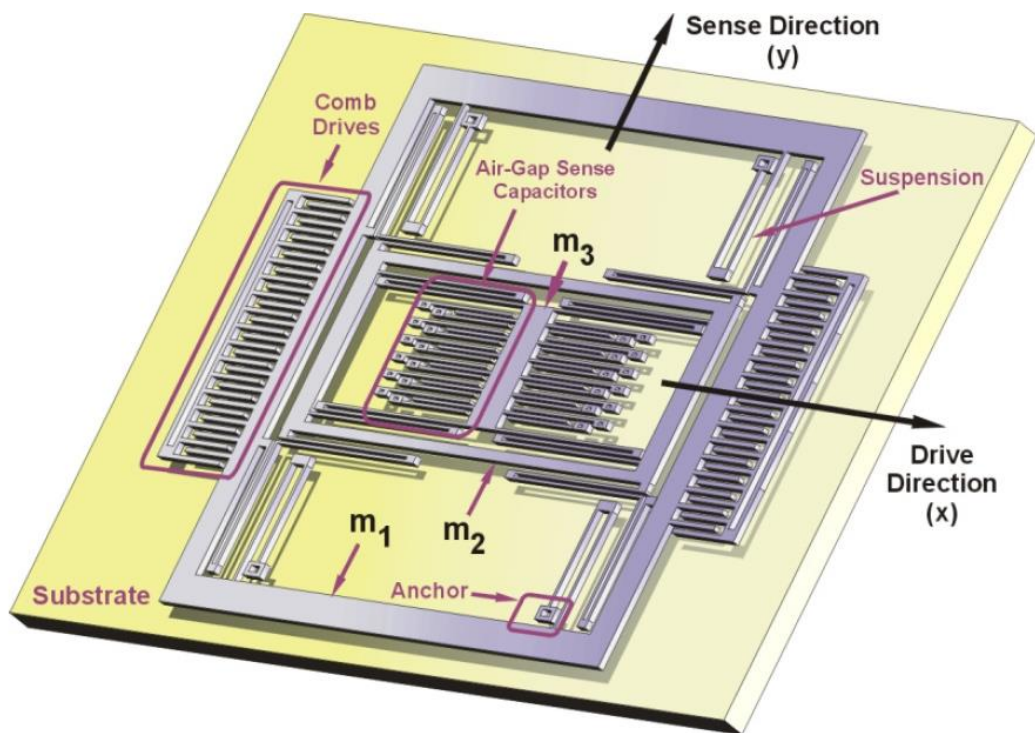


图 1-34 耦合结构传感器

陀螺仪并不是测量角度的，而是测量角度变化量的传感器，也就是角速度。
继续拿第三节中的模型来讲解，如图 1-35 所示：

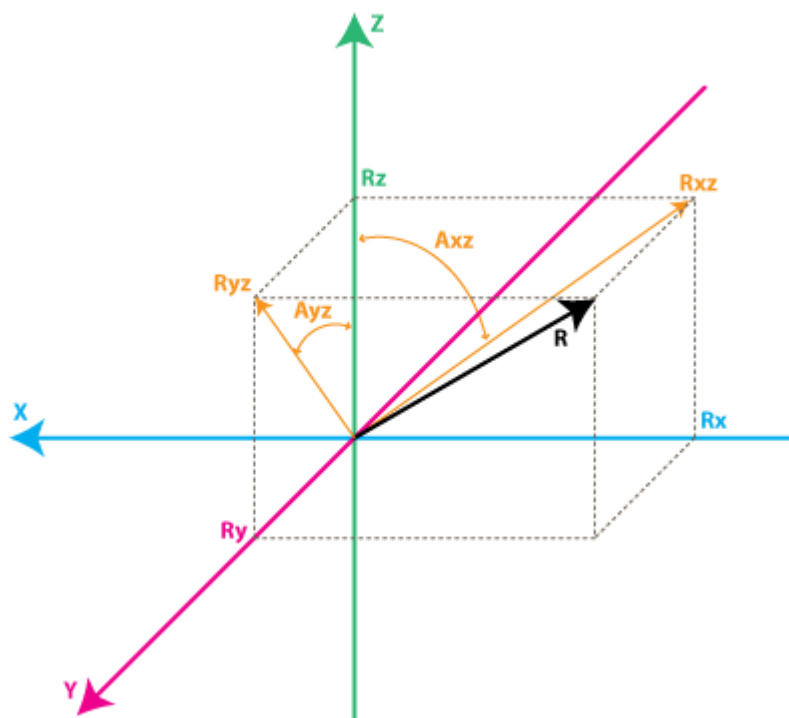


图 1-35 角速度

我们现在给这个模型加上两个角度。也就是图中的 $\angle_{xz}$ 和 $\angle_{yz}$ ，此处 R_{xz} 是惯性力矢量 R 在 XZ 平面上的投影 R_{yz} 是惯性力矢量 R 在 YZ 平面的上投影。此时，我们让这个模型（ R ）围绕 x 轴做逆时针旋转，然后（ R ）再围绕着 y 轴做逆时针旋转。从陀螺仪中可以得到（为了方便计算我们此处都用简单的数值，还是以模拟电压型传感器做例子）：

$$\angle_{xz} \text{ 速率} = 2.00V$$

$$\angle_{yz} \text{ 速率} = 0.50V$$

和加速度一样，陀螺仪也有偏移量，所以在这里也减去陀螺仪的偏移量。比如这里是 $1.00V$ ，那么可以得到较真实电压数据：

$$\angle_{xz} \text{ 速率} = 2.00V - 1.00V = 1.00V$$

$$\angle_{yz} \text{ 速率} = 0.50V - 1.00V = -0.50V$$

然后再根据陀螺仪的灵敏度转化为角速率，比如设置的陀螺仪的灵敏度是 $0.1V/\text{deg/s}$ 那么我们现在就可以得到：

$$\angle_{xz} \text{ 速率} = 1.00V / 0.1V/\text{deg/s} = 10^\circ /s$$

$$\angle_{yz} \text{ 速率} = -0.50V / 0.1V/\text{deg/s} = -5^\circ /s$$

所以陀螺仪算出的数据就是角速度。这就可以解决了上一章快速转动无法得到平滑姿态的问题。那么 $\angle_{yz}$ 速率怎么会有带负号？举个例子，当一架飞行器在一秒钟内顺时针旋转 360° 时，会测出来 $360^\circ /s$ 。那么逆时针旋转自然就是 $-360^\circ /s$ 了，所以 $-5^\circ /s$ 就是逆时针的角速率了。关于传感器方向判定的问题在所有的姿态传感器的寄存器手册里面有规定，所以按照官方的设定方向即可。现在既然知道加速度数值，也知道陀螺仪速率了，如何使用这两个角度数据呢？这就是本书着重要讲的内容。

1.4 磁力计

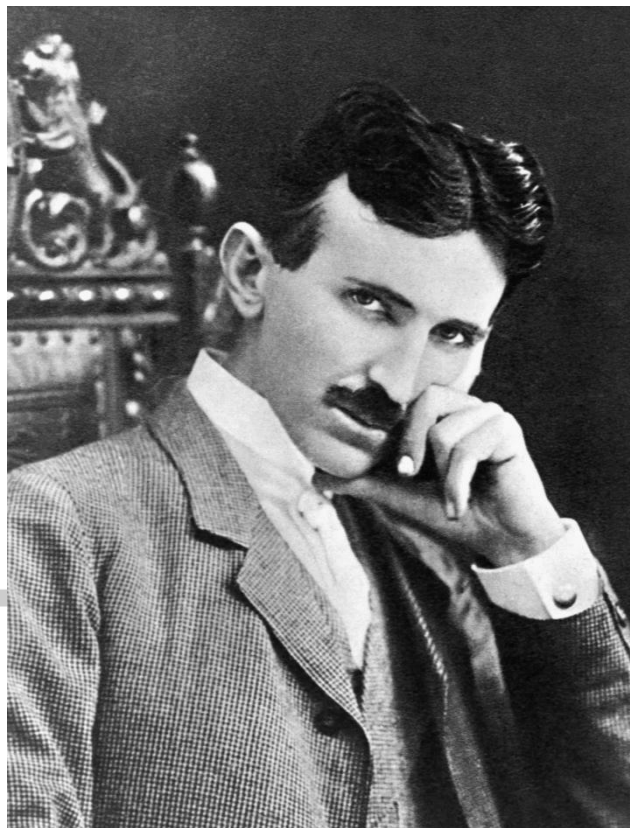


图 1-35 尼古拉·特斯拉，约 1896 年摄

我们这节要计算的磁感应强度大小的单位就是特斯拉(如图 1-35 所示)Tesla 或者高斯 Gauss ($1\text{tesla}=10000\text{gauss}$)。这个单位便是纪念这两位伟大的先驱。

A4 特斯拉是否被“神”化

关于地球磁场的知识大家可能知道的并不多。人毕竟是视觉动物，这种看不见又摸不着的东西，研究起来还是很麻烦。目前我们所掌握的知识差不多有：

1. 从地理课本中我们知道地球磁场的极点和地理的极点之间并不重合，他们之间有着大概 11° 左右的偏差。

2. 地磁和地理位置的关系刚好是相反的。也就是说地理南极附近的是地磁北极 (N)，地理北极附近的是地磁南极 (S)。

现在的电子罗盘和以前的指南针的原理也大不一样。地球磁场的强度只有 $0.4\text{--}0.6\text{Gauss}$ ，这其实是非常非常微弱的磁场。比手机内部扬声器工作时产生的

磁场还小，所以电子设备的抗干扰还是一门技术活啊。而磁场传感器芯片需要探知的就是这个微小强度磁场。

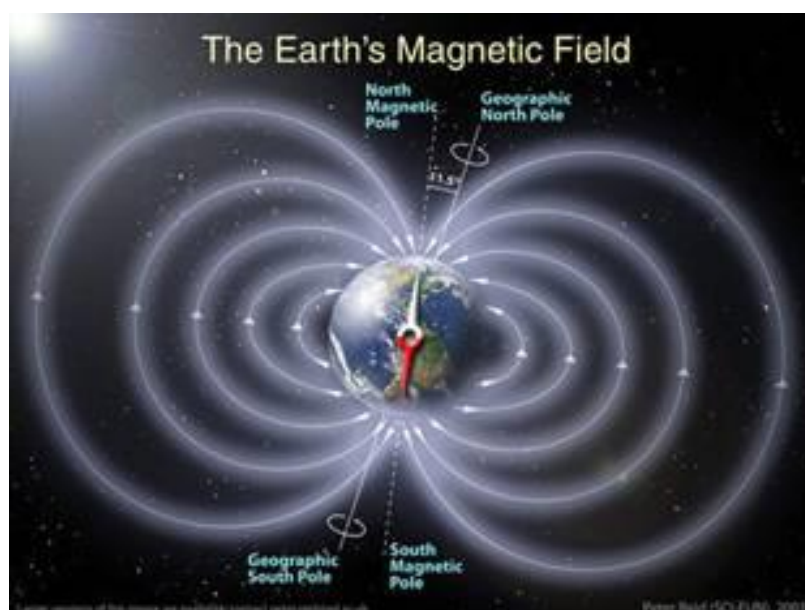


图 1-36 地球磁场分析

和上面一样，下面就来详细讲解一下电子罗盘内部的结构组成原理。（以下部分电子罗盘芯片内部原理来自电子信息港 STM8 社区）

我们以 LSM303 为例子来讲解一下磁力计的内部构造。LSM303 是一块磁力计的传感器。以前没有九轴传感器这样集成的芯片的时候，通常的做法是，姿态电路板上是六轴传感器再单独加上磁力计或者加上气压芯片。其实 MPU9250 也是这样的，只是在 MU6050 的基础上增加了一个磁力传感器罢了。LSM303 这种磁力计，里面还自带了加速度计。

LSM303DLH 中磁力计采用各向异性磁致电阻材料来检测空间中磁感应强度的大小。这种具有晶体结构的合金材料对外界的磁场很敏感，磁场的强弱变化会导致 AMR 自身电阻值发生变化。在制造过程中，将一个强磁场加在 AMR 上使其在某一方向上磁化，建立起一个主磁域，与主磁域垂直的轴被称为该 AMR 的敏感轴，如图 1-37 所示。

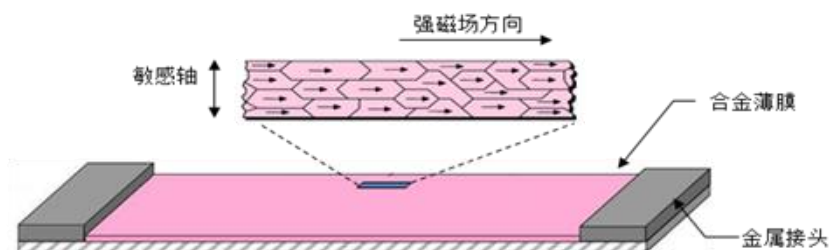


图 1-37 AMR 材料示意图

为了使测量结果以线性的方式变化，AMR 材料上的金属导线呈 45° 角倾斜排列，电流从这些导线上流过，如图 1-38 所示。由初始的强磁场在 AMR 材料上建立起来的主磁域和电流的方向有 45° 的夹角。

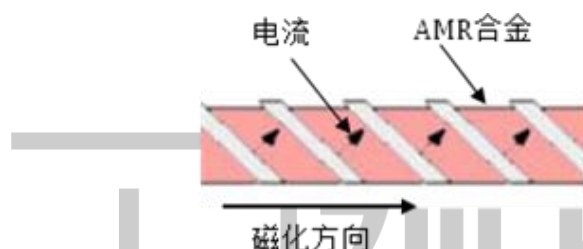


图 1-38 45° 排列的导线

当有外界磁场 H_a 时，AMR 上主磁域方向就会发生变化而不再是初始的方向了，那么磁场方向和电流的夹角 θ 也会发生变化，如图 1-39 所示。

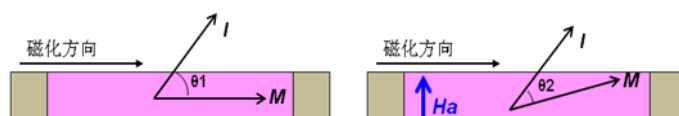


图 1-39 磁场方向的夹角

对于 AMR 材料来说， θ 角的变化会引起 AMR 自身阻值的变化，并且呈线性关系，如图 1-40 所示。

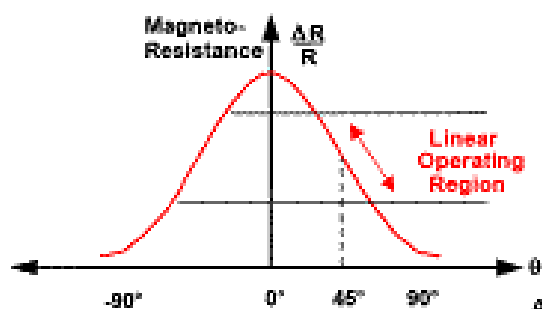


图 1-40 特性曲线

ST 则是利用惠斯通电桥检测 AMR 阻值的变化,如图 1-41 所示, $R_1/R_2/R_3/R_4$ 是初始状态相同的 AMR 电阻,但是 R_1/R_2 和 R_3/R_4 具有相反的磁化特性。

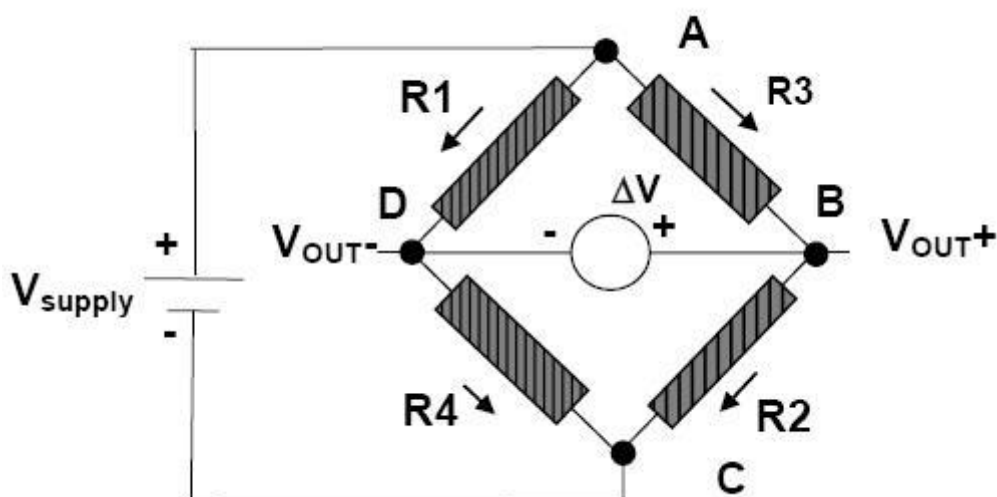


图 1-41 惠斯通电桥

当检测到外界磁场的时候, R_1/R_2 阻值增加 ΔR 而 R_3/R_4 减少 ΔR 。这样在没有外界磁场的情况下,电桥的输出为零;而在有外界磁场时电桥的输出为一个微小的电压 ΔV 。当 $R_1=R_2=R_3=R_4=R$, 在外界磁场的作用下电阻变化为 ΔR 时,电桥输出 ΔV 正比于 ΔR 。所以就根据电压的大小来判定磁场的大小。

我们都知道,磁场计算的时候最大的麻烦就是干扰,这个干扰比加速度和陀螺仪大多了,况且还要探知这么微小的地球磁场。地球的磁场在 0.4-0.6 高斯,而我们平时用手机里面扬声器就差不多 4 高斯,微动马达都至少 6 高斯。所以看智能手机里面电路板和传感器的布局,一般扬声器和马达都放在底部或都集中在顶部,离磁力计远远的。但光是离开一段距离还是不管用,应为手机周边环境中的因素也会干扰我们手机内部的磁力计。比如你把手机放在音响旁边,放在磁铁附近,放在充电器周围都会干扰到磁力计的测量数据。所以在用手机指南针的时候手机基本上都会让我们做现场的矫正。



图 1-42 手机磁场矫正

图 1-23 的这种方法就是 8 字校准法，让用户将设备按照 8 字在空中做来回摆动。其实类似的还有平面校准法和十面校准法（将设备分别朝向不同面的前后等），但不管是用什么方法校准，最终原理其实都是一样的。让我们的传感器在空间中在各个方向上做旋转，最终测量出来的值的范围在空间几何中就是一个圆球（实际上是一个椭球体），所有的采样点都在这个圆球上，而圆心，就是我们需要矫正成的数值，这个具体矫正方案我们将在第 4 章说明。

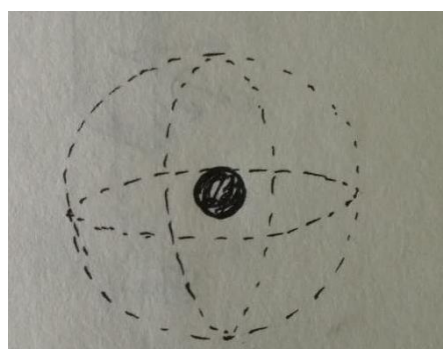


图 1-43 磁力计测量范围形成的球体

这个比较直观的表现 iPhone 的电子罗盘矫正上。如下图 1-44：

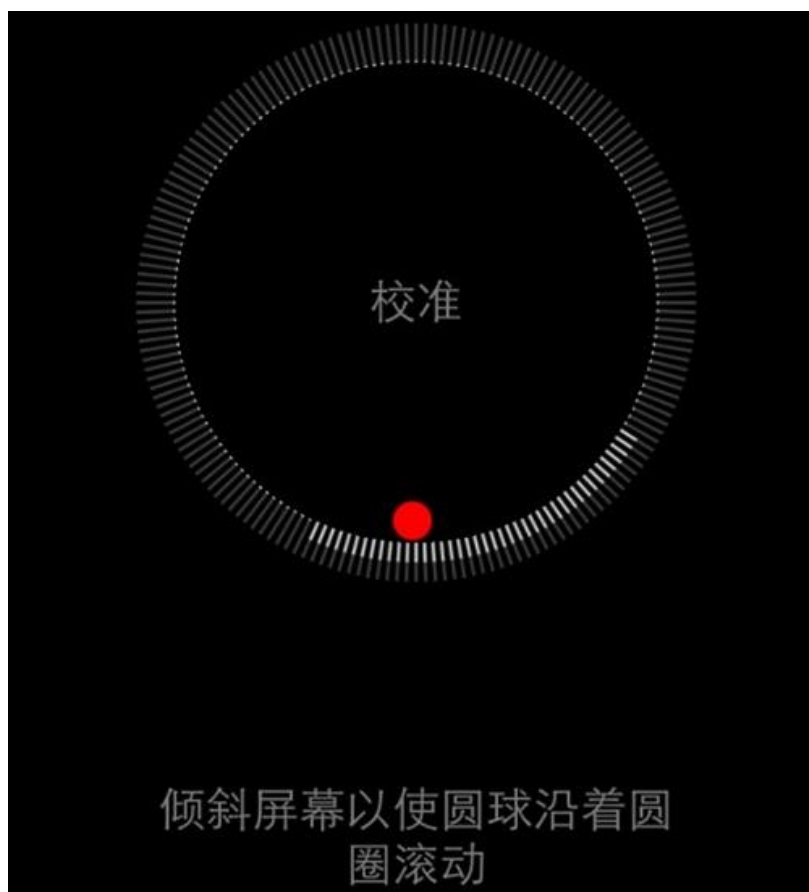


图 1-44 iPhone 罗盘校准

当然在使用磁力计传感器的时候，也是要做校准的。举个例子，水平放置磁力计，我们可以得到数值：（为方便计算，此处也是假设数值）

$X=233$ （和之前一样都是电压转换的对应值比如：2.3V 对应 233）

$Y=248$

$Z=156$

因为这个数值是带有磁场干扰的数值，所以我们必须要减去相应的偏移量。如图 1-45 所示可以看到我们现在测得圆心在 $C(X_n, Y_n)$ 。但是不管我们怎么样，偏移量是差不多的。所以，我们测得的圆心 C 就是本地的偏移量。减去这个偏移量的一个办法就是在接下来测得的数据中直接减去这个圆心的数值。

至于为什么这里是一个圆，我在 A5 为什么磁力计这里的数据是一个圆中解释了。

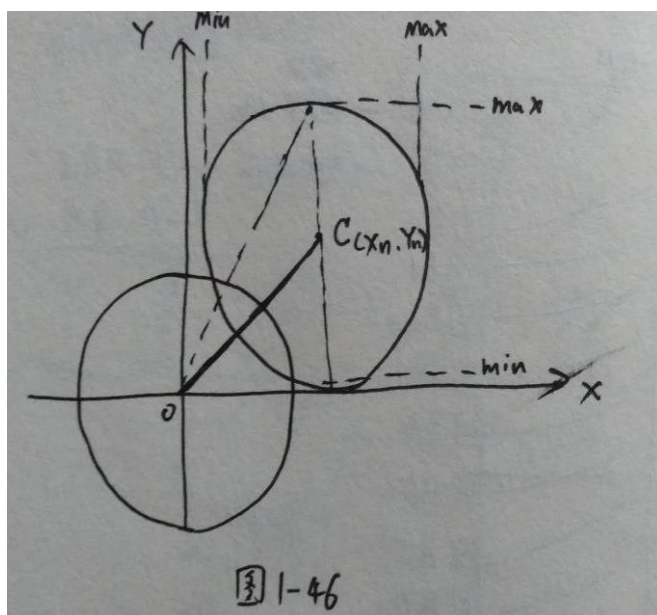


图 1-45 偏离的圆心

将磁力计平放在桌子上，只需将传感器水平旋转一圈，就可以测量 Xy 轴的数据（水平方向为磁力计的 Xy 轴）。此时的数据就会像图中那样形成一个圆一样。比如我们此时得到 x 轴的值是 $0.5V \sim 2.7V$ 之间的数据，也就是对应数值为 $50 \sim 270$ ，那么此时只需要将最小值和最大值相加再除以 2，就可以得到偏移圆的圆心坐标，即：

$$(50+270)/2=160=X_n$$

同理也可以得到 y 轴，假如是 $Y_n=120$ ，再将之前得到的数据直接减去这个偏移的值，就可以得到真正圆的圆心位置，即：

$$X=233-160=73$$

$$Y=248-120=128$$

因为磁力计此时是水平的，所以也可以得到方位角（指南针功能）：

$$\angle \text{方位角} = \arctan(Y/X) = \arctan(128/73) = 60.255^\circ$$

至于北偏南还是东偏南还是什么的就取决于每个传感器具体的规定了。不过既然角度都已经知道了，那么方向也很好确定了。

z 轴的偏移值的计算要绕 x 轴旋转 90° （此时 z 轴水平），然后也是同样远

离可以得到最大最小值，相加除 2 即可得到偏移数值。

那你可能会问我们平时拿手机放在车上导航的时候有时候也不会平躺着手机，也有可能是竖直放置的。那我们又怎么计算呢？这个时候加速度计就派上用场了。通过加速度计算出此时芯片的姿态来，就可以决定到底是该使用磁力计的 XYZ 中的哪两个数据来计算方位角 $\arctan(Y/X)$ 。具体的计算我们将在第 4 章讲解。

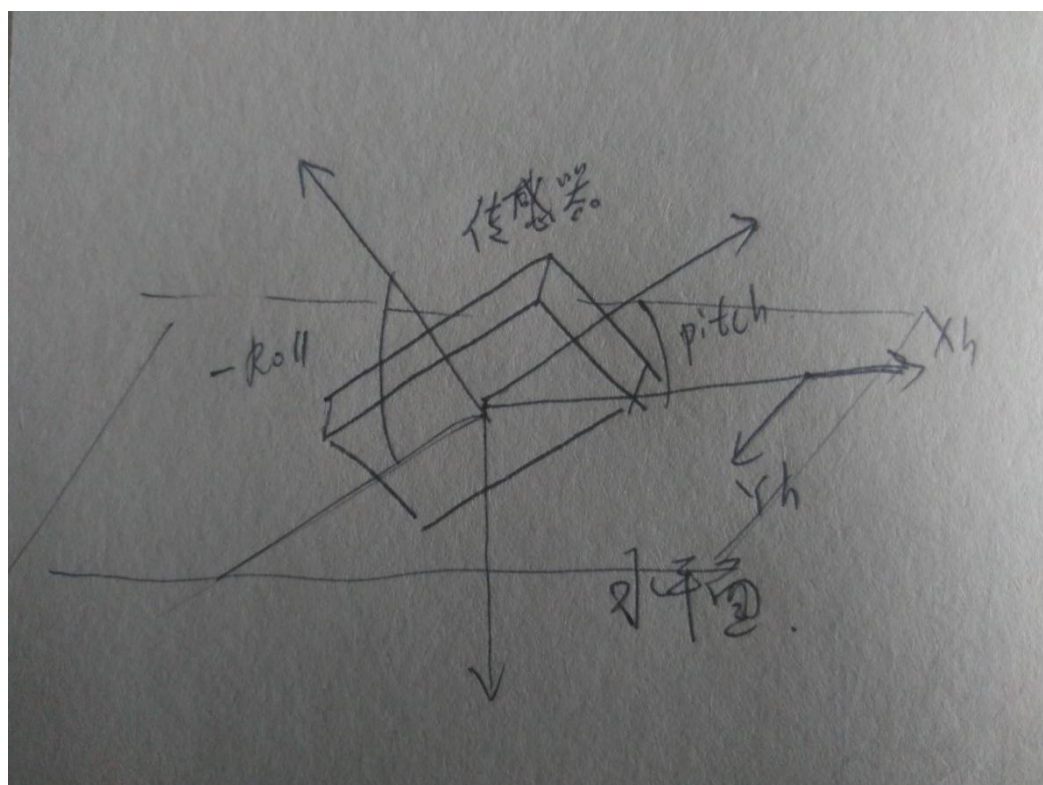


图 1-46 磁场受力图求方位角

当芯片平放的时候，z 轴又可以做什么呢？检测垂直平面方向上的磁场力。也就是说你用一个磁体在手机附件做前后运动的话，这个动作是可以被 z 轴的传感器捕捉到距离和方位的。比如你可以戴个磁力手套并用手势来控制你的手机，通过这个来做一些精巧的结构部件等，是不是也挺好玩？总之，无论什么技术其实都比不上优秀的创意。

本章对九轴姿态芯片进行了简单的讲解，让大家对加速度计、磁力计和陀螺仪的设计原理和功能有个了解。以便更好的帮助我们做产品的开发。

第2章 电路设计

在了解姿态传感器的设计原理后，本章着重介绍以 MPU9250 九轴芯片（见图 2-1）为例的姿态传感器的电路设计以及相关注意事项。为什么本书要以 MPU9250 为例子？

- ☐ 集成度最高
- ☐ 功能最多
- ☐ 具有代表性

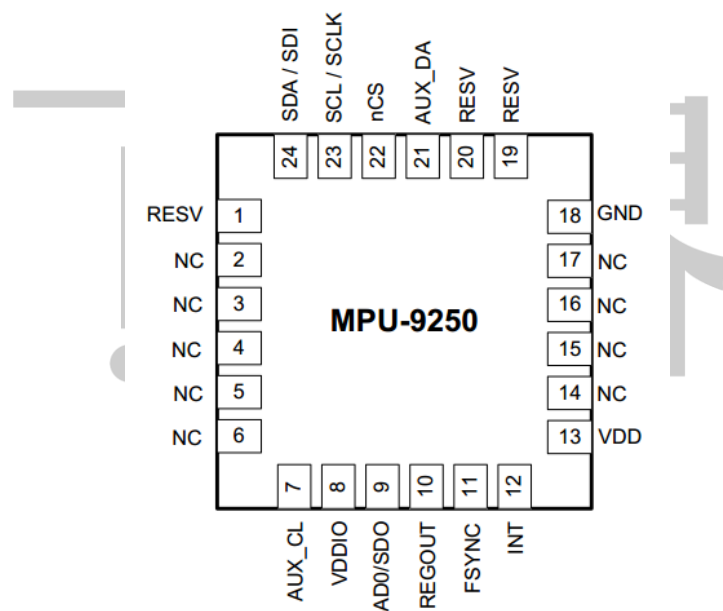


图 2-1 MPU9250 芯片

在硬件高度集成化的今天，硬件工程师的工作可谓是越来越简单化，计算量也会越来越少。这些都不得不归功于硬件的模块化发展。其中最具有代表性的应该属于谷歌的模块化手机了。



图 2-2 谷歌的模块化手机

模块化的最大的一个好处就是抗干扰，易设计。从智能手机内部拆解图的时候就能很明显地感觉到。我们这里以 iPhone 6s 内部主板为例给大家示例说明。

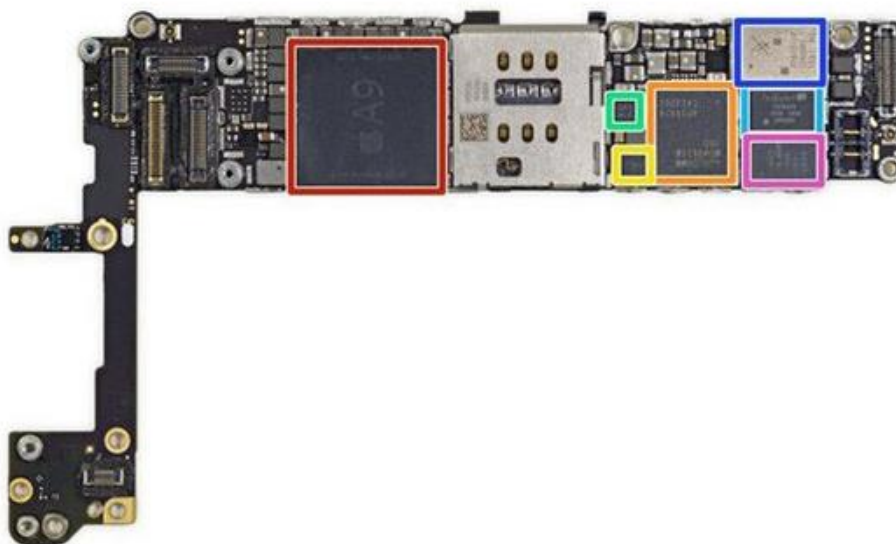


图 2-3 iPhone 6s 电路板

我们在图中能看到被各种色彩框框起来的芯片，那几个芯片其中就有我们本书所讲的姿态传感器。当然，苹果公司的 iPhone 系列作为业内顶尖设计公司的主打产品，我们不难看出这个电路板的设计和布局真的可谓是榨干了电路板的每一滴水，不留丝毫的多余之处。除了被框起来的七个元器件以外，其他的多为电阻电容以及预留的接口，他们井然有序的排列在一起。（这些电阻电容其实都是围绕着他们服务的芯片有计划的排列，其实这就是模块化电路的体现）先说明一下图中模块分别代表的相应元件：

- ❑ 红色：苹果 A9 处理器+三星 2GB LPDDR4 RAM
- ❑ 橘色：高通 MDM9635M LTE Cat.6 调制解调器
- ❑ 黄色：MPU6700 六轴陀螺仪和加速计模块
- ❑ 绿色：Bosch Sensortec 3P7LA 三轴加速度计
- ❑ 浅蓝色：TriQuint TQF6405 功率放大器模块
- ❑ 蓝色：Skyworks SKY77812 功率放大器模块
- ❑ 紫色：Avago AFEM-8030 功率放大器模块

当然这个板子的背面还有很多传感器和闪存等东西，在这里我就不一一说明了。眼尖的可能一下子看到苹果用了两个加速度计：一个是 InvenSense 的，一个是 Bosch 的。其中市面上买不到的 MPU6700 其实对应着 MPU6050 六轴芯片，不过只是专门为苹果定制的。和 6050 相比不但换了封装和引脚，而且也减少了 40% 的空间和一些内部的结构原理。**A6 为什么苹果有 MPU6700 这种买不到型号的芯片**

Chipwork (科技杂志) 认为这是为了节省手机内部可怜的电量而做出的设计，为什么呢？InvenSense 的陀螺仪计耗电 3.2mA，加速计模式下耗电 450uA，而 Bosch 的加速度计才 130uA，而且 Bosch 的加速度芯片启动速度很快，只需要 3ms，而 InvenSense 的需要 30ms，但是 InvenSense 的灵敏度和分辨率都要比 Bosch 高很多，甚至在休眠模式耗电量比 Bosch 还低。所以在设计时，在熄屏或锁屏看电影计步器等轻量应用下，iPhone 默认打开的是 Bosch 的加速度计，但涉及游戏或者姿态控制等重量级应用的时候就会打开 InvenSense 的六轴芯片 (MPU6700)。

然后我们来看看 A9 处理器是怎么体现模块化的？如图 2-9 所示。



图 2-4 A9 模块及周围电路

这是我们把刚才那张图横向放大来看。模块化的设计，功能集成度极高，而且芯片电路厂家出厂前就给你设计好了周围电路的方案，所以你一般用厂家的方案来搭建设计电路不会有什么问题，业内普遍都是遵照这个方法来做，所以像 TI（德州仪器）这种厂商会贩售自己的电源电路的模块化设计等。模块化的标志就是芯片+外围电路。这里 A9 处理器周围的电路既是必要的周围电路。（A9 是苹果自己设计生产的，不对外销售，按照苹果的设计极简主义风格，周围电路也几乎没有什么其他东西，也就是一些滤波电容之类的。）

那么 MPU9250 的参考资料又是怎么给出的参考电路的呢？

如图 2-5 和图 2-6 所示，分别是 I2C 传输方式的接线方案和 SPI 的接线方案。如果只想使用芯片的基本功能的话，可以直接拿来使用，不需要其他设计。

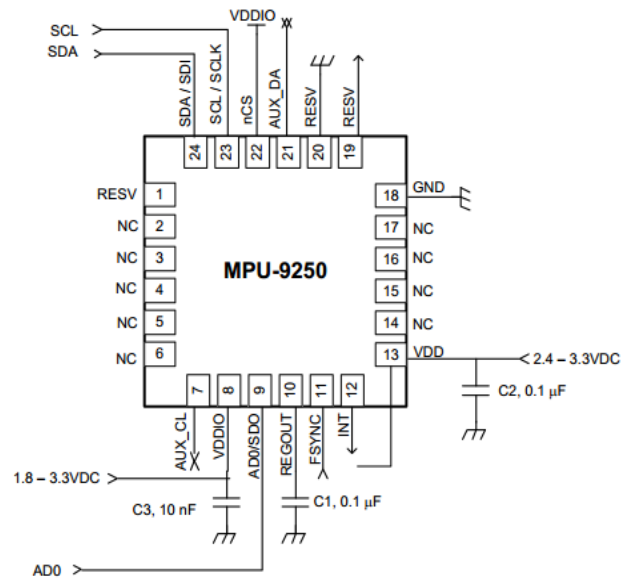


图 2-5 I2C 通信设计方案

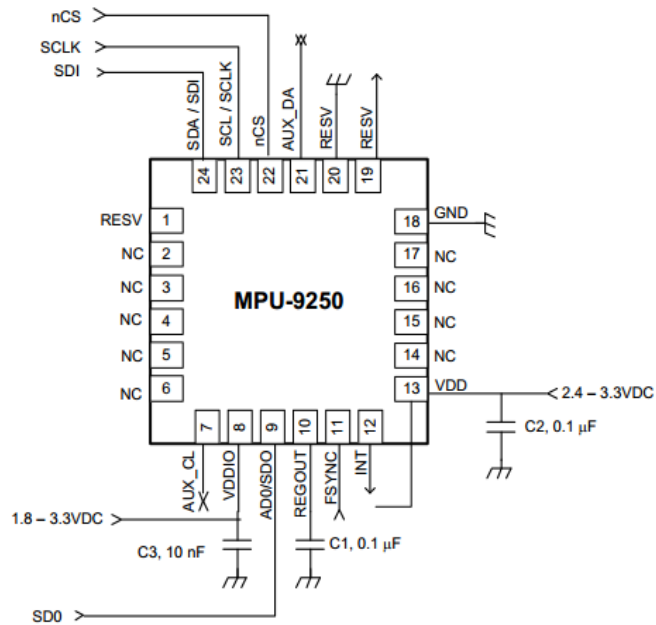


图 2-6 SPI 通信设计方案

2.1 芯片引脚

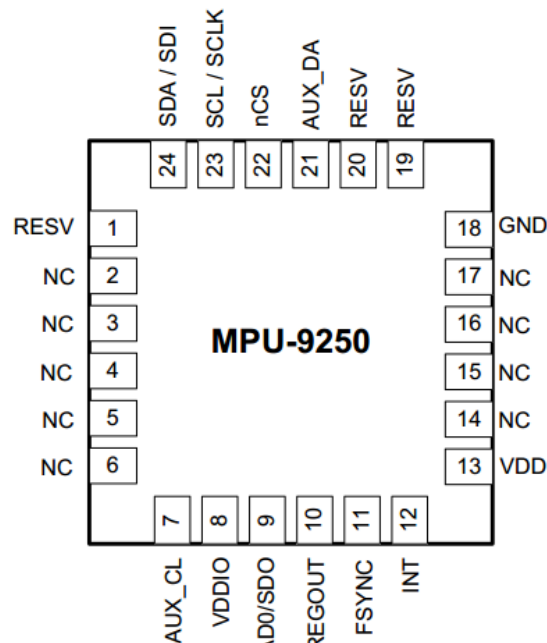


图 2-7 QFN 封装 MPU9250

接下来我们介绍，MPU9250 传感器的引脚及其功能。我们按照引脚顺序来逐个讲解（图中的编号即为引脚编号）：

引脚 1：连接芯片内部引脚 VDDIO，这里不接。

（VDDIO 是指芯片内部引脚的电压，有 3.3V 和 5V 供电的区分的芯片）

引脚 2-6：NC 是 not connect 的缩写，不接任何设备。

引脚 7：如果你是打算使用外部 I2C 辅助器件，就需要使用这个引脚。MPU9250 有一个特色功能——MPU9250 传感器可以作为一个处理器主器件去读取其他 I2C 传感器从器件中的数据并储存起来，最多可以从 4 个辅助传感器读取 24 个字节的数据。为了节约能耗，可以在不开启 MPU9250 的时候也直接读取辅助寄存器的 I2C 数据并传输给 MCU，相当于中继器。（如果使用这个功能的话，那么 MPU9250 主动去读取辅助传感器的功能就没法使用了）有关辅助 I2C 的连接我们将在稍后第 3 节中为大家讲解。

引脚 8：VDDIO 是专门给芯片提供运行所需电压的。电压范围在 1.8-3.3V（电

压的供给的大小其实和传感器的准确度是有一定关系的，本芯片是根据供给电压的多少来开关芯片的一部分功能的，这里建议大家供给传感器提供完整的电压值—3.3v，可以稍微高一点儿，比如 3.5V 左右）

引脚 9：此脚是当 MPU9250 作为从机的地址位。当通过 I2C 通信时，MPU9250 作为从机其地址位为 110100x（B），此时 X 的值取决于引脚 9 的高低电平。（AD 为低电平时 X 为 0，AD 为高电平时 X 为 1，当然此处的高低电平我们可以随意控制和设置）这也就是说我们可以让设备同时连接 2 个 MPU9250 芯片作为从机。当使用 SPI 通信时，此引脚作为 SOMI（关于 SPI 通信协议稍后第 2 节将会讲到）。

引脚 10：调节引脚，连接滤波电容。（多数芯片连接引脚接有电容大多数是为了滤波和稳压或隔离，所以必须要接，这也是周围电路中比较重要的组成部分，在设计手机等精密电子设备的时候还要考虑芯片之间干扰的问题。）

引脚 11：SYNC 同步信号输入端，也就是外部时钟信号输入端。给芯片提供一个外部的时钟振荡，一般芯片在做从机的时候需要。比如量产时可以制作工装单独检测芯片是否正常运行。一般不用时接地。

引脚 12：INT 引脚，即中断引脚。通常连接单片机的外部中断引脚（单片机外部中断此处不再解释）。此芯片的外部中断引脚可以通过寄存器设置。分为以下四种状态：

1. 时钟振荡改变的时候。（通常发生在切换时钟源时 4 发生，通过这个功能可以用在为节能而改变时钟震荡放式时使用）
2. 有新数据可读的时候。（FIFO 寄存器内的数据，当读取到新数据，通过中断告诉单片机，不需要单片机一直等待数据，节约 CPU 读取时间）
3. 加速度的中断功能（运动唤醒功能，比如拿起时唤醒手机的功能时使用）
4. 没有接收到辅助传感器数据的时候

如果不使用这个引脚，或者 MCU 引脚不够用时，传感器的中断的状态也是可以通过寄存器来读取的。

引脚 13：VDD 提供芯片电压的端口，注意此处一定要加上稳压电容使芯片

电压稳定。

引脚 14-17: NC 什么都不接。

引脚 18: GND 接地。

引脚 19: RESV 什么都不用接。

引脚 20: RESV 接地。

引脚 21: AUX_DA 配合引脚 9 一起使用, 如果你的 MPU9250 作为主机去连接其他辅助传感器或芯片, 并通过 I2C 协议通信, 那么此时 21 引脚担任 SDA 线(数据传输信号线)的工作。

引脚 22: nCS 片选端口, SPI 模式下使用的端口。当 22 引脚被选中(高电平时), 主机便可与被选中的这片 MPU9250 开始通信。如果只用这一个芯片的话, 该引脚可以直接连接 VCC, 一直高电平, 以使得传感器一直处于激活状态。

引脚 23: SCL/SCLK 顾名思义, 当 I2C 通信时为 SCL 引脚, (注意此处是与主控芯片 MCU 单片机连接而不是辅助传感器)若使用 SPI 通信时则为 SCLK 引脚。SCL 是时钟信号引脚, 时钟信号是主机设备发出的。

引脚 24: SDA/SDI 当使用 I2C 通信时为 SDA 引脚。当用 SPI 通信时作为 MISO。SDA 为数据引脚, 通过和时钟引脚有序的时序(通信协议)来达到数据传输的目的。

2.2 I2C 和 SPI 通信协议

本小节着重讲解 MPU9250 的两种通信模式(I2C 和 SPI), 如果没有接触过这两种通信模式一定要细读, 我将简单通俗易懂的方式讲解这两种通信模式的机制, 如果了解的话可以直接跳过。

2.2.1 I2C (I2C) 通信协议

为了能够做到远程通信, 人们发明了各种通信协议。比如莫尔斯电码(通过规定好打点的方式来找到对应字母或者数字来取得通信内容)。又例如在电影《火

星救援》中，马克通过探索者号摄像机摇头的方向与地球进行通信。由此可见，只要是通信双方都了解和知道一个共同的规则，大家按照这个规则来传递信息，这个规则就是通信协议。

I2C（I2C 既是 I2C，本文以下均用 I2C 表示）通信也是如此，如果 2 个电子设备之间需要通信，只通过 2 根线，即可完成相应的数据交换。一根是 SDA，一根是 SCL。SDA 是数据线，SCL 是时钟线。这个方案最早是飞利浦公司提出的（但是大家不要忘了 VCC 和 GND，用来提供电源驱动芯片工作）。

具体协议如图 2-8 所示：

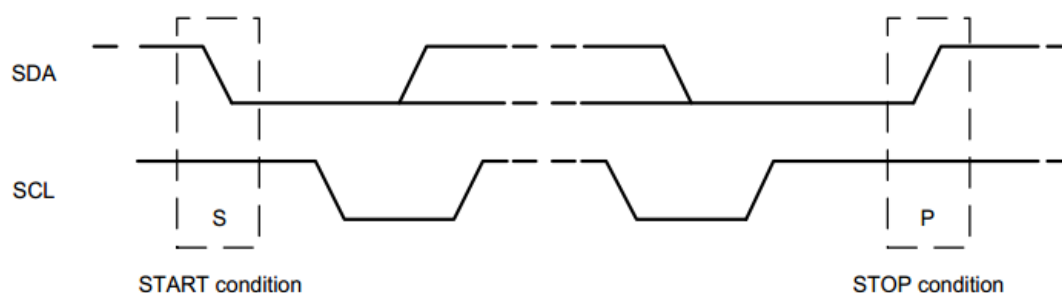


图 2-8 I2C 中 SDA 和 SCL 信号开始结束传输的时序图

我们看到在图中 S 方框所指代的就是 I2C 信号传输的开始条件，P 则指代的 I2C 传输信号的结束条件。信号传输的开始条件为：当 SDA 产生下降沿信号时（电压从 3.3V 降到 0V），SCL 信号保持高电平（保持 3.3V）不变，此时视为 I2C 信号开始传输，然后便可以按照格式发送传输数据。而当 SDA 信号线产生一个上升沿（电压从 0V 到 3.3V）时，SCL 保持高电平不变，此时视作信号传输结束。所以这两个时序之间的数据才是我们要传输的数据。我们再来看一下如图 2-9 所示的数据格式的规则：

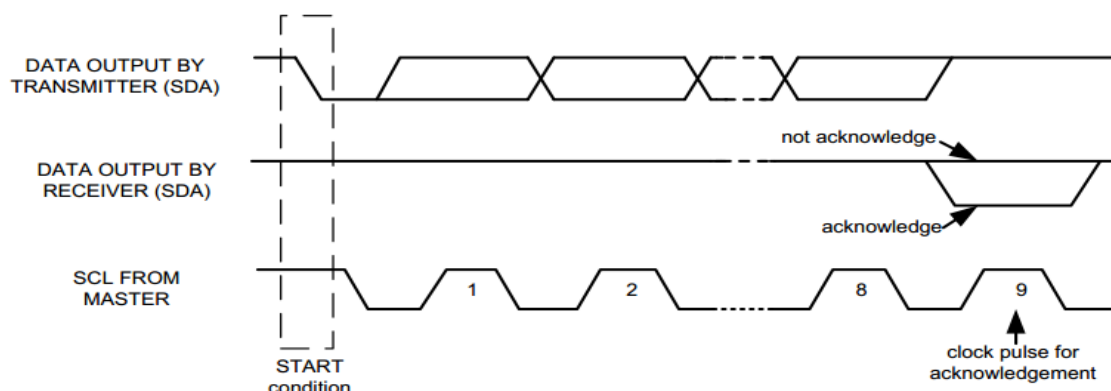


图 2-9 MPU9250 的 I2C 数据传输时序电路

(DATA OUTPUT BY TRANSMITTER 是 I2C 数据发送，DATA OUTPUT BY RECEIVER 是 I2C 数据接收，SCL FROM MASTER 是主机时钟信号，START condition 是 I2C 传输的开始条件，not acknowledge 是高电平，acknowledge 是低电平，clock pulse for acknowledgement 是一个完整的时序脉冲)

上图是官方 datasheet 中关于 MPU9250 的 I2C 时序电路的规定。(所有的 I2C 都是这个时序) 我们看到在主机发出 Start 信号后(也就是 I2C 开始信号)，此时主机会发出从机的地址(7 位)+读写(1 位)+等待的应答信号(1 位) 一共 9 位。解释一下，每一个具有 I2C 功能的传感器都会有一个 I2C 地址，我们传感器的地址是 110100x，比如现在引脚 9 被拉低(也就是 0V)，那么 X=0，即我们 MPU9250 此时地址为 1101000 (B)。读写位表示的是现在这个芯片要做的动作：可以读取其他芯片寄存器内部的值(第 8 位为 1)，也可以写入一个值到芯片的寄存器(第 8 位为 0)。比如我们去读取 MPU9250 的一个寄存器的数值，那么发送的数据就是 11010001，即 0xD1。此时发送的主芯片就会释放 SDA，等待从设备(传感器)应答。所以 I2C 设备只需要 2 根线就可以做到同时连接很多芯片并传输数据。就好比老师上第一课喊名字来认识同学一样。如果你发现主控 MCU 发送的地址(呼叫的名字)和你的一样，此时传感器芯片将会自动去拉低(回应) SDA 的电压(0V)，即表示应答成功。然后我们就看到了如下图 2-10 所示的时序图：

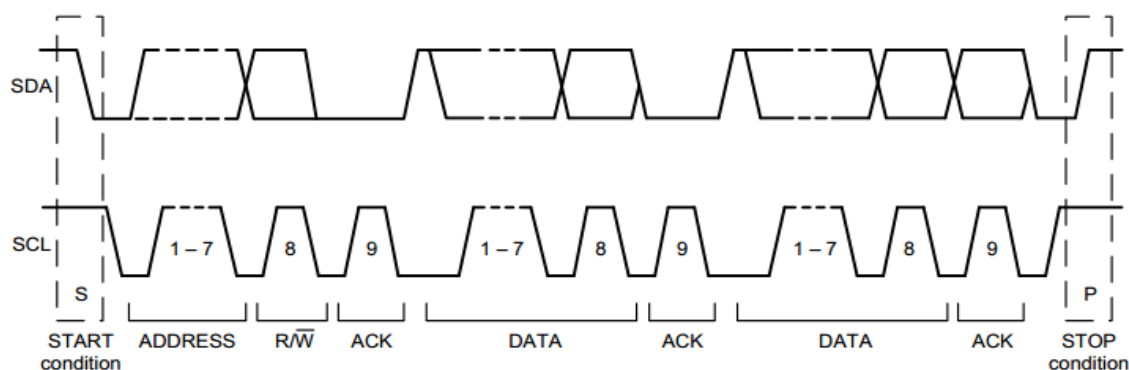


图 2-10 I2C 时序图

图中显示出当从传感器做出响应了（既拉低 SDA）以后，此时主芯片（MCU）就开始传输数据了，就是 7 位数据+1 位读写+1 位响应位，图中连续发送了 2 次 DATA 数据，也就是双字节传输，一共 16 位。

下面来看一下写入 MPU9250 寄存器数据的具体规则，如图 2-11 所示。

Master	S	AD+W		RA		DATA		DATA		P
Slave			ACK		ACK		ACK		ACK	

图 2-11 写多字节传输

主机 Master（MCU）发送开始信号（S）和从机（Slave）的 7 个地址位（AD）再加上 1 位的写入位（W），此时芯片（MPU9250（Slave））产生应答（ACK）。这时，主机输出寄存器地址（RA），从机（MPU9250）再次产生 ACK 应答，传输过程可以随时产生停止信号停止。ACK 响应后，数据（DATA）可以继续输入，除非产生停止位。芯片内部自带的递增寄存器可以自动将数据写入相应寄存。

然后来读取 MPU9250 寄存器的命令传输顺序，如图 2-12 所示：

Master	S	AD+W		RA		S	AD+R			ACK		NACK	P
Slave			ACK		ACK			ACK	DATA		DATA		

图 2-12 读取 MPU9250 寄存器时序表

主机 Master 发送开始信号（S）和从机（Slave）的 7 个地址位（AD）再加上读位（W）。此时，被读取的传感器接受到信号，返回一个 ACK 信号给主机。主机发送寄存器地址信号查询是否有相应的寄存器，从机查询是否有相应寄存

器，如果有则返回 ACK 信号给主机。没有则不返回，主机发现长时间无应答会自动脱机报错（这个功能需要程序设计），主机再次发送开始信号和地址，MPU9250 此时才又会发出应答信号 ACK 和数据。主机再次发回响应信号，从机发回剩余的数据，一共 16 位，2 个字节。直到主机发送 NACK 或停止位后通信停止。（NACK 信号就是第 9 个时钟脉冲 SDA 保持高电平。）A7 关于 I2C 读取 10 位地址问题

关于 I2C 的程序代码在各个单片机的教学中都有实例，所以这里就不再浪费篇幅讲解。

2.2.2 SPI 通信协议

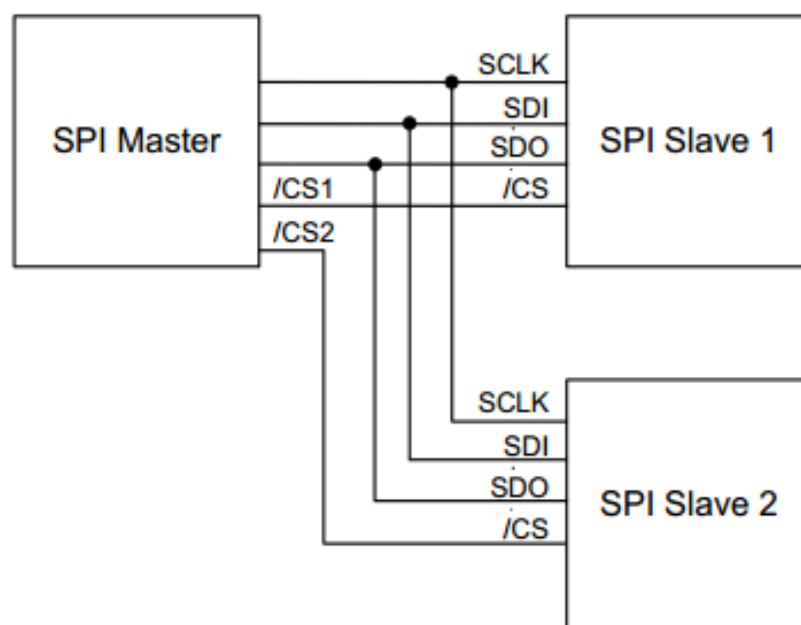


图 2-13 典型 SPI 数据连接方案

如图 2-13 所示，典型的 SPI 数据连接方案—4 线模式。其中包括 2 根数据线和 1 根控制线以及 1 根信号线。MPU9250 在 SPI 模式下工作的时候就不能再使用 I2C 读取辅助寄存器的功能了。简单讲解一下这 4 个引脚的功能。SCLK 和 SCL 一样是 MCU 给传感器提供时钟信号的。SDI 和 SDO 是 2 根单向的数据传输线，这里单向的意思就是一根线只有一个传输方向。比如 SDI 就是 MCU 的数据传递给 MPU9250，SDO 就是 MPU9250 的数据传递给 MCU（注意，在芯片和单片机上 SDI 和 SDO 数据是反的，也写作 MOSI 和 MISO，芯片的 MOSI 要连接 MCU 的 MISO，同

理 MISO 也要连接 MOSI)。这样的话, 就可以做到全双工传输信号, 也就是从主机可以同时向对面发送信号, 速度更快, 效率更高。而刚才讲 I2C 的时候, 里面的 SDA 是双向传输的, 这样一次只能传输一边的信号。这就是属于半双工, 这也是为什么我们传输的数据里面要带读写位。CS 就是片选引脚, 哪个 CS 置低 (0V) 哪个就被选中 (这样的高阻态设计是为了防止设备之间的干扰), 这一点和 I2C 是一样的。

SPI 的工作方式有四种。选择不同的工作方式取决于 2 个寄存器 (CPOL 和 CPHA) 不同的数据配置。CPOL 是规定 SCLK 时钟信号空闲的电平, 当 CPOL=0 时, 空闲电平就是低电平 (0V), 为 1 时即为高电平 (3.3V)。而 CPHA 是决定采样的时间, 当 CPHA=0 时, 表示采样的时间是每周期的第一次时钟跳变沿。当 CPHA=1 时, 则为每周期的第二次时钟跳变采样。如图 2-14 所示。

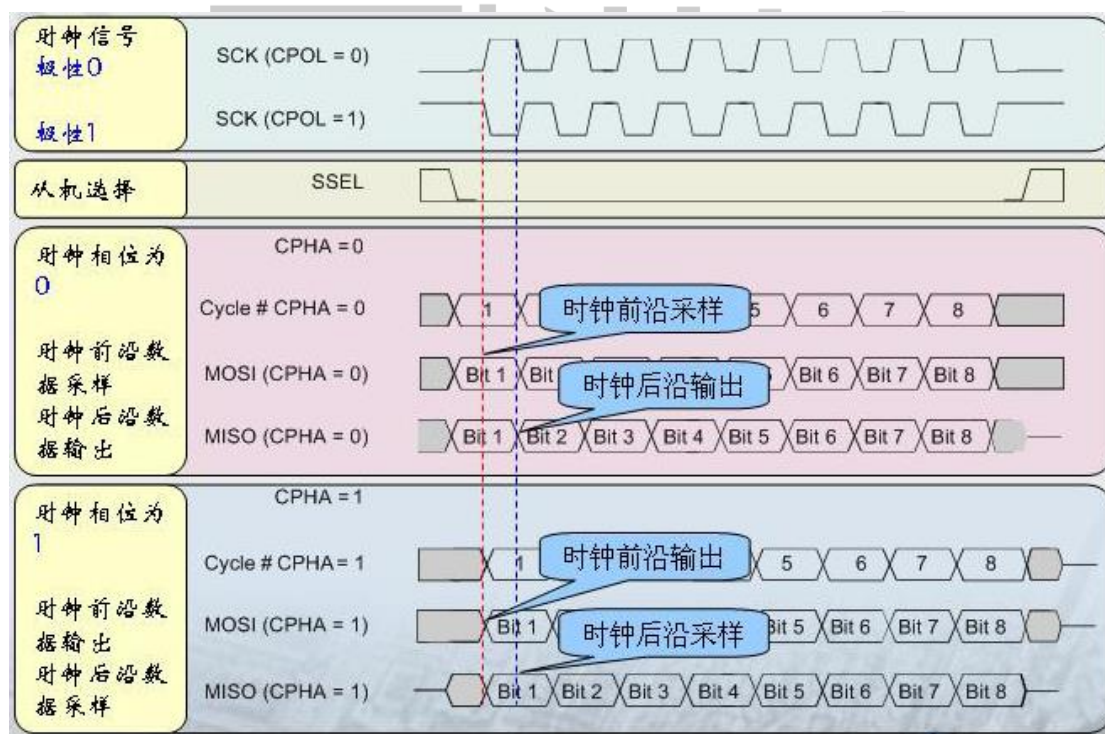


图 2-14 SPI 的时序图 (重新绘制)

也就是说这四种工作方式 (SP0, SP1, SP2, SP3) 即为这 2 个寄存器配置的组合。

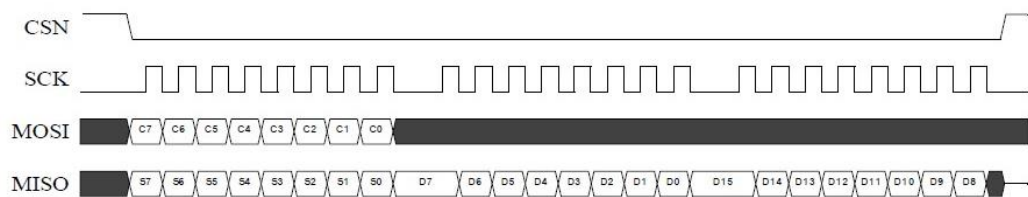


Figure 23. SPI read operation

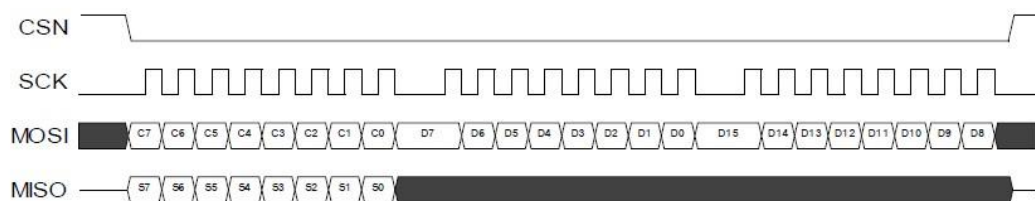


Figure 24. SPI write operation

图 2-15 SPI 读写操作时序图

图 2-15 表现的是 SPI 工作下的读写时序图。参考 I2C 的时序电路的解释，大家能否看出来 SPI 时序电路的读写规律呢？

和 I2C 一样，信号开始位都是在 SCK 低电平的时候，CSN 产生一个下降沿。而停止位则是 CSN 产生上升沿的时候 SCK 处于低电平不变。MOSI 即为主机发送从机接收，而 MISO 则反之。发送数据的时候都是在 SCK 产生上升沿的时候。

注意一点的是数据传输的顺序是从高到低，也就是从 D7 到 D0，而内部字节传输的时候是先传送的低字节，后传输高字节。A8 关于资料的获取和使用的感悟

2.2.3 I2C 与 SPI

SPI 模式相比 I2C 模式占的信号线实在是太多了，所以接线很麻烦。至少要用 3 根（3 根时假设 CS 默认一直拉低 0V），而 I2C 模式只需要 2 根。但越简单的东西，实际使用起来却越麻烦。首先，I2C 是全双工，所以他远远追不上 SPI 的数据传输速度。SPI 一般传输速度能达到 10Mbps，而 I2C 加上高速缓冲区以后最高也不超过 3.4Mbps。所以要想获得稳定高度的传输结果，SPI 是个比较理想的数据接收协议（其实还有 UART，但是在传感器中做串口的话目前来说会把传感器模块搞得很大，所以此处不考虑在传感器中加入串口通信功能，但大家也可

以通过其他芯片自行转换为串口模式)。所以 SPI 模式虽然简单性,却让它在连接多设备的时候会让线路变得更为复杂,而 I2C 区区 2 根线即可连接 N 多设备。

(连接多设备时,需要考虑驱动的问题,主要是 SCL 的驱动需要单独供能,本人实验过用 STM32 驱动多个 MPU6050,在不使用外部驱动的情况下,超过挂载 8 个设备,就会出现信号不正确的现象)

总结一下,虽说这些协议和 PCI-E 或 USB 那些动辄几百上千 M 的速率简直都没法比,但我们不需要一味地追求高速。物尽其用,恰到好处才是我们需要考虑的,就像你给汽车安装一个飞机的引擎,肯定是不合适的。SPI 适合做数据流处理(比如连续不断的大量芯片传输,比如我们火箭调整姿态的时候传输信息的方式)而 I2C 适合小批量数据的传输(比如我们智能手环跑步,睡眠功能的数据传输)。作为一个工程师来说,合适的才是最好的,在这方面不要犯了越大越快越好的错误哟。

2.3 电路布局及注意事项

根据官方给出的电路图可以设计如图 2-16 所示电路,这个连接方案连接了外部辅助芯片的电路,所以看上去要比较多。此电路图连接了辅助芯片 BMP180 (气压传感器),使用 I2C 接口通信。也开放了 SPI 的接口,也就是说,I2C 和 SPI 的接法都可以使用。

2.3.1 电路原理图设计

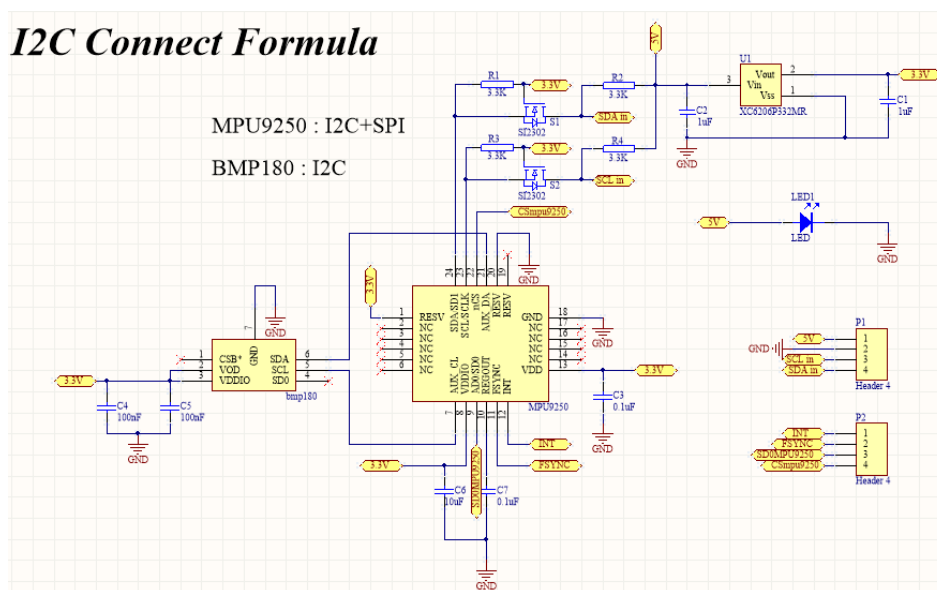


图 2-16 MPU9250 的 I2C 接口电路

下面我们来简单解释一个电路中每个部分的作用和使用方法。先看左边部分的电路：

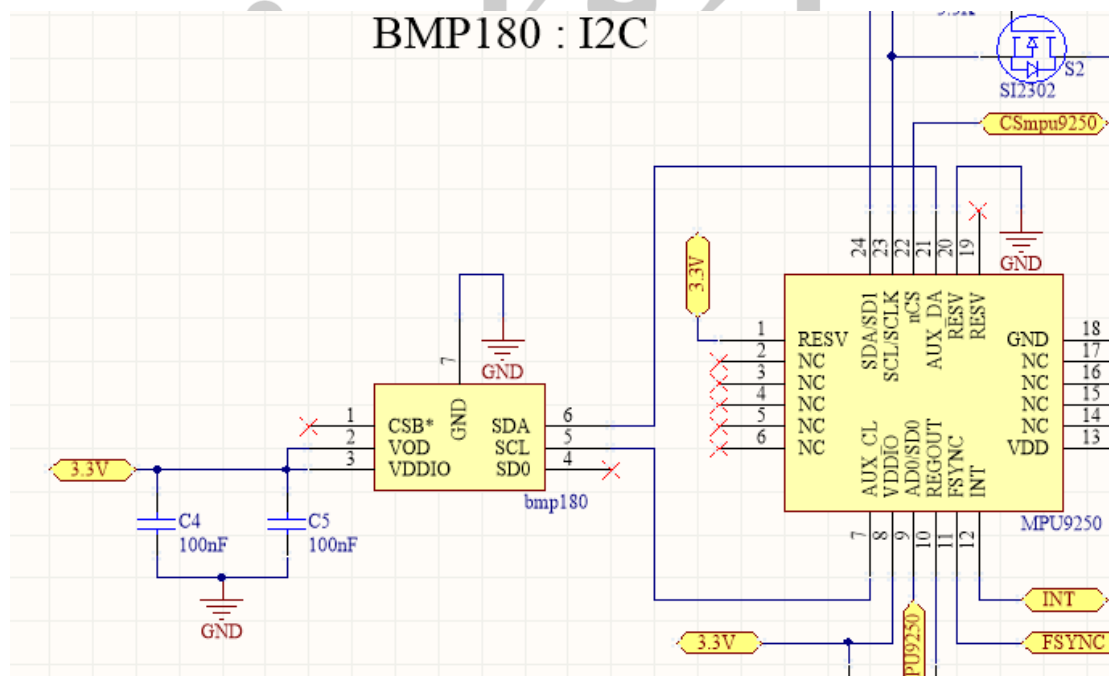


图 2-17 BMP180 辅助芯片连接

这是 MPU9250 的辅助芯片，主要是用于飞控板使用。因为一个飞行器不光要

知道他的姿态和方向，还要知道飞在有多高的地方。而气压芯片可以通过测量大气压的压强来计算出海拔，从而知道飞行器目前飞行的高度。这就是传说中的十轴飞控板。当然现在有很多飞行器上加装了 GPS 芯片，在告诉你经纬度的同时也会告诉你海拔和温度。所以要用 BMP180 的话，算是低成本考虑的首选。而且 BMP180 内部自带温度计。但是像这种 GPS 或者气压湿度传感器带温度计一般都不是都很准，漂移比较大，如果你要测量比较精准的温度，建议还是用工业级的温度传感器。由于 BMP180 采用的也是 I2C 通信协议，所以很自然的我们按照第一节引脚功能的描述把气压芯片的 SCL 和 SDA 连接到了 MPU9250 的第 7 脚和第 21 脚。

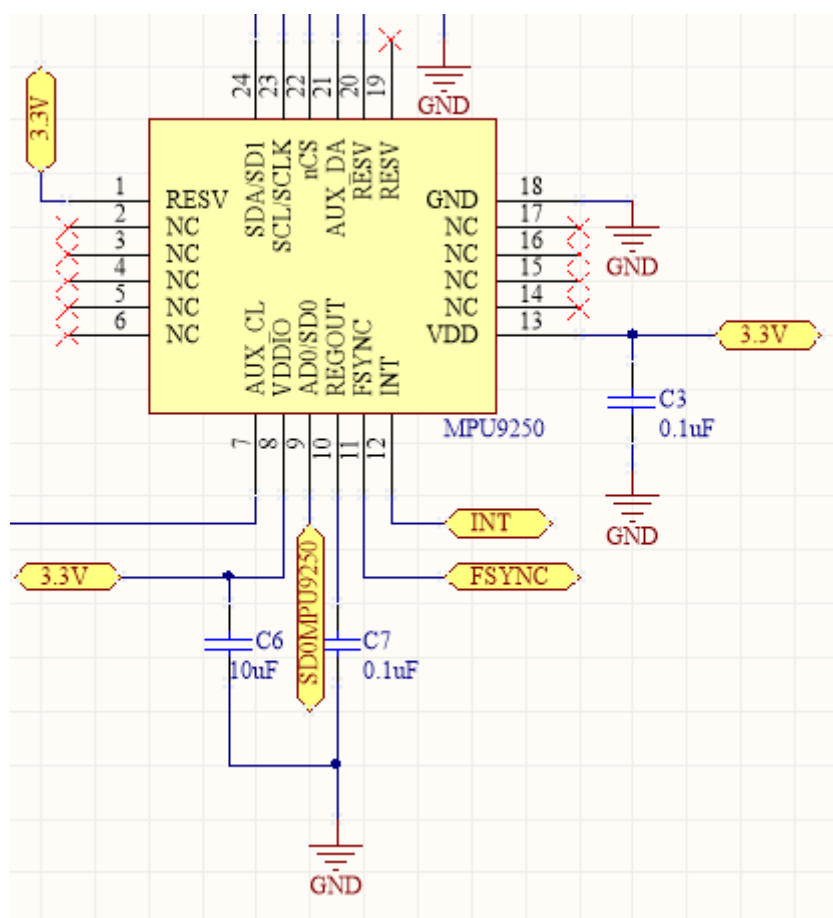


图 2-18 典型电路

这里直接使用官方给出的 I2C 方案来连接 MPU9250 的外围电路。这里如果你只用 I2C 协议通信的话，请酌情去掉多余的外围电路。反之如果你用 SPI 协议的话，也可以酌情去掉 I2C 的多余部分。

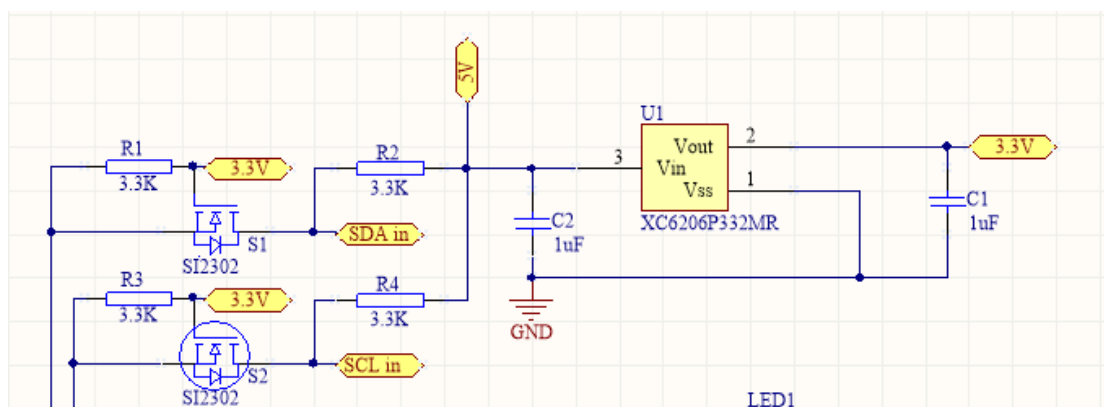


图 2-19 I2C 供电

这里加了一个 MOS 管来驱动 SDA 和 SCL 电路，将 3.3V 的电压转换成了 5V。因为单片机（Arduino 或者 51 系列）是 5V 电压，这里也是需要用户酌情考虑的，如果你的 MCU 工作电压就是 3.3V（比如 TI 的 MSP 系列或者 STM32）那么就不需要这样的转换电路。旁边那个 U1 是稳压芯片，稳的是 3.3V。5V 电压进来（3 口）转换出来（2 口）即为 3.3V。

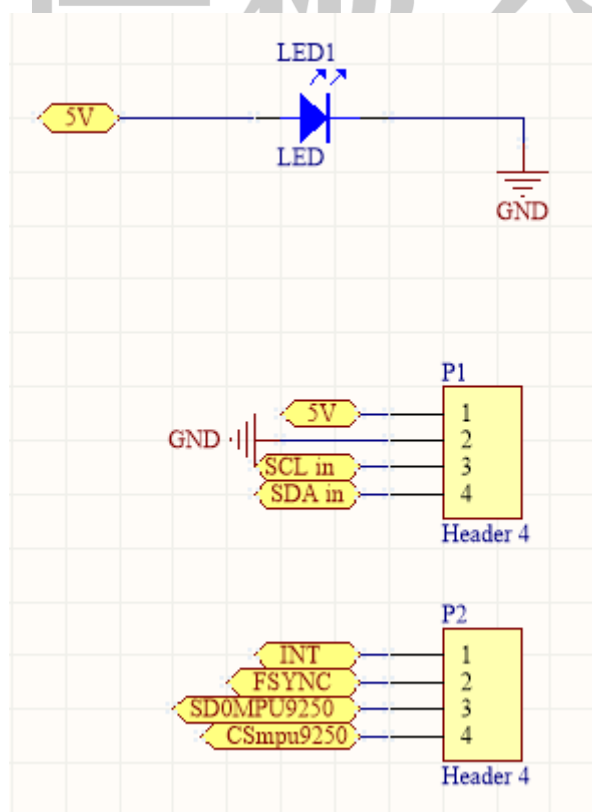


图 2-20 外接电路

这一部分就比较简单，如图 2-20 所示，LED 为电源指示灯，表明芯片供电

是否正常。同时你也可以通过这个 LED 的亮度判断当前芯片电压供给够不够。其他的 P1 和 P2 则是外部链接的引脚。当然如果你只用在单一模式（I2C 或 SPI）的情况下是不需要这么多引脚的。

硬件设计中需要注意的地方：

芯片的典型电路中给出的外围电路中的电容有要求，大家在电容选型的时候要注意一下电容的耐压值。如果耐压值低于线路电压会损坏电容。

元件	标号	规格	数量
滤波电容	C1	陶瓷, X7R, 0.1 μ F $\pm$ 10%, 2V	1
VDD 旁路电容	C2	陶瓷, X7R, 0.1 μ F $\pm$ 10%	1
VDDIO 旁路电容	C3	陶瓷, X7R, 10nF $\pm$ 10%, 4V	1

完成这些后就可以画电路板了，由于线路不多所以我们只需要双层板即可，大家根据自己的需要自己设计板子。

下面我们将电路设计步骤简单讲一下。

2.3.2 电路 PCB 设计

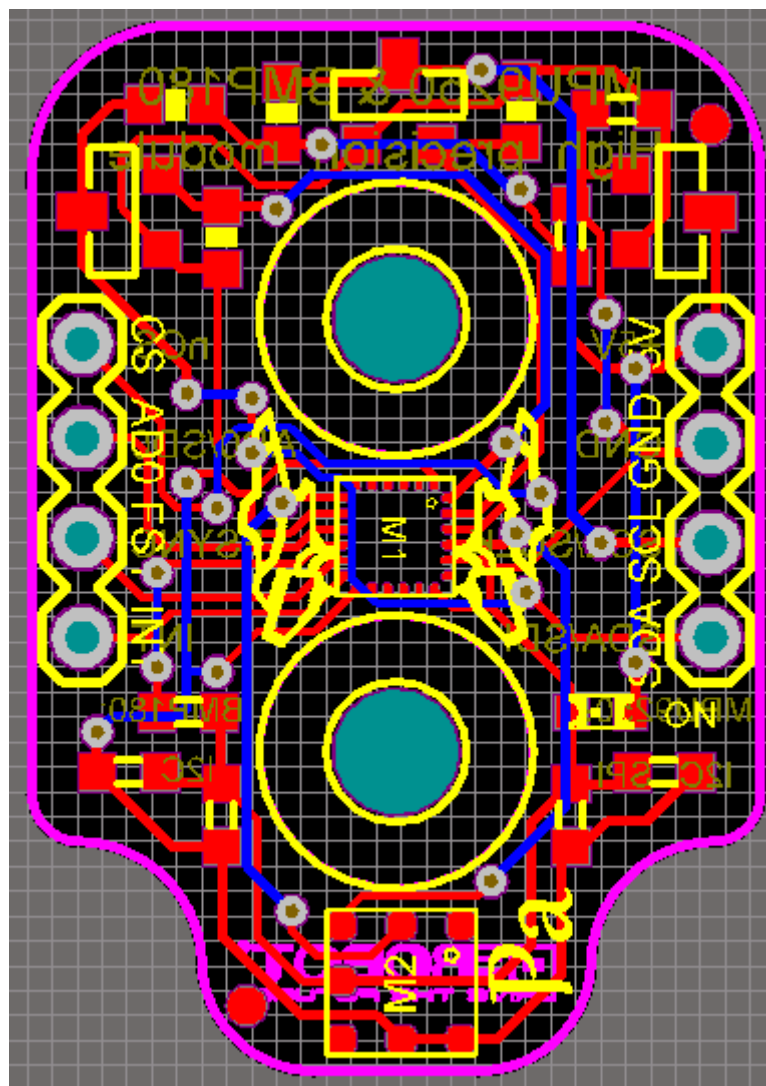


图 2-21 布板画线

大家根据自己的需要自行设计板子的形状大小。这里设计芯片在中间的目的是为了绝对的平衡（其实也可以放在外部，只是为了美学考虑），旁边两个加固孔固定板子。在画板的时候有个地方要注意，稳压电容不要离得电源端太远，要就近安放，这里的设计将对称和稳压兼顾了。

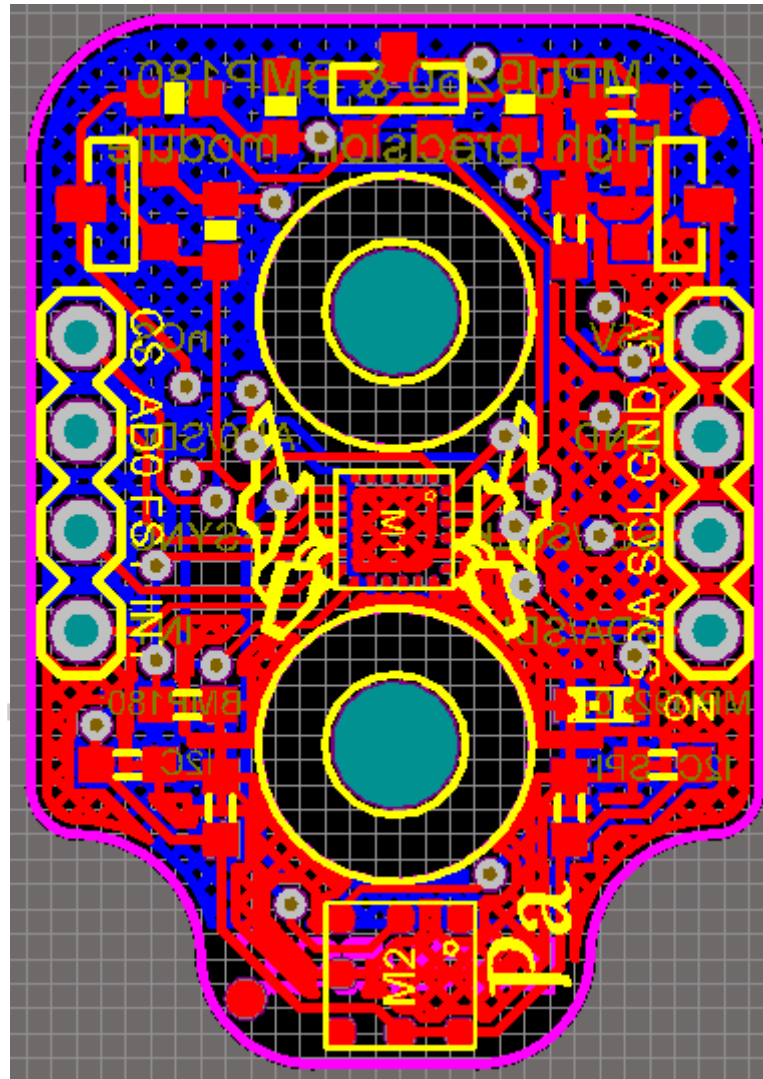


图 2-22 敷铜

画完板子后先滴泪再敷铜。这样在机帖时更好上锡，量产时不要忘了加 MARK 点（为了方便机器定位元器件位置）如图 2-22 所示，左下方那个圆点就是 mark 点。

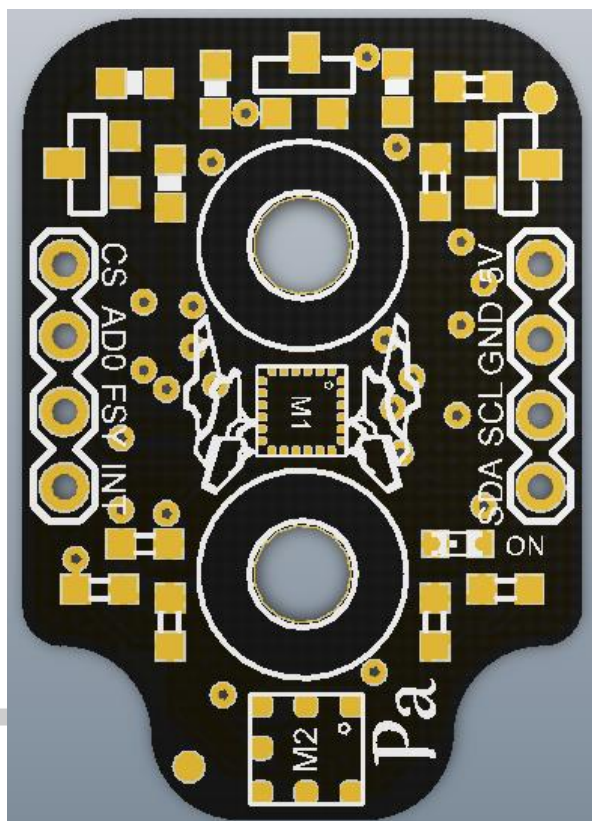


图 2-23 飞控板

图 2-23 就是制作出来的电路板的渲染图。只需要贴片和焊接需要的插件连接上相应的端口，便可以使用。本章的详细电路图和 PCB 工程文件在载附录中有下载链接，可以直接拿来打板加工。以供需要的同学下载使用。

2.4 AD15 简单使用

对于想要简单修改 PCB 文件的同学，这一小节简单讲一下 Altium Designer 软件的使用步骤。

这里我们用 Altium Designer 15（简称 AD15）来举例子。

首先打开 AD15，新建一个 PCB 工程，如图 2-24 所示：

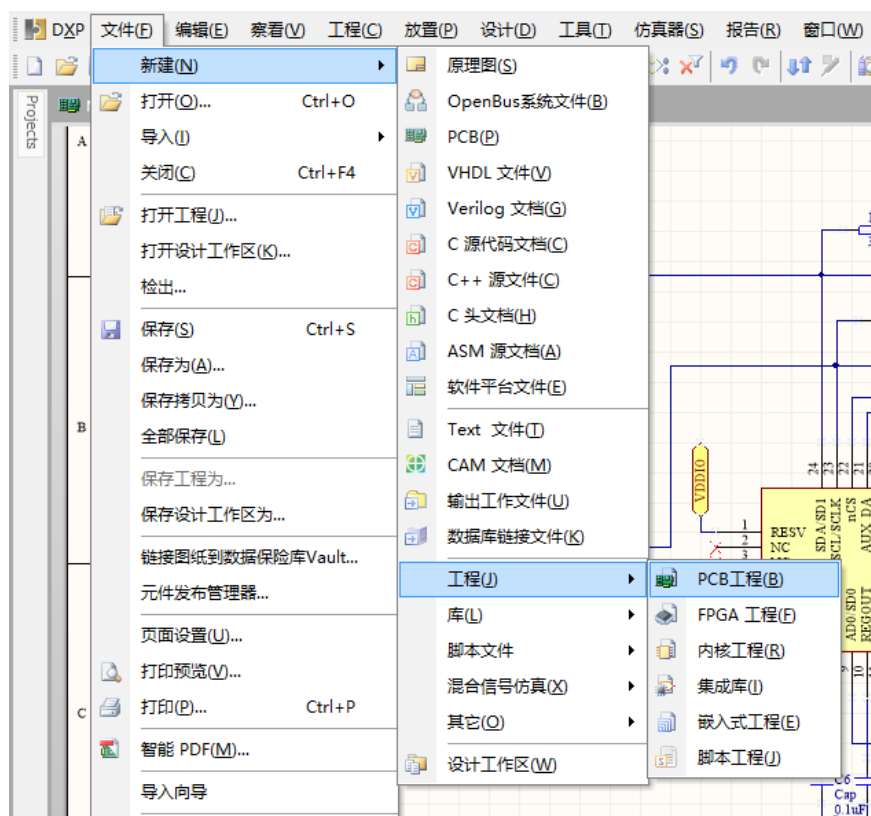


图 2-24 新建工程

此时左边的目录页面就会显示图 2-25 的工程。

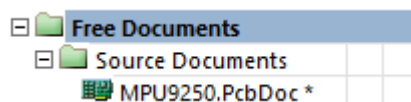


图 2-25 新建工程的列表

然后我们再新建一个原理图和 PCB，如图 2-26 所示：

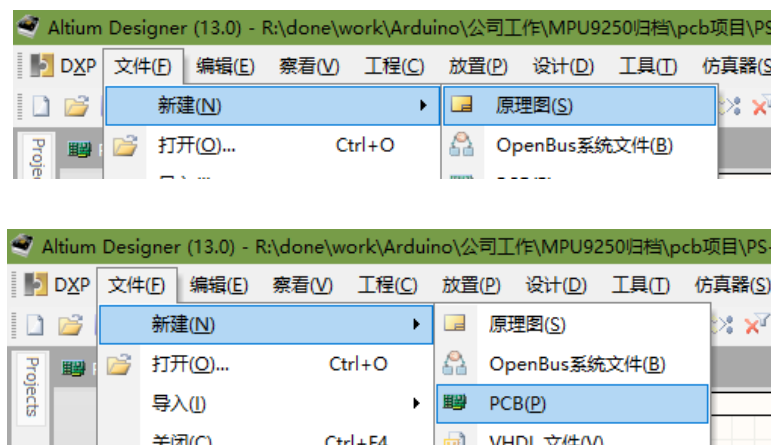


图 2-26 新建 2 个文件

原理图中就可以开始设计电路了，当遇到库文件里面没有的元器件的时候我们就要自己画了，比如 MPU9250。一般官方都会提供相应的封装文件，建议到官方网站下载。如图 2-27 所示点击新建一个原理图库。

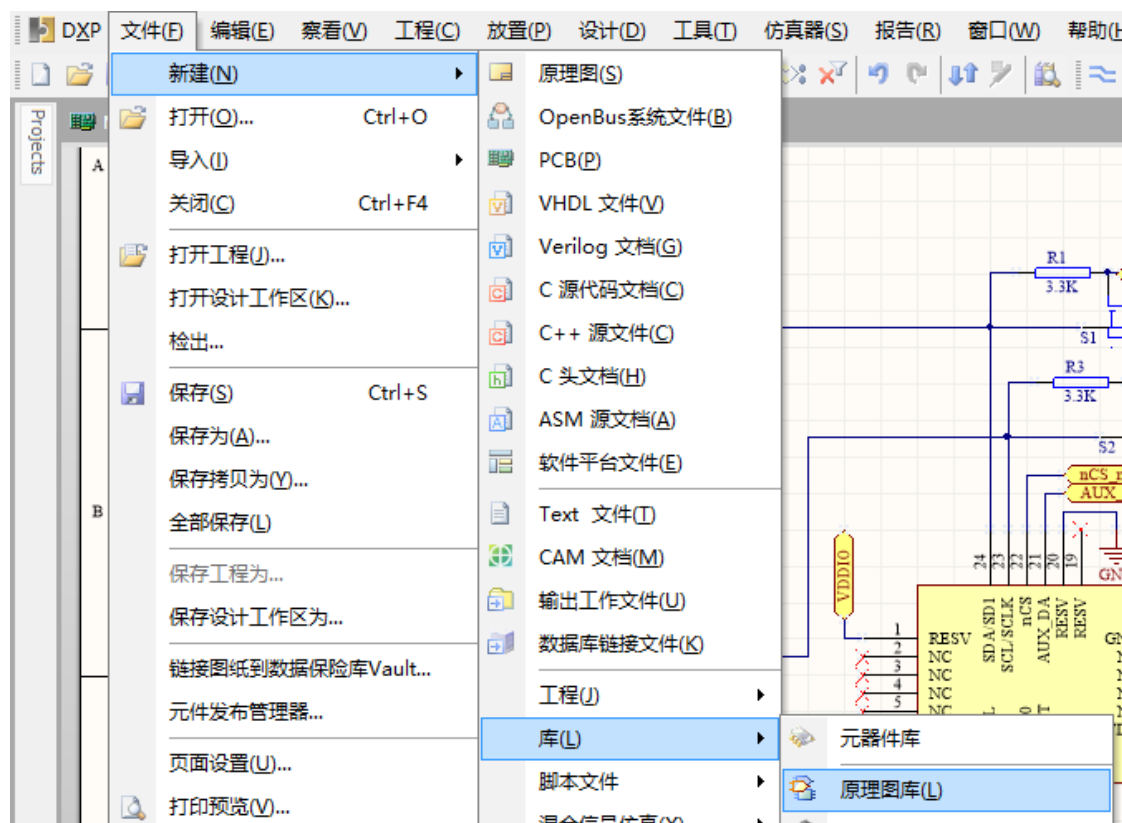


图 2-27 新建原理图库

在原理图库中画好原理图以后在原理图中就可以直接添加了。当电路画好之后别忘了做电路规则检查。然后我们要为刚才新建的那个原理图画一个 PCB 封装。如图 2-28 所示。

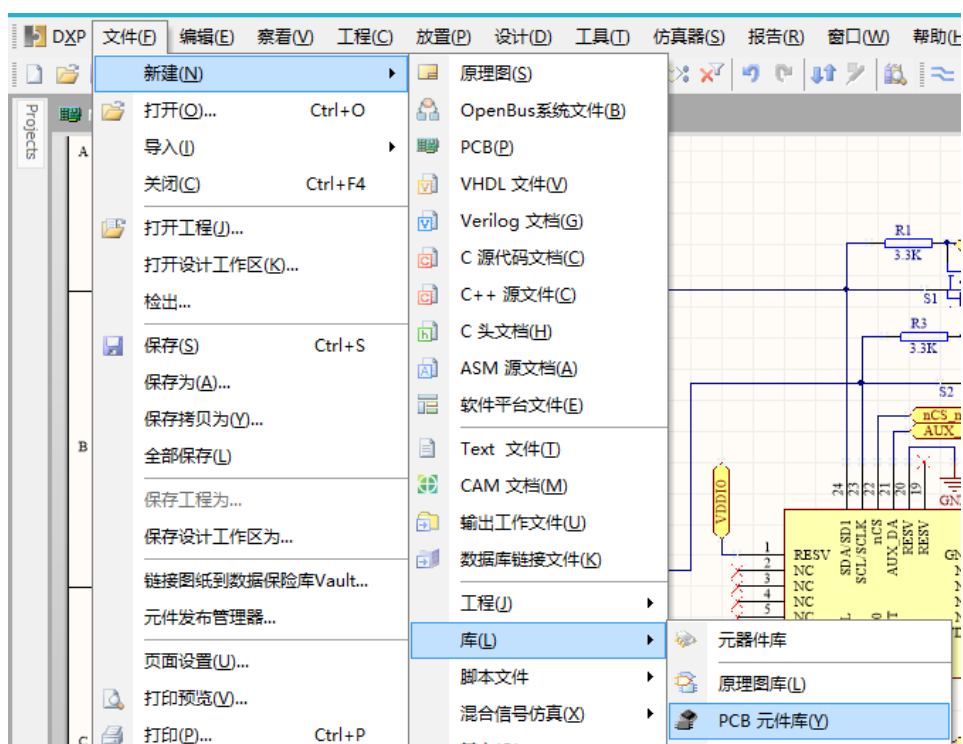
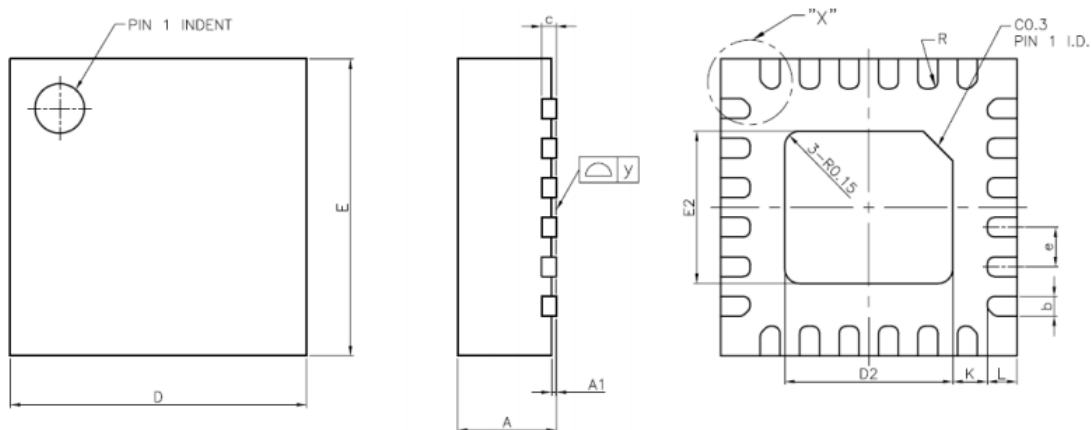


图 2-28 新建 PCB 元件库

元器件的封装官方的 datasheet 都会给出，一般都在末尾处，MPU9250 的封装规格下表所示。



符号	名称	规格 (mm)		
		最小值	正常值	最大值
A	封装厚度	0.95	1.00	1.05
A1	引脚高度	0.00	0.02	0.05

b	引脚宽度	0.15	0.20	0.25
c	引脚高度	-----	0.15REF	-----
D	封装宽度	2.90	3.00	3.10
D2	裸露焊点宽度	1.65	1.70	1.75
E	封装长度	2.90	3.00	3.10
E2	裸露焊点长度	1.49	1.54	1.59
e	引脚到引脚	-----	0.40	-----
f(e-b)	引脚宽度	0.15	0.20	0.25
K	引脚到裸露点	-----	0.35REF	-----
L	引脚长度	0.25	0.30	0.35
R	引脚边半径	0.075	REF	-----
s	边缘引脚半径 距离	-----	0.25REF	-----
y		0.00	-----	0.075

画好芯片的封装后导入到相应的元器件，然后在原理图上生成一个表单并导入到 PCB 中。在 PCB 中设计板子的位置形状以及芯片等元器件的位置。然后开始布线，这里要注意布线的粗细大小距离不要超过生产厂家的规格（一般不要过细，间距太小等），否则生产出来次品会很多或无法使用。

设计好电路板以后，一定要做一次电器规则检查。如图 2-29 所示。

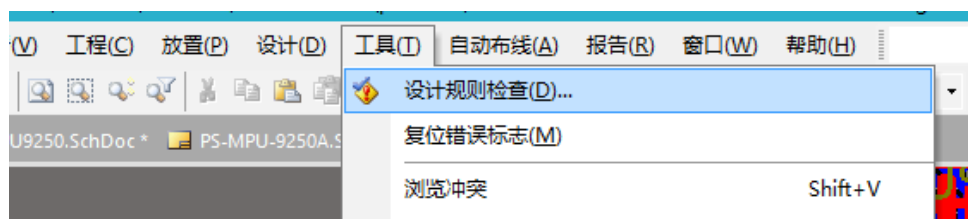


图 2-29 电路设计规则检查

检查完没有问题了以后，就开始滴泪（方便生产的时候锡膏附着）



图 2-30 滴泪

然后敷铜（减小地线阻抗, 提高抗干扰能力;降低压降, 提高电源效率），一般铜为 GND（当有大电流经过时，可以单独给电源信号敷铜），可以用来防止线路烧断线路。

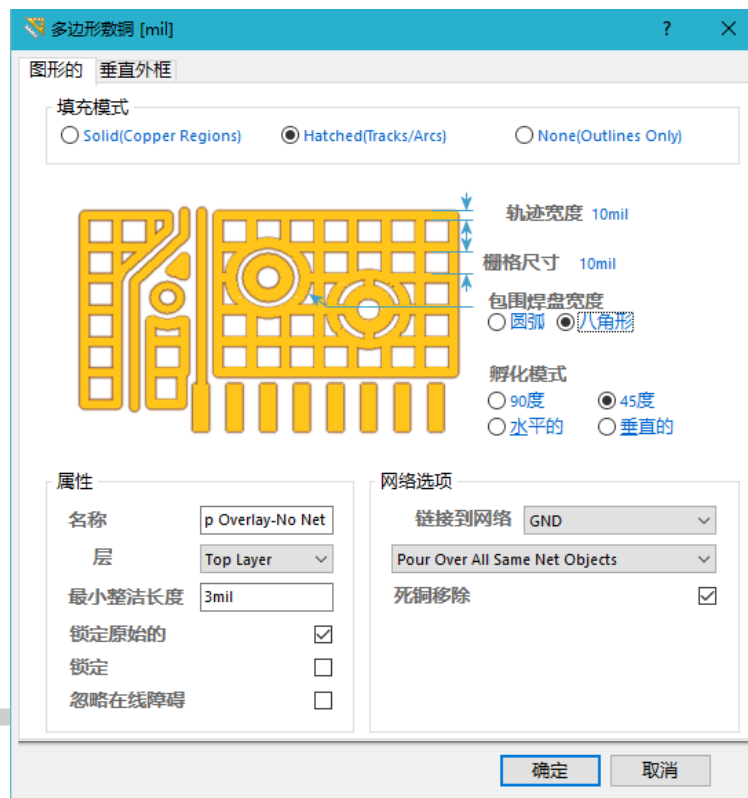


图 2-31 敷铜

然后我们就可以开始导出我们的 PCB 工程给厂家了，别忘了生成 BOM 单，以便整理工程。

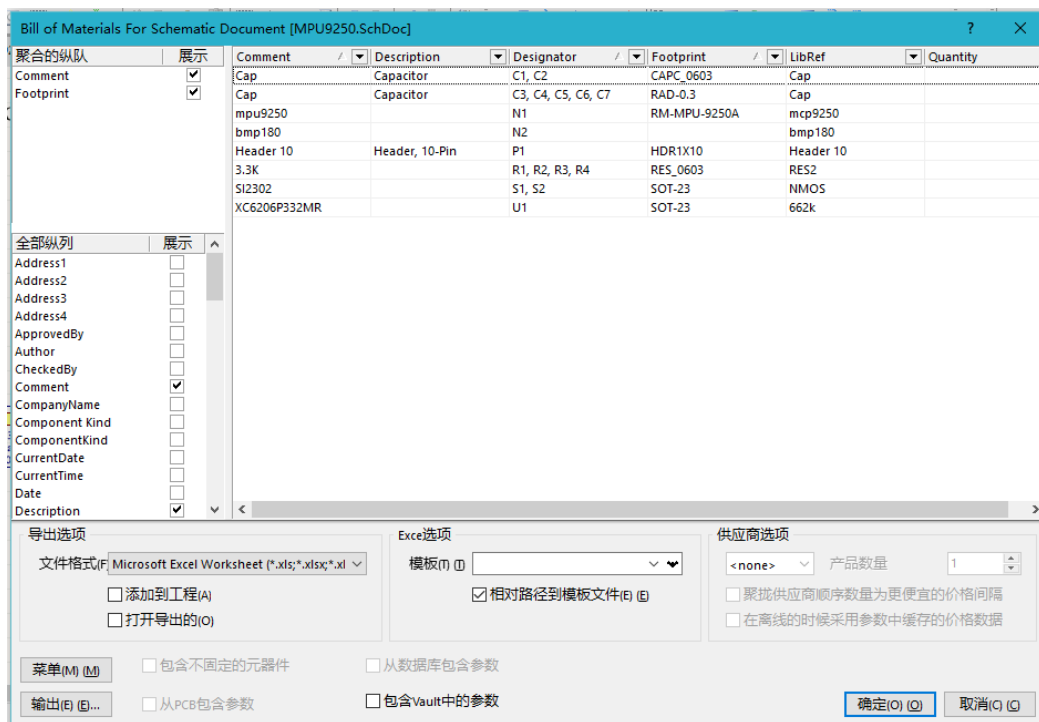


图 2-32 BOM 单生成

本书附件提供了设计完整的 PCB 电路和其硬件 BOM 资料，方便用户直接拿来使用。可以直接寄给生产厂家打板。至此，姿态传感器的硬件部分的介绍就到此结束了，下一章我们要对姿态传感器姿态一百多个处理器寄存器作介绍，看看它到底都有些什么功能。

下!视界

第 3 章 加速度陀螺仪与磁力计

本章，我们来读取姿态传感器中的原始姿态数据。也就是加速度陀螺仪与磁力计中 XYZ 三轴的数据。本章例程 MCU 为 arduino，通讯模式是 I2C 通信。这里先推荐一个安卓 APP 叫作 Dom's Diagnostic Tools（资料提供下载），如图 3-1 所示。安装并打开它。



图 3-1 Dom's Diagnostic Tools

这个软件可以连接并检测搭载安卓设备内大部分传感器的数值。我们可以在其中找到手机内部姿态传感器的数据。

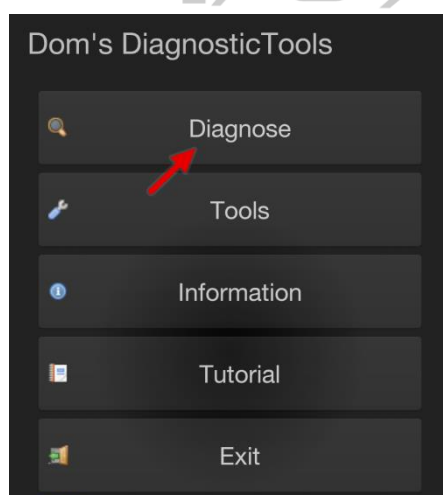


图 3-2 功能菜单

如上图 3-2，进入软件后点击 diagnose（测试）

然后点击 single diagnose（单项测试）

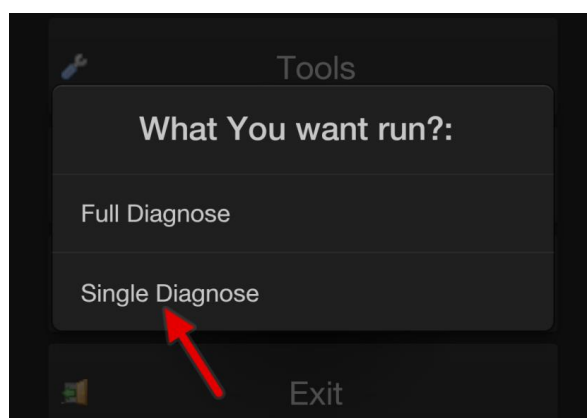


图 3-3 运行模式选择

找到并选中加速度计的测试选项

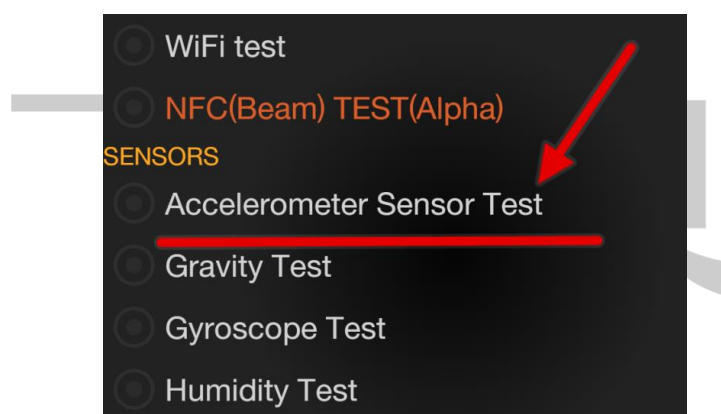


图 3-4 加速度模式

进入后我们就可以看到手机内部加速度计测量出来的数值了。（本人的手机内部搭载的姿态传感器为 MPU6050）如图 3-5 所示。此时我将手机完全平放在桌子上，显示出来的数据 x 轴为 0，Y 为 0，Z 为 9.9 也就是相当于我们平时所说的 G（重力加速度）。因为手机单纯的平放在桌面上，所以我们默认它只受到重力。

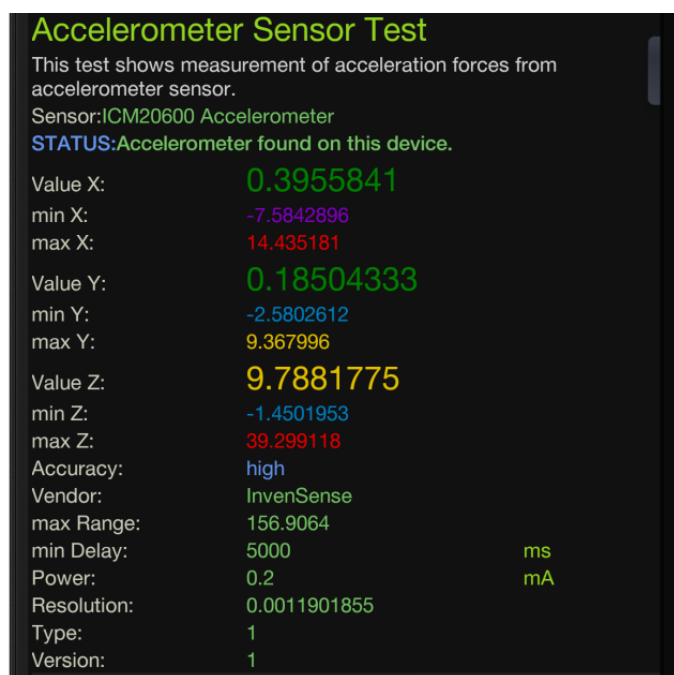


图 3-5 平躺时手机加速度计受力

此时如果把手机按照图 3-6 直立起来的时候就可以看到如图 3-7 所示的数据。

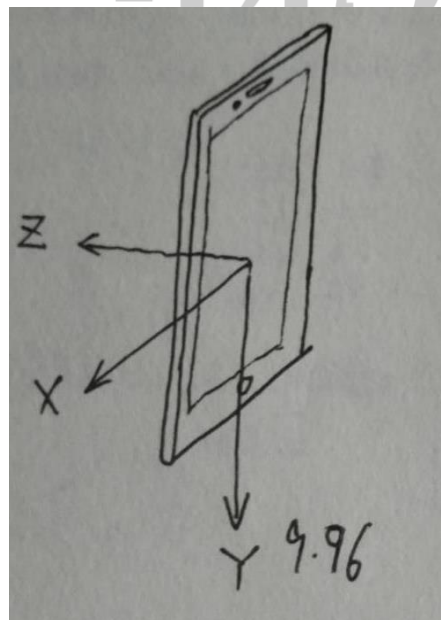


图 3-6 手机直立起来

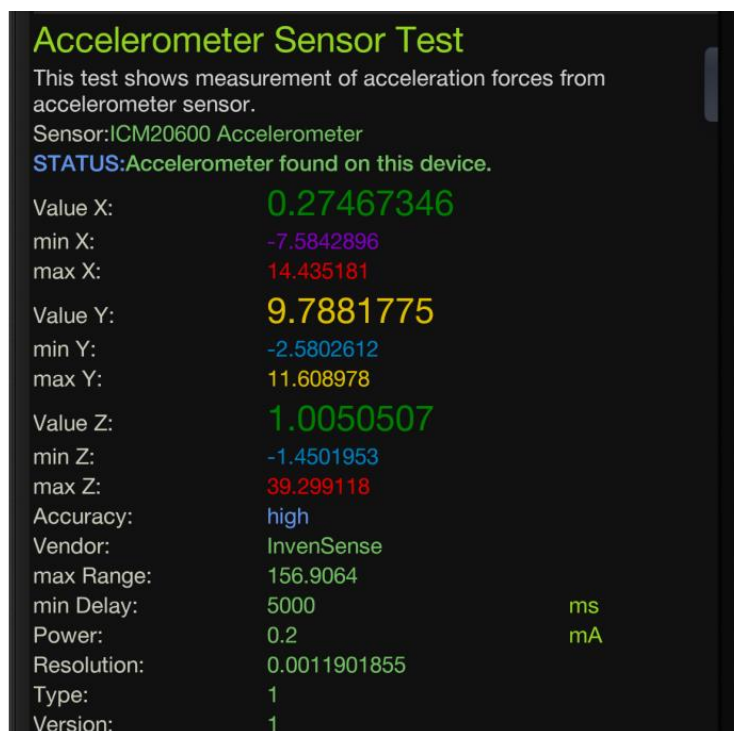


图 3-7 直立手机加速度计受力

我们可以看到此时 y 轴的值变成了 9.9，而其他轴的值趋于 0，也就是说此时芯片 y 轴受重力，而其他轴几乎不受力。

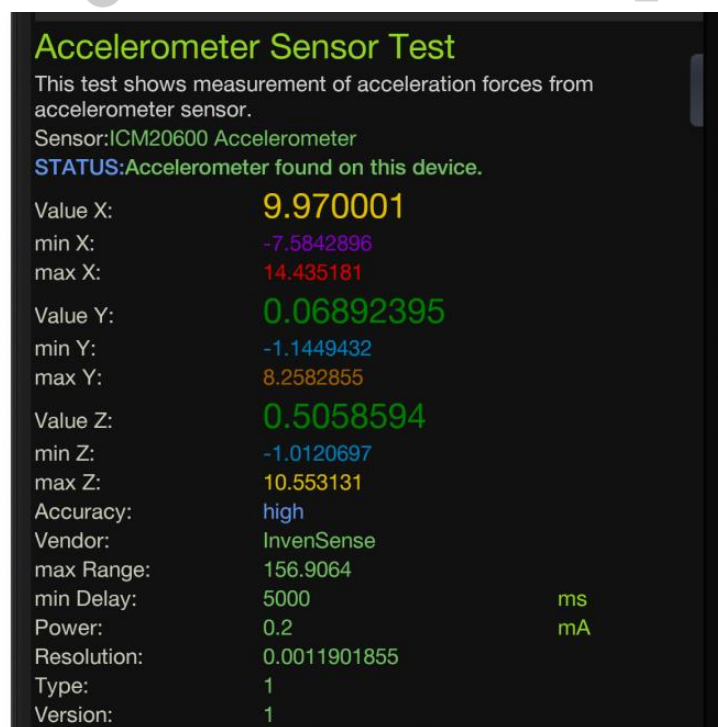


图 3-8 侧立手机时加速度受力

同理，当你把手机竖直侧放的时候 x 轴就会受重力，其 x 轴的数值则会趋于 9.8，如图 3-8 所示。在得到这些数据以后，就可以通过计算得到传感器倾角了。这里面有个最大值 max 和最小值 min 的记录，是测量程范围用的。

下面我们来点击第三个选项——陀螺仪数据检测：

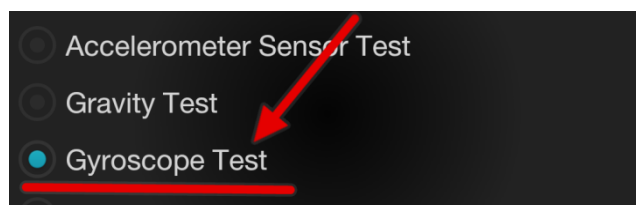


图 3-9 选择陀螺仪

根据刚才得到的 XYZ 轴方向，此时我们将手机绕 X 轴旋转（MPU6050 和 MPU9250 中加速度和陀螺仪 XYZ 是同轴的）如图 3-10 所示：

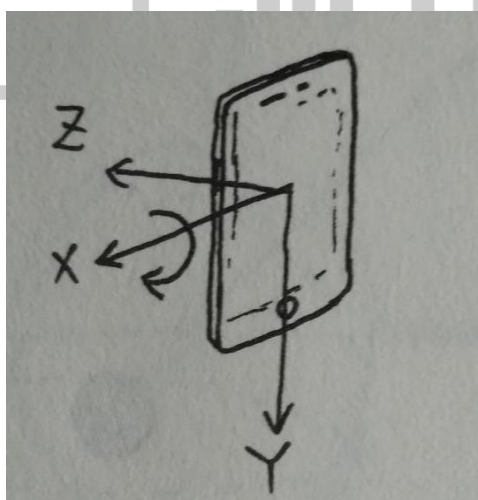


图 3-10 绕 X 轴旋转

此时我们可以得到图 3-11 所示数据：

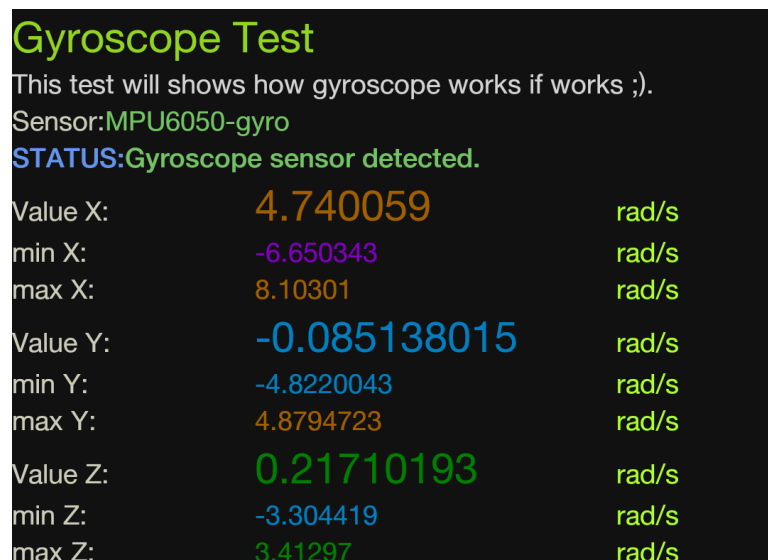


图 3-11 手机绕 x 轴选择时的角速度

可以看到只有 X 的值发生了变化，而其余两个轴还是趋于 0。所以不难知道芯片 x 轴显示的数据含义是绕 x 轴旋转的角速度，得出角速度值为 4.74rad/s 。通过这个就可以得到手机的瞬时姿态变化了。

同理现在将手机平放，如图 3-12 所示，此时手机绕 y 轴旋转。

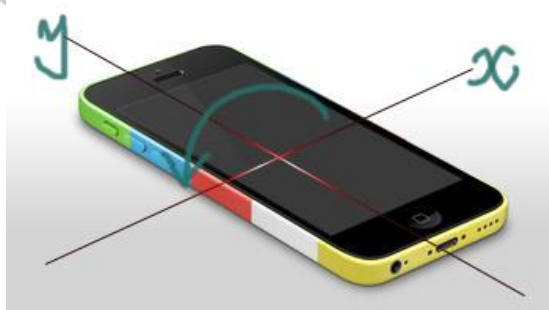


图 3-12 手机绕 y 轴旋转

可以得到如图 3-13 所示数据：

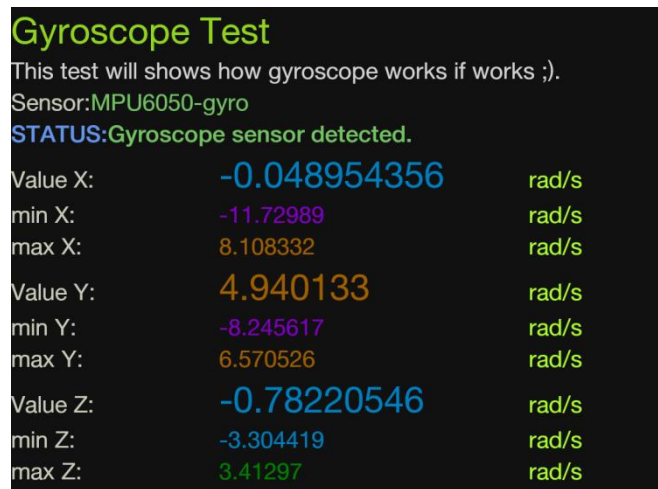


图 3-13 绕 y 轴旋转得到的角速度

很明显可以看到这里显示的是手机绕 y 轴旋转的角速度值。当然 4.94 不为负，表示了绕芯片规定 y 轴的顺时针方向。

现在将手机平放在桌子上，旋转手机。得到的即是 z 轴的数据，如图 3-15 所示。

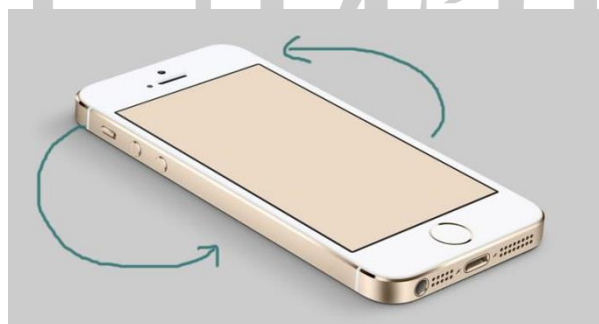


图 3-14 手机绕 z 轴旋转

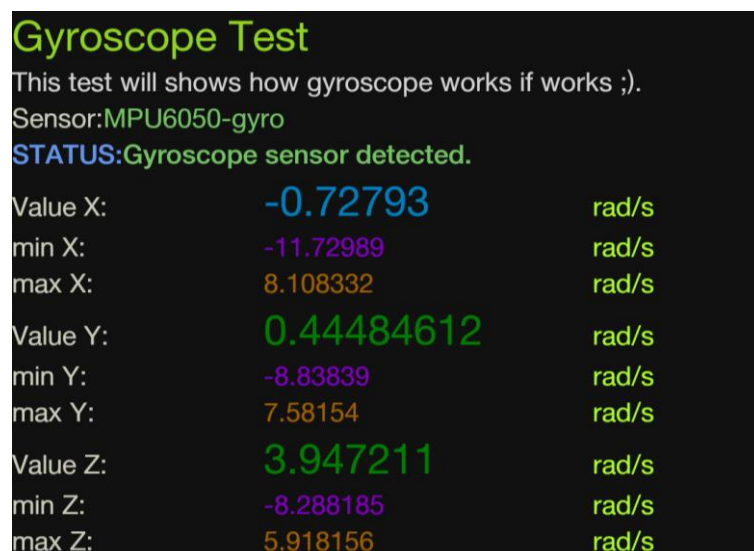


图 3-15 绕 z 轴旋转的角速度

最后我们再来打开磁力计测试，看看磁力计在手机上是如何工作的，如图 3-16 所示：

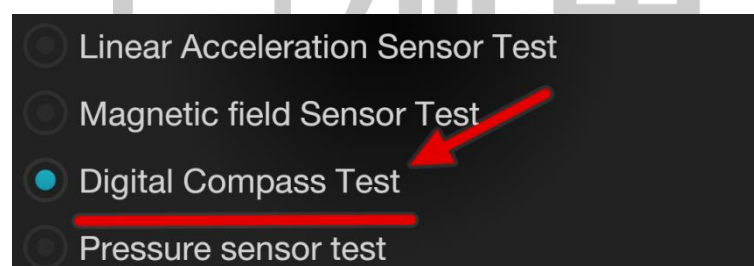


图 3-16 磁力检测

将手机平放在桌面上，然后像图 3-14 一样旋转手机。我们可以得出磁力计的 x 轴是测试手机的东南西北方向的。（MPU6050 和 MPU9250 规定磁力计的 XYZ 轴和加速度陀螺仪的 XYZ 轴不一样！这里磁力计的 x 轴取代了加速度陀螺仪的 z 轴的位置）

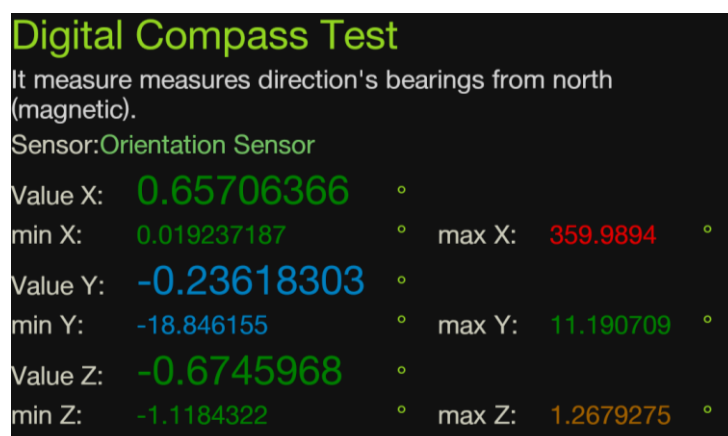


图 3-17 手机磁力计三轴测量数值

我们可以从图中看到 x 轴最大 max X 为 359.9° 最小 min X 为 0.0°，这刚好涵盖了东南西北四个方向的角度。电子地图导航的时候计算运行朝向的方法之一就是通过这种方法，当然还需要加速度计的辅助。

那磁力计的 y 轴又是干什么的呢？还是按照图 3-10 的方式旋转手机。发现磁力计的 y 轴是可以反映倾斜角度变化的量（有点儿像陀螺仪中 x 轴的功能），其实这就是侧面检测地球竖直方向上的磁场强度。

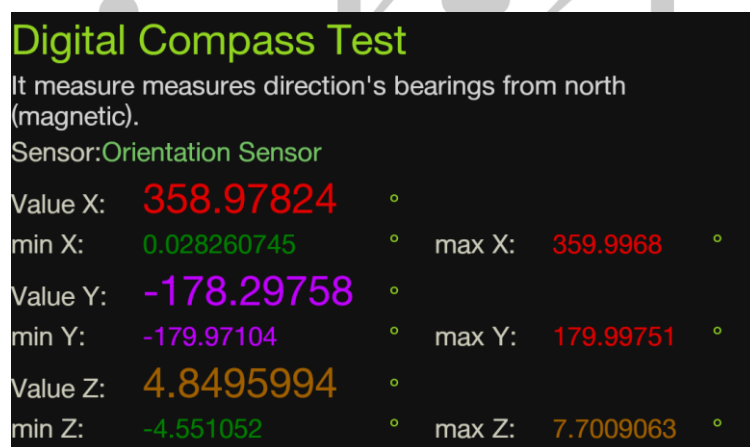


图 3-18 磁力计 y 轴旋转角度

所以磁力计 z 轴和加速度计陀螺仪的 y 轴也是相对应的。如图 3-19 所示。可以随时反映手机 y 轴的倾斜姿态。但由于地球磁场的方向问题，磁场方向和磁力计的 z 轴没什么关系，所以我们一般不用磁力计的 z 轴数据。

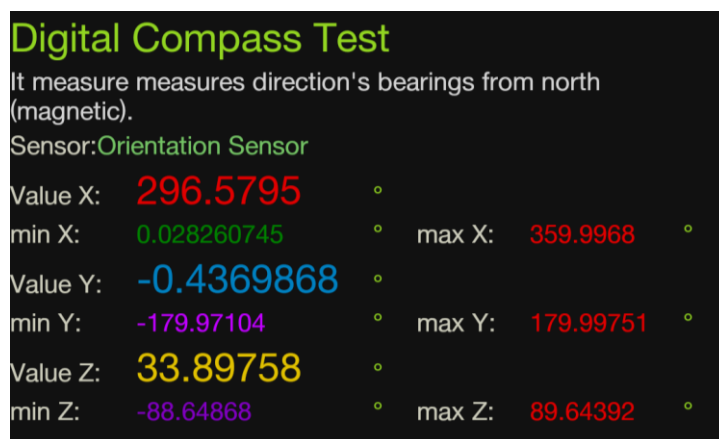


图 3-19 磁力计 z 轴旋转角度

在了解这些以后，接下来先从 MPU9250 中抽出 XYZ 测量的数据，并换算成 APP 中的数据格式。

3.1 寄存器

之前说过，数据都是存在 MPU9250 的寄存器中，如果要提取寄存器中的数据，就要先配置好寄存器的状态。那么首先要了解一下寄存器。寄存器普遍分为以下两种：

- ❑ 配置寄存器：一个寄存器有 8 位，有些功能不止一个寄存器就可以配置完全的，而且每一位都对应着不同的功能。这 8 位用[0:7]表示，[0]就是第一位，[7]就是第 7 位，每一位可以有 0 或 1 两种状态。还有连续几位是配置一个功能的，比如[0: 2]这三位数值配置一个功能。那么就可能有：000，001，010，011，100，101，110，111 分别对应了 8 种状态可以配置。
- ❑ 读取寄存器：有些寄存器是用来保存数据的，比如我们读取加速度的寄存器。加速度的数值保存在寄存中，以 2 进制的方式储存起来。比如加速度寄存器的 x 轴占用一个寄存器，也就是[0:7]，其中[7]是符号位，表示加速度的方向。[0:6]为加速度数值。假如其中的数据为：0000 1010 那么这个值就是：10，然后再进一步换算。如果数据为：1000 1010 那么储存的数据就是：-10。

MPU9250 的全部寄存器资料已经放在了附录 C 中，资料中也提供 PDF 下载。

下面我们简单提几个比较重要的寄存器。

3.1.1 电源管理寄存器（0x6B 0x6C）

位	位名	功能
[7]	H_RESET	1-重置所有寄存器和已经配置数据，重置后此位将自动清 0
[6]	SLEEP	设置时，芯片进入休眠模式（加载 OTP 后，PU_SLEEP_MODE 位将会被写在此位）
[5]	CYCLE	设置后，当 SLEEP 和 STANDBY 没有设置时，芯片将进入休眠和读数的循环状态，此时速率由 LP_ACCEL_ODR 寄存器决定。 说明：当加速度计关闭时，芯片还是会被中断唤醒，但是芯片此时不读值。
[4]	GYRO_STANDBY	当被设置时，陀螺仪驱动电路使能，但传感器不工作。这是专门用在低功耗模式快速使能陀螺仪用的。
[3]	PD_PTAT	PTAT 和 PTAT ADC 掉电
[2:0]	CLKSEL[2:0]	代码 时钟源 0 内部 20MHz 振荡 1 自动选择最佳时钟源-否则使用外部振荡 2 自动选择最佳时钟源-否则使用外部振荡 3 自动选择最佳时钟源-否则使用外部振荡 4 自动选择最佳时钟源-否则使用外部振荡 5 自动选择最佳时钟源-否则使用外部振荡 6 内部 20MHz 振荡 7 停止时钟源并保持 OTP 加载完成后，PU_SLEEP_MODE 状态将会被写入 CLKSEL[0]

这个寄存器主要管理的是芯片的启动和工作状态的寄存器。本寄存器分为 2 个，地址分别为 0x6B 和 0x6C，前者主要是管理芯片的工作模式（上表是 0x6B），后者则是控制加速度和陀螺仪的使能与否。

需要注意的是 Power Management1 中的[2: 0]（0-2 位），一般默认选择内部振荡时钟信号，配置为 0。如果在单独检测芯片工作情况或作为从机时可配置为外部时钟信号，但注意一定要在 11 引脚上接上外部时钟信号，否则芯片将不工作。

此外需要注意的地方还有[4]（第 4 位）和[6]（第 6 位），这里同样都是对休眠模式的设置但是使用场景会不同。[4]（第 4 位）为低功耗加速度模式，使用的场景可以是：设备处于长时间静止状态的时候突然做剧烈运动（加速度计的值进行大量的变化），从而来唤醒芯片。要进入到这种模式，你还需要设置 CYCLE 位为 1，SLEEP 位为 0，TEMP_DIS 位为 1，DIS_XG,DIS_YG,DIS_ZG 位为 1（注意其中设置位包含在 6A 和 6B 两个寄存器中）。[6]（第 6 位）为休眠模式的外部中断唤醒，配合引脚 INT 使用。这种的触发条件就是得有个外部唤醒事件的发生才可以。要进入这个模式时请将 SLEEP 置 1，并连接 12 外部中断引脚。

3.1.2 芯片配置寄存器(0x1A)

寄存器位	寄存器名	功能
[7]	-	保留位
[6]	FIFO_MODE	当设置为 1 时，FIFO 缓冲区满后将不再写入数值。 当设置为 0 时，FIFO 缓冲区满后新数据会不停更新以替换老数据。

[5:3]	EXT_SYNC_SET[2:0]	使能 FSYNC 引脚以采样数据。 <table><tr><td>EXT_SYNC_SET</td><td>FSYNC 位地址</td></tr><tr><td>0</td><td>功能关闭</td></tr><tr><td>1</td><td>TEMP_OUT_L[0]</td></tr><tr><td>2</td><td>GYRO_XOUT_L[0]</td></tr><tr><td>3</td><td>GYRO_YOUT_L[0]</td></tr><tr><td>4</td><td>GYRO_ZOUT_L[0]</td></tr><tr><td>5</td><td>ACCEL_XOUT_L[0]</td></tr><tr><td>6</td><td>ACCEL_YOUT_L[0]</td></tr><tr><td>7</td><td>ACCEL_ZOUT_L[0]</td></tr></table> FSYNC 将会捕捉到一个短的信号并产生一次锁存值，同一个信号只能产生一次锁存值。条件是必须外接第三方驱动设备，而且 fsync 通信速率要低于我们的正常设备的采样率。	EXT_SYNC_SET	FSYNC 位地址	0	功能关闭	1	TEMP_OUT_L[0]	2	GYRO_XOUT_L[0]	3	GYRO_YOUT_L[0]	4	GYRO_ZOUT_L[0]	5	ACCEL_XOUT_L[0]	6	ACCEL_YOUT_L[0]	7	ACCEL_ZOUT_L[0]
EXT_SYNC_SET	FSYNC 位地址																			
0	功能关闭																			
1	TEMP_OUT_L[0]																			
2	GYRO_XOUT_L[0]																			
3	GYRO_YOUT_L[0]																			
4	GYRO_ZOUT_L[0]																			
5	ACCEL_XOUT_L[0]																			
6	ACCEL_YOUT_L[0]																			
7	ACCEL_ZOUT_L[0]																			
[2: 0]	DLPF_CFG[2:0]	若要使用 DLPF，fchoice[1:0] 必须设置成 2’ b11，fchoice_b[1:0]设置成 2’ b00。 详见下面表																		

这个寄存器的[2: 0]是用来配置芯片的数据交换的速率和温度计的参数属性。这里根据具体需要选择具体的配置。若使用高带宽速率传输，请务必打开 FIFO 缓存区。

[5: 3]是用在有外部辅助传感器的时候使用，配合 FSYNC 引脚使用。FSYNV 会读取外部信号并存在一个位中。[5: 3]就是配置这个位储存位置的。

[6]是配置 FIFO 写入数据缓存的，一般我们置 0，以不停地更新最新数据。若置 1 后缓存器存满后可写入 0x00 复位清零。

3.1.3 陀螺仪配置寄存器(0x1B)

位	位名	位功能
[7]	XGYRO_Cten	x 轴陀螺仪自检
[6]	YGYRO_Cten	y 轴陀螺仪自检
[5]	ZGYRO_Cten	z 轴陀螺仪自检
[4:3]	GYRO_FS_SEL[1:0]	陀螺仪全范围选择： 00 = +250dps 01 =+500dps 10 =+1000dps 11 =+2000dps
[2]	-	预留位
[1:0]	Fchoice_b[1:0]	和上一个表格所表示相反。 Fchoice _b 的上表的Fchoice 相反。

这里设置的是陀螺仪的响应速度和带宽延时等特性。[4：3]则是设置陀螺仪的灵敏度范围，也就是第1章所说的 deg/s，那么就需要根据具体情况来设置具体的灵敏度了。

比如测得陀螺仪 x 轴是 4500，如果选择的是+250dps，那么可以得到： $4500/250\text{dps}=18^{\circ}/\text{s}$ 。如果选择的是+2000dps，则得到： $4500/2000\text{dps}=2.25^{\circ}/\text{s}$ ，也就是陀螺仪现在绕 x 轴以 $2.25^{\circ}/\text{s}$ 的速度移动。

[7:5]则是控制陀螺仪自检功能的开关。

3.1.4 加速度计寄存器(0x1C 0x1D)

位	位名	位功能
[7]	ax_st_en	加速度 x 轴自检

[6]	ay_st_en	加速度 y 轴自检
[5]	az_st_en	加速度 z 轴自检
[4:3]	ACCEL_FS_SEL[1:0]	加速度全范围选择： ±2g（00），±4g（01），±8g（10），±16g（11）
[2:0]	—	预留

0x1C 中[4:3]设置加速度灵敏度，比如你此时得到加速度值是 20，你设置的是 2g，那么 $20/2g=10g$ 。如果是 16g，那么得到 $20/16g=1.25g$ 。[7:5]为加速度计自检。

位	位名	位功能
[7:6]	预留	
[5:4]	预留	
[3]	accel_fchoice_b	和下表的 accel_fchoice 相反的
[2:0]	A_DLPFCFG	加速度低通滤波，设置如下表

0x1D 中主要是设置加速度计的采样率和低功耗模式的采样率，如果需要开启低功耗唤醒模式的话，需要配置 30 号寄存器。

磁力计的寄存器由于和加速度陀螺仪不在一起，所以将在本章第三节单独讲解磁力计的寄存器。

3.2 配置

为了方便大家理解步骤和代码，本章这里用 Arduino 的程序来简单举例说明。Arduino 编程非常简单，只要以前用 C 语言开发过单片机的同学应该可以秒懂。**A10 Arduino 的使用和编程**（本实例附录中提供下载，方便大家测试调试）

```
void setup()
{
```

```

Serial.begin(9600);      //设置好波特率

Wire.begin();           //打开串口

mpu9250.reburn();       //配置芯片

mpu9250.readdata();     //读取数值
}

```

这个 setup 里的功能是芯片刚开始上电时 MCU 芯片的初始化的步骤。前两个不用说（第一个设置波特率就是设置芯片与计算机串口的传输速率，单位为波特率，一般有 9600, 115200 等，这里选用的是 9600 波特率。代表一秒钟传输 9600 位数，一个字节有 8 位，那么就有 $9600/8=1200$ 位数据；第二个是 arduino 的串口设置命令，arduino 将与电脑产生连接。Arduino 通过这种方式可以把需要显示的数据打印在电脑的串口助手上，以便调试传感器时使用），然后就可以配置芯片了：

```

void MPU9250::reburn()
{
    I2CWriteByte(MPU9250_ADDRESS, 0x6B, 0x80); //重置寄存器

    delay(100); //延时 0.1s

    I2CWriteByte(MPU9250_ADDRESS, 27, GYRO_FULL_SCALE_250_DPS); //陀螺仪灵敏度设置为 250dps

    I2CWriteByte(MPU9250_ADDRESS, 28, ACC_FULL_SCALE_2_G); //加速度计灵敏度设置为 2G

    I2CWriteByte(MPU9250_ADDRESS, 0x37, 0x02); //旁路模式

    I2CWriteByte(MAG_ADDRESS, 0x0A, 0x11); //设置磁力计通信模式
}

```

重置寄存器配置的情况可以参照第 107 号寄存器也就是电源管理 1 寄存器的配值来参考。此处向 0x6B 配值 1000 0000（0x80）也就是说满足如下情况：

1. 启用内部 20MHz 振荡时钟源
2. PTAT ADC 不掉电
3. 不使用芯片待机模式
4. 不进入休眠模式

5. 重置所有寄存器和数据

重置芯片配置以后，由于本章只单纯地读取数据。并不需要外部中断和配置 I2C 从地址等信息。所以只需要打开加速度和陀螺仪即可。然后进行加速度计和陀螺仪灵敏度配置的选择方式，查阅配置寄存器我们可看到，加速度的灵敏度选择可以配置为 2G，4G，8G，16G。其中这个 G 就是重力加速度，也就是地球的引力。通常这个引力根据所处地球位置的不同而不同，大致介于 9.78m/s^2 到 9.83m/s^2 之间，约为 9.8m/s^2 。那么 2G，4G 前面这个数字 2 或 4 就是 G 的倍数的意思。由于每个轴的数据都是由两个寄存器来保存的，每个寄存器有 8 位，那么测得的值就有 16 位，也就是 $2^{16}=65536$ 这么多，又因为加速度和角速度都是有方向的，所以 65536 会被分成 $-32768\sim 32768$ 。也就是测试出来最大的数值就是 ± 32768 ，然后便可以通过以下这个公式转换：

$$\text{ACC_XYZ} = \text{N} * \text{G} * (\text{ACC_OUTXYZ} / 32768)$$

ACC_XYZ 是每个轴得出的重力加速度的数据；

N 为我们选择的配置倍数（2，4，8，16）；

G 为当地的重力加速度，介于 $9.78\sim 9.83\text{m/s}^2$ 之间；

ACC_OUTXYZ 为测试得到的数据值；

根据公式，可以看出，**倍率越低精度越好，倍率越高表示的范围越大，所以开发时根据具体应用情况来规定精度。**（比如调试飞行器的话，动作幅度就比较大，倍率就需要高一些，这样获得的数据范围会更宽，便于计算。如果 VR 之类的，平时移动幅度没有那么大的话，精度就可以选低一些。）**重要的一点是，在实际根据加速度计算姿态的时候千万不要忘记减掉重力的数据才能测量出物体的姿态来。**

精度在默认情况下为 2G，适用于平衡之类的控制计算。

对比一下 28 号寄存器的值，可以设 `ACC_FULL_SCALE_2_G=0x00`。

那么配置参数：

加速度范围选择为 $\pm 2g$ （00）

加速度各个轴均不自检

同理陀螺仪测试的角速度也是这么配置，陀螺仪的精度分为 $250^\circ/s$ ， $500^\circ/s$ ， $1000^\circ/s$ ， $2000^\circ/s$ 。所以当我们选择为 $250^\circ/s$ 时，若此时某一个轴测出的值为 ± 32768 时，表示这个轴正以 $250^\circ/s$ 的速度运动，但是如果速度高于 $250^\circ/s$ 的时候将不能被正确的测试出来，所以具体选择的参数是多少，也要视具体情况而定。由此可以看出来，选择的数值越高，精度就越小，可测量的范围就变大了很多，这和加速度是一样的。

陀螺仪的计算公式：

$$G_{xyz} = N * (G_{out} / 32768)$$

其中 G_{xyz} 为计算后各个轴的角速度值；

G_{out} 为测得的相应数据值；

N 为我们配置的精度（250，500，1000，2000）

这里欲将精度配置为 250dps，则 27 号寄存器写入 0x00，满足这个要求后寄存器被配置为：

1. 关闭自检功能

2. 选择测量范围为：250dps

3. 陀螺仪将以带宽 3600Hz 间隔延时 0.17s 的速度传输测试数据，温度值将会以 4KHz 间隔 0.04s 的速度传输测试数值。

磁力计的设置也和之前两个一样，这里设置 0x0A 为 0x16 即 0001 0110 。此位为专门控制磁力计测试模式的，根据寄存器表可知现在配置的是：

启用连续测量模式 2 测量磁力数据

16 位数据输出模式（即 $-32768 \sim 32768$ ）

之前讲过 MPU9250 内部实为 2 个传感器，一个 MPU6050，一个 AK8963（磁力计）。其中 MPU6050 可以通过 SPI 和 I2C 的方式通信，而 AK8963 只能通过 I2C 的模式进行通信。所以要利用 I2C 的方式读取磁力计的数值。相当于一个 I2C 总线上挂载 2 个传感器。

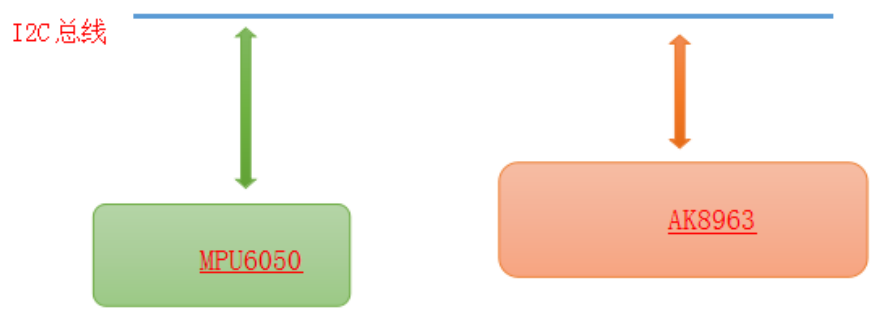


图 3-20 传感器的 I2C 总线挂载

现在将磁力计的 55 号寄存器（0x37）配置为 0x02，开启旁路模式。并将其其他中断功能关闭（官方手册上说只能通过这种模式才能读取磁力计的数据，但经测试不开启旁路模式也可以读取磁力计的数据）。将磁力计的通信模式设置成 0x01，由于磁力计是每掉电一次再上电才能读取新数值，这里在读取磁力计数值的时候先读取磁力计的数据预存位（ST）。

```
uint8_t ST1;

do
{
    I2CRead(MAG_ADDRESS, 0x02, 1, &ST1);
}

while (!(ST1&0x01));
```

这里用 do while 的语句读取 0x02 寄存器来实现。

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
02H	ST1	0	0	0	0	0	0	0	DRDY

重置	0	0	0	0	0	0	0	0
----	---	---	---	---	---	---	---	---

DYDR 在磁力计读取好数据时会置为 1，所以这里用 do while 读取此位是否为 1，当为 1 时才进入接下来读取数据的部分。定义一个局部变量来接收 8 个字节的 XYZ 轴数据值。在读取磁力计数值后还需要再设置一次磁力计，将其设为 0x11，即 16 位，单独测量模式。rawData 是这里设计的缓存器，我们把数据值暂时保存在这个数组里面，方便程序调用。

```
uint8_t rawData[6];      //读取磁力计的数据

I2CRead(MAG_ADDRESS, 0x03, 6, rawData);

I2CWriteByte(MAG_ADDRESS, MAG_CNTL1, 0x11); //单独测量模式
```

注意这里操作顺序不要改变，读完值后要重置一下 MAG_CNTL1 寄存器。

总结一下，这里的设置顺序：

在 reburn 中我们设置了磁力计的通信模式和旁路模式，读取数据时先读取状态位 ST，当数据准备好后读取 XYZ 三轴数据，并重置寄存器通信模式为单独测量模式，如此循环。

以上的配置就是 MPU9250 最简单的配置方式。现在让我们来读取寄存器中测量的数据。

3.3 计算

经过配置寄存器以后，就可以开始读取寄存器内部的数值。由于 MPU9250 内实为 2 颗芯片——加速度陀螺仪和电子罗盘的寄存器是分开的，所以分开计算。首先来看一下加速度和陀螺仪的寄存器数据。

加速度陀螺仪寄存器

地址(16 位)	地址(10 位)	寄存器名字	读写	位
3B	59	ACCEL_XOUT_H	R	[15:8]

3C	60	ACCEL_XOUT_L	R	[7:0]
3D	61	ACCEL_YOUT_H	R	[15:8]
3E	62	ACCEL_YOUT_L	R	[7:0]
3F	63	ACCEL_ZOUT_H	R	[15:8]
40	64	ACCEL_ZOUT_L	R	[7:0]
43	67	GYRO_XOUT_H	R	[15:8]
44	68	GYRO_XOUT_L	R	[7:0]
45	69	GYRO_YOUT_H	R	[15:8]
46	70	GYRO_YOUT_L	R	[7:0]
47	71	GYRO_ZOUT_H	R	[15:8]
48	72	GYRO_ZOUT_L	R	[7:0]

我们先建立数组 Buf，读取传感器的寄存器数据并两两相加存入 ACC, Gyro 中。

```
void MPU9250::readAG()//读取加速度和陀螺仪的数据
{
    uint8_t Buf[14];
    I2CRead(MPU9250_ADDRESS, 0x3B, 14, Buf);

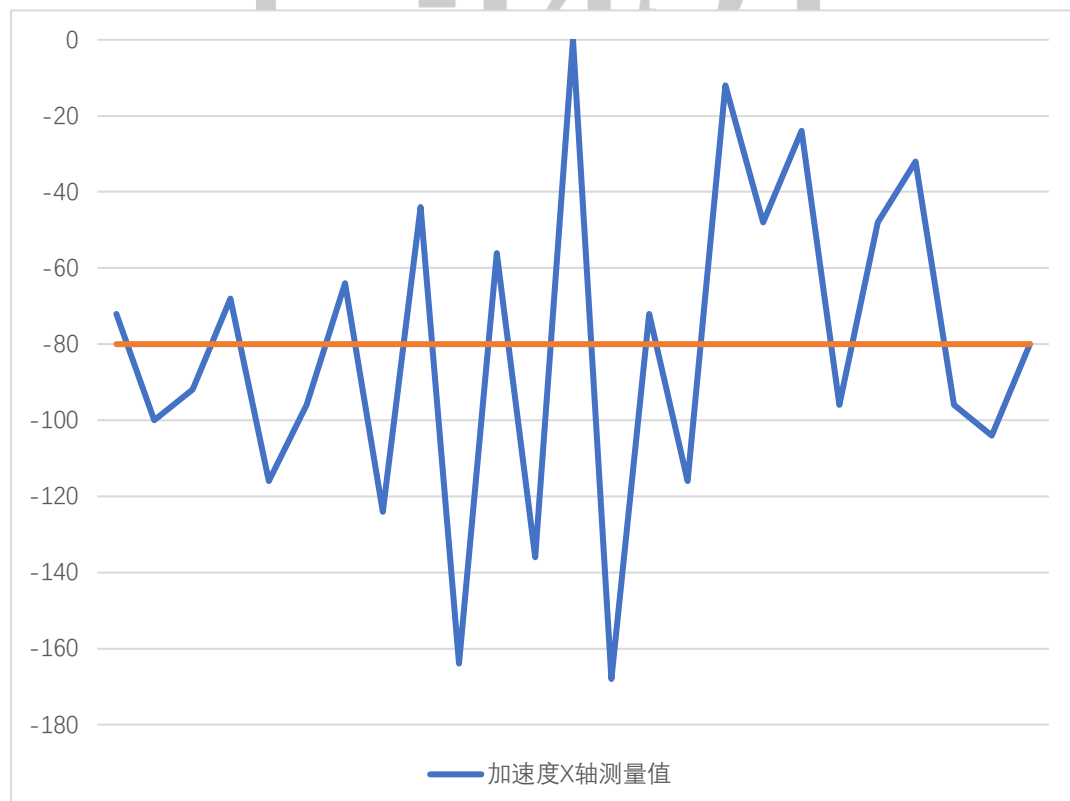
    // 加速度
    ACC_X=(Buf[0]<<8 | Buf[1]);
    ACC_Y=(Buf[2]<<8 | Buf[3]);
    ACC_Z=(Buf[4]<<8 | Buf[5]);

    //陀螺仪
    Gyro_X=(Buf[8]<<8 | Buf[9]);
    Gyro_Y=(Buf[10]<<8 | Buf[11]);
    Gyro_Z=(Buf[12]<<8 | Buf[13]);
}
```

根据寄存器表不难找到存放加速度和陀螺仪的寄存器数值，此处读取从地址 0x3B 开始的 14 个寄存器的值。6 个加速度的值，6 个陀螺仪的值，其中还包括 2 个芯片当前温度的值。（温度的数值主要是用来减少温漂的误差，其实现在已经不需要这个温度的数据，因为输出的数据是已经做过温度偏移矫正了，关于温漂的运用附件资料中会讲解。）

而其他的 12 个数据类似的我们可以用 $ACC_X = (Buf[0] \ll 8 \mid Buf[1])$ 的方法赋值给 ACC_X 即可。这里要注意一下定义数值的类型。因为寄存器为 8 位数据，这 8 位是单纯的位的数值，不带符号，所以此处数组我们定义的是 uint（无符号整形）。而我们最后得到的数据为 16 位，范围是 $(-32768 \sim +32768)$ ，所以合并数据后我们要把数据定义为 int16（有符号）。

现在运行程序就能得到 6 组不停变动的数据，可以通过串口实时监测当前加速度计和陀螺仪的数据。以其中加速度 x 轴的数据为例。水平放置 MPU9250 的传感器，可以得到一组 200ms 内的采集的数值。我把它整理成一段折线图以便大家分析。



可以看到数据在-80 上下 80 之间左右浮动，平均数就是那条橙色的线。而

我们知道水平放置的时候数据不应该是像表中这样浮动的范围，也不应该在-80左右，所以我们现在要做的就是校准这个数值。

要做到绝对的校准，还需要借助其他传感器，这里光用一个精度是达不到那么高的。但是满足日常生活的需要还是绰绰有余的。**A11** 我们这里用一个比较低，但很直接有效的方式将输出的数据规整到正确的数值上去——平均值+offset 的方式。（之后会讲解其他的算法）

平均值可以通过克服图表中那些毛刺的数值，让测量得到的数据比较平稳，不会再像那样幅度的跃动。比如计算 100 次测量得到数值的总和并取平均的方法来克服毛刺。

通过取 100 次测量值来取得加速度各轴的平均值，由于要相加 100 次的结果，所以需要定义一个很大的数，也就是说至少要能容纳 $32768 \times 100 = 3276800$ 这么大的数据，所以这里定义 long 型或者 int32 型的数据。所以还需要定义一个局部变量，主要是为了减少内存占用，何况这么大的数。

```
int32_t AX100=0, AY100=0, AZ100=0, GX100=0, GY100=0, GZ100=0; //定义 6 个局部变量

for(int i=0; i<100; i++) //循环计算 100 次的值累加起来
{
    readAG(); //读取加速度和陀螺仪的数据

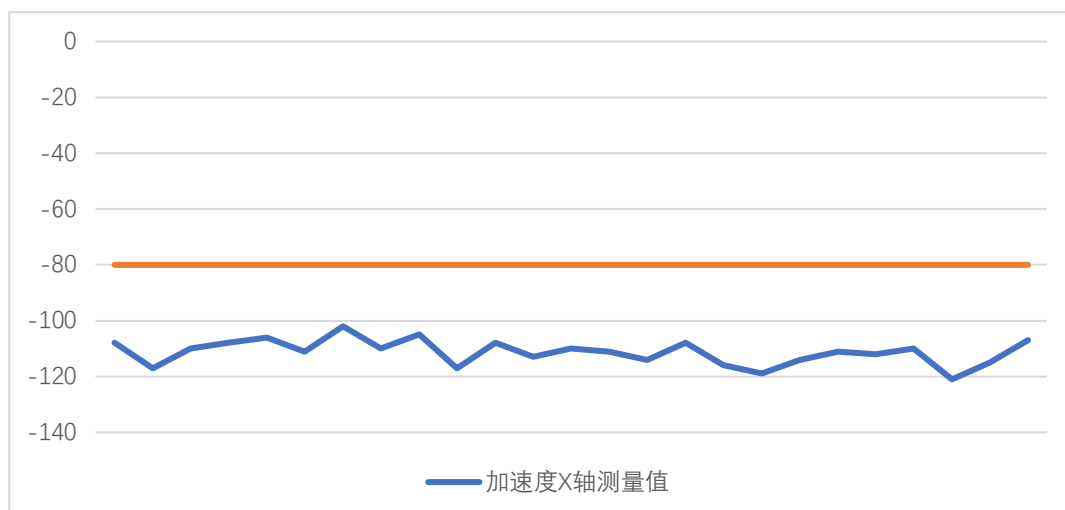
    AX100 += ACC_X;

    AY100 += ACC_Y;

    AZ100 += ACC_Z;
}

ACC_X = AX100/100; //计算 100 次的平均值
ACC_Y = AY100/100;
ACC_Z = AZ100/100;
```

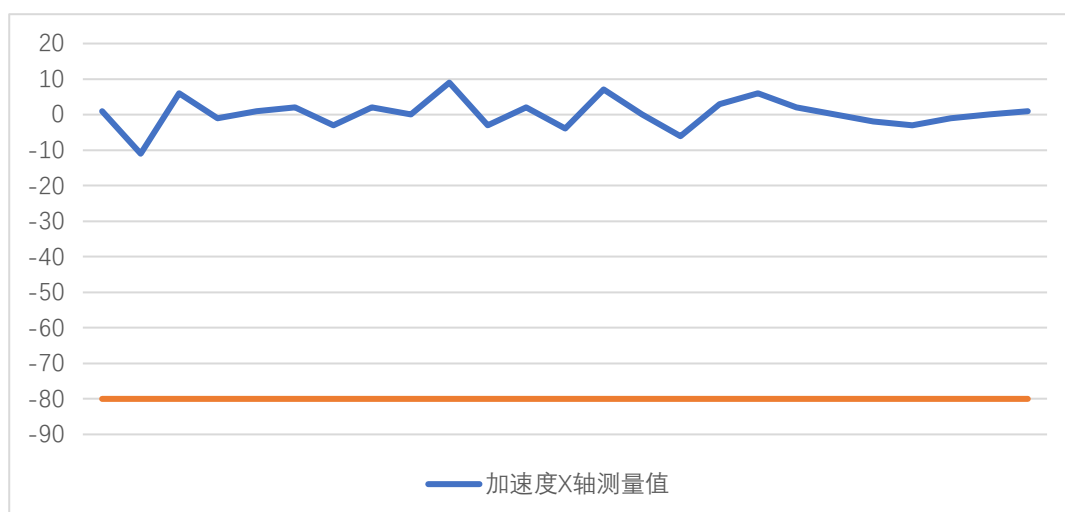
现在把得到的数据再进行一次数据分析，看看现在得到的数据是否达到了我们想要的效果。如下表所示：



相信也不用做过多赘述，现在测量的值的毛刺大大地减少，数值趋近平稳。但是平均值刚才估计错误，实际在图上来看大概在-110 左右波动。现在要做的就是将这条蓝色的线，移动到0 左右，让 X 的值尽量的靠近 0。此处添加个 offset 值专门补偿每个轴的偏移量。

```
ACC_X = AX100/100+XA_offs;
ACC_Y = AY100/100+YA_offs;
ACC_Z = AZ100/100+ZA_offs;
```

然后定义 XA_offs=110，此时再检测加速度 x 轴得到的数值，如下表所示。



以此可以看到，加速度 x 轴输出的数值就差不多达到所需的数值了。同理，现在就开始调教一下 Y 和 Z 轴的数值。Y 的数值比较好调节，这里就不再赘述。主要是 Z 这个地方要注意一下。首先，要知道如下的公式：

$$ACC_Z = N * G * (ACC_Z_OUT/32768) \text{ (m/s}^2\text{)}$$

这个公式就是加速度每个轴的数值转换为单位值的公式。在公式中，N 为选择的加速度倍数，由于之前配置的是 2 倍所以这里 N=2，G 为当地的重力加速度，也就是 9.80m/s^2 ，ACC_Z_OUT 为单轴输出的数据，ACC_Z 为算出的值，单位是 m/s^2 。这里期望 y 轴最后得到的数为 9.8=ACC_Z，此时公式变为：

$$9.8 \text{ (m/s}^2\text{)} = 2 * 9.8 * (ACC_Z_OUT/32768) \text{ (m/s}^2\text{)}$$

所以 ACC_Z_OUT 的值必定是 16384. 如果想要把 ACC_Z 的平均值提升到 16384，还需给 Z 的平均值加 499，赋值给 ZA_offs。

现在把芯片放置在水平面上，XYZ 三轴输出的数值应该分别在 0，0，16384 左右。这样就和想要的数值非常接近了。这里要说明一点，把这个芯片以水平姿态放置，或多或少都会有些偏差，但是这些偏差按照我们这种方式修正以后的计算结果是不会影响计算结果的。

通过以上的修正参数以后，现在要把得到的数值转换成加速度 (m/s^2) 为单位的数值。所用到的公式还是上面提到的这个公式。

$$ACC_Z = N * G * (ACC_Z_OUT/32768) \text{ (m/s}^2\text{)}$$

下面来说明一下这个公式的推导的思路：

我们得到的数据是一个 16 位有符号的整形(int_16)。所以数据变化范围是 215，最高位是符号位。也就是说数据其实是在 $-32768 \sim +32768$ 之间变化。得到的数据范围也在这其中变化，所以此数据对应的加速度数值应该是 $G * (ACC/32768)$ 。当输出为 32768 时，对应的加速度是 9.8m/s^2 ，由于之前配置倍数为 2G，所以此处要 $2 * G * (ACC/32768)$ ，也就是说最多可以检测到 $9.8 * 2 = 19.6\text{m/s}^2$ ，这里就可以明显看出 0x1C 寄存器中配置 2G, 4G, 8G, 16G 的区别。当选择的数值越大时，比如 16G，那么的最大可以测得 $16 * 9.8 = 156.8\text{m/s}^2$ 速度，而一般 12G 的加速度都比导弹火箭等加速度还要快了，由此来看，这个测量范围几乎能测得生活中几乎所有的应用场景。由此我们也能看出测得的范围越大，相应的数值精度就越低。所以，到底选择什么样的范围，还是需要综合一下应用场景最好。

现在把上面的公式带入传感器，还是在传感器正面向上静止不动的时候把 X，Y，Z 数值都测出来，然后把测量的数值绘制成下表。



图 3-21 传感器的实测数据

大家可以看到黑色的 z 轴数值在 9.7800 左右浮动，Xy 轴分别在-0.002 和 0.002 左右浮动。这就是测得的实际数值。

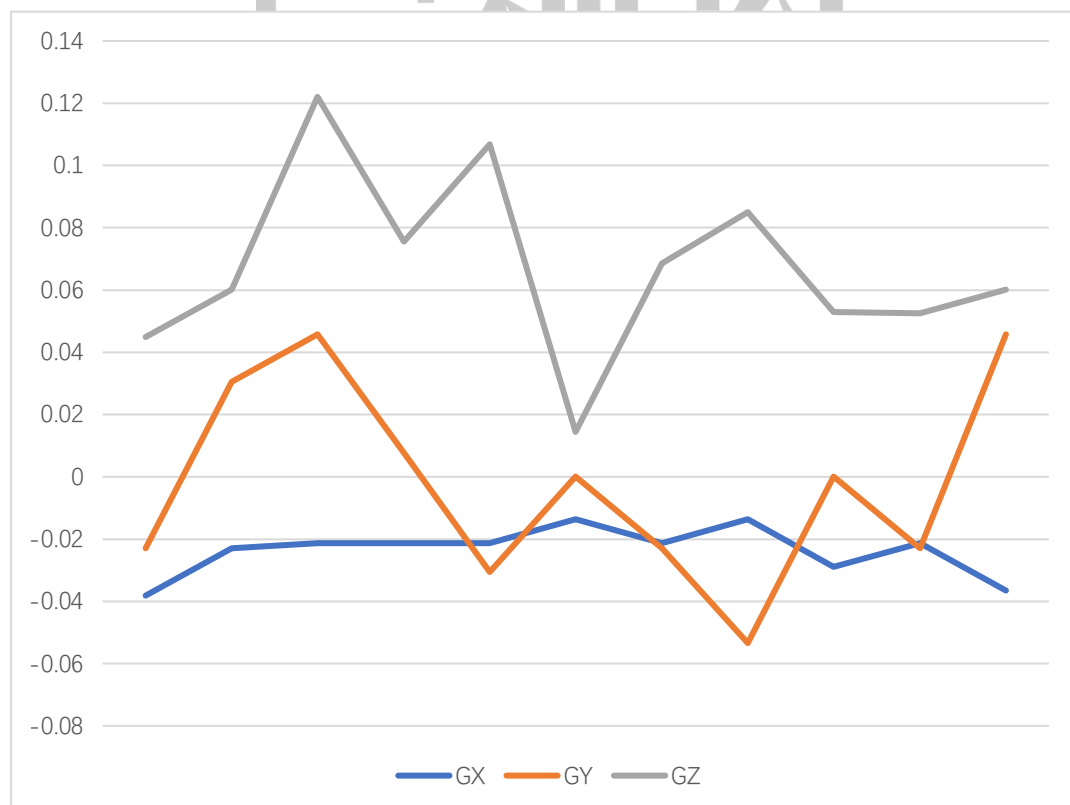
现在，按照同样的方法把陀螺仪的测量数据也转换成相应的格式。读取的部分就不再解释了，和上面读取加速度的方式是一样的。调教的时候，由于把整个

陀螺仪是静止放置在桌子上的，所以三个轴的数值应该为 0，我们要做的就是
把漂移的值减去。调教完成后，我们直接来看陀螺仪的公式：

$$\text{GYRO_X} = N * (\text{GYOR_X} / 32768) (^\circ / \text{s})$$

上式中，X 可以替换为 XYZ 三个轴中任意一轴，在程序配置的时候配置过陀
螺仪的数值。陀螺仪可测量的数值有 $250^\circ / \text{s}$ ， $500^\circ / \text{s}$ ， $1000^\circ / \text{s}$ 和 $2000^\circ / \text{s}$ ，
也就说当选择 $250^\circ / \text{s}$ 时，如果输出是 32768 时，则当前轴的角速度为 $250^\circ / \text{s}$ 。
而我们选择量程为 $2000^\circ / \text{s}$ 时，输出 32768 对应的则是 $2000^\circ / \text{s}$ 。由此也可以
看出，量程越大的数，精度越低。

这里注意一下，陀螺仪不管以什么姿态摆放，静止时三轴读取的数据永远是
接近 0，因为它用来探测的是当前的角速度。按照上面加速度计的方式调节好
offset 值以后，静止放在桌面上得到如下表的数据。图 3-22 是表中三轴所得的
数据。



```

GX=-0.03815 o/s  GY=-0.10681 o/s  GZ=0.06104 o/s
GX=-0.02289 o/s  GY=-0.07629 o/s  GZ=0.12207 o/s
GX=-0.00763 o/s  GY=0.06866 o/s   GZ=0.05341 o/s
GX=-0.02289 o/s  GY=0.01526 o/s   GZ=0.06866 o/s
GX=-0.04578 o/s  GY=-0.04578 o/s  GZ=0.07629 o/s
GX=-0.05341 o/s  GY=0.00763 o/s   GZ=0.09918 o/s
GX=-0.06104 o/s  GY=0.05341 o/s   GZ=0.10681 o/s
GX=-0.01526 o/s  GY=-0.05341 o/s  GZ=0.06104 o/s
GX=-0.06104 o/s  GY=0.00000 o/s   GZ=0.09155 o/s
GX=-0.04578 o/s  GY=0.06104 o/s   GZ=0.07629 o/s

```

图 3-22 传感器实测陀螺仪数据

由于陀螺仪或多或少都会有些误差，所以这些微小的角度我们一般积分计算的时候是不考虑的，一般精确到小数点后一位即可。

上面说过磁力计的配置。现在来读取磁力计的数值，大致方法和加速度计和陀螺仪没差。

```

Mag_X = (rawData[1]<<8 | rawData[0]);
Mag_Y = (rawData[3]<<8 | rawData[2]);
Mag_Z = (rawData[5]<<8 | rawData[4]);

```

先把得到的数据转化为单位为 uT 的数值。

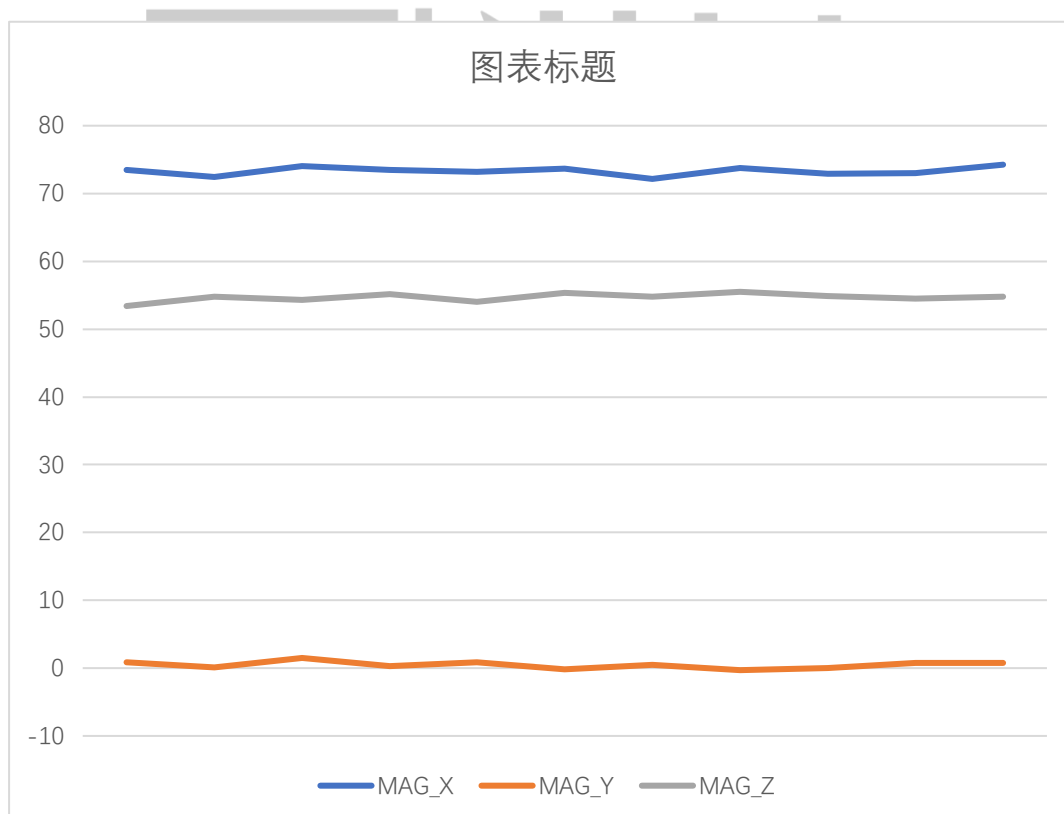
数据测量（每个轴）[15:0]			磁流密度[uT]
2 个寄存器	十六进制	十位	
0111 1111 1111 1000	7FF8	32760	4912(最大)
0000 0000 0000 0001	0001	1	0.15
0000 0000 0000 0000	0000	0	0
1111 1111 1111 1111	FFFF	-1	-0.15

1000 0000 0000 1000	8008	-32760	-4912 (最小)

根据官方提供的数据测量表可以看出，除 0 外，读到的 15 位数（最高位为符号位），每升高 1 个单位，相当于升高 $(4912 - 0.15) / (32760 - 1) = 0.1499\text{uT}$ ，约合 0.15uT 。所以可以直接将所得数据转化为 uT 。

```
Mag_X_uT = Mag_X * 0.15;
Mag_Y_uT = Mag_Y * 0.15;
Mag_Z_uT = Mag_Z * 0.15;
```

此时正面向上静置传感器，可以得到如下表所示的数据。单位是 uT



到这里你可能就会问了，磁场又不是不能感知，怎么做矫正呢？应美盛当然也想到了这一点，所以在磁场的寄存器中还增加了 ASAX, ASAY 和 ASAZ 三个值来调节磁力计数据。因此我们也需要读取这三个值得结果。现在根据官方提供的公式来纠正磁力计的三轴：

$$Hadj = H * \left(\frac{(AsA - 128) * 0.5}{128} + 1 \right)$$

其中 ASA 是三轴矫正读到的数据，H 是磁力计读取得到的数值。Hadj 是当前轴调整过后得到的数据，通过以下代码直接读取 ASA（0x10，0x11，0x12）相应值，并计算出最后结果。（注意 ASA 为 8 位有符号数据）

```
I2CRead(MAG_ADDRESS, 0x10, 3,rawData3); //将 XYZ 的数值放在 rawData 数组中

ASAX = rawData3[0]-128;
ASAY = rawData3[1]-128;
ASAZ = rawData3[2]-128;

Hadjmx=((ASAX/256.00)+1.00)*Mag_X;
Hadjmy=((ASAY/256.00)+1.00)*Mag_Y;
Hadjmz=((ASAZ/256.00)+1.00)*Mag_Z;
```

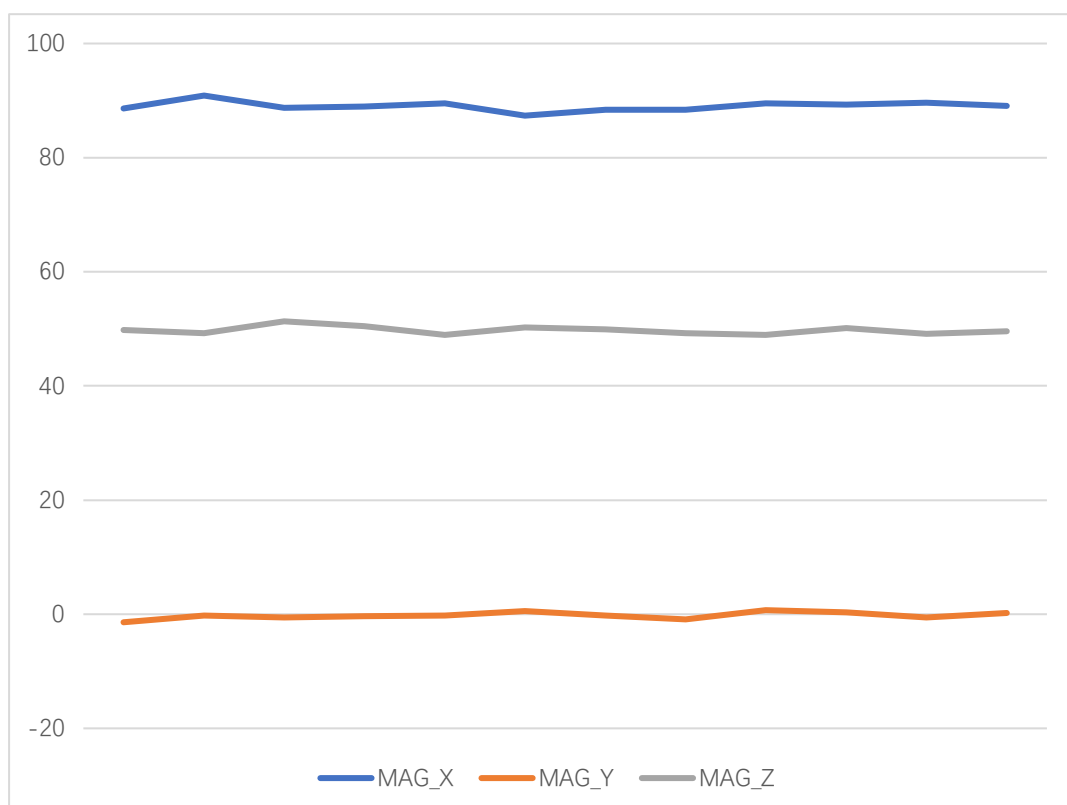
通过读取上面数据的数值，可以发现，不管怎么旋转翻转磁力计，ASAX, ASAY 和 ASAZ 的数据就是不变。笔者这里读取到的数值是 ASAX=172, ASAY=173, ASAZ=163，那么可以把程序化简成：

```
Hadjmx= 1.171875 * Mag_X;
Hadjmy= 1.17578125 * Mag_Y;
Hadjmz= 1.13671875 * Mag_Z;
```

经过矫正计算后，还需要把得到的数据转化一下，转化成单位为 uT 的数值。通过上面的数据表可以看出，磁力计的数据步进为 1：0.15。所以现在把程序改为：

```
Hadjmx= 1.171875 * Mag_X * 0.15;
Hadjmy= 1.17578125 * Mag_Y * 0.15;
Hadjmz= 1.13671875 * Mag_Z * 0.15;
```

现在，将磁力计朝向正北方向，来记录一下得到的数据。如下表所示，单位（uT）。



3.3 磁力计校准和朝向角

虽然说可以通过以上的方式校准了磁力计的数据,但真正磁场的的数据却不一定准。主要是因为,地球磁场的强度实在是太小了,以至于稍微带磁性的东西就会对其造成干扰,比如说变压器,扬声器音响之类的。这就叫硬铁漂移,如图 3-23 所示,可能使得磁场圆心点本身离开造成偏移。

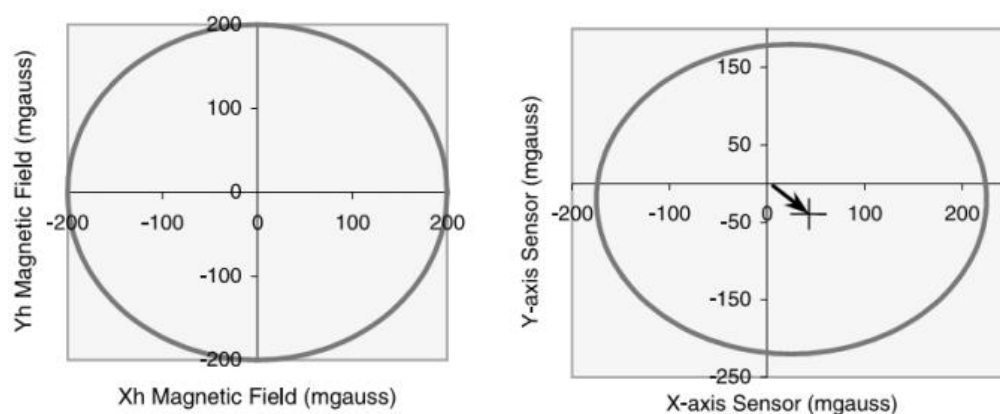


图 3-23 磁力计的偏移现象

而且这些干扰的强度也不一定,有些大,有些小。所以现实生活中的磁场干

扰非常的混乱。这就是之前为什么手机使用指南针功能的时候他都会让你校准一次。

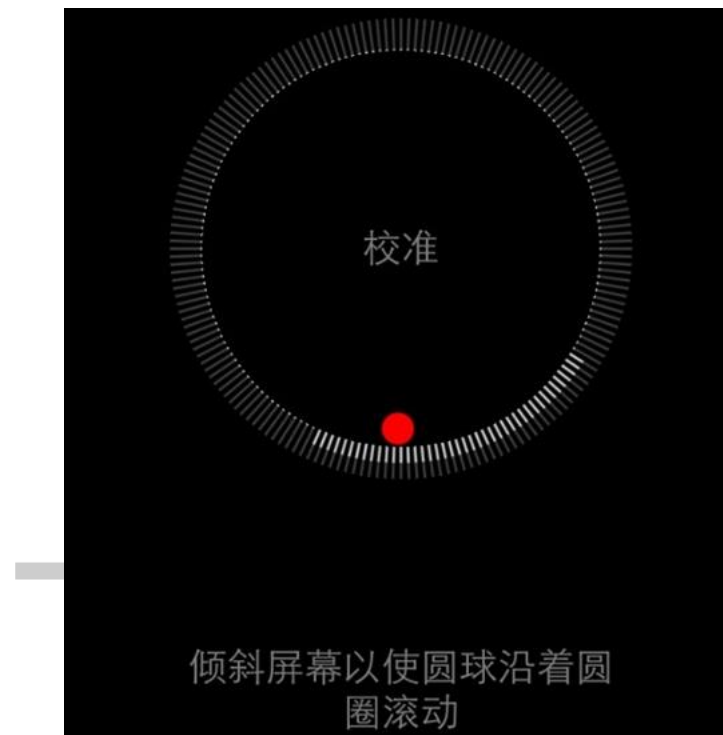


图 3-24 手机的磁力计矫正

我们知道，磁力计一般都是用来检测目前设备朝向的。也就是当作指南针来使用。如图 3-25 所示，假设 F 为当前地球磁场，可以看出计算的朝向角度 H 方向和 z 轴的数据没有任何关系。所以校准方向我们就只需要水平轴的数据就够了。

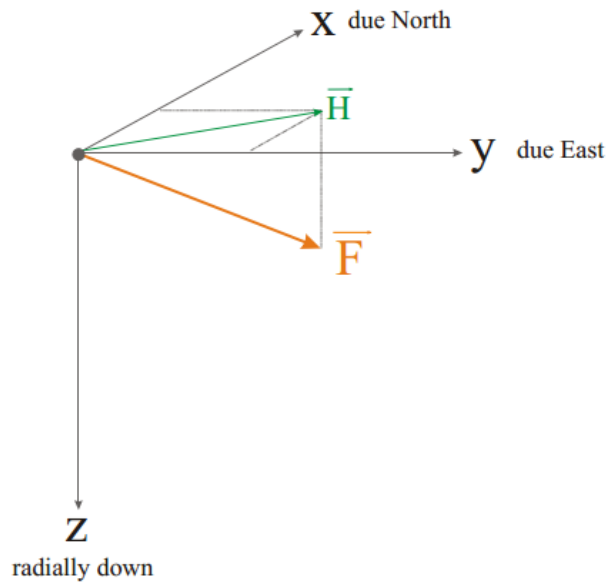


图 3-25 磁力计受地球磁场力分解

XY 的数据由于有干扰，要想去除干扰就需要得知 XY 轴的最大数据和最小数据。现在可以利用 self_test 函数来记录磁力计任意旋转时所测量出来 Xy 轴的最大和最小值。原理如下：

```
if (Mag_X_max < Mag_X_uT) Mag_X_max = Mag_X_uT; // 比较出测量的最大值

if (Mag_Y_max < Mag_Y_uT) Mag_Y_max = Mag_Y_uT;

if (Mag_Z_max < Mag_Z_uT) Mag_Z_max = Mag_Z_uT;

if (Mag_X_min > Mag_X_uT) Mag_X_min = Mag_X_uT; // 比较出测量的最小值

if (Mag_Y_min > Mag_Y_uT) Mag_Y_min = Mag_Y_uT;

if (Mag_Z_min > Mag_Z_uT) Mag_Z_min = Mag_Z_uT;
```

（每个值分别与最大和最小的值比较，如果超越最大或最小，那么覆盖之。）

现在我们开始采集数据：

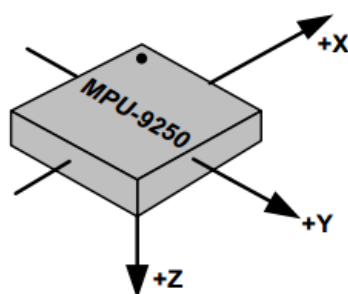


图 3-26 MPU9250 的磁场三轴方向规定

按照如图 3-26 所示中 XYZ 轴的标识，分别绕着 XYZ 轴做 360° 旋转，来采集每个轴上数据的最大值和最小值。或者嫌麻烦你可以直接将传感器在空中绕着圆心随意转动，尽量能覆盖所有的面和所有的点，直到此时你看到 Mag_X_max ， Mag_Y_max 等轴的数据不再增大 Mag_X_min ， Mag_Y_min 不再减小为止。

通过以上的办法，作者测得的数据是：

$$\text{Mag_X_max} = 91.65\mu\text{T} \quad \text{Mag_Y_max} = 51.60\mu\text{T}$$

$$\text{Mag_X_min} = 10.5\mu\text{T} \quad \text{Mag_Y_min} = -27.45\mu\text{T}$$

因为之前说过 z 轴测磁场时用不到，所以我们这里不取 z 轴的数据。到现在为止把校准用到的关键数据测到了。下面我将讲 2 种方法来做芯片磁力计的数据校准。

方法一：

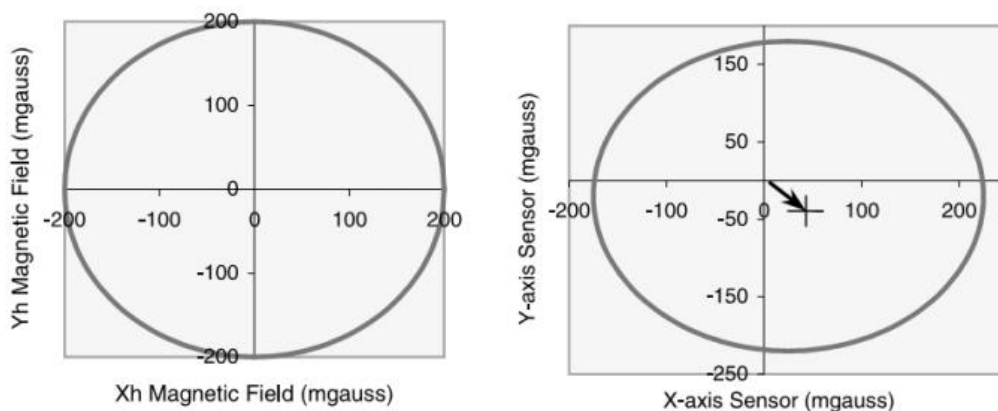


图 3-27 磁场偏移（图片错误，圆球改成）

根据我们之前测得的数据知道偏移量以后可以很容易地找到偏移后原点的漂移位置。因为它的整个值就形成一个圆，你把最大值最小值找了，那么圆心就知道了。只需：

$$(\text{Mag_X_max} + \text{Mag_X_min}) * 0.5 = \text{Mag_X_offset}$$

$$(\text{Mag_Y_max} + \text{Mag_Y_min}) * 0.5 = \text{Mag_Y_offset}$$

所以现在我这块芯片原点漂移到了（计算出圆心）：

$$\text{Mag_X_offset} = (91.65 + 10.5) * 0.5 = 51.075$$

$$\text{Mag_Y_offset} = (51.6 + (-27.45)) * 0.5 = 12.075$$

那么，我们把得到的数据减去偏移量就能得到实际的数据值：

$$\text{Mag_X_uT} = \text{Mag_X_in} - \text{Mag_X_offset}$$

$$\text{Mag_Y_uT} = \text{Mag_Y_in} - \text{Mag_Y_offset}$$

得到的数据就是 X, y 轴得到的较准确数值。（注意这里的数据是笔者这颗传感器的误差值）

$$\text{Mag_X_uT} = \text{Mag_X_in} - 51.075$$

$$\text{Mag_Y_uT} = \text{Mag_Y_in} - 12.075$$

方法二：

第二种方法要用到公式：

$$X_{out} = X_{in} * X_s - X_b$$

$$Y_{out} = Y_{in} * Y_s - Y_b$$

X_{in} 和 Y_{in} 是测得的数据值， X_s, Y_s 是测得数据的比例系数， X_b 和 Y_b 是偏移量。根据之前测得的数据，不难发现 x 轴数据的落差比 y 轴数据的落差要大。

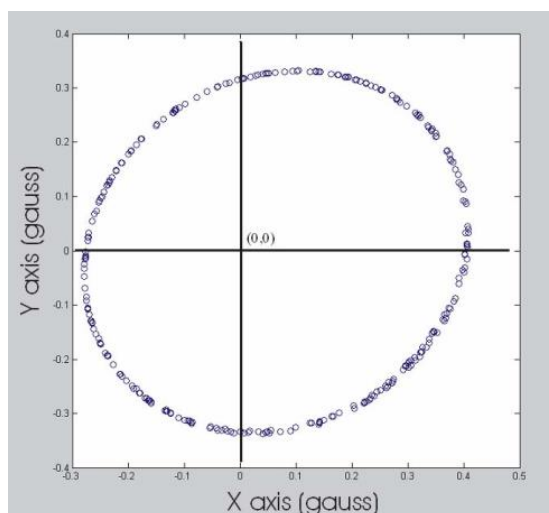


图 3-28 磁场偏移

说明 x 轴的磁力影响比 y 轴要大，因此这里设 $X_s=1$ ，那么

$$Y_s = \frac{(X_{\max} - X_{\min})}{(Y_{\max} - Y_{\min})}.$$

上面测得的数据计算出 $Y_s=1.026565$

$$X_b = X_s \left[\frac{1}{2} (X_{\max} - X_{\min}) - X_{\max} \right]$$

$$Y_b = Y_s \left[\frac{1}{2} (Y_{\max} - Y_{\min}) - Y_{\max} \right].$$

根据以上公式把测得的数据代入，可测得 $X_b=-0.51075$ ， $Y_b=0.12395772375$ ，最后得到公式：

$$X_{out} = X_{in} - 51.075$$

$$Y_{out} = Y_{in} * 1.026565 - 12.395772375$$

这样，就得到了一个比较准确的 X_y 轴磁力计的数据。知道了磁力计数据，就可以计算芯片的朝向了。

3.4 朝向角

C 语言中 $\text{atan2}(y, x)$ 函数可以求得原点和 (x, y) 连线与 x 轴的夹角，也可以理解为复数 $x+yi$ 的辐角。**A12** 返回值的单位为弧度，取值范围为 $(-\pi, \pi]$ ，C 函数允许 x 、 y 同时为零。

这里可以利用这个函数来求得东西南北的角度。将刚才校准计算得到的 X_{out} 和 Y_{out} 的值代入 atan2 函数， $\text{atan2}(Y_{\text{out}}, X_{\text{out}}) = \text{当前弧度}$ 。要得到角度还需要：

$$\text{Heading} = 180 * \text{atan2}(\text{Mag_Y_uT}, \text{Mag_X_uT}) / \pi$$

其中 Mag_Y_uT 和 Mag_X_uT 是计算出来 X 和 y 轴的磁力计数值， π 是圆周率。Heading 计算出来直接是角度值。角度范围如图 3-29 所示。

笔者根据计算出的方位角看一下在正东西南北四个方向时， Xy 轴的区别。总结出为下图：

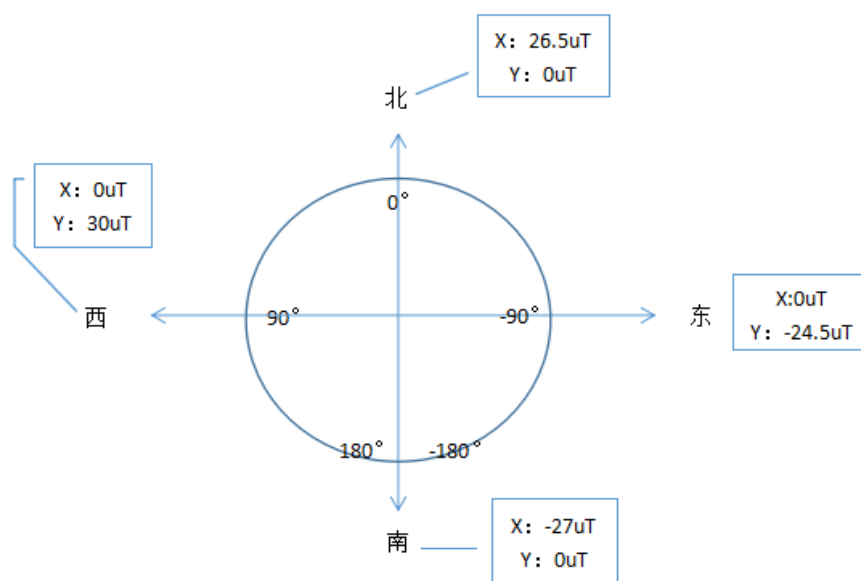


图 3-29 磁力计在各个方向的实测磁力数据

Heading 的数值会在 $+180^\circ$ 到 -180° 之间变化， X 和 Y 的数据在不同的朝向角也不一样。如图 3-30 显示：

```
X=-22.94uT Y=10.27uT heading=155.97o/s
X=-22.80uT Y=10.13uT heading=156.13o/s
X=-22.59uT Y=9.21uT heading=157.90o/s
X=-22.87uT Y=9.77uT heading=156.94o/s
X=-22.24uT Y=9.49uT heading=156.96o/s
X=-23.29uT Y=10.41uT heading=156.00o/s
X=-23.43uT Y=10.69uT heading=155.55o/s
X=-22.94uT Y=9.77uT heading=157.00o/s
X=-24.21uT Y=9.56uT heading=158.52o/s
X=-23.57uT Y=10.13uT heading=156.83o/s
X=-23.22uT Y=10.20uT heading=156.37o/s
X=-22.24uT Y=10.48uT heading=154.85o/s
X=-23.15uT Y=10.20uT heading=156.31o/s
X=-22.59uT Y=10.83uT heading=154.46o/s
X=-23.22uT Y=9.92uT heading=156.96o/s
X=-22.94uT Y=11.04uT heading=154.37o/s
```

图 3-30 程序输出数据

现在，便得到了加速度，陀螺仪和磁力计相对准确的数据。已经做到了本章开始时展示 APP 中显示出的功能。

3.5 自检模式

通过上一节已经得到了 3 组 3 轴数据。那么我们如何判断这 3 组是否有效，传感器是否由于附近的特殊条件或者其他原因导致数据异常不可用呢？在每次上电之间，其实还需要做一次数据检测，来判断得到的数据是否有效。（在应美盛的官方文档中对此功能做出了详细的说明。资料给出了相关的技术文档，感兴趣的可以去看看）

下面，我们根据文档的指示进行芯片数据自检的操作。

3.5.1 加速度计与陀螺仪的自检

首先要使用校准功能要先熟悉几个寄存器，他们分别是：

寄存器号	寄存器功能
0, 1, 2	陀螺仪自检

13, 14, 15	加速度计自检
26, 27, 28, 29	配置寄存器

传感器的具体功能可以查询附录 C 中的内容，这里就不再赘述。

自检的原理：传感器再出厂的时候已通过自测模式校准过，并将校准后的数据存入指定的寄存器中。我们在使用时也要打开自测模式并将其中测量出来的数据通过官方指定的公式与出厂数据进行比较，在一定范围内视作数据正常。

下面来分解自测函数：

```
void MPU9250::SelfTest()//芯片自测
{
    uint8_t Buf[14];
    uint8_t selfTest[3];
    int32_t gAvg[3], aAvg[3], aSTAvg[3], gSTAvg[3];
    float factoryTrim[6];
    uint8_t FS = 0;
```

首先定义数组承载接下来要计算的数据，接下来需要配置加速度计和陀螺仪的采样率，并且配置加速度范围±2g，陀螺仪范围 250dps：

DLPF 配置	LPF	采样率	延迟
2（陀螺）	92Hz	1KHz	3.9ms
2（加计）	92Hz	1KHz	7.8ms

```
I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_SMPLRT_DIV, 0x00); //设置陀螺仪采样率不分频
I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_CONFIG, 0x02); //设置陀螺仪采样 1KHz DLPF 为 32KHz
I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_GYRO_CONFIG, 0x00); //设置陀螺仪采样范围为 250° /s
I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_ACCEL_CONFIG_2, 0x06); //设置加速度采样率为 1KHz,
带宽为 32KHz
```

```
I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_ACCEL_CONFIG, 0x00); //设置加速度采样范围 2G
```

读取 200 次加速度和陀螺仪的值并取平均值

```
for( int ii = 0; ii < 200; ii++) //读 200 次加速度计和陀螺仪数值并取平均

{

I2CRead(MPU9250_ADDRESS, 0x3B, 14, Buf);

aAvg[0] += ((Buf[0] << 8) | Buf[1]);

aAvg[1] += ((Buf[2] << 8) | Buf[3]) ;

aAvg[2] += ((Buf[4] << 8) | Buf[5]) ;


gAvg[0] += ((Buf[8] << 8) | Buf[9]) ;

gAvg[1] += ((Buf[10] << 8) | Buf[11]) ;

gAvg[2] += ((Buf[12] << 8) | Buf[13]) ;

}


for (int ii =0; ii < 3; ii++)

{

aAvg[ii] /= 200;

gAvg[ii] /= 200;

}


/*****普通模式数据*****/

ACC_X=aAvg[0];ACC_Y=aAvg[1];ACC_Z=aAvg[2];

Gyro_X=gAvg[0];Gyro_Y=gAvg[1];Gyro_Z=gAvg[2];

/*****/
```

现在，平均值放进了 aAvg 和 gAvg 数组中。我们来看一下笔者这里得到的数据：

```

aAvgX=-2567 aAvgY=3292 aAvgZ=16123 gAvgX=506 gAvgY=-517 gAvgZ=216
aAvgX=-2559 aAvgY=3289 aAvgZ=16122 gAvgX=508 gAvgY=-518 gAvgZ=210
aAvgX=-2563 aAvgY=3289 aAvgZ=16122 gAvgX=508 gAvgY=-514 gAvgZ=209
aAvgX=-2566 aAvgY=3290 aAvgZ=16120 gAvgX=506 gAvgY=-515 gAvgZ=211
aAvgX=-2561 aAvgY=3287 aAvgZ=16127 gAvgX=508 gAvgY=-516 gAvgZ=209
aAvgX=-2563 aAvgY=3287 aAvgZ=16123 gAvgX=506 gAvgY=-516 gAvgZ=209
aAvgX=-2566 aAvgY=3289 aAvgZ=16128 gAvgX=505 gAvgY=-518 gAvgZ=210
aAvgX=-2561 aAvgY=3293 aAvgZ=16121 gAvgX=507 gAvgY=-518 gAvgZ=208
aAvgX=-2569 aAvgY=3290 aAvgZ=16130 gAvgX=507 gAvgY=-516 gAvgZ=209

```

可以看到，数据还是比较稳定的。现在来配置加速度计和磁力计为自检模式。并再次读取 200 次数据取平均。

```

/*配置并使能加速度计和陀螺仪自检模式，范围 2G 和 250° /s*/

I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_ACCEL_CONFIG, 0xE0);

I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_GYRO_CONFIG, 0xE0);

delay(25);

for( int ii = 0; ii < 200; ii++) //读加速度和陀螺仪自检数值并计算 200 次平均值
{
    I2CRead(MPU9250_ADDRESS, 0x3B, 14, Buf);

    aSTAvg[0] += ((Buf[0] << 8) | Buf[1]) ;
    aSTAvg[1] += ((Buf[2] << 8) | Buf[3]) ;
    aSTAvg[2] += ((Buf[4] << 8) | Buf[5]) ;

    gSTAvg[0] += ((Buf[8] << 8) | Buf[9]) ;
    gSTAvg[1] += ((Buf[10] << 8) | Buf[11]) ;
    gSTAvg[2] += ((Buf[12] << 8) | Buf[13]) ;
}

for (int ii =0; ii < 3; ii++)
{
    aSTAvg[ii] /= 200;
    gSTAvg[ii] /= 200;
}

```

```
// /*******自检模式数据*****//

TACC_X=aSTAvg[0];TACC_Y=aSTAvg[1];TACC_Z=aSTAvg[2];

TGyro_X=gSTAvg[0];TGyro_Y=gSTAvg[1];TGyro_Z=gSTAvg[2];

// /*******//
```

笔者读取到的数据为：

```
aAvgTX=4005 aAvgTY=9600 aAvgTZ=23909 gAvgTX=18649 gAvgTY=20134 gAvgTZ=25184
aAvgTX=3998 aAvgTY=9607 aAvgTZ=23913 gAvgTX=18651 gAvgTY=20133 gAvgTZ=25183
aAvgTX=4005 aAvgTY=9605 aAvgTZ=23914 gAvgTX=18649 gAvgTY=20134 gAvgTZ=25184
aAvgTX=3998 aAvgTY=9601 aAvgTZ=23906 gAvgTX=18648 gAvgTY=20132 gAvgTZ=25183
aAvgTX=4002 aAvgTY=9603 aAvgTZ=23908 gAvgTX=18650 gAvgTY=20134 gAvgTZ=25183
aAvgTX=4002 aAvgTY=9608 aAvgTZ=23914 gAvgTX=18651 gAvgTY=20134 gAvgTZ=25185
aAvgTX=4002 aAvgTY=9605 aAvgTZ=23907 gAvgTX=18650 gAvgTY=20132 gAvgTZ=25183
aAvgTX=3998 aAvgTY=9606 aAvgTZ=23900 gAvgTX=18650 gAvgTY=20134 gAvgTZ=25182
aAvgTX=4003 aAvgTY=9607 aAvgTZ=23907 gAvgTX=18649 gAvgTY=20134 gAvgTZ=25183
aAvgTX=3997 aAvgTY=9610 aAvgTZ=23909 gAvgTX=18649 gAvgTY=20135 gAvgTZ=25182
```

接着关闭自检模式，读取出厂数据：

```
// /*配置加速度计和陀螺仪回到正常模式（初始化）*/

I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_ACCEL_CONFIG, 0x00);

I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_GYRO_CONFIG, 0x00);

delay(25);

// /*******读取传感器自测寄存器数值*****//

I2CRead(MPU9250_ADDRESS, 0x0D, 3, selfTest);

AXT=selfTest[0];

AYT=selfTest[1];

AZT=selfTest[2];

I2CRead(MPU9250_ADDRESS, 0x00, 3, selfTest);

GXT=selfTest[0];

GYT=selfTest[1];

GZT=selfTest[2];

// /*******//
```


工厂数据是不变的，这里笔者读取的是：

```
afactoryX=98 afactoryY=94 afactoryZ=114 gfactoryX=-60 gfactoryY=-50 gfactoryZ=-29
afactoryX=98 afactoryY=94 afactoryZ=114 gfactoryX=-60 gfactoryY=-50 gfactoryZ=-29
afactoryX=98 afactoryY=94 afactoryZ=114 gfactoryX=-60 gfactoryY=-50 gfactoryZ=-29
afactoryX=98 afactoryY=94 afactoryZ=114 gfactoryX=-60 gfactoryY=-50 gfactoryZ=-29
afactoryX=98 afactoryY=94 afactoryZ=114 gfactoryX=-60 gfactoryY=-50 gfactoryZ=-29
afactoryX=98 afactoryY=94 afactoryZ=114 gfactoryX=-60 gfactoryY=-50 gfactoryZ=-29
```

工厂的数据（ST_code）是根据如下公式得来的：

$$ST_code = round\left(\frac{\log(ST_FAC / (2620 / 2^{FS}))}{\log(1.01)}\right) + 1$$

那么现在根据得到得工厂数据进一步的计算（其中 FS 是 26 号寄存器中配置决定，此处是 0）：

$$ST_OTP = (2620 / 2^{FS}) * 1.01^{(ST_code-1)} (lsb)$$

所以我们可以得到：

//计算工厂偏移值，注意，这里计算出来的是出场数据的偏移值，也就是标准偏移量。

```
factoryTrim[0] = (float) (2620/1<<FS)*(pow( 1.01 , ((float)AXT - 1.0) ));
factoryTrim[1] = (float) (2620/1<<FS)*(pow( 1.01 , ((float)AYT - 1.0) ));
factoryTrim[2] = (float) (2620/1<<FS)*(pow( 1.01 , ((float)AZT - 1.0) ));
factoryTrim[3] = (float) (2620/1<<FS)*(pow( 1.01 , ((float)GXT - 1.0) ));
factoryTrim[4] = (float) (2620/1<<FS)*(pow( 1.01 , ((float)GYT - 1.0) ));
factoryTrim[5] = (float) (2620/1<<FS)*(pow( 1.01 , ((float)GZT - 1.0) ));
```

接下来，根据 factoryTrim[x] 得到的值的不同，举要进行不同的比对：

当 factoryTrim[x] 不为 0 时：

轴	通过指标
陀螺仪 X	(自测值 / 出厂值) > 0.5
陀螺仪 Y	(自测值 / 出厂值) > 0.5

陀螺仪 Z	$(\text{自测值} / \text{出厂值}) > 0.5$
加速度 X	$0.5 < (\text{自测值} / \text{出厂值}) < 1.5$
加速度 Y	$0.5 < (\text{自测值} / \text{出厂值}) < 1.5$
加速度 Z	$0.5 < (\text{自测值} / \text{出厂值}) < 1.5$

//在得到标准的偏移量后, 可以通过 $((\text{自测模式读取值} - \text{正常模式读取值}) / \text{工厂标准偏移值}) * 100\%$

```
for (int i = 0; i < 3; i++)
{
    destination[i]=100.0*((float)(aSTAvg[i] - aAvg[i]))/factoryTrim[i];

    destination[i+3]=100.0*((float)(gSTAvg[i] - gAvg[i]))/factoryTrim[i+3];
}
```

此处笔者得到的数据为:

```
factoryTrim[0]=95.52 factoryTrim[1]=95.49 factoryTrim[2]=96.43 factoryTrim[3]=1270.40 factoryTrim[4]=1309.91 factoryTrim[5]=1284.47
factoryTrim[0]=95.48 factoryTrim[1]=95.51 factoryTrim[2]=96.44 factoryTrim[3]=1270.68 factoryTrim[4]=1310.03 factoryTrim[5]=1284.47
factoryTrim[0]=95.52 factoryTrim[1]=95.49 factoryTrim[2]=96.33 factoryTrim[3]=1270.75 factoryTrim[4]=1309.59 factoryTrim[5]=1284.57
factoryTrim[0]=95.48 factoryTrim[1]=95.51 factoryTrim[2]=96.39 factoryTrim[3]=1270.61 factoryTrim[4]=1309.91 factoryTrim[5]=1284.42
factoryTrim[0]=95.49 factoryTrim[1]=95.51 factoryTrim[2]=96.45 factoryTrim[3]=1270.68 factoryTrim[4]=1309.78 factoryTrim[5]=1284.37
factoryTrim[0]=95.49 factoryTrim[1]=95.51 factoryTrim[2]=96.50 factoryTrim[3]=1270.82 factoryTrim[4]=1309.91 factoryTrim[5]=1284.31
factoryTrim[0]=95.46 factoryTrim[1]=95.54 factoryTrim[2]=96.48 factoryTrim[3]=1270.61 factoryTrim[4]=1309.78 factoryTrim[5]=1284.67
factoryTrim[0]=95.61 factoryTrim[1]=95.51 factoryTrim[2]=96.46 factoryTrim[3]=1270.47 factoryTrim[4]=1309.84 factoryTrim[5]=1284.31
factoryTrim[0]=95.55 factoryTrim[1]=95.56 factoryTrim[2]=96.34 factoryTrim[3]=1270.68 factoryTrim[4]=1309.97 factoryTrim[5]=1284.21
factoryTrim[0]=95.49 factoryTrim[1]=95.57 factoryTrim[2]=96.44 factoryTrim[3]=1270.40 factoryTrim[4]=1309.65 factoryTrim[5]=1284.31
factoryTrim[0]=95.45 factoryTrim[1]=95.46 factoryTrim[2]=96.36 factoryTrim[3]=1270.47 factoryTrim[4]=1309.91 factoryTrim[5]=1284.57
factoryTrim[0]=95.53 factoryTrim[1]=95.49 factoryTrim[2]=96.43 factoryTrim[3]=1270.47 factoryTrim[4]=1309.84 factoryTrim[5]=1284.52
factoryTrim[0]=95.50 factoryTrim[1]=95.57 factoryTrim[2]=96.31 factoryTrim[3]=1270.61 factoryTrim[4]=1309.91 factoryTrim[5]=1284.31
factoryTrim[0]=95.62 factoryTrim[1]=95.57 factoryTrim[2]=96.39 factoryTrim[3]=1270.61 factoryTrim[4]=1310.03 factoryTrim[5]=1284.47
factoryTrim[0]=95.52 factoryTrim[1]=95.51 factoryTrim[2]=96.36 factoryTrim[3]=1270.40 factoryTrim[4]=1309.84 factoryTrim[5]=1284.52
```

全部通过, 说明数据有效。

当 $\text{factoryTrim}[x]=0$ 时:

轴	通过条件
陀螺仪 X	$ \text{自测差值} \geq 60\text{dps}$
陀螺仪 Y	$ \text{自测差值} \geq 60\text{dps}$

陀螺仪 Z	$ \text{自测差值} \geq 60\text{dps}$
加速度 X	$225\text{mg} \geq \text{自测差值} \leq 675\text{dmg}$
加速度 Y	$225\text{mg} \geq \text{自测差值} \leq 675\text{dmg}$
加速度 Z	$225\text{mg} \geq \text{自测差值} \leq 675\text{dmg}$

当上两项都通过了还需要核对一下陀螺仪的偏移量：

轴	通过条件
陀螺仪 X	$ \text{偏移量} \leq 20\text{dps}$
陀螺仪 Y	$ \text{偏移量} \leq 20\text{dps}$
陀螺仪 Z	$ \text{偏移量} \leq 20\text{dps}$

那么我们只需要读取 19-24 号寄存器的数据即可：

```
void MPU9250::Offsetget()//偏移量检测
{
    uint8_t Buf[6];

    int16_t gAvg[3];

    I2CRead(MPU9250_ADDRESS, 0x13, 6, Buf);

    gAvg[0]= Buf[1]<<8 | Buf[0];
    gAvg[0]= Buf[2]<<8 | Buf[3];
    gAvg[0]= Buf[4]<<8 | Buf[5];

    GXOFF=gAvg[0];
    GYOFF=gAvg[1];
    GZOFF=gAvg[2];
}
```

```
}
```

这里笔者得到的是 0，也是全部通过：

```
GXOFF=0 GYOFF=0 GZOFF=0  
GXOFF=0 GYOFF=0 GZOFF=0  
GXOFF=0 GYOFF=0 GZOFF=0  
GXOFF=0 GYOFF=0 GZOFF=0  
GXOFF=0 GYOFF=0 GZOFF=0
```

由此，可以测得此传感器数据输出正常，本次测量的数据均有效。

有人可能要问了，除了自测模式以外，寄存器中的 OFFSET 偏移量又是干啥的呢？附录 A 中 **A13** 将会讲到。

下视界

第4章 姿态

到现在为止，我们已经可以得到传感器数据并能将其转换为加速度，陀螺仪和磁力计的单位数据了。但是为了得到传感器的姿态数据，还需要将测得的数据再转换成角度值才可以，但是由于直接使用姿态的角度数据在程序中计算比较复杂，所以要转换成其他的形式数学模型才便于计算。所以，本章将给大家讲解三种姿态表示方法，分别为欧拉角、方向余弦和四元数，将上一章得到的 XYZ 值转化成这样的角度表达式。

现在我们设定一个正方体：

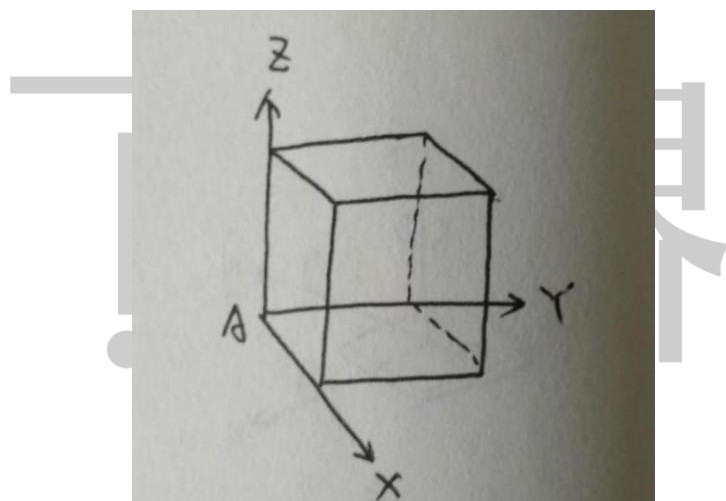


图 5-1 正方体建立坐标系

按照如图 5-1 所示的方式给这个正方体建立一个三角坐标系，就可以在数学上形容它当前的姿态。如果不用数学的方式描绘这个正方体的状态，而用纯语言文字描述又应该怎么说呢——正方体正面向上直立在水平面上。

现在，稍微绕着 A 点旋转一下正方体的位置，正方体的姿态变成如下图 5-2。那么又应该怎么形容立方体现在的姿态呢？

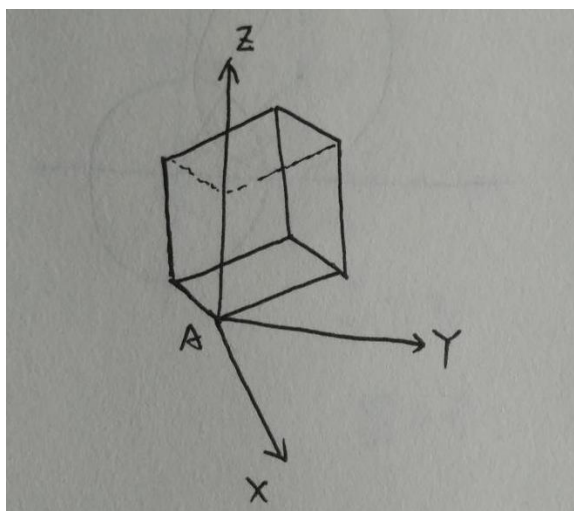


图 5-2 旋转后的立方体

仔细想想？是不是觉得不好形容。为了更好理解，现在把立方体换成一架飞机并加上角度。如图 5-3 所示，现在比较直观了。

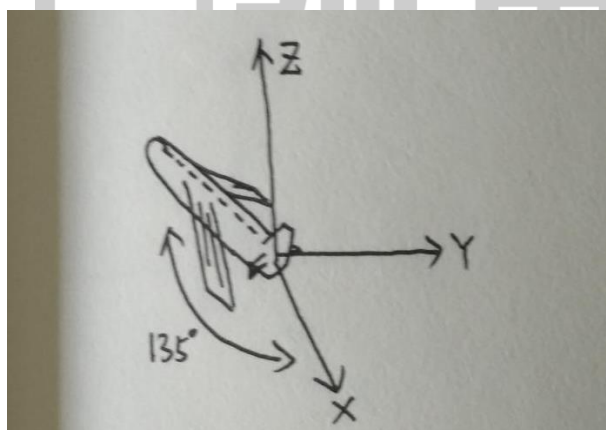


图 5-3 旋转 135° 的飞机

我们可以说飞机向着自己朝向的东北方向旋转了 135° 。但是光说旋转了 135° ，谁知道是以哪个角度旋转的 135° 呢。所以要将肉眼看到的東西更好的表达出来是要用到数学模型的。那么怎么把这个旋转的方式和旋转的方向角度用数学的方式表达出来呢？图中建立的三维直角坐标系就是帮我们定位角度用的。如图 5-4 所示：

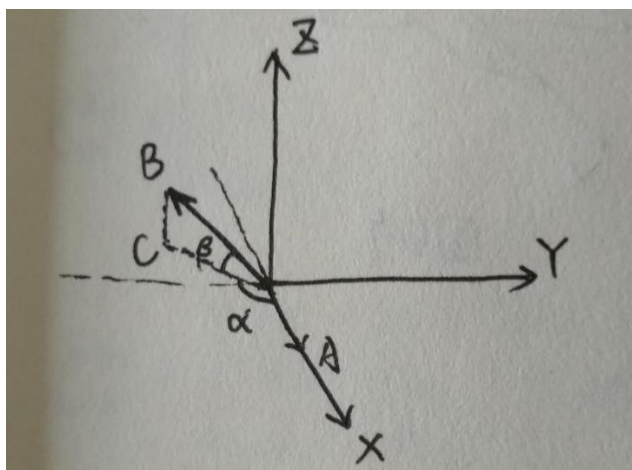


图 5-4 角度分析

把飞机所处的平面做成三位空间模型，飞机替换为单位向量，这样，通过向量绕每个轴旋转方向的不同角度来描述飞机的最终朝向。就可以描述出飞机的旋转方式。图中我们可以看到飞机是从 A 点旋转到 B 点的。之前我们说 A 和 B 的夹角是 135° ，那么现在就可以通过旋转的方式分解这个角度。A 点绕 z 轴顺时针旋转 α 角度到 C 点，然后 C 点旋转 β 角度到 B 点。**注意，这里 C 到 B 的旋转其实是 C 分别绕 x 轴和 y 轴旋转角度的合角度。**虽然图中没有画出来，但是可以想象一下，C 绕着 y 轴旋转一定角度，然后此时再绕着 x 轴旋转一定角度，就可以得到 B 这个位置的点。这样，通过描述向量与每个轴之间的旋转角度的关系就是数学中表达空间向量运动的一种方法，而这三个与轴的角度 α ， β ， γ 就是我们下面要讲到的欧拉角。而这个欧拉角不就是我们用陀螺仪测得的数据吗？

5.1 欧拉角

根据上面的例子我们知道，向量的旋转是可以按照一定顺序来描述的。比如上面的例子就是按照 Z-Y-X 的顺序。那么是不是还可以按照 X-Z-Y 的方式来旋转？答案是肯定的，这样旋转虽然过程改变了，但是结果却还是一样的。那我们马上就会总结出以下这几种旋转方法：X-Y-Z, X-Z-Y, Y-Z-X, Y-X-Z, Z-X-Y, Z-Y-X 这六种方式吧。事实上它还有 Z-X-Z, X-Y-X, Y-Z-Y, Z-Y-Z, X-Z-X 以及 Y-X-Y 的这些方式。你可能会蒙住，怎么会只旋转两个轴就可以旋转到坐标内的点？现在，我们就以 Z-X-Z 为例子，来看一下坐标系是如何在只用 2 个轴就能进行旋转的。

如下图 5-5 所示，我们要将蓝色坐标系 XYZ 旋转到红色坐标系 XYZ，先绕 z 旋转 α 角度，此时之前的 x 变为现在的 N，那么我们现在开始绕着新的 x (N) 旋转 β 角，此时蓝色的 z 转动到了红色的 Z。然后此时坐标轴再绕着 Z 旋转 γ 角，绿色 N 的位置 x 就转动到了红色 X 的位置。即可得到图中红色 XYZ 位置的图形。

看完以后你可能会觉得不对啊，刚才举的例子是向量旋转，而这里又变成了坐标系旋转。并不能说明向量旋转到了我们想要的位置。但其实这就是欧拉角旋转的两种不同的思路。一种是向量绕轴旋转，另一种是向量固定不动，坐标轴平面自行旋转。这两种方式都可以实现我们想要的效果。

维基百科上对欧拉角的定义是这么说的：

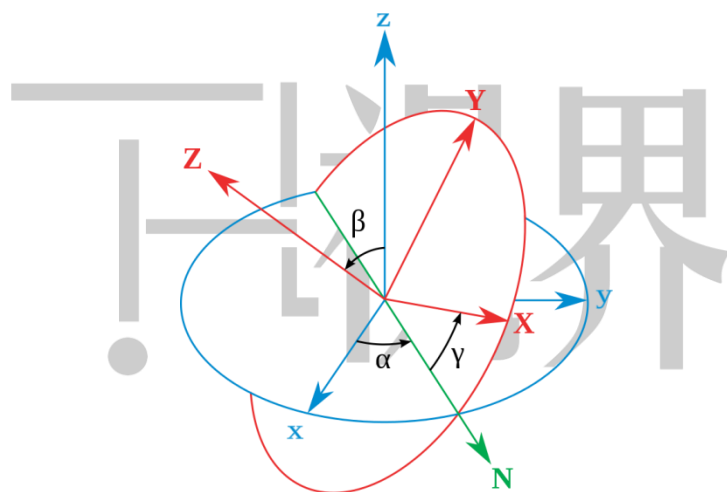


图 5-5 蓝色 XYZ 坐标系旋转到红色 XYZ 坐标系

The axes of the original frame are denoted as x, y, z and the axes of the rotated frame are denoted as X, Y, Z . The geometrical definition (sometimes referred to as static) begins defining the line of nodes as the intersection of the planes xy and the XY (it can also be defined as the common perpendicular to the axes z and Z and then written as the vector product $N = Z \times z$). Using it, the three Euler angles can be defined as follows:

α is the angle between the x axis and the N axis (x-convention. It could also be defined between y and N , called y-convention).

β is the angle between the z axis and the Z axis.

γ is the angle between the N axis and the X axis
(x-convention).

Euler angles between two reference frames are defined only if both frames have the same handedness.

注：图中三个欧拉角 α ， β ， γ ，蓝色的轴是 xyz -轴，红色的轴是 XYZ -坐标轴。绿色的线是交点线 (N)。现在我们设定 xyz -轴为参考系的参考轴。那么称 xy -平面与 XY -平面的相交为交点线，我们用英文字母 (N) 表示。 zxz 顺规的欧拉角可以静态地这样定义：

α 是 x -轴与交点线的夹角，

β 是 z -轴与 Z -轴的夹角，

γ 是交点线与 X -轴的夹角。

也就是说，对于在三维空间里的参考系，都可以用欧拉角来表示。这里定义说参考系就是实验室参考系，是静止不动的。而坐标系则固定于刚体，坐标系是随着刚体的旋转而旋转的。也就是说有两个参考坐标系，一个是空间内的，一个是物体自己的。

欧拉角有多重定义，分别是静态定义和动态定义。动态定义就是 $Z-X-Z$ 这种，绕着固定于刚体的坐标轴的三个旋转的复合；静态定义则是绕着实验室参考轴的旋转复合，就是 $Z-Y-X$ 这种。

那么既然是 $Z-Y-X$ 的话，其中的旋转是有子父级关系的，按照 $Z-Y-X$ 的顺序排序的话， Z 是最大的父级，相当于环套环，大家还记得第 1 章时候的那个图吗？

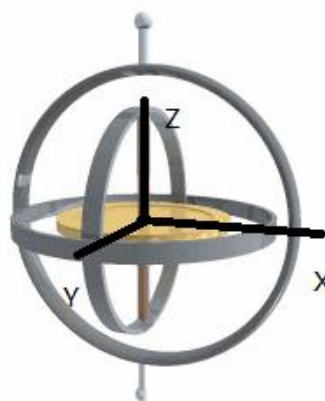


图 5-6 陀螺仪

这个图可以是 X-Y-Z 顺序，大家想象一下。当刚体绕着 x 轴旋转任意角度的时候，Y 和 Z 也要跟着旋转。但刚体绕着 y 轴旋转的时候 x 轴就不会旋转了，而 z 轴就需要跟着 y 轴进行旋转。当绕着 z 轴旋转的时候，X 和 Y 都不需要旋转。所以说，**欧拉角的旋转是需要规定旋转顺序的**。要不然别人不知道你顺序的情况下，得到的数据就是错误的。在航天航空领域，飞机旋转的位置都是有统一规定的。

说到飞行器和飞机，这两年玩的人是越来越多，而且已经从土豪的玩具变成了平价玩具。我相信很多玩家或多或少的会接触 Yaw, Pitch 和 Roll 这样的值。而且我也相信很多人都会把欧拉角和 Yaw, Pitch 和 Roll 弄混。以为 Yaw, Pitch 和 Roll 就是对应着欧拉角中的 α ， β 和 γ 。下面就给大家讲一下其中的区别，以免大家混淆。

Yaw、Pitch 和 Roll

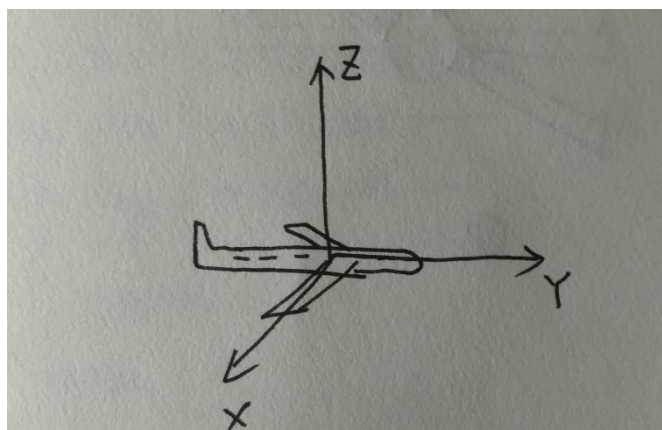


图 5-7 静止的飞机建立坐标系

如图 5-7 所示，假设一个飞机场地上停着一架飞机，虽然没画机场。我们按照图中所示的方式，给飞机建立他自己的直角坐标系。

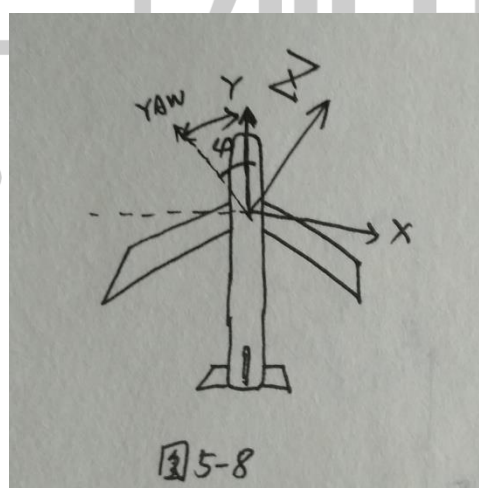


图 5-8 飞机俯视图 YAW 角

图 5-8 是飞机的俯视图，现在让这架飞机绕着坐标系的 z 轴旋转 ϕ 角，这个 ϕ 角就是我们刚才所提到的 Yaw，俗称朝向角。对应的是飞机航行的朝向。这个 Yaw 决定了飞机飞行方向的东南西北。

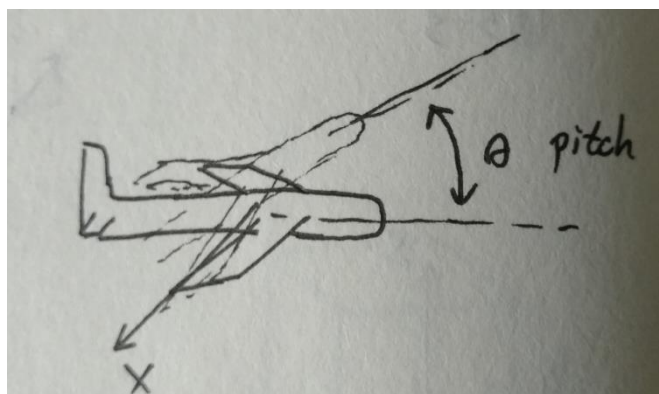


图 5-9 飞机侧视图 PITCH 角

当绕着飞机的 x 轴旋转，通过侧视图图 5-9 可以看到，相当于在控制飞机朝上飞还是向下降落。而这个 θ 角就是 Pitch，俗称俯仰角。

当然，剩下的这个绕 y 轴旋转所得到的 γ 角，就是 Roll 角，俗称横滚角，可以控制飞机向左飞或者向右飞。如图 5-10 所示。注意，下图为飞机正视图，是从机尾处平视得到图形。可以参考轴坐标箭头方向判断。

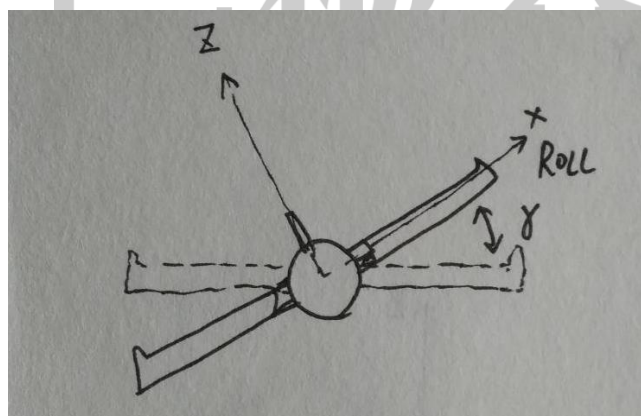


图 5-10 飞机正视图

所以不难看出 Yaw、Pitch 和 Roll 这三个角度的数据是物体当前朝向分别与初始状态（参考坐标系）的三轴所形成的三个平面（ X - Y , X - Z , Y - Z ）之间的夹角，那是不是欧拉角呢？答案是正确的，这其实就是欧拉角旋转中的一种方案：Z-Y-X 旋转，也就是 Yaw-Pitch-Roll。为什么四轴这种航天的要用这个来计算呢？这就是为了方便统一，航天方面采用的方案 ZYX；而在航空领域用的是 Z-Y-Z 的旋转模式。

接下来讲解一下怎么用利用数学表达式来表达向量或者姿态角现在旋转姿

态，数学上用的方法是方向余弦。

5.2 方向余弦

顾名思义，方向余弦就是向量在方向上分别与每个轴夹角的余弦值。那么，上一章中飞机的例子就有三个余弦角度，如图 5-11 所示。

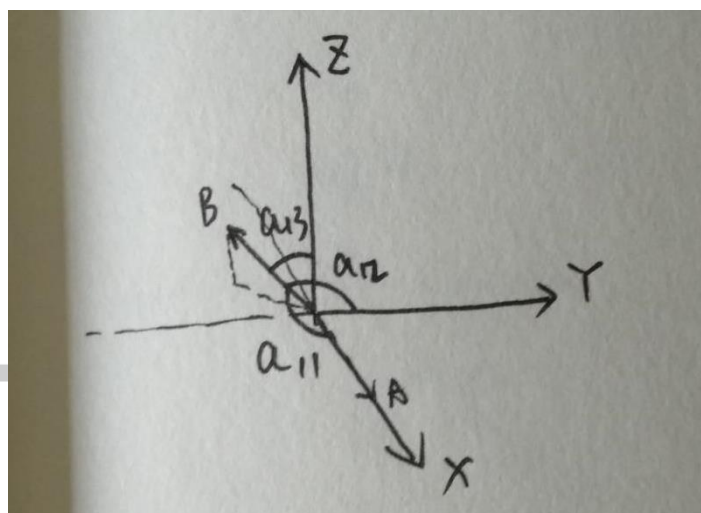


图 5-11 方向余弦

其中 a_{11} 、 a_{12} 和 a_{13} 是向量 B 分别与 xyz 的夹角。而方向余弦就是指这个角度的余弦值。

同理，我们找三个相互垂直的向量，图中 B 向量变成转动后的 x 轴，再加上转动后的 y 和 z 轴，放在坐标系中，就相当于坐标系转动过的位置，这就是动态欧拉角。刚体自身的参考系 xyz 与刚体旋转过后的最终形态 $x_1y_1z_1$ 的夹角就应该有 $3 \times 3 = 9$ 个余弦值（ x_1 分别到 xyz 夹角， y_1 分别到 xyz 夹角， z_1 分别到 xyz 的夹角）。

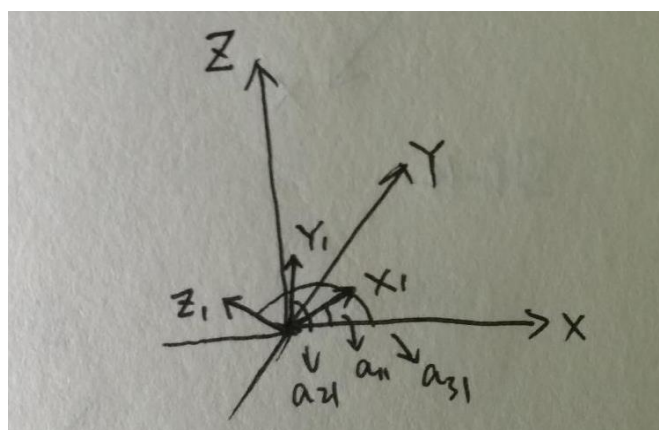


图 5-12 两个坐标系， z_1 分别与 xyz 的夹角

上图 5-12 显示， z_1 、 y_1 、 x_1 分别与 x 轴的夹角为 a_{31} 、 a_{21} 和 a_{11} 。现在，我们可以用矩阵的方式来表示 $x_1y_1z_1$ 在 x 的投影（余弦值）：

$$C_x^{x_1} = \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \end{bmatrix}$$

所以把在 yz 上的投影表示出来，可得：

$$C_x^{x_1} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

其中 x 和 x_1 的意思是坐标系从 x 变到 x_1 坐标系的变换矩阵。所以他们之间存在以下的关系：（为了方便计算角度，我们都设置最初坐标系为单位向量）

$$C_x^{x_1} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{array}{l} \rightarrow \text{在 } x_1 \text{ 上的投影余弦} \\ \rightarrow \text{在 } y_1 \text{ 上的} \text{——} \\ \rightarrow \text{在 } z_1 \text{ 上的} \text{——} \end{array}$$

$\uparrow$ $\uparrow$ $\uparrow$
 x 轴上单位 y 轴上单位 z 轴上单位

关于矩阵函数公式的推导原理，这里就不详细的叙述了，如果有兴趣的同学可以去维基百科或《惯性导航》中查阅相关资料。

由于以上例子中欧拉角旋转是绕轴旋转的，所以还需要得到原始坐标分别按照 x - y - z 三个轴旋转的公式。所以我们需要知道先绕一个轴旋转的公式，再乘以

另外两个即可。下图是推导中绕 z 轴旋转的过程的坐标变换关系图。

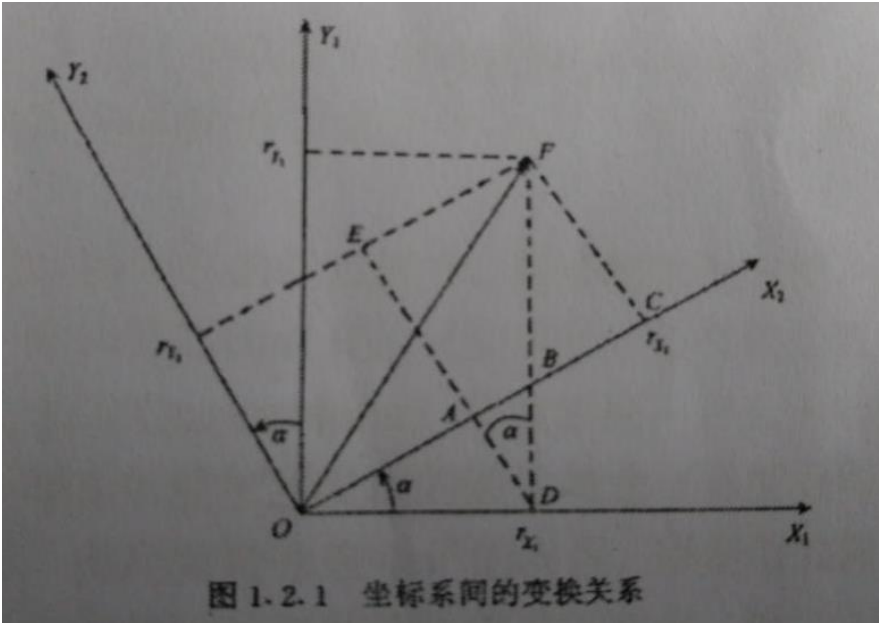


图 5-13 角度转换坐标关系图

根据图 5-13，可以得到如下关系式：

$$C_1^2 = \begin{bmatrix} \cos \alpha & \sin \alpha & 0 \\ -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

→ X_2 轴上的投影

→ Y_2 轴上的投影

→ Z_2 轴上的投影

→ Z_1 轴上的单位 1

→ Y_1 轴上的单位 1

→ X_1 轴上的单位 1

→ X_2 轴上的投影

→ Y_2 轴上的投影

→ Z_2 轴上的投影

$$C_{x_1}^x = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$\rightarrow$ 在 x_1 上的投影的余弦
 $\rightarrow$ 在 y_1 上的投影的余弦
 $\rightarrow$ 在 z_1 上的投影的余弦

$\uparrow$ $\uparrow$ $\uparrow$
 x 轴上的单位 y 轴上的单位 z 轴上的单位

接合上面的这两个公式。这是绕 z 轴旋转的关系式，可以看到 x_1 在 x_2 上投影的余弦值就是 $\cos\alpha$ （注意：由于 x_1 为单位 1，所以本来应该是 $\cos\alpha*1$ ，此处简写成 $\cos\alpha$ ，后面也是一样。所以这里的 $\cos\alpha$ 并不是个夹角值，而是 x_1 的单位向量在 x_2 上投影的长度值），所以此处的 a_{12} 、 a_{22} 和 a_{21} 的数值还是比较好理解的。然后再来看一下 a_{31} 。 a_{31} 是指 x_1 上的单位 1 长度的向量在 z 轴上的投影，由于是绕着 z 轴转动的，所以 x 和 y 一直处于垂直关系，长度为 0，因此，无论绕着 z 轴转动多少度， a_{31} 处值一直为 0。所以 z 上的单位长度在 x_2 上的投影长度也为 0。又因为 xyz 三轴是相互垂直的，所以 y 轴上的长度也应该为 0。（ a_{23} 和 a_{32} 也为 0）明白了这些， a_{33} 为什么是 1，大家也应该就明白了吧。（因为绕 z 轴转动， z_2 和 z_1 是完全重合的，长度不变，所以还是为 1）。至此，就把绕着 z 轴旋转的矩阵关系推导了出来。注意，推导出来的矩阵里面只有一个角的角度值，那么根据欧拉角的旋转性质，我们要描述一个刚体的姿态运动，还需要知道它绕每一个轴的旋转角度。然后，我们就可以得到坐标系分别绕 zyx 轴旋转的矩阵函数：

$$C_a^1 = \begin{bmatrix} \cos\Psi & -\sin\Psi & 0 \\ \sin\Psi & \cos\Psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad C_1^2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & \sin\theta \\ 0 & -\sin\theta & \cos\theta \end{bmatrix} \quad C_2^3 = \begin{bmatrix} \cos\gamma & 0 & -\sin\gamma \\ 0 & 1 & 0 \\ \sin\gamma & 0 & \cos\gamma \end{bmatrix}$$

那么如果要表达一个完整的旋转姿态，就只需要把这三个角度矩阵相乘就可以了。注意，矩阵乘法在相乘的顺序上其实是有规定的。怎么旋转就怎么相乘。 $A \times B \times C$ 和 $A \times C \times B$ 是不一样的意义和结果。

我们直接将刚才得到的数据相乘，可以得到以下关系式，这里得到数据是绕 $z-y-x$ 的顺序旋转的：

$$C_n^b = C_2^b C_1^n C_n^1 = \begin{bmatrix} \cos\gamma & 0 & -\sin\gamma \\ 0 & 1 & 0 \\ \sin\gamma & 0 & \cos\gamma \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & \sin\theta \\ 0 & -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} \cos\Psi & -\sin\Psi & 0 \\ \sin\Psi & \cos\Psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos\gamma\cos\Psi + \sin\gamma\sin\Psi\sin\theta & -\cos\gamma\sin\Psi + \sin\gamma\cos\Psi\sin\theta & -\sin\gamma\cos\theta \\ \sin\Psi\cos\theta & \cos\Psi\cos\theta & \sin\theta \\ \sin\gamma\cos\Psi - \cos\gamma\sin\Psi\sin\theta & -\sin\gamma\sin\Psi - \cos\gamma\cos\Psi\sin\theta & \cos\gamma\cos\theta \end{bmatrix}$$

注意，其中 α 、 β 和 γ 就是欧拉角的三个角度！

但是，光规定好旋转的顺序，欧拉角在角度计算上还是会出现问题。这就是欧拉角的一大致命缺点 Gimbal Lock（万向节死锁）。

5.3 万向节死锁

万向节死锁是指当把向量按照某一特殊情况轴旋转到一个特殊角度的时候，向量此时会丢失一个自由度，使得无法朝某一个方向转动，应为他无论怎么旋转都没有作用，就相当于有两个轴重合。

下面我们用一个例子来讲解一下这种情况：

假设在水平地面上部署了反导弹系统，他是专门用来拦截人的导弹的。为了更好的拦截，需要炮口（向量方向）一直对准可疑飞行物，以便不时之需。以它为基准建立起一个三维空间的坐标系。如图 5-14 所示，假设炮口的朝向一开始是平行于 y 轴的，方向朝着正东方。

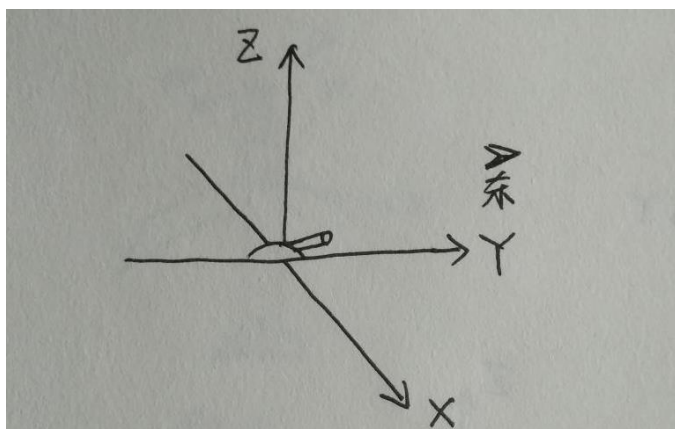


图 5-14 炮基建立坐标系， y 对应正东方

此时，从正东方的方向飞过来一架飞机，这架飞机现在是由东向南沿着 y 轴

笔直飞行。此时炮口会随着飞机飞行位置的变化而不断改变自己的姿势。所以当飞机飞到距离导弹越来越远的时候，炮口与 y 轴的夹角会越变越大，直到 90° （飞机飞到导弹正上方）。如图 5-15 所示。

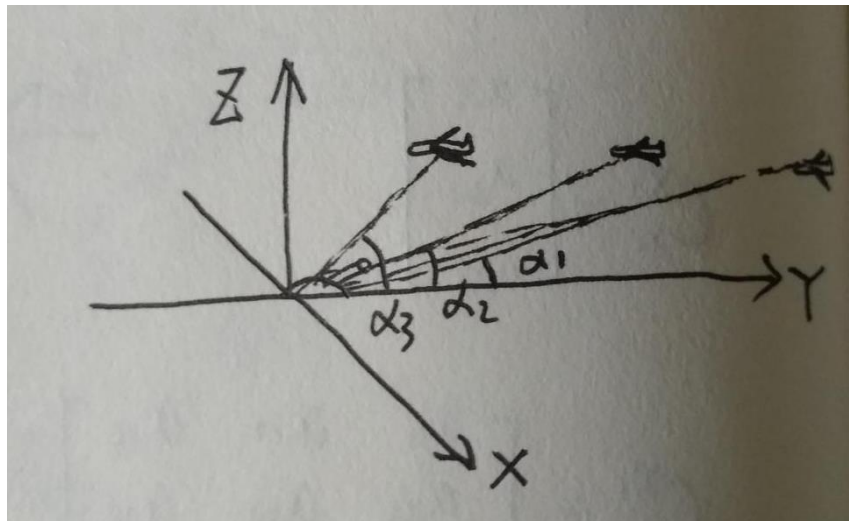


图 5-15 正东方飞来一架飞机

假设这里我们是按照 x - y - z 顺序旋转的（再次说明，从一个朝向移动到另一个朝向必须在 x - y - z 内一套完整的旋转完成）。当导弹在夹角不断增大的时候是绕着 x 轴旋转的。也就是说当飞机飞到导弹的正上方的时候相当于导弹绕着 x 轴旋转了 90° ，可以得到如图 5-16 所示的新坐标 $x_1y_1z_1$ 。

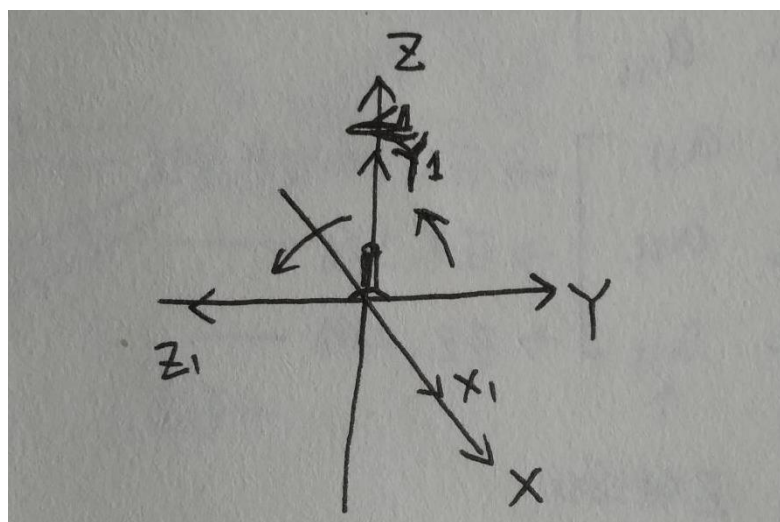


图 5-16 炮基坐标系的变化

此时飞机突然改道向南方飞去，按理说这个时候坐标轴应该绕 y_1 旋转 90° 。

但是此时我们得到如图 5-17 所示的坐标系。

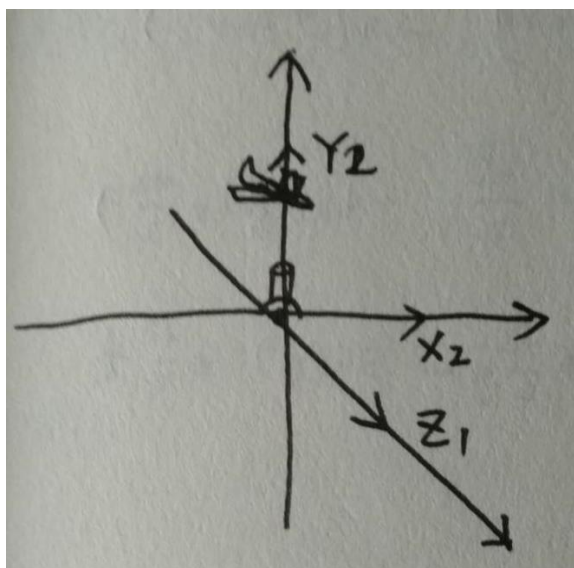


图 5-17 飞机紧急转向

不难发现，如果绕着 y_1 旋转 90° 的话，此时该绕着 z_2 旋转了。但是我们无论如何绕着 z_2 旋转都没法追踪飞机的位置了。这种情况就是万向节死锁。所以刚才如果想要能继续正常运行的话，必须旋转 180° 或者 360° 才可以。一般的解决方案就是在这种情况下做单独分析。在这种改变方向的情况下，做单独处理。比如让他变成 $x-z-y$ 转动模式，或者在绕 y_1 旋转时不让其旋转。但是不要以为用 $x-z-y$ 旋转模式就万事大吉了，因为无论用什么模式，都会有一个节点产生万向节死锁。

我相信对于万向节死锁，很多做游戏，或做动画的同学应该比较熟悉，因为他们在设计摄像头位置的时候是要考虑到这个问题的。在 Maya 或者 3d Max 中镜头的方向就是通过欧拉角控制的。所以也会面临欧拉角的万向节死锁的问题。这里给大家介绍国外有个叫作 THE GUERRILLA CG PROJECT 的课程讲解 CG 动画制作的，其中有一段对万向节死锁的问题进行了动画式的讲解。我在书本里没法用动画来演示万向节死锁的步骤。大家可以在这里观看（我也会在附件中提供下载观看）：

优酷地址：http://v.youku.com/v_show/id_XNzkyOTIyMTI=.html

附录中有个万向节死锁的程序，读者们可以下载下来亲自感受一下。PS：只能在 Windows 上运行。

现在可以得出如下的几个简单结论：

1) 欧拉角就是坐标系在初始状态旋转到最终状态中分别按照规定的旋转顺序分别绕轴旋转的三个角；

2) 欧拉角和 Yaw、Pitch、Roll 不是指同一种角；

3) 欧拉角在旋转的时候，是按照规定好的顺序来旋转，中途不能随意修改。

4) 欧拉角有万向节死锁的问题，所以在程序中需要单独解决这种问题。

除了向量余弦的方法，有一种表达方式，算是简化版的欧拉角。简化了欧拉角的弊端，使得角度旋转变得无比简单，而且还避免掉了万向死锁的问题。这就是四元数。

5.4 四元数

四元数精妙在哪里呢？它能把欧拉角三步完成的工作集中在一步完成。第一节讲欧拉角旋转的时候说过，不管是静态定义还是动态定义，都要将向量或坐标轴分别绕轴旋转。四元数现在就做到了只绕一个轴旋转，并转动一次就能达到同样的效果。可以这样理解：坐标系 A_1 绕着一个轴 v 转动了 α 角度，变成了我们需要的刚体新坐标体系 A_2 ，如图 5-18 所示。

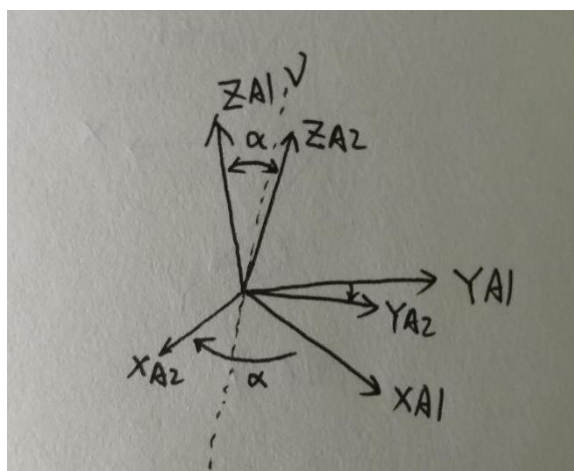


图 5-18 坐标系 $xyzA_1$ 绕 v 旋转 α 角变成 $xyzA_2$ 坐标系

那么现在描述物体旋转的方式就变成了：坐标系统一个轴转动了一定的角度。如果这个轴是 $v(x, y, z)$ ，角度 α 就是 w ，那么就可以把旋转的方式用四元数表示：

$$\vec{q} = \{w, \vec{v}\} = [w \ x \ y \ z]^T$$

四元数其实是种四维空间的函数，由数学家威廉·卢云·哈密尔顿(William Rowan Hamilton, 1805-1865)在 1843 年提出的数学概念。思维方式就是利用一个高维空间的数学表达式去计算一个低维空间数学的思想。

比如 $y=1$ 就是一维空间函数，它只是一条线，一条平行于 x 轴的线。而形如 $y=x^2+1$ 的函数就是二维空间函数，它涉及了 x 和 y 组成的平面中点和线。但是当我们把 x 设为 0 时，这个二维空间的图形照样可以表示一维空间中的图形和数据。三维空间 ($1=x^2+y^2+z^2$) 和四维以此类推。而我们要用的四元数就是利用四维空间的数据去计算三位空间的数据。你可能会问为什么要用四维去计算三维？基于之前的惯性思维，所有的人都自然的以为 n 维空间的向量变换需要 n 维复数来表示。但多年以来数学家们证明出根本不存在这样的三维复数来表示三维空间变换，所以哈密尔顿才提出用四元数来表示。

哈密尔顿提出要想使用四元数来表示三维复数需要满足：

- 1) 必须包含四个分量；
- 2) 不满足乘法交换律。

那这个复数又是什么呢？不知道大家还记不记得 $i^2=-1$ 这条公式。没错，这就是高中学的虚数。虚数的发明解决了很多数学等式解算的问题，比如 $x^2=-1$ ，本来是无解的，不过有了以上那条现在可以让 x 等于 i 。假设，此时有了 $z=bi$ ，然后在这个虚数上再加上个实数 a ，组合出来就是复数：

$$z=a+bi \quad a, b \in R, i^2=-1$$

甚至可以认为所有的实数和虚数都是用这种方式表示的。只是表示实数时 b 永远为 0，而表示虚数时 a 永远为 0。关于复数的基本计算公式，这里就不详细

叙述了，作者把复数的常用计算公式放在了附录中，请移步至 A14 关于复数计算，下面需要用到相关复数的计算。

那四元数又是如何表示的呢？在之前得到的数据上稍微修改--引入两个虚数，得到的数就是四元数，四元数的一般写法是：

$$q=s+x_i+y_j+z_k \quad s,x,y,z \in R \text{-----} (1)$$

s 为绕行的角度， xyz 分别是要绕的轴坐标值， q 为四元数。

$$\text{可以写作：} z=a+b_i+c_j+d_k \quad a,b,c,d \in R$$

式 (1) 中四元数有如下基本性质：

$$\begin{aligned} ij=k & \quad jk=i & \quad ki=j \\ ji=-k & \quad kj=-i & \quad ik=-j \\ i^2=j^2=k^2=ijk=-1 \end{aligned}$$

汉密尔顿发现复数的 i, j, k 之间的关系非常像向量叉积的规则。因为向量之间正交也满足：（注意一下公式中 xyz 均为向量。）

$$x \times y = z$$

$$y \times z = x$$

$$z \times x = y$$

$$y \times x = -z$$

$$z \times y = -x$$

$$x \times z = -y$$

由于 x, y, z 三轴之间满足这个关系，和复数一样。所以汉密尔顿就把上面的 i, j, k 三个虚数当作 xyz 坐标系上的三个向量来使用。

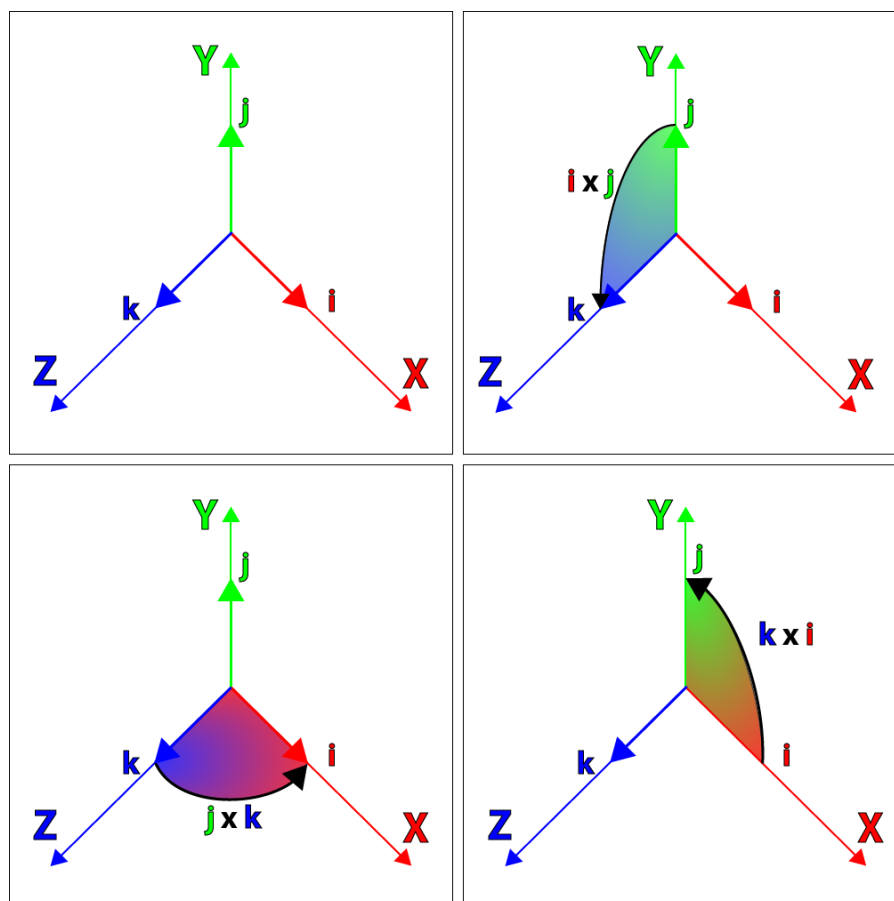


图 5-19 复数相乘在坐标系的表达

如上图所示，四元数中的三个虚数 ijk 可以分别当笛卡尔坐标系 xyz 三轴的单位向量值使用。

如果用有序对的形式来表示四元数：

$$[s, v] \quad s \in R, \quad v \in R^3$$

其中 v 就是刚才的三个虚数值，所以也可以写成：

$$q = [s, xi + yj + zk] \quad s, x, y, z \in R$$

s 就是转动的角度值。就用这种四元数来表示刚体旋转的角度和位置。当不转动的时候，也就是 s 为 0 时，此时四元数为纯四元数。也就是 $q = [0, v]$ 。这里需要特别注意，由于我们是利用四维函数计算三维角度，所以在做计算的时候要保证计算出来的结果为纯四元数，并且单位长度为 1，这样就可以保证旋转结果值处于纯三维平面中。

举个例子，现在陀螺仪测得 xyz 的数据分别是 1、2、3，那么对应的值应该就是 $p=[0,i+2j+3k]$ ，现在我们将 p 绕 v 向量转动 θ 角度，那么设 $q=[\theta,v]$ ，因此 $p'=qp$ ，意思就是 p 绕 q 右方向旋转了 θ 角度。当然，你理解成 q 绕 p 左方向旋转了 θ 角度也可以。因为要保证算出来的数据是在三维空间向量内，所以我们要保证得数是纯四元数。经汉密尔顿推导发现， p' 右乘 q 的逆，得到的数据就会变成纯四元数。所以该等式变成： $p'=qpq^*$ ，这是很重要的一步。四元数有很多性质，但是为了计算旋转角度只需要简单的加减和积的公式即可，所以在接下来进一步计算姿态数据之前，要先说明 2 个比较重要的公式，等会儿要用到：（四元数的计算公式 **A15 四元数的计算公式**）

四元数的积

$$\begin{aligned} q_a q_b &= [s_a, \mathbf{a}][s_b, \mathbf{b}] \\ &= (s_a + x_a i + y_a j + z_a k)(s_b + x_b i + y_b j + z_b k) \end{aligned}$$

经过推导可以得到：

$$\mathbf{a} = x_a i + y_a j + z_a k$$

$$\mathbf{b} = x_b i + y_b j + z_b k$$

$$\mathbf{a} \cdot \mathbf{b} = x_a x_b i^2 + y_a y_b j^2 + z_a z_b k^2$$

$$\mathbf{a} \times \mathbf{b} = (y_a z_b - y_b z_a)i + (z_a x_b - z_b x_a)j + (x_a y_b - x_b y_a)k$$

最后可以得到：

$$q_a q_b = [s_a s_b - \mathbf{a} \cdot \mathbf{b}, s_a \mathbf{b} + s_b \mathbf{a} + \mathbf{a} \times \mathbf{b}]$$

具体的推导细节可以参阅维基百科线性代数及其相关内容。不仅加法和积，还要用到四元数的逆。逆的表示方法是 q^{-1} 或者 q^* （如果是单位四元数的话）， q^* 是共轭四元数。其中的关系满足：

$$q^{-1} = \frac{q^*}{|q|^2}$$

四元数的共轭四元数去除四元数的范数。共轭四元数就是把虚数向量取反（向量前加负号），也就是：

$$q = [s, \mathbf{v}]$$

$$q^* = [s, -\mathbf{v}]$$

四元数的范数就有点儿像向量模长的那个意思，是用来规范化四元数用的。具体的计算的方式是：

$$q = [s, \mathbf{v}]$$

$$|q| = \sqrt{s^2 + v^2}$$

当四元数 q 左乘 c 时--这里 c 是一种四元数，也写成 $[s, \mathbf{v}]$ 的形式。当 s 为 0（即角度为 0），那么 c 就是个纯实数， cq 相乘就相当于改变了 q 的伸缩比例，但角度没有发生任何变化。当 s 不为 0，且 $\mathbf{v}$ 有数值， cq 相乘不但改变了比例，也改变了旋转方向，而且向量的长度都会按照相应的比例进行伸缩。

下面就来举例详细讲解一下四元数旋转时的计算过程：

$$\hat{\mathbf{v}} = \frac{\sqrt{2}}{2}\mathbf{i} + \frac{\sqrt{2}}{2}\mathbf{k}$$

首先设一个点 $p=[0,2i]$ ，将他绕 $\hat{\mathbf{v}}$ 旋转 45° ，我们设 $q=[\cos\theta, \sin\theta \mathbf{v}]$ 。设旋转后的点 p' ，则有 $p'=qp$ 。

所以原式 $p'=[\cos\theta, \sin\theta\mathbf{v}][0, p]$

根据和积公式可以得到 $=[-\sin\theta\mathbf{v} p, \cos\theta p + \sin\theta\mathbf{v} \times p]$

把 $\mathbf{v}$ 和 $\theta=45$ 带入上式可以得到：

$$p' = [-\frac{\sqrt{2}}{2}(\frac{\sqrt{2}}{2}\mathbf{i} + \frac{\sqrt{2}}{2}\mathbf{k}) \cdot (2\mathbf{i}), \frac{\sqrt{2}}{2}2\mathbf{i} + \frac{\sqrt{2}}{2}(\frac{\sqrt{2}}{2}\mathbf{i} + \frac{\sqrt{2}}{2}\mathbf{k}) \times 2\mathbf{i}]$$

$$= [-1, \sqrt{2}\mathbf{i} + \mathbf{j}]$$

注意，这里得到的 p' 已经不是纯四元数了，角度值为-1，之前讲过，四元数是在四维空间内的旋转的，为了使得计算出的数据在三维空间中（纯四元数没有角度值，可以看作是三维空间。纯四元数的三维被汉密尔顿看作四维的一个子集。就好比四川省是中国的一个省份一样），必须让他变成纯四元数，需要如下步骤：

1. 需要右乘 q 的逆。首先我们写出逆的方程：

$$q = [\cos\theta, \sin\theta(\frac{\sqrt{2}}{2}\mathbf{i} + \frac{\sqrt{2}}{2}\mathbf{k})]$$

$$q^{-1} = [\cos\theta, -\sin\theta(\frac{\sqrt{2}}{2}\mathbf{i} + \frac{\sqrt{2}}{2}\mathbf{k})]$$

注意这里 q 为单位四元数，根据性质有 $q^{-1}=q^*$ （ q 的逆等于 q 的共轭），此时计算应为 $qpq^{-1}=p'q^{-1}$ ，那么将 θ 的值 45° 代入到 q^{-1} 中可以得到：

$$q^{-1} = [\frac{\sqrt{2}}{2}, -\frac{\sqrt{2}}{2}(\frac{\sqrt{2}}{2}\mathbf{i} + \frac{\sqrt{2}}{2}\mathbf{k})]$$

$$\frac{1}{2}[\sqrt{2}, -\mathbf{i} - \mathbf{k}]$$

2. 在得到 q^{-1} 以后可得到 qpq^{-1} 值为：

$$qp = [-1, \sqrt{2}\mathbf{i} + \mathbf{j}]$$

$$qpq^{-1} = [-1, \sqrt{2}\mathbf{i} + \mathbf{j}] \frac{1}{2} [\sqrt{2}, -\mathbf{i} - \mathbf{k}]$$

$$= \frac{1}{2} [-\sqrt{2} - (\sqrt{2}\mathbf{i} + \mathbf{j}) \cdot (-\mathbf{i} - \mathbf{k}), \mathbf{i} + \mathbf{k} + \sqrt{2}(\sqrt{2}\mathbf{i} + \mathbf{j}) - \mathbf{i} + \sqrt{2}\mathbf{j} + \mathbf{k}]$$

$$= \frac{1}{2} [-\sqrt{2} + \sqrt{2}, \mathbf{i} + \mathbf{k} + 2\mathbf{i} + \sqrt{2}\mathbf{j} - \mathbf{i} + \sqrt{2}\mathbf{j} + \mathbf{k}]$$

$$= [0, \mathbf{i} + \sqrt{2}\mathbf{j} + \mathbf{k}]$$

3. 现在得到的数据就是纯四元数了。我们来做一个范数检查：

$$|qpq^{-1}| = \sqrt{1^2 + \sqrt{2}^2 + 1^2} = \sqrt{4} = 2$$

和 $p=[0, 2\mathbf{i}]$ 的范数 ($\sqrt{2^2}=2$) 完全一致。说明得到的这个点 p' 绕 q 旋转后的大小没有伸缩。如下图 5-17 所示，就是刚才计算出来的向量图形。

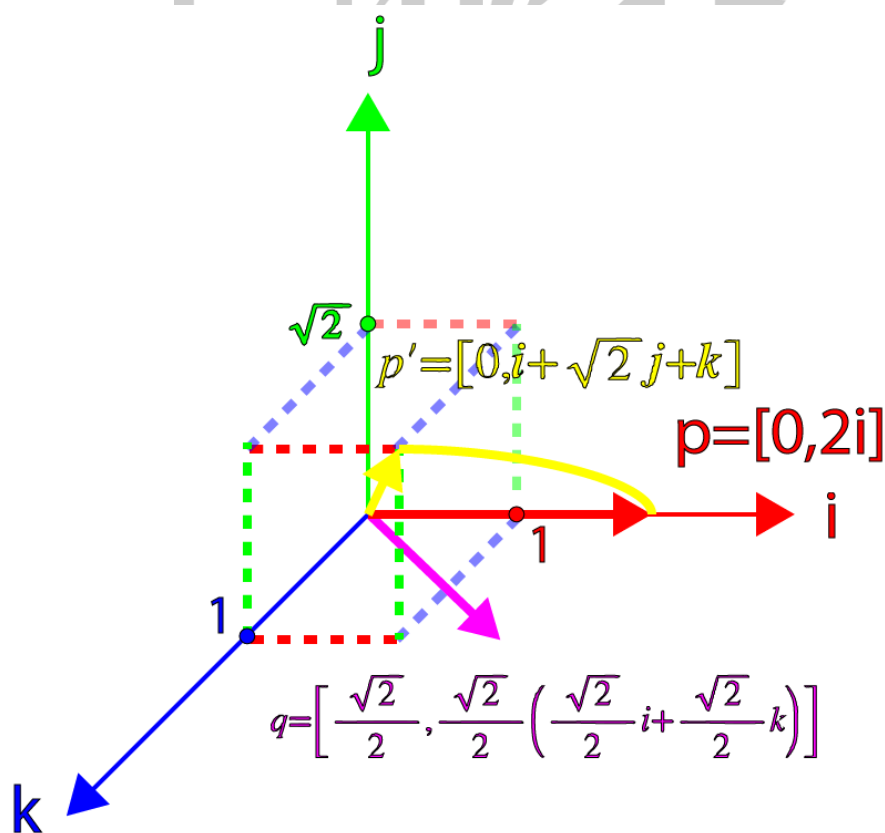


图 5-17 四元数旋转的图像表示

所以可以看到，这个结果是一个纯四元数，并且原四元数的向量的范数也保持住了。但是还有一个问题：向量被旋转了 90 度而不是 45 度。这刚好是我们需要的度数的两倍！为了正确地让一个向量绕某个轴向量旋转某个角度，必须以目标角度的一半来计算。因此，构造出了下面的四元数：

$$q = [\cos \frac{1}{2} \theta, \sin \frac{1}{2} \theta \hat{v}]$$

4. 这就是旋转四元数表达方式，所以我们计算四元数的时候就用 $\theta/2$ 来计算，才能得到要旋转的值。那么，旋转轴的四元数表达式可以这样表示：

$$q = (q1, q2, q3, q4) = (\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2}) * vx, \sin(\frac{\theta}{2}) * vy, \sin(\frac{\theta}{2}) * vz),$$

在秦永元的《惯性导航》这本书里 292 页也有这个结果的具体推导过程，如果有兴趣的同学可以去看一看。下面我们来用实例做个比方：

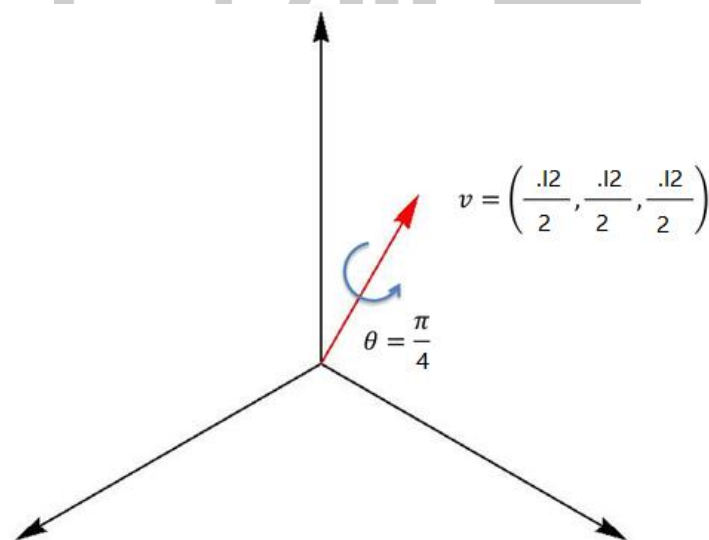


图 5-18 绕 v 旋转 45°

如图 5-18 所示，假设绕 v 向量 $((\sqrt{2}/2), (\sqrt{2}/2), (\sqrt{2}/2))$ 旋转 45° ，那么此时向量就应该改写成写成：

$$\begin{aligned}
 q_u &= (\cos \frac{\theta}{2}, \sin \frac{\theta}{2} * vx, \sin \frac{\theta}{2} * vy, \sin \frac{\theta}{2} * vz) \\
 q_u &= (\cos \frac{\pi}{8}, \sin \frac{\pi}{8} * \frac{\sqrt{2}}{2}, \sin \frac{\pi}{8} * \frac{\sqrt{2}}{2}, \sin \frac{\pi}{8} * \frac{\sqrt{2}}{2}) \\
 q_u &= (\cos \frac{\pi}{8}, \sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2} i + \sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2} j + \sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2} k)
 \end{aligned}$$

然后共轭就改成是：

$$\begin{aligned}
 q_u^{-1} &= (\cos(\frac{\theta}{2}), -\sin(\frac{\theta}{2}) * vx, -\sin(\frac{\theta}{2}) * vy, -\sin(\frac{\theta}{2}) * vz) \\
 q_u^{-1} &= (\cos(\frac{\pi}{8}), -\sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2}, -\sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2}, -\sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2}) \\
 q_u^{-1} &= (\cos(\frac{\pi}{8}) - \sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2} i - \sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2} j - \sin(\frac{\pi}{8}) * \frac{\sqrt{2}}{2} k)
 \end{aligned}$$

现在来总结一下四元数计算需要的步骤：

首先定义四元数：

w 为一个点，或者你想旋转的向量，定义为 $w(w_x, w_y, w_z)$

q 为你要绕的旋转轴， θ 为你要旋转的角度，定义为：

$$q = (q_1, q_2, q_3, q_4) = (\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2}) * vx, \sin(\frac{\theta}{2}) * vy, \sin(\frac{\theta}{2}) * vz)$$

那么有：

$$qw = (0, w_x, w_y, w_z)$$

$$qw = 0 + w_x * i + w_y * j + w_z * k$$

代入公式：

之前讲过旋转四元数的计算： $qw' = q * qw * q^{-1}$

其中 w' 为旋转过后变成的那个点

再把算出得到的数转化成纯四元数：

$$qw'=(0, wx',wy',wz')$$

$$=0+wx'*i+wy'*j+wz'*k$$

提取纯四元数中的后三位：

$$w'=(wx',wy',wz')$$

上面得到的 wx' 、 wy' 和 wz' 就是我们需要的值，这就是三维空间向量旋转后点或向量的位置坐标，也就是第 4 章讲过的陀螺仪和加速度计算得的数值。所以，根据得到的这些数据后就可以反推刚体的旋转角度了，我们把四元数计算出来的角度转换为欧拉角，这样的话，不但可以更好理解刚才旋转的姿态，而且每个轴的计算也会更加方便。

四元数比欧拉角好处多，它不仅简化了运算量，特别是在程序上编程，而且还节省了 MCU 很多的运算开支，并且让需要储存的变量减少为四个。避免了 Gimbal Lock（万向节互锁）的问题。是计算传感器姿态的最佳方法。

5.5 四元数的角度转换

通常的转换思路是四元数→余弦矩阵→欧拉角，将四元数用欧拉角的方式表示出来。其实从这点也可以看出，这三个数的值其实是可以相互转换的。具体的推导过程就不说了，这里直接给出推导出来的结果，以便大家直接利用。（关于四元数与欧拉角和余弦矩阵的转换关系式 **A16 欧拉角，余弦向量，四元数之间的转换**）

首先规定一下四元数为：

$$q=[w \ x \ y \ z]^T$$

$$|q|^2=w^2+x^2+y^2+z^2=1$$

然后给旋转轴和旋转角度定义一个四元数：

$$w=\cos(\alpha/2)$$

$$x=\sin(\alpha/2)\cos(\beta_x)$$

$$y=\sin(\alpha/2)\cos(\beta_y)$$

$$z=\sin(\alpha/2)\cos(\beta_z)$$

α 是旋转角度， $\cos\beta_x$ ， $\cos\beta_y$ ， $\cos\beta_z$ 是旋转轴在三轴上的投影大小。那么欧

拉角到四元数的转换关系就是：

$$q = \begin{bmatrix} w \\ x \\ y \\ z \end{bmatrix} = \begin{bmatrix} \cos(\varphi/2)\cos(\theta/2)\cos(\psi/2) + \sin(\varphi/2)\sin(\theta/2)\sin(\psi/2) \\ \sin(\varphi/2)\cos(\theta/2)\cos(\psi/2) - \cos(\varphi/2)\sin(\theta/2)\sin(\psi/2) \\ \cos(\varphi/2)\sin(\theta/2)\cos(\psi/2) + \sin(\varphi/2)\cos(\theta/2)\sin(\psi/2) \\ \cos(\varphi/2)\cos(\theta/2)\sin(\psi/2) - \sin(\varphi/2)\sin(\theta/2)\cos(\psi/2) \end{bmatrix}$$

也可得四元数到欧拉角的转换关系：

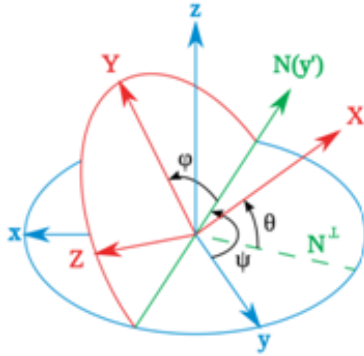


图 5-19 坐标轴欧拉角变换

$$\begin{bmatrix} \varphi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} \arctan \frac{2(wx + yz)}{1 - 2(x^2 + y^2)} \\ \arcsin(2(wy - zx)) \\ \arctan \left(\frac{2(wz + xy)}{1 - 2(y^2 + z^2)} \right) \end{bmatrix}$$

当然，在上一章讲过，程序中避免 arctan 值为 0，所以此处我们把 arctan 改成了 arctan2：

$$\begin{bmatrix} \varphi \\ \theta \\ \psi \end{bmatrix} = \begin{bmatrix} \text{atan2}(2(wx + yz), 1 - 2(x^2 + y^2)) \\ \arcsin(2(wy - zx)) \\ \text{atan2}(2(wz + xy), 1 - 2(y^2 + z^2)) \end{bmatrix}$$

因此，在程序中计算角度时，利用四元数推导出来的这个公式，只需要得到 q_0 , q_1 , q_2 和 q_3 的值即可求出欧拉角了。以下便是将上述推导得出的等式转化为程序的形式：

首先在进行四元数计算的时候，一定要保证四元数为单位四元数，模为 1。因此我们需要先把四元数归一化。

```
norm=sqrt(q0*q0+q1*q1+q2*q2+q3*q3);
```

```
q0= q0/norm;
```

```
q1= q1/norm;
```

```
q2= q2/norm;
```

```
q3= q3/norm;
```

其中的 q_0 , q_1 , q_2 和 q_3 就对应着上式中的 $wxyz$, 代入上式中我们就可以得到如下公式（以下为程序中的公式）：

```
Q_ANGLE.Z=atan2(2*q1*q2+2*q0*q3,-2*q2*q2-2*q3*q3+1)*Rad; //yaw
```

```
Q_ANGLE.Y =asin(-2*q1*q3+2*q0*q2)*Rad;// pitch
```

```
Q_ANGLE.X = atan2(2 * q2 * q3 + 2 * q0 * q1, -2 * q1 * q1 - 2 * q2 *  
q2 + 1) * Rad; //roll
```

这便是四元数与欧拉角的转换关系式了，那么既然求出欧拉角我们要确保欧拉角的数据不会超出范围，那么有：

```
if(Q_ANGLE.X>90||Q_ANGLE.X<-90)  
{  
  
if(Q_ANGLE.Y>0) Q_ANGLE.Y=180-Q_ANGLE.Y;  
  
if(Q_ANGLE.Y<0) Q_ANGLE.Y=-(180+Q_ANGLE.Y);  
  
}
```

这便是通过四元数来计算欧拉角的方法了。但我们在实际运用中确实通过欧拉角或姿态角来推导四元数的值—— q_0 , q_1 , q_2 和 q_3 。下面来让通过程序实例来让大家明白，这个计算的运作机制。

以四元数的初始化为例，来讲解一下四元数的使用方法：（这里的程序是用在芯片初始化时使用的，目的是得到芯片初始化时姿态的四元数数值，以便与后来的角度变化做计算）


```

void init_quaternion(void)
{
    unsigned long timestamp;

    signed short int accel[3], mag[3];

    float init_Yaw, init_Pitch, init_Roll;

    int i;

```

以上为我们定义要用到的数据寄存器，赋值。

```

    if(!i2cread(MPU9250_Addr, Accel_Xout_H, 6, data_write))
    {
        accel[0]=(signed short int)((data_write[0]<<8) | data_write[1]);
        accel[1]=(signed short int)((data_write[2]<<8) | data_write[3]);
        accel[2]=(signed short int)((data_write[4]<<8) | data_write[5]) + Accel_Zout_Offset;

```

判断有没有读取到数据，然后把读取到的数据放到 accel 的数组里，并纠偏。

```

        init_ax=(float)(accel[0] / Accel_4_Scale_Factor);
        init_ay=(float)(accel[1] / Accel_4_Scale_Factor);
        init_az=(float)(accel[2] / Accel_4_Scale_Factor);

        printf("ax=%f, ay=%f, az=%f", init_ax, init_ay, init_az);

```

单位转化，将读取的重力加速度转化为 m/s^2 的数据。并串口输出。

```

        for(i=0;i<5;i++)
        {
            mpu_set_bypass(1);

            mpu_get_compass_reg(mag,&timestamp);

            init_mx =(float)mag[1]-8;

            init_my =(float)1.046632*mag[0]-1.569948;

            init_mz =(float)-mag[2];

            mpu_set_bypass(0);

            printf("mx=%f,my=%f,mz=%f \n\r", init_mx, init_my, init_mz);
        }

```

读取磁力计的数据，并纠偏。通过串口打印出来。到目前为止都是前两章讲

到的结果，现在如果按照航空的旋转方法计算的话，就可以规定旋转方向是 z-x-y，这样的话，就可以计算刚体目前的姿态角——roll, pitch, yaw 角：

```
init_Roll=-atan2(init_ax,init_az);

init_Pitch=asin(init_ay);

init_Yaw=atan2(init_mx*cos(init_Roll)+init_my*sin(init_Roll)*sin(init_Pitch)+init_mz*sin(
init_Roll)*cos(init_Pitch),init_my*cos(init_Pitch)-init_mz*sin(init_Pitch)); //yaw 是朝向
角，需要借助磁力计的数据计算出可靠的角度。

if(init_Yaw < 0){init_Yaw = init_Yaw + 2*3.141593;}

if(init_Yaw > 360){init_Yaw = init_Yaw - 2*3.141593;}//将 Yaw 角控制在 0~360°之间。
```

以上的方法，求出了刚体的欧拉角，那么此时我们可以根据四元数和欧拉角转换的算法，带入欧拉角数据来求得四元数的 q_0, q_1, q_2, q_3 。注意，这里四元数转化的公式必须和之前欧拉角规定的旋转方向一致才行，否则转化出来是错误的。

由于刚才计算欧拉角是按照航天的 Yaw-Pitch-Roll 的顺序来设计的，那么此处计算也要按照 z-y-x 的方法来旋转。

我们知道绕着 z 轴旋转的旋转矩阵为：（Yaw）

$$C_{\psi} = \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

而绕着 x 轴旋转的旋转矩阵：（Pitch）

$$C_{\theta} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & \sin \theta \\ 0 & -\sin \theta & \cos \theta \end{bmatrix}$$

绕着 y 轴旋转的矩阵：（Roll）

$$C_{\varphi} = \begin{bmatrix} \cos \varphi & 0 & -\sin \varphi \\ 0 & 1 & 0 \\ \sin \varphi & 0 & \cos \varphi \end{bmatrix}$$

那么将它们按照 $z-y-x$ 的顺序相乘遍可得到如下余弦矩阵：

	E	N	ξ
X	$\cos \psi \cos \varphi - \sin \psi \sin \theta \sin \varphi$	$\sin \psi \cos \varphi + \cos \psi \sin \theta \sin \varphi$	$-\cos \theta \sin \varphi$
Y	$-\sin \psi \cos \theta$	$\cos \psi \cos \theta$	$\sin \theta$
Z	$\cos \psi \sin \varphi + \sin \psi \sin \theta \cos \varphi$	$\sin \psi \sin \varphi - \cos \psi \sin \theta \cos \varphi$	$\cos \theta \cos \varphi$

以下便是余弦矩阵与四元数对应的矩阵：

$$A = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

此时把四元数矩阵对应一下余弦矩阵就可以得到以下求出 $q_0 \sim q_3$ 的计算方法了。（此处方案不为唯一，相当于找最简表达式能表示出三个欧拉角，然后反推 $q_0 \sim q_3$ 的值）

```
q0 = cos(0.5*init_Roll)*cos(0.5*init_Pitch)*cos(0.5*init_Yaw) -
sin(0.5*init_Roll)*sin(0.5*init_Pitch)*sin(0.5*init_Yaw);

q1 = cos(0.5*init_Roll)*sin(0.5*init_Pitch)*cos(0.5*init_Yaw) -
sin(0.5*init_Roll)*cos(0.5*init_Pitch)*sin(0.5*init_Yaw);

q2 = sin(0.5*init_Roll)*cos(0.5*init_Pitch)*cos(0.5*init_Yaw) +
cos(0.5*init_Roll)*sin(0.5*init_Pitch)*sin(0.5*init_Yaw);

q3 = cos(0.5*init_Roll)*cos(0.5*init_Pitch)*sin(0.5*init_Yaw) +
sin(0.5*init_Roll)*sin(0.5*init_Pitch)*cos(0.5*init_Yaw);

printf("Yaw=%f,Pitch=%f,Roll=%f\n\r",init_Yaw*57.295780, init_Pitch*57.295780,
init_Roll*57.295780); //此处将输出的数据乘以 57.2958 就可以将弧度值换算成角度值。

}
```

至此，就得到了刚体目前的 q_0 ， q_1 ， q_2 和 q_3 ，可以通过上位机直接表示物体的旋转了。当然，在计算的过程中还需要不断地更新这 4 个数值来精确表达出物体是如何转动的，以及转动后的姿态角度。那么，加速度，陀螺仪和磁力计又是怎么融合在一起的呢？下一章，我们将把九轴芯片中得到的数据通过滤波融合的

方式，变成我们需要更新的四元数信息。来表现刚体的运动姿态。

下视界

第6章 姿态融合

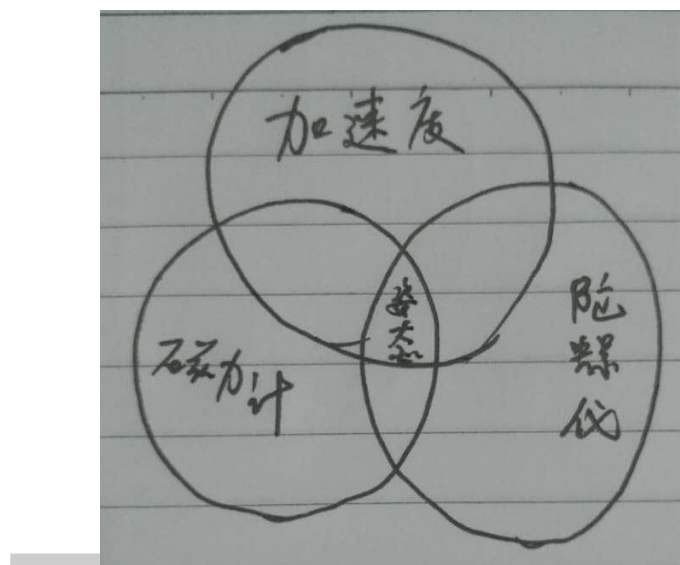


图 6-1 融合

到目前为止我们已经能将测得的加速度计，陀螺仪和磁力计的数据进行换算变成我们需要的姿态数据，并以四元数的方法表示出来。但是得到的数据却都是单独的，陀螺仪是陀螺仪，加速度计是加速度计，磁力计是磁力计，他们之间仿佛并没有什么必然的联系一般。但在第一章我们曾经说过关于他们三者之间的联系。所以，我们在这一章就要将他们之间的数据联系起来。得到一个更可靠的姿态角度——姿态融合算法。

如果说在姿态融合算法中最重要的是什么？那当然是陀螺仪。在第一章我们曾讲过陀螺仪是计算实时性姿态的工具。它显示的是传感器分别绕每个轴的实时旋转角速度。那么，如果我们用积分的方式来计算陀螺仪的角速度的话，那么累计出来的就是弧长，也可以累积出旋转的角度。很多市面上卖的工业角速度仪里面就自带了积分计算，众所周知的是光靠积分角速度来计算物体姿态的方法是靠不住的。因为积分时间长后必然会产生漂移，导致数据不准确，偏移越来越大。所以我们不能只相信一个陀螺仪来计算刚体的姿态。这时候，加速度计就可以上场了，通过加速度计，便可以随时检测刚体目前的姿态，仿佛时间暂停似得。然后用此时的姿态去修正陀螺仪的姿态，便可以得到一个相对更准确的姿态数据。而磁力计的作用是计算刚体的朝向角，也就是 Yaw 值，所以 Yaw 值是需要三个传

传感器同时补偿才能得到的。

但是要获得准确的数据首先需要解决的是确保得到数据的有效性。尽管我们之前讲过用平均值和设置 offset 点的方法来去除噪声，但这个方法会使得得到的数据也不是很准。比如我现在给出一组数据：

1	2	3	4	5	6	7	8	9	10
1.5	1.4	1.5	1.7	3.2	1.4	1.3	1.2	1.4	1.6

以上这 10 个数据大家看一下，第 5 个数据出现了数据偏移，比如上表是测得加速度的数值。综合 1~10 的前后数据来看，其实这个时候数据应该是不可取的。也就是有用的数据其实只有 9 个。但是如果按照我们之前取 10 个数的平均的方法，平均下来是 1.62；如果按照 9 位有效值的平均数则是 1.44，相差就有 0.18，但是这个误差也不小，可以影响到系统的稳定了。

你或许会想，如果设计一个程序来判断这 10 组数据中误差很大的数据，然后把它们剔除掉，就可以了。但是说起来容易，做起来并不好做。首先第一点，这样的程序需要时时刻刻不停的比较和对比传感器取得的数据再判断。我们知道在数学程序计算中如果 10 个数值，每两个都要做对标的话，大概要进行 $(1+10)*10/2=55$ 次运算，这样的话不但会耽误传感器的读取效率，还会占用 CPU 的时间。然后第二点，你无法对接下来要遇到的情况作出正确的判断。比如，我们稳定向右加速的飞行器突然遇到一股强劲的气流。此时得到一个突变的数据：（表中数据假设为 y 轴横向受力）

1	2	3	4	5	6	7	8	9	10
1.1	1.4	1.8	1.6	7.4	8.8	9.7	4.9	1.3	1.0

这里测得的加速度数据假设是根据不可抗力的干扰造成数据漂移，观察一下，他的数据并不稳定。先是突然增大，然后又突然减小。如果，这 10 组数据是在 1 秒内检测到的，变数就会非常多，可能是突然迎面来了场风，可能是其他物体突然推了它一把，总之一切皆有可能。这个时候你如果再用上面的那个方法

来排除特殊值的话数据就更不可靠了。所以，我们务必要找出一种方法可以很好地解决这种姿态数据跳变且不准的情况。下面我们就来介绍一下著名的卡尔曼滤波算法。

6.1 卡尔曼滤波



图 6-2 卡尔曼

卡尔曼滤波是一种通过已有的数据来预测未来数据的一种手段。最初是用来做阿波罗飞船的轨道预测，后来由卡尔曼本人发表滤波器的相关论文而闻名于世，2009 年的时候卡尔曼本人还获得了奥巴马的颁发的美国国家科学奖章，以奖励他在科学上为人类所做出的贡献。

提到滤波，就是过滤我们得到的数据值中不想要的那些数据。在硬件电路上也有高通滤波，低通滤波和带通滤波的方法。之前讲的平均或者消除的方法也是程序上一种滤波的方法，不过这都是传统的滤波方式了。他们的缺点都是要把想要的信号和噪声分离开来非常麻烦，况且很多信号中本身自带的噪声很难分离。比如加速度的数据，就算是芯片自己直接测量出来的数值，而没有经过任何算法的加工，得出的数据也不是准确的。如果这个传感器在一架飞行器上面的话，他受到风吹时产生的震荡，受到阳光照射产生温度的变化，或者周围磁场不均匀（高压电线，或者基站等影响）都会导致测量出来数据的带上噪声，而这些噪声却是完全不可控的。所以，我们既然不能很好地去掉噪声，干脆把它利用起来。

卡尔曼滤波是怎么利用这些数据呢？他通过各种复杂的推导得出了 5 个公式。通过这 5 个公式可以直接根据现有的数据预测出下一次要得到的数据，然后根据下一次测到的数据再和预测的数据做对比，从而得到一个相对比较满意的数值。在笔者看来，说白了就是加权平均数。要想知道具体的 5 个公式推导，请移步至附录 A 中的 A17——**卡尔曼滤波五大公式**了解。如果你只想直接拿来用的话，请允许我接下来用个简单的例子来给你解释一下卡尔曼滤波的用法和思想。

比如说，我们煮鸡蛋，锅里面有一个传感器，是温度传感器。假设我们要把水烧开到 100°C 。我们知道他一定是这样一个函数图像：

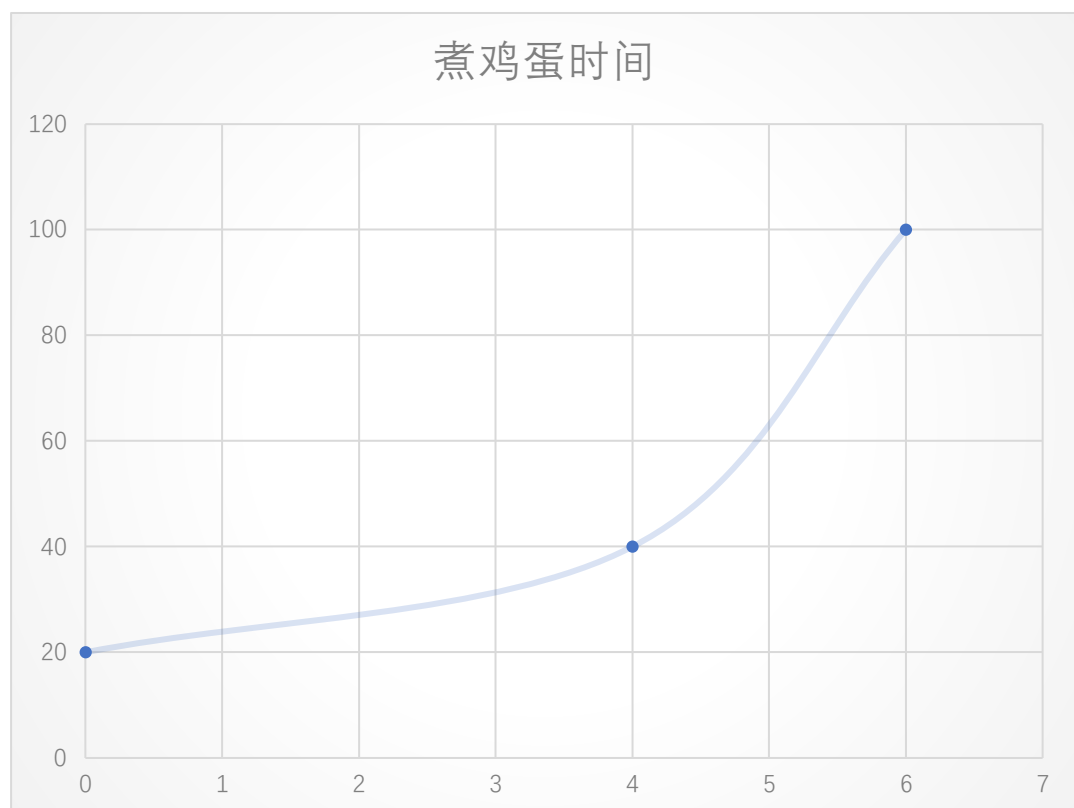


图 6-3 煮鸡蛋的时间的水温

在 0 分的时候假设水是常温 20°C ，那么传感器测得的数据就是 20°C 左右。然后根据卡尔曼滤波那 5 个公式来进行推导，估测出 4 分钟的时候大概在 43°C ，然而到了 4 分钟的时候，传感器测出的水目前的温度是 40°C 左右，此时锅内的水有可能在循环上下的流动，所以导致测得的数据也不一定非常准确。这个时候就可以通过权重来计算出最终的答案。比如说，我们要更相信预测值一些，让预测值占比 90%，那测量值占比 10%。这样算下来 4 分钟的时候得到的温度就是

$=43*90\%+40*10\%=42.7^{\circ}\text{C}$ ，得到的这个 42.7°C 也就是下一个时段的预估值的依据了。那么你可能想问，怎么根据数据决定哪边权重要高一些呢？其实这就是卡尔曼滤波器的高明之处了，他的数据完全符合正态分布。下面我来用一个例子来讲明。（本例程来源于 Understanding the Basis of the Kalman Filter Via a Simple and Intuitive Derivation，由于原版是英文版而且含有很多公式，为了让大家能轻易地看明白，我在这里用白话和大家说清楚）

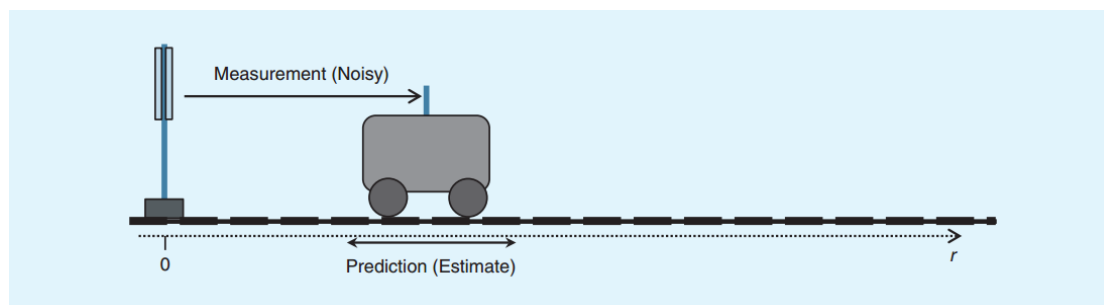


图 6-4 测量小车行走距离（measurement 是测量值（带噪声）prediction 是预测值）

如图 6-4 所示，这里有一个小车向前走。左边有个超声波传感器发射的基站，专门用来测量小车行走距离的。当然，测量的数据会有噪声干扰。噪声的来源可能是轮子不稳，也有可能是前后轮子受力不均，地上有石子，车身重量左右不平衡等各种影响。

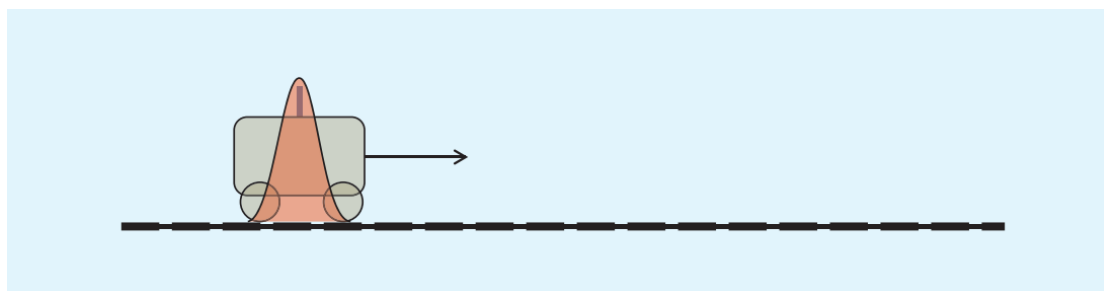


图 6-5 小车行走

我们把它的距离看做是正态分布，如上图 6-5 所示。此时可以获得 t_0 时刻小车的位置。

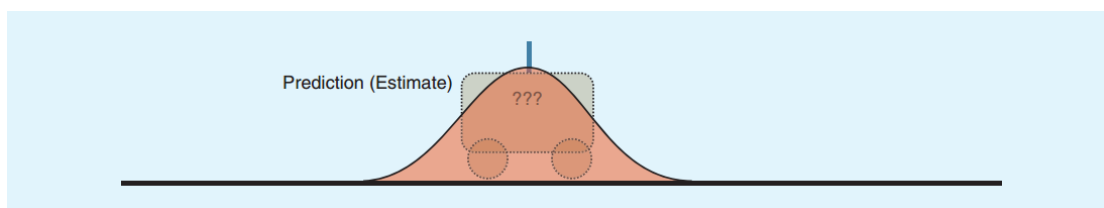


图 6-6 小车的测量值，带误差

然后利用卡尔曼滤波里面的公式，根据 t_0 时刻的位置来估算出 t_1 时刻的位置。如上图所示，你会发现这个正态分布图形两边变宽了。没错，因为不确定因素变多了。所以符合的范围就更大了。这个就有点儿像区间值一样， $\pm x$ 。

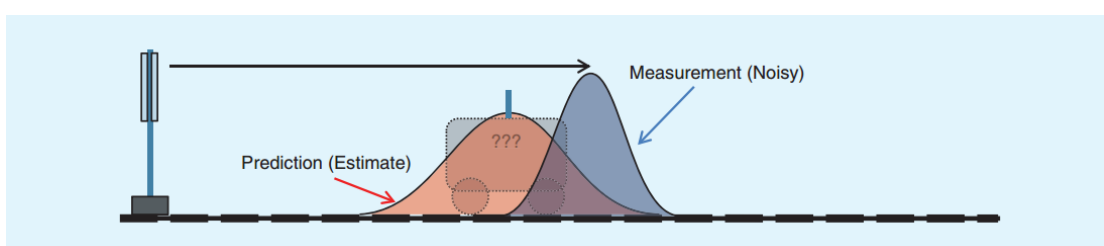


图 6-7 小车行走的测量值和估算值比较

然后，我们在 t_1 时刻测量得的实际距离如上图紫色位置所示。此时我们既有了估测值，又有了测量值。那么，我们开始加权配比。

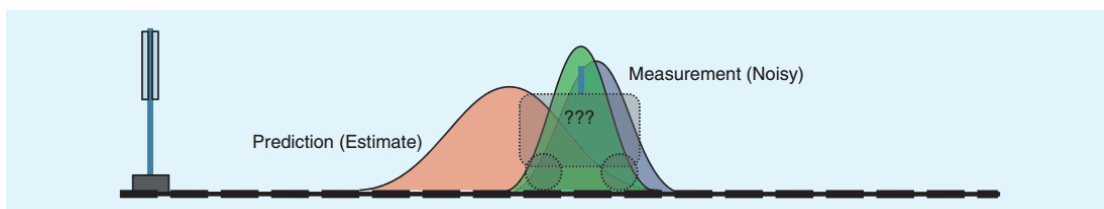


图 6-8 绿色的为小车的加权值

如上图 6-8，绿色的正态分布图形就是我们加权配比后计算出来的位置。所以绿色部分就是 t_1 时刻小车的位置。这个加权值在卡尔曼滤波里面叫作卡尔曼增益。是在公式里面计算出来的。

好了，了解了卡尔曼滤波的思想，其实我们都可以不用管公式就可以在 C 语言中直接使用卡尔曼滤波了。使用起来非常简单。我们现在来看一下卡尔曼滤波的核心程序，它一共分为 7 步：

```
Kalman::Kalman() {
```

```

    Q_angle = 0.001f;

    Q_bias = 0.003f;

    R_measure = 0.03f;

    angle = 0.0f; // 重置角度值

    bias = 0.0f;

    P[0][0] = 0.0f;

    P[0][1] = 0.0f;

    P[1][0] = 0.0f;

    P[1][1] = 0.0f;
};

// 三组数 newAngle 是角度值, newRate 是角度每秒的值, dt 是每秒钟的时间

float Kalman::getAngle(float newAngle, float newRate, float dt) {

    /* 第一步 */

    rate = newRate - bias;

    angle += dt * rate;

    // 预测并更新误差协方差

    /* 第二步*/

    P[0][0] += dt * (dt * P[1][1] - P[0][1] - P[1][0] + Q_angle);

    P[0][1] -= dt * P[1][1];

    P[1][0] -= dt * P[1][1];

    P[1][1] += Q_bias * dt;

    // 离散式卡尔曼滤波方程更新, 并计算出卡尔曼增益

    /* 第四步*/

    float S = P[0][0] + R_measure; // 预测误差

    /* 第五步*/

    float K[2]; // 卡尔曼增益, 这是一个 2*1 的向量

    K[0] = P[0][0] / S;

    K[1] = P[1][0] / S;

    // 计算角度值和误差值 - 测量数据更新

```

```

/* 第三步*/

float y = newAngle - angle; // 角度差

/* 第六步 */

angle += K[0] * y;

bias += K[1] * y;

// 计算并更新估计误差和协方差

/* 第七步*/

float P00_temp = P[0][0];

float P01_temp = P[0][1];

P[0][0] -= K[0] * P00_temp;

P[0][1] -= K[0] * P01_temp;

P[1][0] -= K[1] * P00_temp;

P[1][1] -= K[1] * P01_temp;

return angle;//返回角度值
};

```

计算卡尔曼滤波，只需要调用 `float Kalman::getAngle(float newAngle, float newRate, float dt)` 这个程序就可以了。输入角度值 (roll, pitch, yaw)，还有测得的角速度值和间隔时间，程序就会自动算出并返回出当前最佳角度值。结合一下第四章所讲的计算角度的方法和第 5 章讲的计算欧拉角的方法，我们来看一个卡尔曼滤波的例子。（本例程适用于 arduino）

```

#include <Wire.h>

#include <Kalman.h>

#define RESTRICT_PITCH // pitch 角度在±90°之内

Kalman kalmanX; // 当前 roll 的角度

Kalman kalmanY; // 当前 pitch 的角度

/* 芯片数据 */

double accX, accY, accZ;

double gyroX, gyroY, gyroZ;

int16_t tempRaw;

```

```

double gyroXangle, gyroYangle; // 只用陀螺仪计算出的角度

double compAngleX, compAngleY; // 使用互补滤波器计算出来的角度

double kalAngleX, kalAngleY; // 卡尔曼滤波计算出来的角度

uint32_t timer;

uint8_t i2cData[14]; //I2C 的缓存器

    到此，程序在运行开始之前设置了后面需要用到数据的缓存器。

void setup() {                                // 程序设置部分开始

    Serial.begin(115200);

    Wire.begin();

#ifdef ARDUINO >= 157

    Wire.setClock(400000UL); // I2C 速率设置为 400kHz

#else

    TWBR = ((F_CPU / 400000UL) - 16) / 2; // 设置 I2C 速率为 400kHz

#endif

    i2cData[0] = 7; // 设置采样速率 1000Hz - 8kHz/(7+1) = 1000Hz

    i2cData[1] = 0x00; // 关闭 FSYNC 功能并设置 260 Hz 加速度滤波, 256 Hz 陀螺仪滤波, 8 KHz 采样率

    i2cData[2] = 0x00; // 设置陀螺仪的采样速率 ±250deg/s

    i2cData[3] = 0x00; // 设置加速度的采样速率为 ±2g

    while (i2cWrite(0x19, i2cData, 4, false)); // 一次写 4 个寄存器

    while (i2cWrite(0x6B, 0x01, true)); // 关闭休眠模式

    while (i2cRead(0x75, i2cData, 1));

    if (i2cData[0] != 0x68) { // 读取自己的地址

        Serial.print(F("Error reading sensor"));

        while (1);

    }

    delay(100); //等待传感器反应

    到这里为止，通过 I2C 设置完成了传感器的各项初始化数值和配置。

    /* 设置卡尔曼和陀螺仪的初始角度*/

    while (i2cRead(0x3B, i2cData, 6));

```

```

    accX = (int16_t)((i2cData[0] << 8) | i2cData[1]);

    accY = (int16_t)((i2cData[2] << 8) | i2cData[3]);

    accZ = (int16_t)((i2cData[4] << 8) | i2cData[5]);

    到这里为止，通过 I2C 读取了三轴加速度数据并储存在 accXYZ 三个数值当中。

    // atan2 输出值在 $-\pi$ 到 $\pi$ 之间，这个之前是讲过的。然后转换为弧度值。

#ifdef RESTRICT_PITCH

    double roll = atan2(accY, accZ) * RAD_TO_DEG;

    double pitch = atan(-accX / sqrt(accY * accY + accZ * accZ)) * RAD_TO_DEG;

#else

    double roll = atan(accY / sqrt(accX * accX + accZ * accZ)) * RAD_TO_DEG;

    double pitch = atan2(-accX, accZ) * RAD_TO_DEG;

#endif

    根据当前的姿态来算出当前 roll 角和 pitch 角，并转换成弧度值。

    kalmanX.setAngle(roll); // 设置初始角度

    kalmanY.setAngle(pitch);

    gyroXangle = roll;

    gyroYangle = pitch;

    compAngleX = roll;

    compAngleY = pitch;

    timer = micros();

}

```

到这里为止，初始化完成了卡尔曼滤波器里面各个寄存器的配置。分别得到了，卡尔曼初始 roll 角度，初始 pitch 角度。直接通过测量数据计算出来的 pitch 和 roll 角，还有滤波后得到的 pitch 和 roll 角，以及测量间隔时间。

```

void loop() {

    /* 更新全部数值 */

    while (i2cRead(0x3B, i2cData, 14));

    accX = (int16_t)((i2cData[0] << 8) | i2cData[1]);

    accY = (int16_t)((i2cData[2] << 8) | i2cData[3]);

```

```

accZ = (int16_t)((i2cData[4] << 8) | i2cData[5]);

tempRaw = (int16_t)((i2cData[6] << 8) | i2cData[7]);

gyroX = (int16_t)((i2cData[8] << 8) | i2cData[9]);

gyroY = (int16_t)((i2cData[10] << 8) | i2cData[11]);

gyroZ = (int16_t)((i2cData[12] << 8) | i2cData[13]);;

double dt = (double)(micros() - timer) / 1000000; // 计算差距时间

timer = micros();

```

读取加速度和陀螺仪温度计的数值，以及得到这次读取和上一次读取这些值之间的时间间隔值。

```

// atan2 输出值在 $-\pi$ 在 $\pi$ 之间，然后将数据转换成弧度值

#ifdef RESTRICT_PITCH

    double roll = atan2(accY, accZ) * RAD_TO_DEG;

    double pitch = atan(-accX / sqrt(accY * accY + accZ * accZ)) * RAD_TO_DEG;

#else

    double roll = atan(accY / sqrt(accX * accX + accZ * accZ)) * RAD_TO_DEG;

    double pitch = atan2(-accX, accZ) * RAD_TO_DEG;

#endif

```

计算出 roll 和 pitch 角度值

```

double gyroXrate = gyroX / 131.0; // 转成 deg/s

double gyroYrate = gyroY / 131.0; // 转成 deg/s

#ifdef RESTRICT_PITCH

    //解决万向节死锁问题，角度更替

    if ((roll < -90 && kalAngleX > 90) || (roll > 90 && kalAngleX < -90)) {

        kalmanX.setAngle(roll);

        compAngleX = roll;

        kalAngleX = roll;

        gyroXangle = roll;

    } else

        kalAngleX = kalmanX.getAngle(roll, gyroXrate, dt); // 卡尔曼滤波计算角度

```

```

if (abs(kalAngleX) > 90)

    gyroYrate = -gyroYrate; //适合于加速度计度数

    kalAngleY = kalmanY.getAngle(pitch, gyroYrate, dt);

```

以上是当角度变化范围限死在 $\pm 90^\circ$ 的时候，根据卡尔曼滤波得到 roll 和 pitch 的角度。

```

#else

    // 这里也是角度更替，避免角度 180 到-180 突变

    if ((pitch < -90 && kalAngleY > 90) || (pitch > 90 && kalAngleY < -90)) {

        kalmanY.setAngle(pitch);

        compAngleY = pitch;

        kalAngleY = pitch;

        gyroYangle = pitch;

    } else

        kalAngleY = kalmanY.getAngle(pitch, gyroYrate, dt); // 用卡尔曼滤波器计算角度

        if (abs(kalAngleY) > 90)

            gyroXrate = -gyroXrate; //根据加速度的值，来改变速率

            kalAngleX = kalmanX.getAngle(roll, gyroXrate, dt); // 卡尔曼滤波器计算角度

#endif

```

以上是当角度变化范围限死在 $\pm 90^\circ$ 的时候，根据卡尔曼滤波更新 pitch 和 roll 的角度。

```

gyroXangle += gyroXrate * dt; // 陀螺仪角度积分计算

gyroYangle += gyroYrate * dt;

```

计算陀螺仪角度值，根据积分来算出当前旋转的角度

```

compAngleX = 0.93 * (compAngleX + gyroXrate * dt) + 0.07 * roll; //更新当前物体最新的 roll
角度值

compAngleY = 0.93 * (compAngleY + gyroYrate * dt) + 0.07 * pitch; //更新为当前物体最新的
pitch 角度值

```

权重配比计算最佳角度值

```

// 飘逸太大时重置陀螺仪角度值

```



```

if (gyroXangle < -180 || gyroXangle > 180)

    gyroXangle = kalAngleX;

if (gyroYangle < -180 || gyroYangle > 180)

    gyroYangle = kalAngleY;

```

更新角度值，新角度就是当前的 ROLL 和 PITCH 角度

```

/* 输出数据 */

#if 0 //置 1 时使能数据输出

    Serial.print(accX); Serial.print("\t");

    Serial.print(accY); Serial.print("\t");

    Serial.print(accZ); Serial.print("\t");

    Serial.print(gyroX); Serial.print("\t");

    Serial.print(gyroY); Serial.print("\t");

    Serial.print(gyroZ); Serial.print("\t");

    Serial.print("\t");

```

输出加速度值和陀螺仪数据

```

#endif

    Serial.print(roll); Serial.print("\t");

    Serial.print(gyroXangle); Serial.print("\t");

    Serial.print(compAngleX); Serial.print("\t");

    Serial.print(kalAngleX); Serial.print("\t");

    Serial.print("\t");

    Serial.print(pitch); Serial.print("\t");

    Serial.print(gyroYangle); Serial.print("\t");

    Serial.print(compAngleY); Serial.print("\t");

    Serial.print(kalAngleY); Serial.print("\t");

```

输出卡尔曼滤波算出角度值和测量角度值和欧拉角

```

#if 0 //当为 1 时输出温度数据

    Serial.print("\t");

```

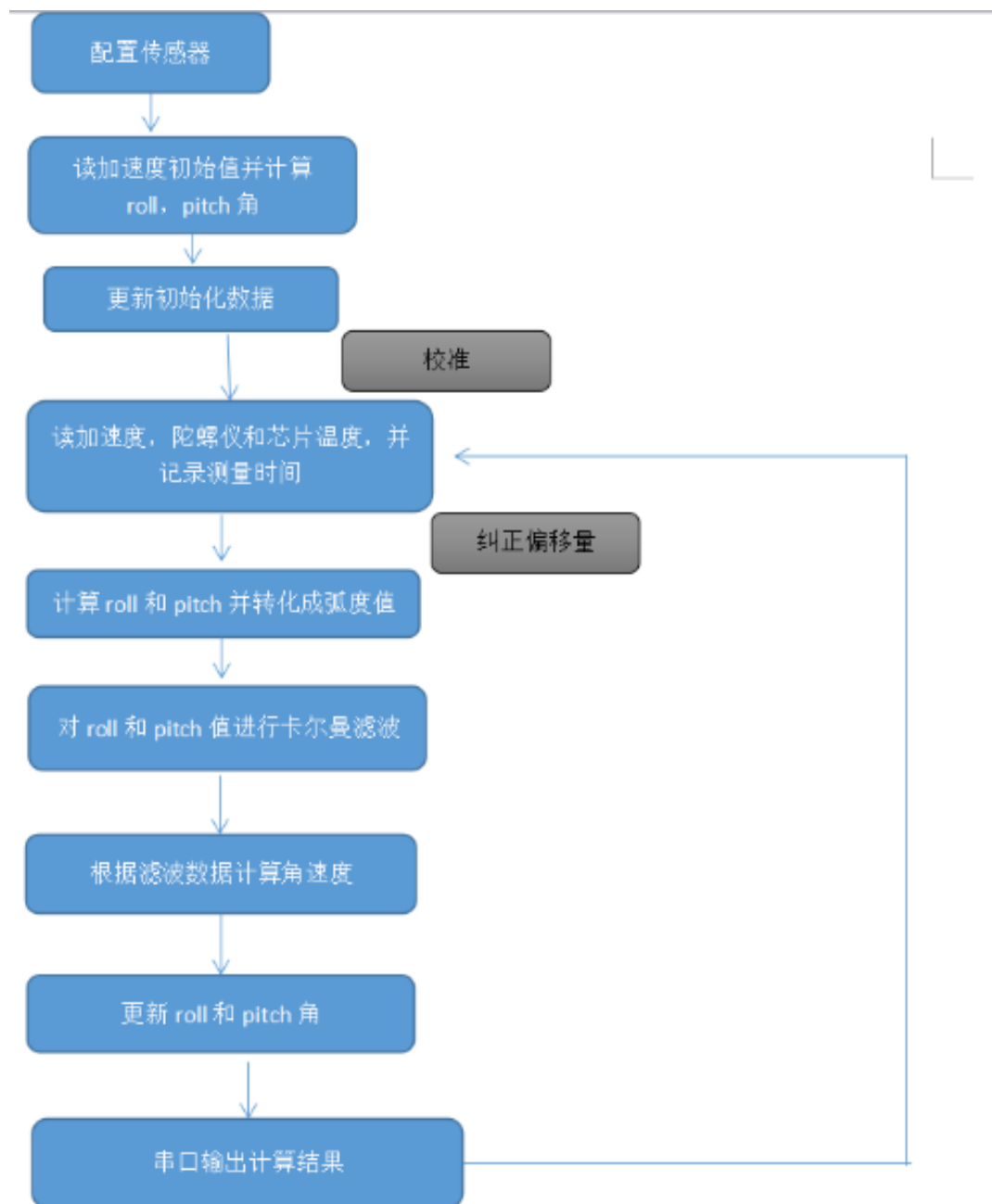
```
double temperature = (double)tempRaw / 340.0 + 36.53;  
  
Serial.print(temperature); Serial.print("\t");  
  
#endif
```

输出温度数值

```
Serial.print("\r\n");  
  
delay(2);  
  
}
```

最后得出来的 pitch 和 roll 角的角度是经过卡尔曼滤波优化了的。现在我把整个程序的思路结构整理在下面，大家一看就明白了。

下视界



通过以上的这个示例程序，不难看出卡尔曼滤波在程序中的使用思路和场景。他并没有像我们之前一样，对每个加速度值和角速度值进行滤波，而是直接对算出的 roll 和 pitch 角进行滤波，并根据之前的 roll 和 pitch 直接得出当前传感器的最优 roll 和 pitch 数值。上面的黑框部分其实并不在程序里面，但是在程序里面加上会好很多。

现在总结一下卡尔曼滤波使用需要的条件：

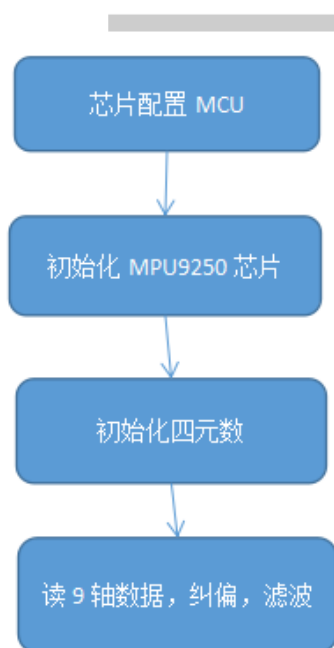
1. 你要更新的数据；

2. 上一次更新的数据；
3. 更新间隔的时间；
4. 数据变化的速率。

知道这点就可以用卡尔曼滤波了。顺便再说一下角度积分的算法，虽然示积分，但其实就是将 $^{\circ}/s$ （度每秒） $\times \Delta t$ （测量间隔时间）=角度（单位时间内变化的角度），一个简单的乘法。再加上之前的角度值，就可以得到累计的角度值。角度+上个角度=当前角度值。

6.2 捷联惯导算法

到目前为止，我们做到了：



在得到了准确的四元数数据后，就可以开始做数据融合了。

通常市面上的数据融合是利用捷联惯导算法。是一个德国人写的，并开源出来。分为两种：

1. 加速度，陀螺仪和磁力计共同补偿；
2. 加速度和陀螺仪补偿。

首先我们以 AHRSupdate 算法（第一种）为例子来讲解捷联惯导的思路和用

法。

先大概说一下思路：AHR 算法主要是陀螺仪为主，加速度和磁力计为辅测量物体姿态的方案。

1. 先通过磁力计将机体坐标系的数值转换为世界坐标系，补偿后转换回机体坐标系。

2. 再将测量的加速度转换为机体坐标系，求出误差。

3. 再将误差补偿给陀螺仪计算得到的机体坐标系的值。

4. 将坐标转换为四元数，最后得到 Yaw, Pitch, Roll。

看到这里你可能回懵，为什么会有世界坐标系和机体坐标系。之前怎么没讲过。为什么要在这两者之间作转换？

这是因为我们的地球是椭圆的，我们的机体在地球上的坐标系并不能完全的对应地理的东西南北。我举个例子。如图所示：

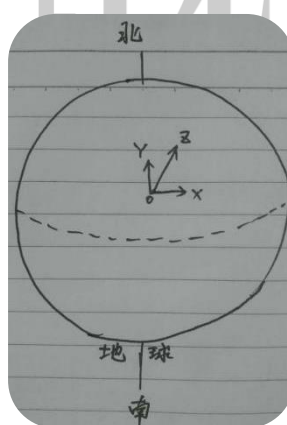


图 6-10 地球上的机体坐标系

我们在地球上如图 6-10 所示的地方，对一个物体建立直角坐标系。它的 x 轴对应着这里的正东方，它的 y 轴对应着正北方，它的 z 轴一直指向天空。我们现在沿着正东方一直走，现在用线将这个路径画下来。见下图 6-11

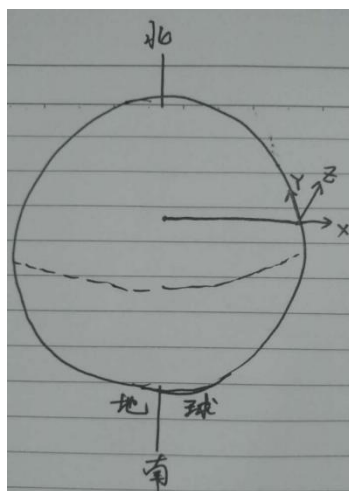


图 6-11 沿着 x 轴前进的机体路径

可以看到，虽然一直向着机体的 x 轴一直前行，但是已经偏离了东方的距离了。我们再来把地球摊开来，看的更直观一点儿。如图 6-12 所示。

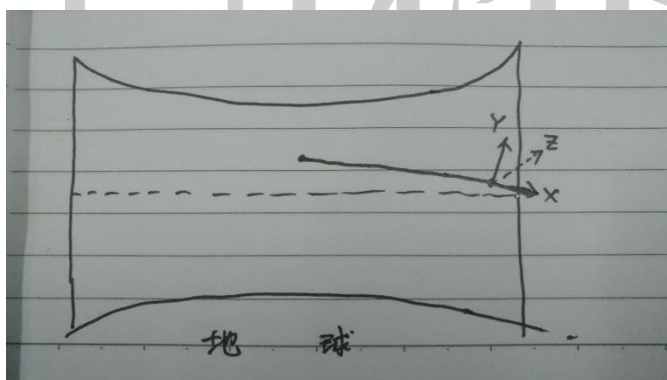


图 6-12 摊开的地球中路径的位置

所以说，我们在计算姿态的时候是不能只考虑物体坐标系的，必须要将物体坐标系转化为世界坐标系，找到误差的值，并纠正，才能走在正东的方向。下面，我们就来看一下程序是如何实现的吧：

```

void AHRSupdate(float gx, float gy, float gz, float ax, float ay, float az, float mx, float
my, float mz)//根据函数后缀可以知道，这个函数需要 9 轴数据的测量值，而函数最后会返回 yaw,
pitch, roll 三个值。

{

    float norm, halfT; //归一数值

    float hx, hy, hz, bz, by;          //测量值

    float vx, vy, vz, wx, wy, wz;      //预测值

    float ex, ey, ez;

    float Pitch, Roll, Yaw; //欧拉角

```

定义四元数需要计算的数值。

```

float q0q0 = q0*q0;

float q0q1 = q0*q1;

float q0q2 = q0*q2;

float q0q3 = q0*q3;

float q1q1 = q1*q1;

float q1q2 = q1*q2;

float q1q3 = q1*q3;

float q2q2 = q2*q2;

float q2q3 = q2*q3;

float q3q3 = q3*q3;

```

预先计算好四元数计算时需要的数据，这里 q0, q1, q2, q3 是通过测量值加速度计算出来的欧拉角转化为四元数所得，在得到加速度计算出的初始姿态角后，可以通过陀螺仪和磁力计来补偿四元数的计算误差。

```

norm = invSqrt(ax*ax + ay*ay + az*az);

ax = ax * norm;

ay = ay * norm;

az = az * norm;

norm = invSqrt(mx*mx + my*my + mz*mz);

mx = mx * norm;

```

```
my = my * norm;
mz = mz * norm;
```

通过以上的步骤可以把数据单位向量化，便于坐标系转换。为什么要做坐标系转换呢？在捷联惯导中，需要的数值不仅有测量值的加速度，陀螺仪和磁力计，还需要重力和地磁场的数值。前三者是机体坐标系中的数据，反应的是机体的姿态和转动角度。后两者是地理坐标系需要的数据，反映了机体在真正的地理位置中所处的位置。所以通过他们之间数据的相互转换来找到机体和地理数据之间的误差，就可以通过这个误差来修正机体的欧拉角。注意，这里使用了测量值的加速度计和磁力计的数据。所以 $axyz$ 和 $mxyz$ 都是机体数据。

```
hx = 2*mx*(0.5 - q2q2 - q3q3) + 2*my*(q1q2 - q0q3) + 2*mz*(q1q3 + q0q2);
hy = 2*mx*(q1q2 + q0q3) + 2*my*(0.5 - q1q1 - q3q3) + 2*mz*(q2q3 - q0q1);
hz = 2*mx*(q1q3 - q0q2) + 2*my*(q2q3 + q0q1) + 2*mz*(0.5 - q1q1 - q2q2);
```

此处 hx , hy 和 hz 是将 mx , my 和 mz 的机体坐标系转换到世界坐标系上得到的数据。公式的出处见上一章余弦矩阵和四元数转换处：

$$A = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

如上，第一列的数据是机体坐标系在世界坐标系上 x 轴的投影，第二列是 y 轴，第三列是 z 轴。所以，现在大家就能理解 $hxyz$ 的公式了吧。公式中还有个问题：为什么是 $0.5 - q_2q_2 - q_3q_3$, $0.5 - q_1q_1 - q_3q_3$, $0.5 - q_1q_1 - q_2q_2$ ，这个 0.5 是哪里来的？那是因为四元数的计算还有条件： $q_0q_0 + q_1q_1 + q_2q_2 + q_3q_3 = 1$ 。这里得到的 $hxyz$ 就是测量地磁场转成的世界坐标系三轴数值。

但是我们知道，地理坐标系中和地球磁场是南北方向，而东西方向是没有磁场的。如图 6-9 所示：

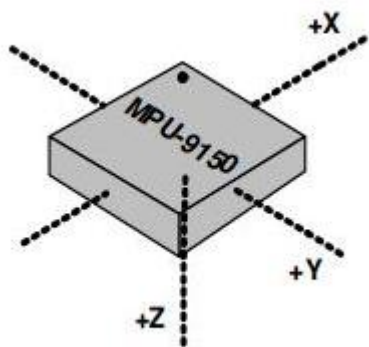


图 6-9 MPU9250 的磁场轴方向规定

这是 MPU9150 中磁力计的规定方向，我们设 y 方向是北方，那么理论上来说 x 方向应该没有磁场，也就是说 x 应该为 0. z 方向还是不变。也就是 (0, 1, x) 所以我们就有：（以下 bxyz 是地理磁场表达式）

$$(b_x * b_x) + (b_y * b_y) = ((h_x * h_x) + (h_y * h_y))$$

因为 b_x 东西方向磁场为 0，所以公式可以简化为：
 $(b_y * b_y) = ((h_x * h_x) + (h_y * h_y))$

所以得到： $b_y^2 = ((h_x * h_x) + (h_y * h_y))$ 。如果设定的是 x 方向为北方那么，那么就是 $b_x^2 = ((h_x * h_x) + (h_y * h_y))$ 。

有人会问 b_z 去哪了？地球上每一处都有竖直方向上的水平分量，但是由于各种不可控因素，所以我们不得而知准确数据，也就没法利用这个做融合补偿（参见第一章），所以此处直接将测量值搬过来。则有：

```
bz=hz

by = sqrtf((hx*hx) + (hy*hy));

bz = hz;
```

在得到世界坐标系的数据后怎么去补偿融合呢？我们此处使用的方法就是将现在这个世界坐标系的数值转换回机体坐标系，这就是机体坐标系通过磁力计获得的三轴数据。那么我们把加速度算得的数据换算成机体坐标系的数据，我们就有了 2 个机体坐标系的数据。而他们的数据肯定是不同的，这样，他们的差

值不就是误差吗？所以此处还是利用四元数的姿态余弦矩阵：

$$A = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

和之前一样：第一列是 x 轴，第二列是 y 轴，第三列是 z 轴。但是不同的是，知道了地理坐标系中地磁 b_x 为 0。现在我们的公式里面含 b_x 项的数据就可以省略掉了，那么现在就可以得到：（wxyz 是地理磁场推算出的机体坐标系）

```
wx = 2*by*(q1q2 + q0q3) + 2*bz*(q1q3 - q0q2);
wy = 2*by*(0.5 - q1q1 - q3q3) + 2*bz*(q0q1 + q2q3);
wz = 2*by*(q2q3 - q0q1) + 2*bz*(0.5 - q1q1 - q2q2);
```

同理，加速度的理想值，当机体不动的时候（将实时的状态看作静止），物体仅受重力的影响。那么，不难得到（0，0，1）。

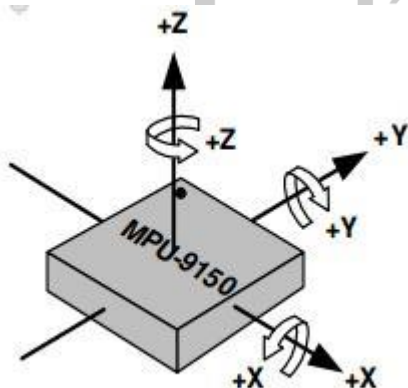


图 6-10 MPU9250 的陀螺仪旋转方向规定

也就是说，物体只受到了重力的影响。那么 x 方向，y 方向都为 0，只有 z 方向是 1。那么按照同理的方法再代入到四元数矩阵。

$$A = \begin{bmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}$$

把带 a_x (为 0) 和 a_y (为 0) 的省略掉, 剩下的 a_z 可得: (v_{xyz} 是地理坐标系重力转换为机体加速度坐标系的三轴数据)

```
vx = 2*(q1q3 - q0q2);  
vy = 2*(q0q1 + q2q3);  
vz = q0q0 - q1q1 - q2q2 + q3q3;
```

到这为止, 理想值 (v_{xyz} , w_{xyz}) 和测量值 (b_{yz} , a_{xyz}) 都计算出来了, 这些都是向量值。向量之间的误差, 我们用向量叉积来计算。这里就有加速度的叉积和磁力计的叉积。分别将计算出来的误差再相加, 就得出总的误差值 (e_{xyz} 就是叉积):

```
ex = (ay*vz - az*vy) + (my*wz - mz*wy);  
ey = (az*vx - ax*vz) + (mz*wx - mx*wz);  
ez = (ax*vy - ay*vx) + (mx*wy - my*wx);
```

此处, 计算出来的 e_{xyz} 也是机体上的误差, 而我们要用这个误差去补偿陀螺仪测量值, 而陀螺仪也是机体坐标系, 所以我们可以用这个数据去补偿陀螺仪。

简单讲一下叉积的问题, 两个向量的误差, 怎么算? 我们在高中学过: 误差值 $= A \times B = |A| |B| \sin(\alpha)$, 当 $\alpha = 90^\circ$ 时, 是没有误差的, 误差值为 0。所以得到的 e_{xyz} 是一个角度值, 就是误差的角度。

```
halfT=GET_NOWTIME(); //姿态更新的半周期
```

这里是程序读取姿态的间隔时间, 比如 MCU 每 10ms 读取一次传感器, 那么 $halfT=5ms$ 。

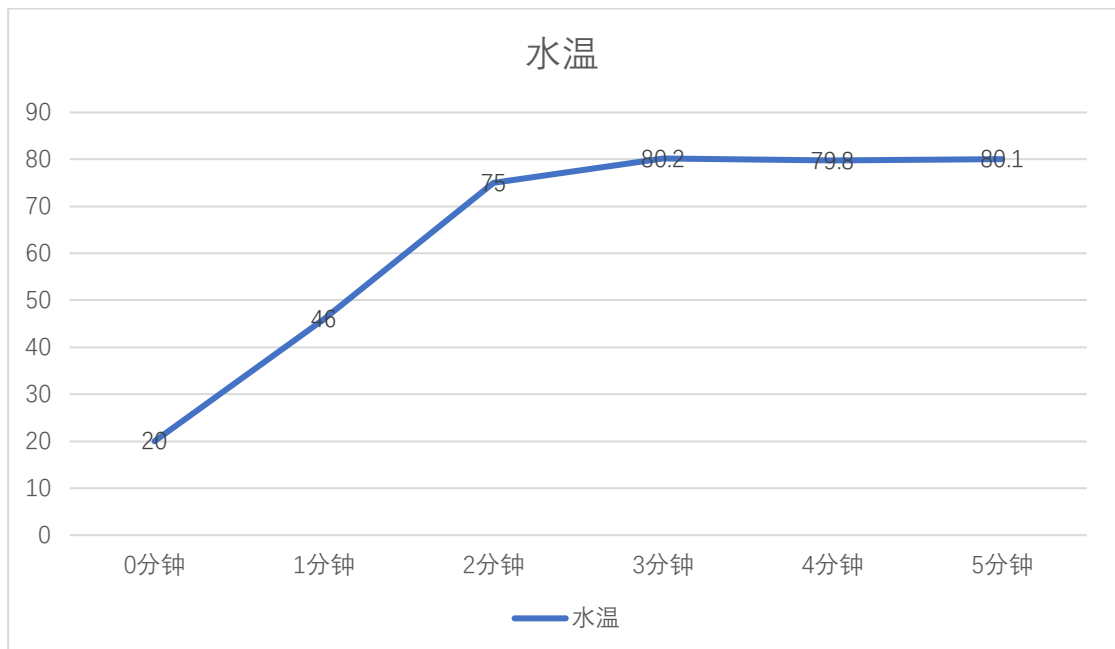
```
if(ex != 0.0f && ey != 0.0f && ez != 0.0f) //当误差不为 0 时执行  
{  
/* 以下对 exyz 误差值进行积分 */  
    exInt = exInt + ex*Ki * halfT;  
    eyInt = eyInt + ey*Ki * halfT;  
    ezInt = ezInt + ez*Ki * halfT;  
/* 以下对陀螺仪进行 PI 修正 */  
    gx = gx + Kp*ex + exInt;
```

```
gy = gy + Kp*ey + eyInt;  
  
gz = gz + Kp*ez + ezInt;  
  
}
```

这个地方使用到了 PID 的调节，我们可以通过控制 K_p 和 K_i 数值大小进行调节加速度和磁力计对陀螺仪的补偿速度。

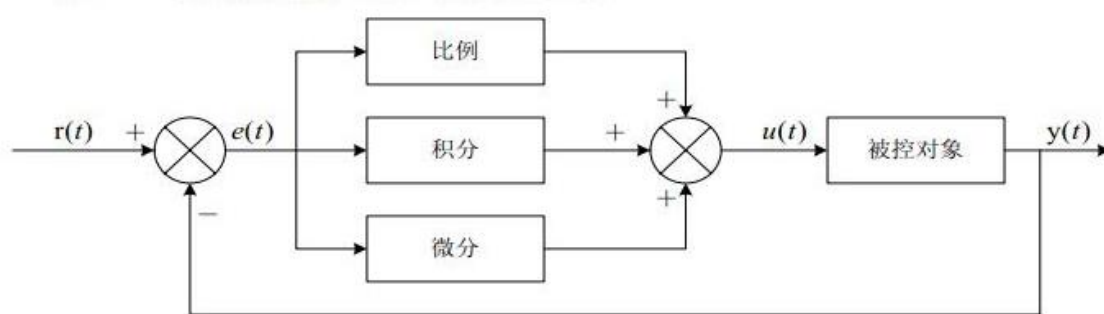
那么 PID 又是干什么的呢？下面举一个简单的例子来讲解一下 PID 的使用环境。

以烧水为例子。现在有口锅，底下用煤气灶来燃烧，锅内烧着水。现在把火打开，火焰的温度大概在 500°C 左右。现在我们用一个温度计放在锅内的水中，要求是锅内的水的温度准确控制在 80 摄氏度整。所以为了保持锅内水的温度精准的在 80 摄氏度，火苗不能是一直在加热状态的。那么怎么去考虑开关火？是不是要在看到温度计快到 80 的时候就去关火。因为关火后不是马上水就凉了，余温会将锅内温度升直 80 度，也有可能稍微超出点儿，当看到余温要结束了，温度趋于下降的时候，再把火打开。让余温再回来，使锅内的温度再要趋于升高到 80 度。就这样，会变成一个来回再 80 度附近徘徊的线。上例中调节开关火时间和火苗大小就是我们要调节的 PID。



现在来回顾一下刚才例子中用到了哪些东西。第一个是温度计给我们的反馈

信号，我们通过实时观察这个反馈的温度信号来决定什么时候关火的。第二个，就是对于温度升高速率的判断意识。比如升温的速度大概再 $2^{\circ}\text{C}/\text{s}$ ，升温速度停止大概在 5 秒内就可以停止，此时就可以觉得在 71 摄氏度左右的时候就可以关火了。因为 $2*5=10$ 度，也就是说关火后大概温度还会升高 10 度，但不可能最后一秒突然就没有温度升高了，所以可以在 71 摄氏度的时候关火，给它一个预留的时间。同理，降温的时候开火也可以这么判断。像这样控制通过反馈数值实时调节的方式就是 PID，PID 的数据通常要自己调节的，因为在实际的生活过程中，数据都不是恒定的，特别是这种精准控制的东西。比如我买的两口锅来烧水，都不一定加热的速度一模一样。所以，要想精准控制这些就需要单独调节 PID 的数值。下面来看一下 PID 调节的思路：



那么调节的东西是哪些呢？比例，微分和积分。也就是 K_p , K_i 和 T_d 。公式如下：

$$u(t) = K_p \left[e(t) + \frac{1}{T_i} \int_0^t e(t) dt + T_d \frac{de(t)}{dt} \right]$$

其中： K_p —— 控制器的比例系数
 T_i —— 控制器的积分时间，也称积分系数
 T_d —— 控制器的微分时间，也称微分系数

PID 的具体推导我们就不在这里说了，感兴趣的话请大家自行翻阅相关书籍来学习（自动控制方面）。我们在这里说一下怎么使用，以程序中调节陀螺仪角度为例：

$exInt = exInt + ex * Ki * halfT$ 和 $gx = gx + Kp * ex + exInt$ 合并在一起就变成了

$$g_x = g_x + K_p * e_x + e_x \text{Int} + e_x * K_i * \text{halfT}$$

就是 $g_x = g_x + \text{原数} + \text{比例} + \text{积分}$ ；可以通过调节 K_P 和 K_I 来调节陀螺仪数据。
怎么调节呢？先来看看他们的功能：

（1）比例系数 K_p

① 对动态性能的影响 比例系数 K_p 加大，使系统的动作灵敏，速度加快， K_p 偏大，振荡次数加多，调节时间加长。当 K_p 太大时，系统会趋于不稳定，若 K_p 太小，又会使系统的动作缓慢；

② 对稳态性能的影响 加大比例系数 K_p ，在系统稳定的情况下，可以减小静差，提高控制精度，但是加大 K_p 只是减少静差，不能完全消除。

（2）积分系数 K_i

① 对动态性能的影响 积分系数 K_i 通常使系统的稳定性下降。 K_i 太大，系统将不稳定； K_i 偏大，振荡次数较多； K_i 太小，对系统性能的影响减少；而当 K_i 合适时，过渡特性比较理想；

② 对稳态性能的影响 积分系数能消除系统的静差，提高控制系统的控制精度。但是若 K_i 太小时，积分作用太弱，以致不能减小静差。

（3）微分系数 K_d

微分控制可以改善动态特性，如超调量减少，调节时间缩短，允许加大比例控制，使静差减小，提高控制精度。但当 K_d 偏大或偏小时，超调量较大，调节时间较长，只有合适的时候，才可以得到比较满意的过渡过程。

当然， K_P, K_I, K_D 的值不是无限大的，它也是有一定范围的。通常在调节数据的时候都是遵从比例—微分—积分的顺序来调节的。先调节比例系数 K_P ，从小到大，其余两个置 0。看看当达到什么数值的时候现象反应差不多快要达到自己的理想值。比如烧水，我们调节火苗大小，比例系数 K_P 就是火苗火焰的大小的比例值，当 K_P 为 1 的时候我们调节火苗就是一次大一倍，当 K_P 为 10 的时候就是一次大 10 倍。当我们觉得差不多的时候开始调节积分数据 K_I ，比如 K_I 是调节火焰变化的速率，此时要稍微减少 K_P 的值才行。最好是之前的 0.8-0.9 左

右。然后 KP 从一个很小的值开始调节，注意，这个时候应该系统要趋于平稳了。所以 KP 的数据不宜过大。当调节好了 KP 以后还是不满意，可以再加上微分调节，原理同上。这个地方的 KP 和 KI 在自己调节的时候一般范围不宜过大，KP 也就在 $1\sim 7$ 之间（有时候也在 $1\sim 12$ 之间），KI 就在 $0.001\sim 0.8$ 之间。

通过对陀螺仪数据的 PID 补偿，我们就得到了一个比较准确的陀螺仪数值了。接下来就可以利用陀螺仪的数据对四元数进行更新了。四元数更新的算法是之前没有讲过的，叫作 Runge-Kunta 法（一阶龙库法），是四元数 q_0 , q_1 , q_2 和 q_3 的一种更新方法，原理和推导步骤就不讲了。具体程序如下：

```
q0 = q0 + (-q1*gx - q2*gy - q3*gz)*halfT;
q1 = q1 + (q0*gx + q2*gz - q3*gy)*halfT;
q2 = q2 + (q0*gy - q1*gz + q3*gx)*halfT;
q3 = q3 + (q0*gz + q1*gy - q2*gx)*halfT;
```

更新完毕后，得到的四元数就是转动后的新角度四元数了，现在还是将四元数归一化，以便计算欧拉角。

```
norm = invSqrt(q0*q0 + q1*q1 + q2*q2 + q3*q3);
q0 = q0 * norm;      //w
q1 = q1 * norm;      //x
q2 = q2 * norm;      //y
q3 = q3 * norm;      //z
```

然后便可代入公式，求出 yaw, pitch 和 roll 的角度。

```
Yaw = -atan2(2*q1*q2 - 2*q0*q3, -2 * q1 * q1 - 2 * q3 * q3 + 1) * 57.3;

if(Yaw < 0 ) {Yaw = Yaw + 360;}

if(Yaw > 360 ) {Yaw = Yaw - 360;}
```

这里是对 yaw 角的实际值做纠正，输出正常范围的结果。

```
Pitch = asin(2*q2*q3 + 2*q0*q1) * 57.3;
Roll  = -atan2(-2*q0*q2 + 2*q1*q3, -2 * q1 * q1 - 2 * q2* q2 + 1) * 57.3;
```

这里的 57.3 度是弧度转角度的数值， $180/\pi=57.3^\circ$ ，得出的欧拉角为角度数据。

```
printf("Yaw=%f, Pitch=%f, Roll=%f \n\r", Yaw, Pitch, Roll);//通过串口打印结果  
}
```

至此，姿态融合的部分就结束了，通过这样的方式不但避开了万向节死锁的问题，而且对数据的融合补偿达到了最大化利用，大家可以利用算出来的欧拉角上传到上位机表示出现在机体的姿态角了。但是美中不足的是这个程序融合了加速度，陀螺仪和磁力计的数据，所以当旁边有磁场干扰的时候还是会对数据有干扰，比如电池，变压器附近等，那么如果干扰大的话就调低磁力计的占重比。不过还有一种融合模式，它只用了加速度和陀螺仪的数值（适用于 MPU6050），叫作 IMU 融合算法。参见附录 A 中 A18—IMU 姿态融合算法。

积分运算的问题就出在时间长了它会产生漂移，所以要想进行更进一步稳定的运算我们还需要借助其他的传感器。比如气压，比如 GPS 等。根据不同的使用情况决定。下一章，我们就来探讨在不同的应用场景中，我们应该如何调教我们的姿态传感器。

第7章 应用

经过前几章的探讨我们学习了姿态传感器 MPU9250 开发所需要的理论，硬件，算法等各种知识，本章就将这些知识全部汇总起来，一起来做姿态传感器的开发和应用。

首先，根据前面所学的知识我们来写一个总程序，将 MPU9250 的功能全部发挥出来。那么它需要满足：芯片配置→数据读取→数据滤波→计算欧拉角→欧拉角更新转换为四元数→姿态融合→结果输出等这一系列功能。

7.1 总程序

为了大家方便理解，作者以 Arduino 程序作为讲解例程：（此例程由于使用了自测功能，所以没有再加入卡尔曼滤波程序）

```
void setup()    //配置设置
{
    Wire.begin();    //串口初始化
    Serial.begin(38400); //串口波特率配置
    delay(1000); //等待 1000ms MCU 反应
    byte c=readByte(MPU9250_ADDRESS, WHO_AM_I_MPU9250); //c 是 MPU9250 的地址
    if (c > 0) //如果成功读取到传感器地址，证明 MCU 和传感器连接成功
    {
        MPU9250SelfTest(SelfTest); //传感器自测，读取测量值和出厂值的误差，关于传感器自检，请参考第 3 章
        calibrateMPU9250(gyroBias, accelBias); //将加速度和陀螺仪的平均值做偏移矫正，方法参见第 4 章
        delay(1000); //等待 MCU 响应
        initMPU9250(); //初始化 MPU9250, 关闭休眠模式，加速度陀螺仪全范围测量，采样率为 1kHz，开启 I2C 旁路模式，参见第 2 章
        byte d = readByte(AK8963_ADDRESS, WHO_AM_I_AK8963); //d 为磁力计地址
        initAK8963(magCalibration); //初始化磁力计，配置有效位并读取校准磁力计
```

```
}
}
```

以上是 MPU9250 的程序配置部分，接下来为程序循环计算部分。用于姿态数据更新。

```
void loop() //用于程序循环更新数据使用，相当于一个大的 while(1) 循环
{
    if (readByte(MPU9250_ADDRESS, INT_STATUS) & 0x01) //如果读取到数据更新完毕发送中断信号后执行
    {
        readAccelData(accelCount); //读取加速度计数值，参见第 4 章
        getAres(); //获取范围
        ax = (float)accelCount[0]*aRes; // - accelBias[0]; //转换成角度并校准
        ay = (float)accelCount[1]*aRes; // - accelBias[1];
        az = (float)accelCount[2]*aRes; // - accelBias[2];
        readGyroData(gyroCount); //读陀螺仪数据
        getGres(); //获取范围
        gx = (float)gyroCount[0]*gRes; //转化成角度值
        gy = (float)gyroCount[1]*gRes;
        gz = (float)gyroCount[2]*gRes;
        readMagData(magCount); //读磁力计数据
        getMres(); //获取范围
        magbias[0] = +470; //矫正数据
        magbias[1] = +120;
        magbias[2] = +125;
        mx = (float)magCount[0]*mRes*magCalibration[0] - magbias[0]; //转换数值及调教
        my = (float)magCount[1]*mRes*magCalibration[1] - magbias[1];
        mz = (float)magCount[2]*mRes*magCalibration[2] - magbias[2];
    }
    Now = micros(); //记录当前时间
```

```

deltat = ((Now - lastUpdate)/1000000.0f); //计算数据更新时间, 参见第 6 章

lastUpdate = Now;

sum += deltat;

sumCount++;

MadgwickQuaternionUpdate(ax, ay, az, gx*PI/180.0f, gy*PI/180.0f, gz*PI/180.0f, my, mx, mz);
//一阶卡库龙法姿态融合四元数, 第 6 章内容

yaw = atan2(2.0f * (q[1] * q[2] + q[0] * q[3]), q[0] * q[0] + q[1] * q[1] - q[2] * q[2] - q[3] * q[3]); //将融合后的四元数换算成欧拉角, 第 5 章内容

pitch = -asin(2.0f * (q[1] * q[3] - q[0] * q[2]));

roll = atan2(2.0f * (q[0] * q[1] + q[2] * q[3]), q[0] * q[0] - q[1] * q[1] - q[2] * q[2] + q[3] * q[3]);

pitch *= 180.0f / PI; //将欧拉角转换成弧度值, 第 5 章内容

yaw *= 180.0f / PI;

roll *= 180.0f / PI;

}

```

现在我们就得到了欧拉角和四元数的值了, 再把这些数据发送给上位机端口, 就可以在电脑上显示芯片的姿态了。

因为, 使用了上位机进行传输, 那么此处就需要自己规定一套传输协议, 以免传输错误。因此有:

```

send_out(rpy, 3, 0x45); //将计算出来的欧拉角上传至上位机-roll,
pitch, yaw

```

```

send_out(Q, 4, 0x65); //将计算更新出的四元数上传至上位机--q0~q3

```

注意这里的 0x45 和 0x65 是我们使用的上位机相应数值的地址。(上位机和具体程序下载参见附录二) 而发送的格式为:

```

void send_out(int16_t *data, uint8_t length, uint8_t send)
{
    uint8_t TX_DATA[20], i=0, k=0;

    memset(TX_DATA, 0, (2*length+5)); //清零缓存器 TX_DATA

    TX_DATA[i++] = 0x5A; //帧头

```

```

TX_DATA[i++]=0x5A; //帧头

TX_DATA[i++]=send; //发送命令

TX_DATA[i++]=2*length; //发送数据的个数

for(k=0;k<length;k++) //将数据存入缓存 TX_DATA 中
{
    TX_DATA[i++]=(uint16_t) data[k]>>8;

    TX_DATA[i++]=(uint16_t) data[k];
}

USART_Send(TX_DATA, 2*length+5);
}

```

当上位机接收到数据后，通过上位机对欧拉角和四元数的分析，将芯片姿态呈现在用户眼前。（上位机在附件中有下载）如图 7-1 所示：

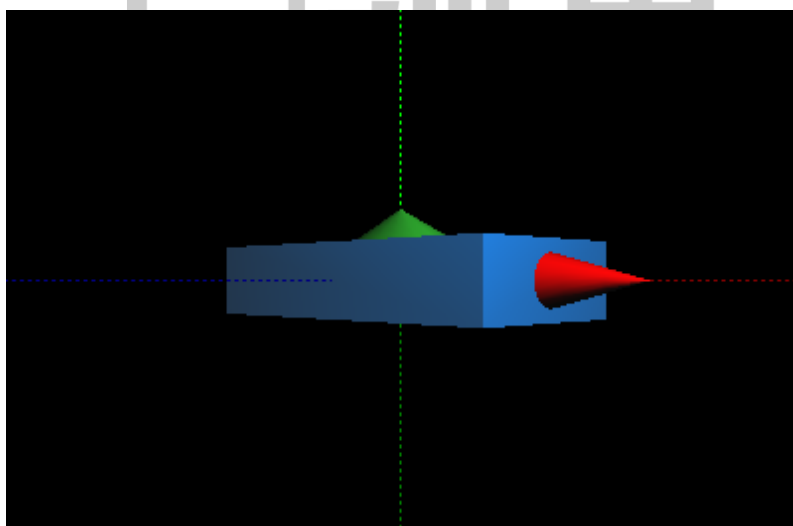


图 7-1 上位机显示的当前姿态

现在为止，大家对姿态传感器的认识应该达到了入门的阶段了。作者有多年做机器人的经验了，其实不管是做机器人还是做智能硬件产品，归根结底都是在调试传感器，通过你所知道的各种算法，谁把传感器的数据调节的最好，谁就能做出最好的产品。下面我就以当前行业中使用比较多的一些智能硬件为例程，简单给大家讲解一下姿态传感器调试开发的思路。

7.2 平衡车



图 7-2 小米 9 号平衡车

平衡车的发展自从出台规定不让上路后就开始走下滑路线。但是我相信凡是用过平衡车的同学应该还是会觉得平衡车是个不错的代步工具，除了太重不太好携带以外。我们先来看看平衡车实现了哪些难点？

首先，既然是平衡车，他就一定要平衡。也就是说他要保证竖直向上的这个轴要一直处于与地面垂直的状态，这是自动的，不管你是刚上电启动的时候，还是站下平衡车的时候。也就是说能让平衡车在无外部压力的情况下保持静止竖直的站在地面上。

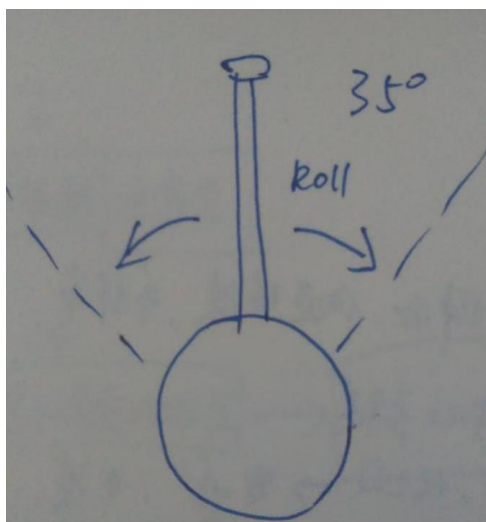


图 7-3 平衡车侧视角度分析图

平衡车在设计的时候有一个绝对倾角值，当角度大于这么多时，平衡车就不再好平衡了。这就可以是 Roll 角。当我们站上平衡车的时候，其实身体是摇晃的。这个时候平衡车一定要做到能抵消和克服这种晃来晃去的力。以保持平衡车的稳定，这样使用者站上去才不会倒。要做到的也就是平衡 Roll 这个角的数值在一个范围内，如图 7-4 所示。

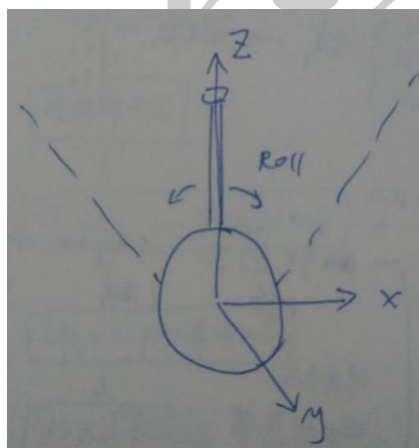


图 7-4 平衡车侧面角度倾斜分析

平衡车基本功能：可以随意加速和减速。我们站在平衡车上时，当然不可能用遥控器来控制。所以，为了更加方便地控制平衡车，代步车设计的通过检测踏板上姿态传感器的姿态来控制平衡车的稳定加速减速，甚至倒退。比如你想前进，那么你就可以将身体稍稍前倾，此时人的重心向前，脚连着踏板，脚也稍稍前倾，指尖处向下压，那么踏板的前部也开始下倾。这样，捕捉到姿态传感器的姿态变

化，平衡车为了平衡姿态，所以它要向前进，以补偿下倾的角度。这样，就完成了平衡车的前进程序。当然，后退左倾右倾都是同理。



图 7-5 平衡车

平衡车左转右转的功能，有些是用杆子控制的（比如图 7-2 中的小米平衡车 9 号），有些是还是用的压力传感器控制的（见图 7-5）。

根据上面的描述，我们不难猜测到，在平衡车里只需要有加速度和陀螺仪传感器就足够了。加速度传感器刚好可以检测当前平衡车的姿态和状态，而且平衡车的倾倒方向只有前后两种可能。（图 7-4）所以，只需要检测出平衡车倾倒的方向即可。

现在我们就给大家分析一段开源平衡小车的程序。来了解一下在平衡车中，姿态传感器是怎么使用的。

```
void setup() {  
  
    Wire.begin();  
  
    Serial.begin(9600);  
  
    //电机配置  
  
    pinMode(IN1, OUTPUT);  
  
    pinMode(IN2, OUTPUT);  
  
    pinMode(IN3, OUTPUT);  
}
```

```
pinMode(IN4, OUTPUT);
```

```
pinMode(ENA, OUTPUT);
```

```
pinMode(ENB, OUTPUT);
```

以上部分是打开串口，配置波特率和电机的 IO 口。因为是平衡车，所以有两个电机需要控制，一个电机最少需要两个引脚控制（伺服电机，一个控制 PWM 波，一个控制顺时针或逆时针旋转方向）。我们需要把测得的姿态角度转化为电机的 PWM 波，来控制电机的前进后退，以便带动平衡车的行进方向。

```
TWBR = ((F_CPU / 400000L) - 16) / 2; //设置 I2C 速率
```

```
i2cData[0] = 7; //设置采样率为 1000Hz
```

```
i2cData[1] = 0x00; //设置加速度计和陀螺仪的采样率
```

```
i2cData[2] = 0x00; // 设置陀螺仪的测量范围，250° /s
```

```
i2cData[3] = 0x00; // 设置加速度的采样范围，2G
```

```
while (i2cWrite(0x19, i2cData, 4, false)); // 配置进去
```

```
while (i2cWrite(0x6B, 0x01, true)); // PLL 模式，关闭休眠
```

```
while (i2cRead(0x75, i2cData, 1));
```

```
if (i2cData[0] != 0x68) { //读取传感器地址，如果读取失败，串口显示读取传感器失败
```

```
Serial.print(F("Error reading sensor"));
```

```
while (1);
```

```
}
```

确认芯片是否地址寻址正确。

```
delay(20); // 给芯片反应时间
```

以上的步骤则是配置芯片的初始化及其数据输出模式。这些在第三，四章的

讲过。

```
while (i2cRead(0x3B, i2cData, 6));  
  
accX = (i2cData[0] << 8) | i2cData[1];  
  
accY = (i2cData[2] << 8) | i2cData[3];  
  
accZ = (i2cData[4] << 8) | i2cData[5];
```

读取加速度的值，并合并。不做单位转换。

```
double roll = atan2(accX, accZ) * RAD_TO_DEG;
```

计算 roll 角度的数据，并转换为角度值。这里要搞清楚的是，因为平衡车的转弯方向有且只有两个方向，所以我们只需要一个欧拉角的姿态就可以了。应该是 pitch，但此处用 roll，并且计算的方式是求取 x 和 z 轴之间的夹角。所以不难猜到他是以 x 为正方向设定。如图 7-6 所示，所以 roll 的角度反应的是平衡车与数直角度的夹角。

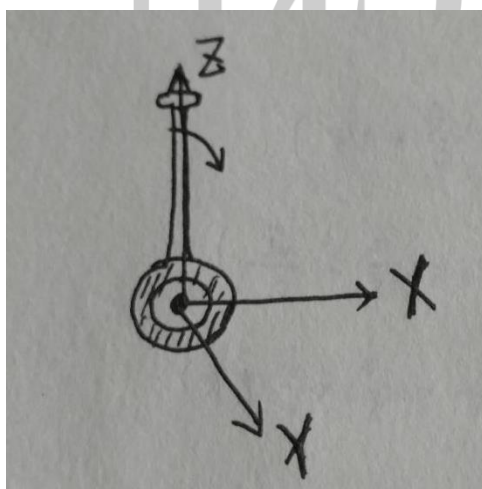


图 7-6 平衡车受力分析图

```
k1=43;  
  
k2=1.4;  
  
k3=-0.88 ;  
  
k4=-0.55;
```

以上设定的是调节平衡的PID的数值，PID在第6章讲过。

```
EnCoderInit(); //开启中断

timer = micros(); //记录程序运行时间

}
```

以上部分为配置部分，单片机的配置和芯片的配置，接口的配置的传输方式的配置。以及得到了roll角度的初始化数据，以便更新周期的数据。

```
void loop()

{

    control(); //控制命令
```

通过串口来控制平衡车前进，后退或停止。

```
while (i2cRead(0x3B, i2cData, 14));
```

while的这种写法的意思是直到读到14个数据后才执行下一步。但是鉴于我们是用在平衡车中，所以并不是所有的数据都需要。而是只需要三个数值：

```
accX = ((i2cData[0] << 8) | i2cData[1]);

accZ = ((i2cData[4] << 8) | i2cData[5]);

gyroY = (i2cData[10] << 8) | i2cData[11];
```

以上为读取的加速度XZ是为了计算roll方向的姿态倾斜角，而陀螺仪Y刚好表示的是陀螺仪的roll转角。

```
double dt=(double)(micros()-timer)/ 1000000; //计算时差

timer = micros(); //记录当前时间

double roll = atan2(accX, accZ) * RAD_TO_DEG-move_flag;
```

根据测得的加速度数据计算当前的roll角度值，并转换为弧度制并补偿。

```
double gyroYrate = -gyroY / 131.0; // Convert to deg/s
```

计算出陀螺仪的角速度值，转换成弧度值。

```
Kalman_Filter(roll, gyroYrate);
```

卡尔曼滤波通过陀螺仪数据来更新 roll 的数据。得到较准确的 roll 角度。

```
if(abs(angle)<35){
```

当角度大于 35° 的时候，人不可能站稳，所以在大于 35° 时，角度值都是不予以考虑的。如果不是倒了，要不然就是数据出错了。

```
pwm_calculate();
```

根据 roll 的数值，通过 PID 来调节控制轮子电机的速率 pwm 波

```
PWMD();
```

根据刚才计算出来的 PWM 波（控制电机的方波），进一步判断并控制电机运行的方向。这里面就需要 PID 调节。

```
Serial.print("pwm:");//串口输出 pwm 波数据
```

```
Serial.print(pwm); Serial.print("\n");
```

```
}
```

```
else{
```

```
    analogWrite(ENB, 0); //PWM 调速 a==0-255
```

```
    analogWrite(ENA, 0 ); //PWM 调速 a==0-255
```

```
}
```

当大于 35° 时，平衡车不动。

```
delay(2);
```

以上为串口打印 pwm 数值以辅助 PID 数据调节。

```
}
```

所以在平衡车的开发上，我们只需要考虑一个轴的角速度和欧拉角就可以平

稳的控制平衡车了。

7.3 智能手环



智能手环的主要功能在记步。现在微信和微博上都有记录步数的功能。那么我们就需要分析一下手环戴在手上时，走路，跑步，坐下，静止时手部位置不同的变化。这个数据量要想精准还要做大量的实际测试实验，根据测量的数据做分析。至于检测睡眠质量的功能，人的睡眠分为浅度睡眠，中度睡眠和深度睡眠。手环通过检测你的睡姿以及心跳来判断你当前的休眠情况。

综上所述，智能手环里面需要的传感器也就加速度计和陀螺仪，以及心率检测传感器。

看一下手环的设计结构：

我们先来看一下戴在人手上的手环平时的位置和可能的情况：（如图 7-7 所示）

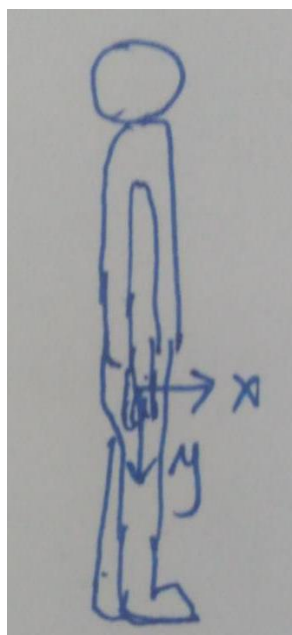


图 7-7 人手佩戴智能手环的坐标建立

如上图 7-7 所示，一般手环都是戴在人手臂的这个位置。所以在我们平时走路的时候，手臂会自然下垂。手环的姿态传感器中，至少有一个轴是差不多垂直于地面的。

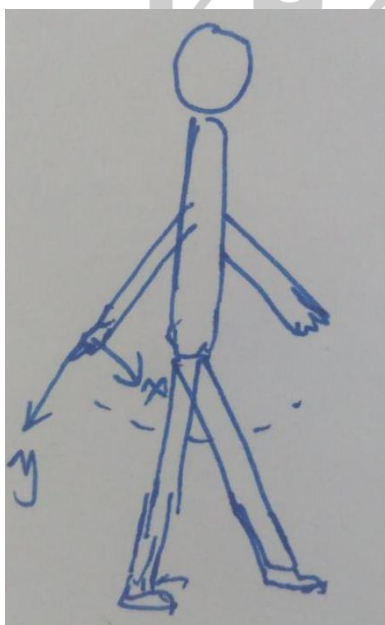


图 7-8 人行走时坐标变化

所以，垂直于地面的可能是 x 轴或 y 轴中任何一个。根据芯片在手环中摆放位置确定（如图 7-8 所示，设定为 y 轴向下），那么来分析一下，人在正常走路

的时候，陀螺仪计会怎么样。

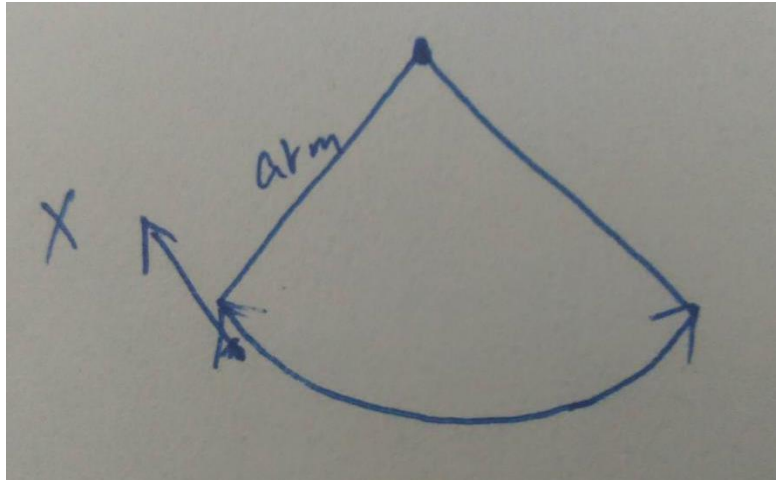


图 7-8 简化的分析图

按照图中的假设坐标来看。人在走路的时候，手臂是一个弧度摆。在一定的角度内做来回绕 z 轴摆动。

不难发现， x 轴的方向一直是弧线运动轨迹的切线。所以，理论上来说， z 轴加速度的值，变化的范围应该是最大的。这里有一组实测的表，显示了人在正常走路的时候，手环检测到的加速度计三轴的数据。

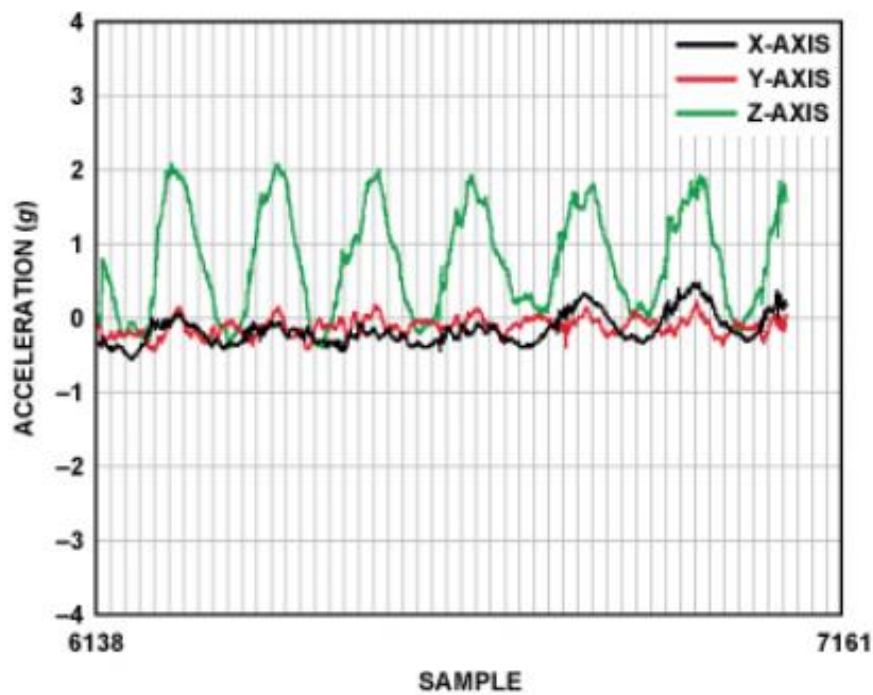


图 7-9 人走路加速度三轴实测

我们可以看出其中数据的曲线变化，不管怎么走。其中 z 轴的数据成周期性变化。当我们把手往后甩的时候，加速度是减小的，然后就像钟表一样，当手臂回到垂直位置的时候反而是最快的。这样的话测得的数据就是一个近似的正弦值。一次来回摆动的图像就应该是一高一低，也就是一起一伏，这就是人走一步的记录。正常人一步的距离差不多是 80cm 到 100cm 左右，就可以根据这个计算走路的里程。我们还可以根据数据的最大值和最小值判断走路的快慢，走路的速度等等。

如上所说，智能手环的开发需要大量的实验数据，这里就不掰开讲解了。

7.4 智能手机

自从乔布斯推出 iPhone 以后，MEMS 的姿态传感器就开始走向繁荣富强，特别是应美盛公司。第一次演示带磁力计功能的手机时，乔布斯再舞台上给大家展示了一个搭积木的游戏。当他拿着手机开始原地转动的时候，手机游戏里面的结构体也开始转动，当时这一个演示惊艳了多少人。让大家开始了解到手机里面的这颗磁力传感器是多么的重要。

这些年来，姿态传感器在智能手机里扮演着举足轻重的角色。从赛车游戏，到搭积木，再到运动记录，甚至现在的 VR 和 AR。更有厂商将其做到了人性化的地步。比如拿起唤醒，平躺熄屏，甚至当传感器检测到 iPhone 在掉落过程中，驱动马达震动，让手机背部着地而不至于损坏屏幕的功能。可谓是在用户体验上，真的是不怕做不到，就怕想不到。

关于智能手机的应用太多，也不是很难，只是需要大量实验数据，所以我们在这里就不分析了。因为下面我们要讲解比这些更复杂的飞行器。

7.5 飞行器



图 7-10 四旋翼飞行器

飞行器是个暴利行业。无人机市场和自动驾驶一样都是科技行业的新宠儿，覆盖量巨大，延伸到各行各业。例如亚马逊就把无人机当快递员一样进行快递的投放工作，又快又方便。在农业上，无人机可以喷洒农药，更精细，不像滑翔机会导致大量的浪费还有农药不均等问题。甚至俄罗斯将机枪和飞行器合二为一，美国用飞行器（只有指甲盖那么大）执行侦察任务等等。不管怎么样，技术是要用于造出来造福于人类的，而不是用来自相残杀的。

我们在这里给大家闲聊一下无人机的硬件结构和程序。

无人机的原理其实很简单，但是要想控制很好却没那么容易。这里拿四轴飞行器来举例说明。

如图 7-10 所示，四轴飞行器。包含正反螺旋桨，四颗无刷马达，四个电子调速器（简称电调），一块锂电池和飞行控制板（传感器就在其中）组成。

飞行器最重要的还是机身，因为机身的长短，材质，大小等直接决定了螺旋桨的长度，从而间接影响到马达的选择，这样又会决定负载的重量大小。不同的材质会改变机身震动的频率。飞行器的飞行，靠的是螺旋桨提供的升力，分为两组，旋转方向相反，这个我们稍后讲解。目的是为了达到水平方向的旋转力矩达

到平衡。桨越长，升力就越大，相应的阻力也会变大，因此扭力也就大，马达也就需要越厉害（如图 7-11 所示，线圈的匝数，和电压以及绕线圈的方式都会对马达产生影响，匝数越少，扭力越小，转速越快）。



图 7-11 电机

电子调速器，简称电调。如图 7-12 所示。



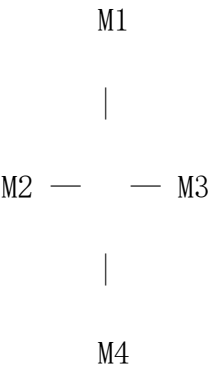
图 7-12 电调

这东西就好比伺服电机里面的控制器，它里面自带驱动电路。所以只需要输入 PWM 波就可以控制无刷马达。PWM 全称脉冲宽度调变技术（Pulse Width Modulation）。锂电池使用大电压 11.1V，2200mAh 即可。

飞控板的配置：微控制器（MCU）2 个（一个主控，一个计算姿态），MPU9250（包含加速度计，陀螺仪和磁力计），气压计，GPS，无线模块（无线传输信号，方便与地面基站做协调 250Kbps 差不多由 1Km 的传输距离），SD 模块（存储数

据）。

飞行器无非就是控制四旋翼的平稳起飞，飞行和降落。说白了就是控制四个电机合理旋转的过程。



按照上面的排列方式，我们看这四个电机的排列方向。首先如果要想让这个飞行器起飞，是不是让 m1，m2，m3 和 m4 同时朝同一个方向运动？答案是否定的。你们想想，当四个轴的方向都朝着同一个方向的时候会怎么样？答案是会向着同一个方向不停地旋转。所以，为了克服掉这个相互作用力，就需要让不是对角的电机转向不同来平衡掉这个扭力。

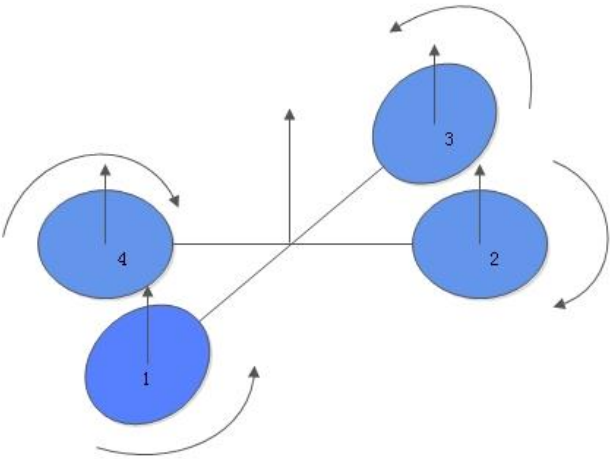


图 7-13 飞控旋翼的正确飞行方向

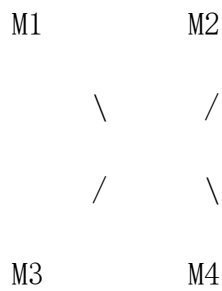
要想控制好飞行器的话，只能对边控制。也就是说将 m1 和 m4 看作一组，将 m2 和 m3 看作一组来控制。现在你来推断一下要想让飞行器做俯仰（pitch）运动，怎么办？

假设绕着 m2 和 m3 这根轴运动的轴就是 pitch 角，那么我们是应该保持

m2 和 m3 电机的速度不变，而去改变 m1 和 m4 的电机的转速呢。当我们把 m1 加速，m4 减速的时候，飞行器会向内部转动。而我们把 m1 减速，m4 加速的时候，飞行器会向外转动。同理我们不难判断出来：

	M1	M2	M3	M4	(H 为 Hold 保持)
正俯仰	+		H	H	-
反俯仰	-		H	H	+
正横滚	H	+		-	H
反横滚	H	-		+	H
顺航向	+		-	-	+
反航向	-		+	+	-
上升	+	+	+	+	
下降	-	-	-	-	

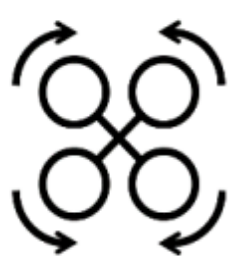
如果你细心的话，就会发现我们平时在市面上看到卖的四轴飞行器飞行姿态并不是按照横杆的平行方向飞的（十字方向）。而是和这个方向再倾斜 45° 飞的（X 字方向）。也就是这样：



那么这时候的俯仰怎么做呢？还是和刚才一样，只不过，m1 和 m2 这次是一起控制，m3 和 m4 是一组。m1 和 m2 加速，m3 和 m4 减速，飞行器就会向外旋转，也就是上面所说的 pitch 角。同理，我们可以得出：

	M1	M2	M3	M4
正俯仰	+	+	-	-

反俯仰	-	-	+	+
正横滚	-	+	-	+
反横滚	+	-	+	-
顺航向	+	-	-	+
反航向	-	+	+	-
上升	+	+	+	+
下降	-	-	-	-



上升下降



前後左右移動



順逆時針旋轉

图 7-14 四旋翼飞行器的飞行方向

其实，不论是十字的还是X形的，都是可以用的。只不过X要稳定一些，但是十字的要好受力分解一些，可以方便做一些高难度的动作。知道了这些，就可以开始着手控制飞行器了。想让飞行器做横滚角运动，应该怎么办呢？看看四轴中对于单轴的测算：

$$\text{ROLL} = (\text{希望的角度} - \text{当前的角度}) * \text{比例系数} + \text{微分系数} * \text{角速度}$$

希望的角度就是摇杆要控制的角度，玩过无人机或者以前遥控赛车的人都知道，推动摇杆的时候的幅度就是控制机器运动方向的。我们在这里通过摇杆的反馈数值计算来得出希望的角度。比如我希望向前飞一点儿，那么我推摇杆的程度就可以换算成要向前飞行的角度。该飞行多少角度呢？那就是（希望的角度-当前的角度）*比例系数，这个比例系数是通过实验测试出来的。现在这个数据就是要旋转的角度了。那么再加上之前通过陀螺仪测算出来的角度，将其相加在一

起就可以得出现在要转动的 roll 角度。知道角度了以后就可以用来控制电机了。

```
Motor[2] = (int16_t)(Thr - Pitch - Roll - Yaw );           //M3
Motor[0] = (int16_t)(Thr + Pitch + Roll - Yaw );           //M1
Motor[3] = (int16_t)(Thr - Pitch + Roll + Yaw );           //M4
Motor[1] = (int16_t)(Thr + Pitch - Roll + Yaw );           //M2
```

以上是通过单轴的方法去控制飞行器的姿态，那么怎么去精准的控制电机的 PWM 波，使其完美的运行呢？主要是通过 PID 算法。

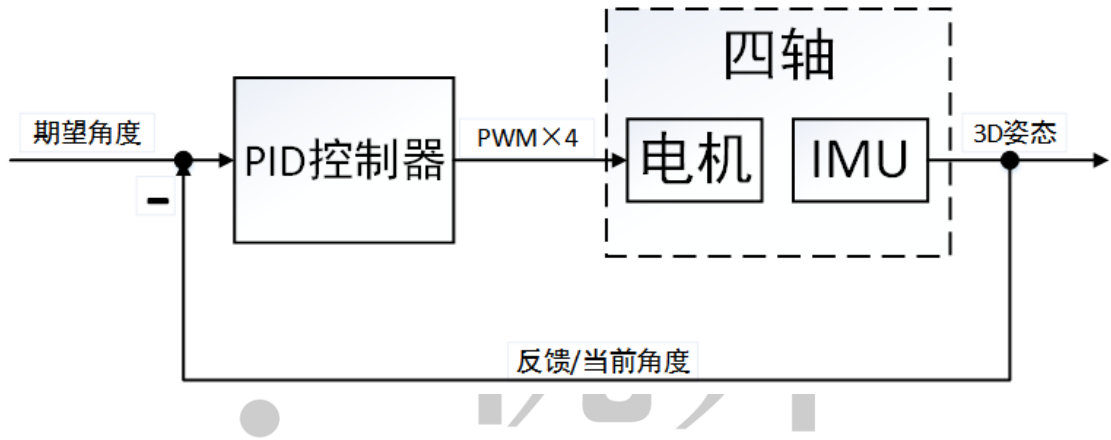


图 7-15 PID 控制反馈图

如上图 7-15 所示，将设备输入的期望角度通过 PID 来转化为电机控制的 PWM 波信号。通过控制 PWM 波信号来控制四轴的四个电机。然后通过反馈回来的姿态角度来实时调节电机的方向。现在我们把以上文字转换为公式：

```
当前角度误差    =    期望角度    -    当前角度
单环PID_P项     =    Kp           *    当前角度误差
当前角度误差积分及其积分限幅
单环PID_I项     =    Ki           *    当前角度误差积分
当前角度的微分
(原理上为当前角度误差-上次角度误差,实际上角度的微分就是角速度,恰好由陀螺仪给出)
单环PID_D项     =    Kd           *    当前角速度的微分(直接用陀螺仪输出)
单环PID_输出    =    单环PID_P项 + 单环PID_I项 + 单环PID_D项p://blog.csdn.net/nemo11990
```

上图去水印

但是这里面我们只考虑了角度的信息，而没有考虑到角速度的信息。所以我们在这里还需要增加角速度的一环。如图 7-16 所示：

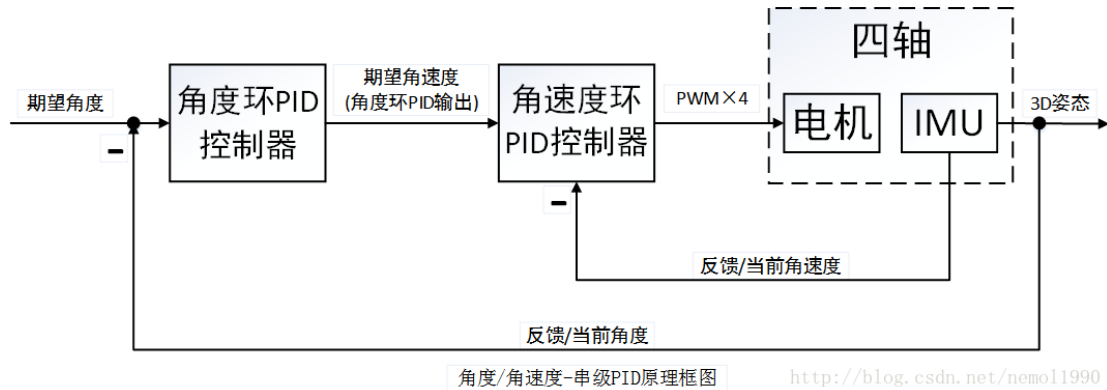


图 7-16 增加角速度的闭环 PID

这就是传说中的串级 PID 控制算法。

$$\begin{aligned} \text{当前角度误差} &= \text{期望角度} - \text{当前角度} \\ \text{外环PID_P项} &= \text{外环Kp} * \text{当前角度误差} \\ \text{当前角度误差积分及其积分限幅} & \\ \text{外环PID_I项} &= \text{外环Ki} * \text{当前角度误差积分} \\ \text{外环PID_输出} &= \text{外环PID_P项} + \text{外环PID_I项} \\ \text{当前角速度误差} &= \text{外环PID_输出} - \text{当前角速度(直接用陀螺仪输出)} \\ \text{内环PID_P项} &= \text{内环Kp} * \text{当前角速度误差} \\ \text{当前角速度误差积分及其积分限幅} & \\ \text{内环PID_I项} &= \text{内环Ki} * \text{当前角速度误差积分} \\ \text{当前角速度的微分(本次角速度误差-上次角速度误差)} & \\ \text{内环PID_D项} &= \text{内环Kd} * \text{当前角速度的微分} \\ \text{内环PID_输出} &= \text{内环PID_P项} + \text{内环PID_I项} + \text{内环PID_D项} \end{aligned}$$

在调节 PID 的时候最好遵守先内环在外环的方式来调节。下面就来根据实时调节 PID 的记录来感受一下调剂 PID 时，飞行器的不同表现：

内环 P：从小到大，拉动四轴越来越困难，越来越感觉到四轴在抵抗你的拉动；到比较大的数值时，四轴自己会高频震动，肉眼可见，此时拉扯它，它会快

速的振荡几下，过几秒钟后稳定；继续增大，不用加人为干扰，自己发散翻机。

特别注意：只有内环 P 的时候，四轴会缓慢地往一个方向下掉，这属于正常现象。这就是系统角速度静差。

内环 I： 前述 PID 原理可以看出，积分只是用来消除静差，因此积分项系数个人觉得没必要弄的很大，因为这样做会降低系统稳定性。从小到大，四轴会定在一个位置不动，不再往下掉；继续增加 I 的值，四轴会不稳定，拉扯一下会自己发散。

特别注意：增加 I 的值，四轴的定角度能力很强，拉动他比较困难，似乎像是在钉钉子一样，但是一旦有强干扰，它就会发散。这是由于积分项太大，拉动一下积分速度快，给的补偿非常大，因此很难拉动，给人一种很稳定的错觉。

内环 D： 这里的微分项 D 为标准的 PID 原理下的微分项，即本次误差-上次误差。在角速度环中的微分就是角加速度，原本四轴的震动就比较强烈，引起陀螺的值变化较大，此时做微分就更容易引入噪声。因此一般在这里可以适当做一些滑动滤波或者 IIR 滤波。从小到大，飞机的性能没有多大改变，只是回中的时候更加平稳；继续增加 D 的值，可以肉眼看到四轴在平衡位置高频震动(或者听到电机发出滋滋的声音)。前述已经说明 D 项属于辅助性项，因此如果机架的震动较大，D 项可以忽略不加。

外环 P： 当内环 PID 全部整定完成后，飞机已经可以稳定在某一位置而不动了。此时内环 P，从小到大，可以明显看到飞机从倾斜位置慢慢回中，用手拉扯它然后放手，它会慢速回中，达到平衡位置；继续增大 P 的值，用遥控器给不同的角度给定，可以看到飞机跟踪的速度和响应越来越快；继续增加 P 的值，飞机变得十分敏感，机动性能越来越强，有发散的趋势。

我们现在来看一下外环控制的代码，以下部分注解和代码来源于。（推荐一下爱无人机网，虽然没有合作，但是是一个干货很多的好网站）建议大家看一下，对于 PID 的调节会有更多心得。

(<http://bbs.loveuav.com/forum.php?mod=viewthread&tid=42>)

```

void ANO_FlyControl::Attitude_Outter_Loop(void)
{
    int32_t errorAngle[2];
    Vector3f Gyro_ADC;
    //计算角度误差值, 角度误差值=期望值-此刻姿态值
    //constrain_int32 作用: 32 位整型数限幅, 使其控制输入的最大飞行倾角不大
    于 25 度 (如果控制量比 25 度大, 飞机早就坠毁了)
    //rc.Command[ROLL]: 遥控数据

    //imu.angle.x: 此刻的角度
    //1. 得到轴姿态的角度差 (errorAngle)
    //2. 这个角度差值进行限幅 (constrain_int32) (正负 FLYANGLE_MAX) (限幅必
    须有, 否则剧烈打舵时容易引发震荡) 作为角速度控制器期望值 (target_rate)
    errorAngle[ROLL] = constrain_int32((rc.Command[ROLL] * 2) ,
    -((int)FLYANGLE_MAX), +FLYANGLE_MAX) - imu.angle.x * 10;
    errorAngle[PITCH] = constrain_int32((rc.Command[PITCH] * 2) ,
    -((int)FLYANGLE_MAX), +FLYANGLE_MAX) - imu.angle.y * 10;
    //获取此时陀螺仪上的角速度, 取角速度的四次平均值
    Gyro_ADC = mpu6050.Get_Gyro() / 4;
    /*
    得到外环 PID 输出 (角速度的差值) (实质是相当于内环的 P 比例项) ----->
    3. target_rate 与陀螺仪得到的当前角速度作差, 得到角速度误差 (RateError)
    乘以 kp (外环控制系数 pid[PIDLEVEL] ---> (280, 0, 0, 0)) 得到给内环的 P。
    */
    //横滚 roll: 外环控制。输入为角度, 输出为角速度。RateError[ROLL] 作为内
    环的输入。
    RateError[ROLL] = pid[PIDLEVEL].get_p(errorAngle[ROLL]) - Gyro_ADC.x;
    //Gyro_ADC.x: 陀螺仪 x 轴的值
    //俯仰 pitch: 外环控制。输入为角度, 输出为角速度。RateError[PITCH] 作为
    内环的输入。

```



```
RateError[PITCH] = pid[PIDLEVEL].get_p(errorAngle[PITCH]) -
Gyro_ADC.y;//Gyro_ADC.y:陀螺仪 y 轴的值
/*
```

偏航（Yaw）的控制算法和前两者略有不同，是将打舵量（遥控数据量 rc.Command[YAW]）和角度误差的和作为角速度内环的期望值，

这样可以获得更好的动态响应。角速度内环和横滚与俯仰的控制方法一致，参数（积分限幅值会很小，默认只有万分之8）上有不同。*/

//航向 yaw：外环控制。输入为角度,输出为角速度。 RateError[YAW] 作为内环的输入。

```
RateError[YAW] = ((int32_t)(yawRate) * rc.Command[YAW]) / 32 - Gyro_ADC.z;
//Gyro_ADC.z:陀螺仪 z 轴的值
}
```

内环控制代码

```
//飞行器姿态内环控制：输入为角速度，输出为 PWM 增量
//内环的效果就是：减小 P 比例控制带来的震荡
void ANO_FlyControl::Attitude_Inner_Loop(void)
{
int32_t PIDTerm[3];
//PIDROLL、PIDPITCH、PIDYAW 是枚举类型，也就是 0、1、2，也就是下面的 pid 、
PIDTerm 就是 3 组 PID
for(u8 i=0; i<3;i++)
{
//现象：当油门低于检查值时积分清零，重新积分
//猜测：这里应该是担心飞机没飞起来时就开始有积分，会导致起飞时不稳定
if ((rc.rawData[THROTTLE]) < RC_MINCHECK)
pid.reset_I();
//get_pid 函数：return get_p(error) + get_i(error, dt) + get_d(error,
dt);----->这里实际就是一个完整的 PID
//PID_INNER_LOOP_TIME: 2000us--->0.2ms 积分微分时间，每隔 0.2ms 操作积
```

```

分和微分,RateError 是外环计算的结果（从外环算出）
//得到内环 PID 输出，直接输出转为电机控制量
PIDTerm = pid.get_pid(RateError, PID_INNER_LOOP_TIME);
}
//对 YAW 角继续处理，加入遥控控制
//在 I 值小于限幅值（这个值大概在 5%油门）或者 rate_error 与 i 值异号时将
rate_error 累加到 I 中。
PIDTerm[YAW] = constrain_int32(PIDTerm[YAW], -300,
abs(rc.Command[YAW]), +300 + abs(rc.Command[YAW]));
//PID 输出转为电机控制量
motor.writeMotor(rc.Command[THROTTLE], PIDTerm[ROLL], PIDTerm[PITCH],
PIDTerm[YAW]);
}

```

来看一段实例代码通过调节 PID 的方式控制

```

void Quadrotor_Control(const float Exp_Pitch, const float E
xp_Roll, const float Exp_Yaw)

```

函数用途：四旋翼控制函数，用于 PWM 计算和输出

```

入口：const float Exp_Pitch      期望倾仰角

      const float Exp_Roll       期望横滚角

      const float Exp_Yaw        期望偏航角

```

备注：当前控制环为姿态控制环

```

void Quadrotor_Control(const float Exp_Pitch, const float E
xp_Roll, const float Exp_Yaw)

{

    s16 outputPWM_Pitch, outputPWM_Roll, outputPWM_Yaw;

```

// 这就是得到偏差角度数据的公式，就是上面我们讲的期望值-
当前值

```
g_Attitude_Error.g_Error_Pitch = Exp_Pitch - g_Pitch;
```

```
g_Attitude_Error.g_Error_Roll = Exp_Roll - g_Roll;
```

```
g_Attitude_Error.g_Error_Yaw = Exp_Yaw - g_Yaw;
```

```
// 当控制角度 Roll 或者 Pitch 超过 55° 时，放弃这次控制
```

```
if (fabs(g_Attitude_Error.g_Error_Pitch) >= 55 || fa  
bs(g_Attitude_Error.g_Error_Roll) >= 55)  
{  
    PWM2_LED = 0; //指示灯亮起  
    PWM_Set(0, 0, 0, 0); //停机  
    return ;  
}
```

```
PWM2_LED = 1;
```

```
//指示灯熄灭
```

// 当误差角度值小于 3° 时，判定为基本稳定, 另一种指示灯亮
起

```
if (fabs(g_Attitude_Error.g_Error_Pitch) <= 3 && fab  
s(g_Attitude_Error.g_Error_Roll) <= 3)
```

```
    PWM4_LED = 0;
```

```
else
```

```
    PWM4_LED = 1;
```

```

// 积分运算与积分误差限幅

if (fabs(g_Attitude_Error.g_Error_Pitch) <= 20)    //
当pitch角度在20° 以下时

{    //Pitch

    //累加误差

    g_Attitude_Error.g_ErrorI_Pitch += g_Attitude_E
rror.g_Error_Pitch;

    //限制大小，不能超过最大值

    if (g_Attitude_Error.g_ErrorI_Pitch >= PITCH_I
_MAX)
        g_Attitude_Error.g_ErrorI_Pitch = PITCH
_I_MAX;
    else if (g_Attitude_Error.g_ErrorI_Pitch <= -
PITCH_I_MAX)
        g_Attitude_Error.g_ErrorI_Pitch = -PITCH
_I_MAX;

}

if (fabs(g_Attitude_Error.g_Error_Roll) <= 20)

{    //Roll

    //累加误差

    g_Attitude_Error.g_ErrorI_Roll += g_Attitude_Er
ror.g_Error_Roll;

    //防止误差过大

    if (g_Attitude_Error.g_ErrorI_Roll >= ROLL_I_M

```

AX)

```
g_Attitude_Error.g_ErrorI_Roll = ROLL_I  
_MAX;
```

```
else if (g_Attitude_Error.g_ErrorI_Roll <= -R  
OLL_I_MAX)
```

```
g_Attitude_Error.g_ErrorI_Roll = -ROLL_  
I_MAX;
```

```
}
```

```
// --- PID 运算---
```

// 注意此处的 KD 项使用的是陀螺仪的数据，这样做的好处是可以直接将角速度作为微分值，符合角度增量的效果。

```
outputPWM_Pitch = (s16)(g_PID_Kp * g_Attitude_Error.g  
_Error_Pitch + g_PID_Ki * g_Attitude_Error.g_ErrorI_Pitch - g  
_PID_Kd * g_MPU6050Data_Filter.gyro_x_c);
```

```
outputPWM_Roll = (s16)(g_PID_Kp * g_Attitude_Error.g_  
Error_Roll + g_PID_Ki * g_Attitude_Error.g_ErrorI_Roll - g_PI  
D_Kd * g_MPU6050Data_Filter.gyro_y_c);
```

```
outputPWM_Yaw = (s16)(g_PID_Yaw_Kp * g_Attitude_Error  
.g_Error_Yaw);
```

我们需要重点调节的就是这里的 Kp、Ki 了，在这里还是和之前我们说的一样，先开始调节 Kp，然后再调节 ki。找到差不多的数值，然后再回来调节 Kp 和 Ki，看看还有没有更合适的。注意，这里要说明一点，我们在调节上面东西。我们调的不是飞行器两个杆子是不是平行了，PID 根本不知道关于飞行器杆子事情，我们调的是每个轴的数值，让 roll、pitch 和 yaw 趋于平衡。

```
// --- 给出 PWM 控制量到四个电机-->X 模式控制
```

//然后代入我们之前提到的公式,来将三轴的角度转换为四个电机的 PWM 波,特别注意,这里输出反相了,因为误差是反的。

```
g_motor1_PWM = g_BasePWM + outputPWM_Pitch +  
outputPWM_Roll + outputPWM_Yaw;
```

```
g_motor2_PWM = g_BasePWM - outputPWM_Pitch +  
outputPWM_Roll - outputPWM_Yaw;
```

```
g_motor3_PWM = g_BasePWM - outputPWM_Pitch -  
outputPWM_Roll + outputPWM_Yaw;
```

```
g_motor4_PWM = g_BasePWM + outputPWM_Pitch -  
outputPWM_Roll - outputPWM_Yaw;
```

```
// --- PWM 反向清零,因为没有反转
```

```
if (g_motor1_PWM < 0)  
    g_motor1_PWM = 0;
```

```
if (g_motor2_PWM < 0)  
    g_motor2_PWM = 0;
```

```
if (g_motor3_PWM < 0)  
    g_motor3_PWM = 0;
```

```
if (g_motor4_PWM < 0)  
    g_motor4_PWM = 0;
```

```
// --- PWM 限幅
```

```
if (g_motor1_PWM >= g_MaxPWM)  
    g_motor1_PWM = g_MaxPWM;
```

```
if (g_motor2_PWM >= g_MaxPWM)
```

```

        g_motor2_PWM = g_MaxPWM;

    if (g_motor3_PWM >= g_MaxPWM)

        g_motor3_PWM = g_MaxPWM;

    if (g_motor4_PWM >= g_MaxPWM)

        g_motor4_PWM = g_MaxPWM;

    if (g_Fly_Enable)                                // 允许起飞, 给出
PWM
        PWM_Set(g_motor1_PWM, g_motor2_PWM, g_motor3_PWM, g_motor4_PWM);
    else
        PWM_Set(0, 0, 0, 0);                        // 停机
}

```

控制了 PWM 波后，直接输出给电机，就可以控制无人机转动了。综上所述，我们来总结一下无人机的控制流程：

读取传感器→矫正测量值→转换单位→卡尔曼滤波→姿态融合→平衡控制（PID）

7.6 VR 设备

自从 2015 年以来，AR 和 VR 就像是两匹脱缰的黑马，冲入了科技圈的视野。但是很多人只闻其名不知其实，我在这里就简单地给大家普及一下 VR/AR。

VR 是虚拟现实（Virtual Reality）的缩写，他就是在在一个什么都没有的虚拟世界里建立一个虚假的世界，人们通过戴上头盔，仿佛身临其境的感觉。这种技术就是 VR 技术。现在市面上卖的头盔普遍分为两种。一种是谷歌 GOOGLE Cardboard 还有一种是 HTC VIVE。这两种又有什么区别呢？

其实大家不难发现，从使用方式和价格上这两个东西都是差的天差地别的。GOOGLE Cardboard 是谷歌公司的一个开源项目，用纸做的一个眼镜盒子，里面有两个凸透镜。



图 7-17 谷歌盒子

人们可以将手机放进前方的槽里面，手机上播放左右眼不同的画面，人们通过前方的一组眼镜就可以看到图中的图像，仿佛在同一个场景中一样。这个原理有点儿像多年前的红蓝 3D 眼镜。然后此时你可以左右旋转你的脑袋，手机画面中的图像也会和你的姿势一样去显示相应位置的图像，仿佛置身其中一般。不需要额外的电子硬件，只靠手机内部的姿态传感器就能实现这个功能，就这么一个小小的发明，成本不足 5 美元，却带起了一批又一批的新兴企业，有时候你不得不感叹某些公司的伟大。



图 7-18 HTC VIVE

HTC VIVE 在这个的基础上又新加了功能，因为不管你手机再厉害，也不可能测得出来我有没有向前向后移动吧。HTC VIVE 就可以精准的测出你的位移距离。这样不管是你用它玩游戏还是参观虚拟世界，都将能进一步获得真实感。还有一点就是 HTC VIVE 增加了 2 个手柄，那两个手柄你虽然捏在手上，但是也会出现在 HTC VIVE 的画面中，适用于任何场景的任何地方，有时候可以变成双枪，有时候可以变成羽毛球拍，也可以变成画笔。HTC VIVE 之所以贵，还因为它还自带了屏幕，头盔上配有单独的姿态传感器，所以说成本也不低。VR 现在在硬件上的瓶颈就是续航和屏幕分辨率了。像素太低会导致观看东西有缺乏真实感。而为了解决续航问题，甚至现在开发出了 VR 的续航背包。

AR 则是增强现实技术（Augmented Reality）的简称，顾名思义，在现有的东西上增加没有的东西的技术，就是增强现实技术。



图 7-19 AR 现实增强



图 7-20 AR 的互动

比如现实生活中，我坐在一个桌子前，桌子上什么都没有，但是我通过佩戴相应的设备，可以看到桌子上有一台笔记本电脑，甚至我都可以操作它。现实的墙上什么都没有，但是我可以通过设备看到墙上有一幅画。现实生活中没有女朋友，我可以通过设备看到面前站着个女朋友，并可以和她亲切互动。AR 的领先者几乎是 AR 行业目前的佼佼者，因为研究 AR 的难度要远远高于 VR。微软的 Hololens 应该说是目前引领着整个 AR 行业的发展。



图 7-21 微软 AR 眼镜 HoloLens

外形酷炫，将投影和屏幕合二为一，加上骨传导技术，再加上姿态传感器（加速度，磁力计和陀螺仪），虽说我没有用过，但是估计续航是个问题，所以要期待一下今年高通的 4 米有效的无线充电芯片了。

简单分析一下关于 VR 头盔的应用。头盔在头上旋转的位置其实很有限。可以根据这些姿态来判断位置，也就是说需要加速度计，磁力计和陀螺仪的 9 轴辅助定姿态。但是要想知道头部转动不比机器慢，而且头部转动完全是没有预期，没有预判的，天知道你下一步又会往哪转，所以这样显得陀螺仪优势就不大了。特别是有些人会做一些杂乱无章的扭头姿势，这样陀螺仪积分计算的时候很容易就会积错。所以我们平时在玩 VR 游戏或者看 VR 视屏的时候，很多人都会发现，手机屏幕会自动往一个方向偏，造成不愉快的体验。关于 VR 的调试，我们就不讲了，和飞行器其实大同小异，只是把控制 PWM 改成了显示画面不同的动画罢了。

7.7 磁力控制器

这个东西是我自己设想的。

上面讲了姿态传感器的各种应用，无非都是在姿态上面做文章。总而言之都

是在利用加速度计和陀螺仪的数据来获取刚体当前的姿态。或去保持平衡，或去控制转动。但是大家都没有在磁力计上面下功夫，磁力计总是被当作一个可怜的指南针来使用。但是既然电子罗盘能测出 xyz 方向上不同方向的磁场强度，那么我们不妨就用它来做一个应用。

总是说看不见的东西是最难调节的，磁场就是看不见又确确实实存在的那么一个东西。所以磁场发现的才那么晚。我们之前讲到过在计算 YAW 角度或者磁场角度的时候竖直方向上的数值是不用的。但是，我们想想，既然这个东西能在有磁场靠近的情况下有干扰，那么为什么我们不用这一特性来捕捉周围有强磁干扰的物体呢？

可以把这种磁场设备戴在手指上。靠近手机（因为手机里面有磁力计）或者 MPU9250 传感器，这下，磁力计三个轴都有用了。通过磁力计三个轴的磁场大小我们可以判定出手指距离传感器的位置。

当我们用手指来做手势的时候，通过磁力计就可以算出比划的是什么动作，就可以做出相应的反应。这里的三轴磁力计主要的共组是定位。通过 xyz 轴不同的场强，确定手指的位置（通过向量的方法），这个反而还是比较好计算的。难点在于将测得的位置数据分析比对，然后判断出刚才是什么手势。这个东西，笔者还在研究中，研究成果会公布在微博和相关网站上。

至此，姿态传感器的例用就暂告一段落，但是不要高兴得太早，这仅仅是入门。接下来各位还要继续钻研各种算法来应用传感器，将其开发到你想要的最佳状态。技术是学不完的，所以不在多，而在精。就像人生，要做的事情太多太多，但是留给我们的时间和精力却太少太少。所以重要的不是你会了多少技术，而是你要去做什么！

附录 A 答疑

之前在各个章节向读者提出的问题或者额外补充的知识或者讨论将在这一章得到解答。希望读者们带着追求真理的心去学习和探索。

A.1 傅科是如何设计地球自转装置的

旋转的陀螺第一性质是保持旋转轴不变，如果把陀螺放置在万向支架上，而将基座放在地球上，地球旋转而陀螺的轴不旋转，由于支架会保持陀螺仪位置平衡旋转，所以用不了多久，陀螺相对于支架就会发生变动，这就说明了地球的自转。

后来傅科又加以改进，制作出了傅科摆，这就是我们在天文馆里经常能看到的那个巨大的摆。时刻提醒着人们地球是自西向东自转着。如图 A-1 所示。



图 A-1 科傅摆

A.2 芯片读取数值的不同

根据之前介绍的芯片内部原理不难发现，芯片主要得到的是电压或电容值。而市面上大多数的传感器数据传输就 2 种方式，一种就是电压值或电容值这样的模拟数据，可以通过单片机的模数转换（AD）接口或者 AD 转换芯片来直接读取。比如液位传感器，温湿度传感器等。还有一种就是传感器内部自带计算芯片，它将读取到的模拟量转换成数字量储存在寄存器中，只需要直接读取相应的寄存器就可以了。这样的芯片，得到的数据多半都是被芯片内部处理过。比如滤波和温漂等问题，解决了我们很多麻烦。现今市面上大部分传感器都是是 I2C 或 SPI 接口，有些还有串口，网口之类的。当然看你需要的速度来决定使用的接口。以上的例子中速度是：网口>串口>SPI>I2C。

本书例解的姿态传感器芯片是数字接口，传输口有 I2C 和 SPI 两种。具体使用方法我们将在第 2 章中讲解。

A.3 交变科里奥利力在陀螺仪中的使用

绝大多数 MEMS 陀螺仪依赖于由相互正交的振动和转动引起的交变科里奥利力。振动物体被柔软的弹性结构悬挂在基底之上。整体动力学系统是二维弹性阻尼系统，在这个系统中振动和转动诱导的科里奥利力把正比于角速度的能量转移到传感模式（如图 A-2 所示）。

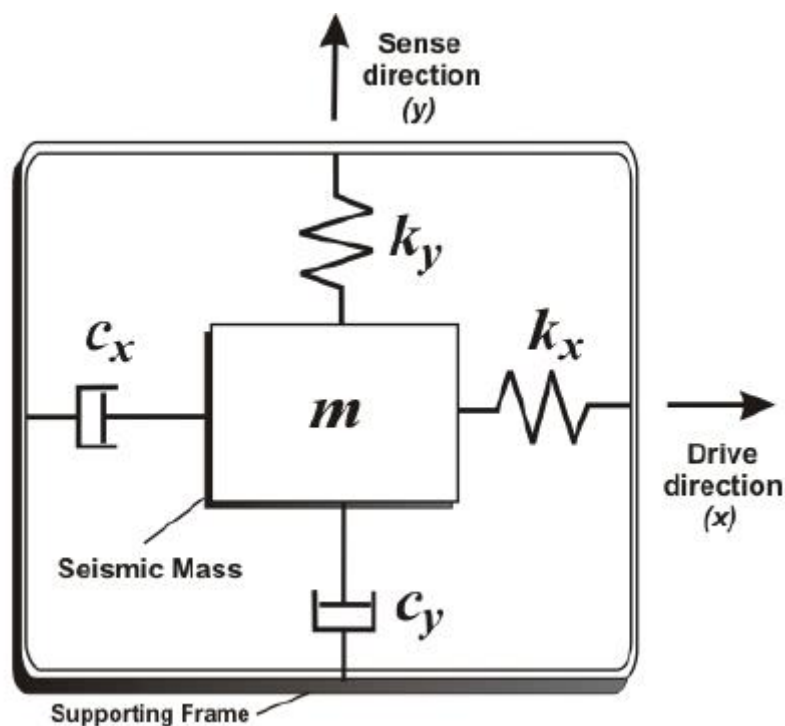


图 A-2 陀螺仪的内部振动

通过改进设计和静电调试使得驱动和传感的共振频率一致，以实现最大可能的能量转移，从而获得最大灵敏度。大多数 MEMS 陀螺仪驱动和传感模式完全匹配或接近匹配，它对系统的振动参数变化极其敏感，而这些系统参数会改变振动的固有频率，因此需要一个好的控制架构来做修正。如果需要高的品质因子(Q)，驱动和感应的频宽必须很窄。增加 1%的频宽可能降低 20%的信号输出。(如图 A-3 所示) 还有阻尼大小也会影响信号输出。(如图 A-4 所示)

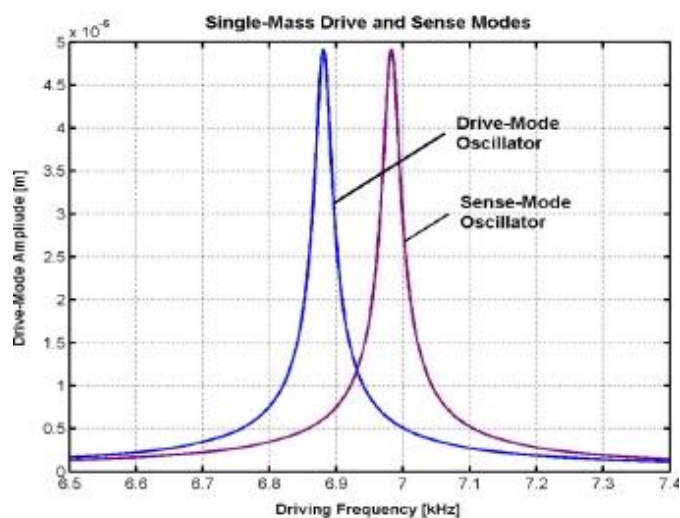


图 A-3 信号与频率的关系

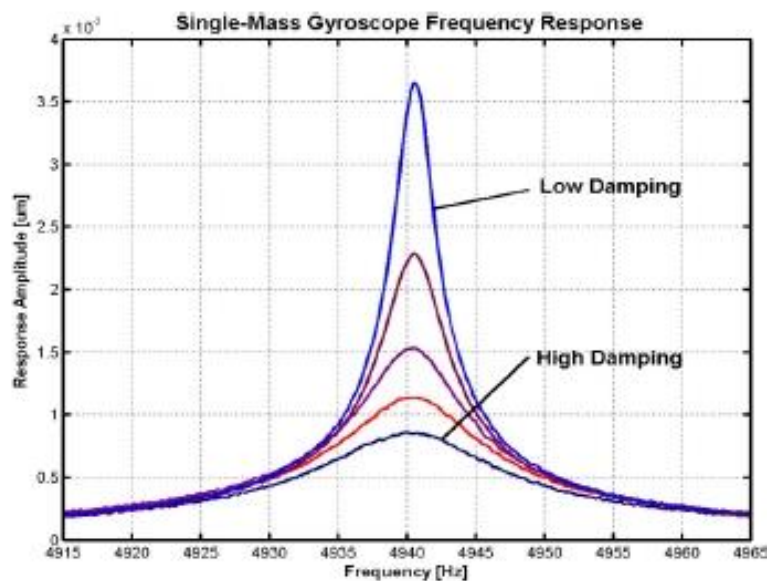


图 A-4 阻尼大小和频率的关系

A. 4 关于特斯拉是否被“神”化

《大象工会》里面有一篇文章讲的是特斯拉的历史功过问题。我还是比较赞同的，这里我把对于特斯拉的一些想法在这里和大家交流一下。关于特斯拉个人的历史故事我就不在这里 copy 百度百科或者维基百科的东西过来了，关于他的资料和纪录片，电影等等相关资料非常多，大家直接在网上就可以查阅。真正值得讨论的地方其实是特斯拉本人的晚年生活。

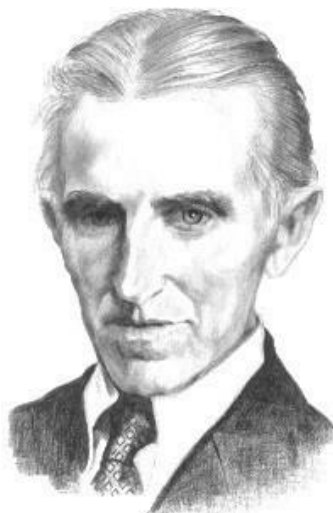


图 A-5 老年特斯拉

为什么特斯拉被神化呢？在特斯拉的晚年，穷困潦倒的特斯拉淡出了人们的视野，仿佛从事着某些神秘的工作。而且他经常会提出一些在当时的人们来看非常匪夷所思的理论。比如无限供电，远距离信号传输，死光等。这样神秘的行踪和超现代化的假设提出，再加上他年轻时做的交流电，大地震荡器等，使得他的行踪变得扑朔迷离，没人知道他干了些什么。世界未解之谜里面有个关于通古斯大爆炸的猜想，有不少人怀疑他是元凶，甚至传言说他可以控制闪电球集中在某处释放能量。

然而关于神化的东西中被证实的有：伦琴发现 X 射线其实还是比他要早；他也并没有建成世界上第一座水力发电站，只是发电站中运用了很多他的专利技术；而且特斯拉也没有发明雷达。最著名的应该属摩根关闭的那个无线电传输塔的事件，被人误传为那是全球供电塔，关闭的原因是因为马可尼率先完成了这个实验。但是虽然如此我们还是不否认他一生 700 多项个人专利的伟大成就，但是在物理基础理论上没有太多的贡献，大部分都是应用物理。所以，根据以上种种，我觉得人们神话他的可能性要更多一些。因为人们总是愿意帮助弱者造势。而且他是真的很厉害，人们宁愿相信他就像自己想象的那样。不管怎样，生活在这 100 年来的人们，都不得不对特斯拉为人了在电力上做的贡献而感谢。

A. 5 为什么磁力计这里的数据是一个圆

我们平时测得的磁场的值是在一个范围内变化的，可能是 x 轴是在 $-20 \sim 30$ 之间 y 轴是 $-30 \sim 20$ 之间（注意这里没有考虑 z 轴是因为我们此时水平放置，无倾斜角，不用考虑 z 轴的值）。那么现在我们得到的数值点会在刚才那段数值之间变化，把那些值投射到平面直角坐标系当中来看的话就是一个圆。但是我们测出的圆并不是理想的位置，因为有干扰，所以我们需要去找现在的圆心位置，然后把测得的值减去我们现在这个偏移的圆心位置就是我们理想的那个圆心了。

A. 6 为什么苹果有 MPU6700 这种买不到型号的芯片

能让芯片厂单独专门定制芯片的必须是大批量的单子，比如批发 5 千万颗芯

片，非常有利可图。在给苹果，三星，索尼这样的大公司做订单的时候，很多厂家都纷纷让出红利，在竞标的时候就会打得头破血流，科技巨头会渔翁得利。

A.7 关于 I2C 读取 10 位地址问题

我们都知道 I2C 设备都有一个 7 位的地址，那么也就是说可以连接 127 种不同的 I2C 设备。但事实上，现有的 I2C 的芯片远远多于这些，所以为了防止在一条总线上会出现同一地址的传感器，很多设备开始使用双重地址——7 位地址加引脚地址。那么现在我们的地址帧则变成两个字节长。所以规定第一个字节前五位最高有效位用作 10 位地址标识，约定是“11110”，如图 A-6 所示。也就是说现在我们发送开始信号后，将开始输出 11110+A9 A8+读写位，然后才是 A7-A0 的地址。

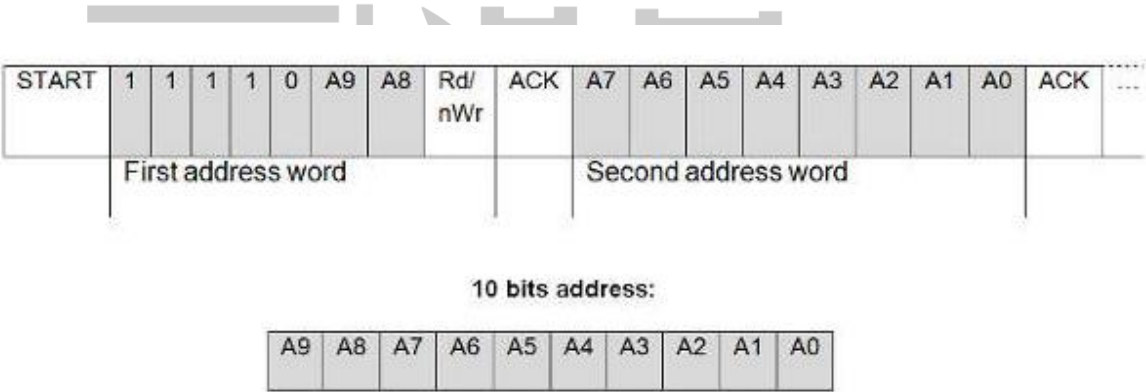


图 A-6 I2C 十位地址时序图

由于现在越来越多的传感器等设备都开始出现 10 位地址位了，所以大家在使用前还是要看清楚。

A.8 关于资料的获取和使用的感悟

随着开源事业的不断升级，我们总能在网上免费得到我们想要的程序等各种资料。其中 github 是一个比较有名气的网站。全球无数的开发者在上面交流并分享自己的程序和设计。

不管怎么样，经过开发者们这么多年不断的努力，很多东西的开发默然变得非常简单。还记得我之前说的硬件模块化么，其实软件的模块化也是如此。比如

我们之前所说的 I2C 和 SPI 通信协议。本来你需要写的程序会是：（以下为 SPI 读一个字节的函数）

```
uint8 SPI_Read_OneByte(void)

{

    uint8 i;

    uint8 temp = 0;

    for(i=0;i<8;i++)

    {

        temp <<= 1;           //读取 MISO 8 次输入的值，存入 temp。
之所以不放在“SCK = 0”语句之后的位置是因为：读取最后 1byte 的最后
一位(即 LSB)之后，不能再左移了

        SCK = 1;

        if(MISO)               //读取最高位，保存至最末尾，通过左
移位完成读整个字节

            temp |= 0x01;

        else

            temp &= ~0x01;

        SCK = 0;               //下降沿来了(SCK 从 1-->0)，MISO 上
的数据将发生改变，稳定后读取存入 temp

    }

    return temp;

}
```

你本来要写这么一大坨，但是现在好了，别人已经帮你写好了。所以只需要下载下来，并在你想要用到的地方写上 `uint8 SPI_Read_OneByte(void)`，当然别忘了添加头文件.h。这么调用函数，既更方便，又缩短了开发周期。何乐而不为呢？如果每个人都把自己的一部分成果与他人分享，我相信这个世界的科技会极速发展。所以我们始终要感谢那些愿意开源的人们。

A.9 关于 QFN 封装

芯片的封装类型有很多，我们这里使用的 MPU9250 是 QFN 封装。他的引脚在地下并没有突出来，这种芯片一般比较脆弱，要用吹焊的方法焊接（手工），而且很容易损坏，也不容易焊下来。所以在设计 QFN 封装的芯片的时候，尽量留出空间，方便日后拆卸调试和研究。

这种封装比较恼火的地方就是拆，一般都用热风枪吹焊下来，但不可长时间高温吹。会有损坏芯片的可能。

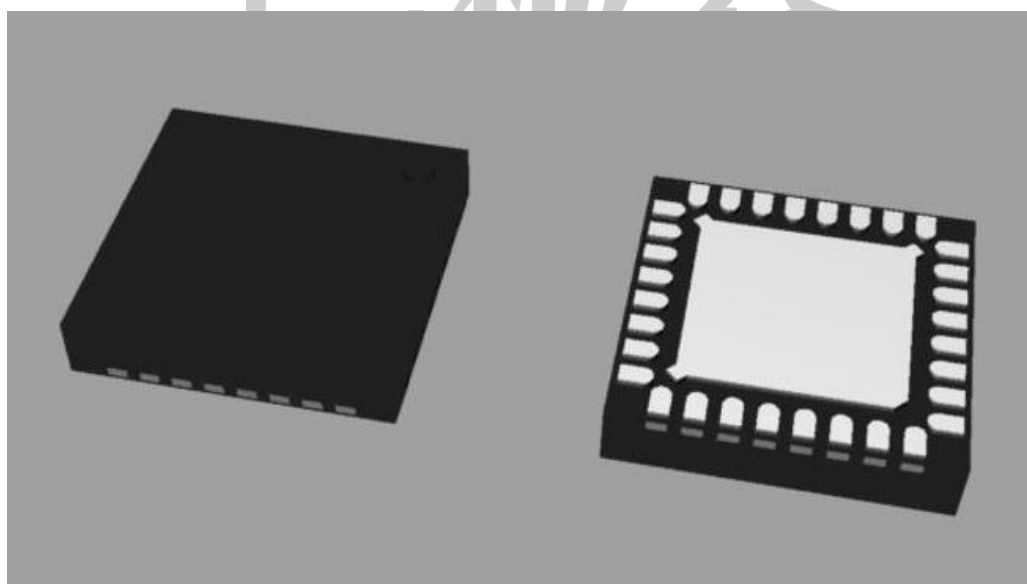


图 A-7 QFN 封装芯片

其他种类的封装这里就不再赘述，感兴趣的可以去网上查找。

A.10 Arduino 的使用和编程

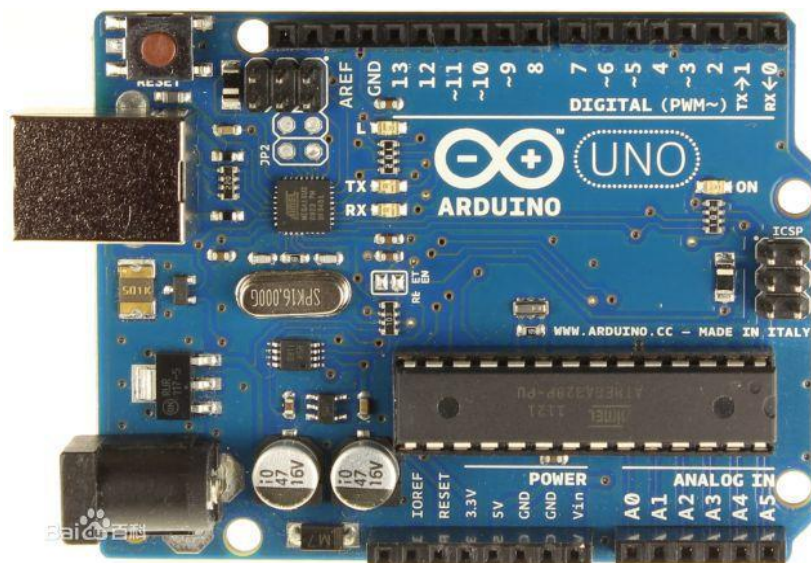


图 A-8 Arduino uno

Arduino 是开源世界非常受欢迎的一种 MCU，非常适合艺术工作者发明者或其他行业的人通过简单的编程来达到一些自己的目的。他最大的优点就是编程简单。让不会的人都能轻易上手。操作简便，一句代码就能顶 STM32 中大段篇幅的代码。感兴趣的同学可以去社区看看：<http://www.arduino.cn/>。

本书中的代码为了大家好理解所以几乎全部使用 arduino 编程，更方便大家移植。下面简单讲一下 arduino 的编程结构：

```
Void setup () {}
```

顾名思义，就是配置部分，这一部分的代码只会在单片机上电时运行一次。以配置好所有的东西，比如姿态传感器的四元数和初始角度值就时在这个步骤得到的。

```
Void loop () {}
```

循环函数，运行完配置函数后，此函数开始循环运行。相当于 while 函数，用在你需要不停更新数据的地方。比如姿态传感器的话我们就需要不停的更新四元数的数据。就可以将四元数更新函数放在这里面。

总体来说就这上面两步的结构，没有什么其他的特别东西，非常容易学习。

A.11 马斯克和 NASA 的 XSPACE 传感器精度

艾隆马斯克一直是美国科技界和新闻界的宠儿，他发明了 paypal，投资了特斯拉，创建了 XSPACE，建立了太阳能公司，可谓是将所有电影里面的未来产业都包揽了。

可是就是这样梦幻般的公司在创业的路上也是步步维艰，作者看过马斯克的传记。他在回忆起自己刚开始做 XSPACE 的时候，是非常不受专业领域信任的，处处受鄙视。就类似当年的维珍航空。NASA 更是不相信民用市场的传感器也能媲美军事产业的传感器。你们可能不相信吧，猎鹰火箭上面搭载的传感器居然也是应美盛的 MPU050 和 MPU9250 这样的传感器。在最初的几年，猎鹰火箭经常发射失败；现在的结果相信大家也看到了，NASA 已经和 XSPACE 签订了合作协议。为什么 NASA 不信任马斯克？就是因为民用市场的传感器精度问题，但马斯克就是认为民用传感器的精度已经达到。这些年，随着姿态传感器飞速发展，他的精度已经今非昔比了。马斯克用实际行动向 NASA 证明了自己判断趋势是多么准确。

A.12 关于 atan () 和 atan2 () 函数

在 C 语言的 math.h 或 C++ 中的 cmath 中有两个求反正切的函数 atan(double x) 与 atan2(double y, double x) 他们返回的值是弧度 要转化为角度再自己处理下。

前者接受的是一个正切值（直线的斜率）得到夹角，但是由于正切的规律性本可以有两个角度的但它却只返回一个，因为 atan 的值域是从 $-90^\circ \sim 90^\circ$ 也就是它只处理一四象限，所以一般不用它。

第二个 atan2(double y, double x) 其中 y 代表已知点的 Y 坐标 同理 x，返回值是此点与远点连线与 x 轴正方向的夹角，这样它就可以处理四个象限的任意情况了，它的值域相应的也就是 $-180^\circ \sim 180^\circ$ 了

例如：

例 1：斜率是 1 的直线的夹角

```
cout<<atan(1.0)*180/PI;//45°
```

```
cout<<atan2(1.0,1.0)*180/PI;//45° 第一象限
```

```
cout<<atan2(-1.0,-1.0)*180/PI;//-135° 第三象限
```

后两个斜率都是 1 但是 atan 只能求出一个 45°

例 2：斜率是-1 的直线的角度

```
cout<<atan(-1.0)*180/PI;//-45°
```

```
cout<<atan2(-1.0,1.0)*180/PI;//-45° y 为负 在第四象限
```

```
cout<<atan2(1.0,-1.0)*180/PI;//135° x 为负 在第二象限
```

常用的不是求过原点的直线的夹角 往往是求一个线段的夹角 这对于 atan2 就更是如鱼得水了

例如求 A(1.0, 1.0) B(3.0, 3.0) 这个线段 AB 与 x 轴正方向的夹角

用 atan2 表示为 atan2(y2-y1, x2-x1) 即 atan2(3.0-1.0, 3.0-1.0)

它的原理就相当于把 A 点平移到原点 B 点相应变成 B' (x2-x1, y2-y1) 点 这样就又回到先前了

例三：

A(0.0, 5.0) B(5.0, 10.0)

线段 AB 的夹角为

```
cout<<atan2(5.0,5.0)*180/PI;//45°
```

A. 13 OFFSET 寄存器的使用

我们这里用一段代码来向大家展示 OFFSET 的使用方法

```
#define XG_OFFSET_H      0x13  //用户定义的偏移量
#define XG_OFFSET_L      0x14
#define YG_OFFSET_H      0x15
```

```

#define YG_OFFSET_L      0x16

#define ZG_OFFSET_H      0x17

#define ZG_OFFSET_L      0x18


#define XA_OFFSET_H      0x77

#define XA_OFFSET_L      0x78

#define YA_OFFSET_H      0x7A

#define YA_OFFSET_L      0x7B

#define ZA_OFFSET_H      0x7D

#define ZA_OFFSET_L      0x7E


//设备初始化后计算加速度和陀螺仪平均值时使用

// 读取复位后将 offset 值写入加速度和陀螺仪寄存器中

void calibrateMPU9250(float * dest1, float * dest2)
{
    uint8_t data[12]; //读取数据保存

    uint16_t ii, packet_count, fifo_count;

    int32_t gyro_bias[3]  = {0, 0, 0}, accel_bias[3] = {0, 0, 0};


    // 重启设备

    writeByte(MPU9250_ADDRESS, PWR_MGMT_1, 0x80);

    delay(100);


    // 自动设置时钟频率, 电源复位

    writeByte(MPU9250_ADDRESS, PWR_MGMT_1, 0x01);

    writeByte(MPU9250_ADDRESS, PWR_MGMT_2, 0x00);

    delay(200);


    // 配置设备偏移量计算寄存器

```



```

writeByte(MPU9250_ADDRESS, INT_ENABLE, 0x00); // 关闭所有中断

writeByte(MPU9250_ADDRESS, FIFO_EN, 0x00); //关闭 FIFO

writeByte(MPU9250_ADDRESS, PWR_MGMT_1, 0x00); //开启内部时钟源

writeByte(MPU9250_ADDRESS, I2C_MST_CTRL, 0x00); //关闭 I2C 主机

writeByte(MPU9250_ADDRESS, USER_CTRL, 0x00); // 关闭 FIFO 和 I2C 主模式

writeByte(MPU9250_ADDRESS, USER_CTRL, 0x0C); // 重置 FIFO 和 DMP

delay(15);

// 配置陀螺仪和加速度计的偏移配置

writeByte(MPU9250_ADDRESS, CONFIG, 0x01); // 运行频率 188 Hz

writeByte(MPU9250_ADDRESS, SMPLRT_DIV, 0x00); //采样率 1 kHz

writeByte(MPU9250_ADDRESS, GYRO_CONFIG, 0x00); // 设置陀螺仪 250 °/s

writeByte(MPU9250_ADDRESS, ACCEL_CONFIG, 0x00); // 设置加速度测量范围为 2 g


uint16_t gyrosensitivity = 131; // 131 LSB/degrees/sec

uint16_t accelsensitivity = 16384; // 16384 LSB/g


//配置 FIFO 捕获加速度计和陀螺仪数计算的偏移数据

writeByte(MPU9250_ADDRESS, USER_CTRL, 0x40); // 使能 FIFO

writeByte(MPU9250_ADDRESS, FIFO_EN, 0x78); // 使能陀螺仪和加速度计的 FIFO （最大为
512 字节）

delay(40); // 等待采集 40 组数据共 480 字节


//采样计算结束后，关闭 FIFO 读取

writeByte(MPU9250_ADDRESS, FIFO_EN, 0x00); // 关闭 FIFO

readBytes(MPU9250_ADDRESS, FIFO_COUNTH, 2, &data[0]); // 读取 FIFO 采样数据

fifo_count = ((uint16_t)data[0] << 8) | data[1];

packet_count = fifo_count/12;// 取平均值


for (ii = 0; ii < packet_count; ii++) {

```

```

    int16_t accel_temp[3] = {0, 0, 0}, gyro_temp[3] = {0, 0, 0};

    readBytes(MPU9250_ADDRESS, FIFO_R_W, 12, &data[0]); // 读取平均值

    accel_temp[0] = (int16_t) (((int16_t)data[0] << 8) | data[1] ); // 合并为 16 位数据
    accel_temp[1] = (int16_t) (((int16_t)data[2] << 8) | data[3] );
    accel_temp[2] = (int16_t) (((int16_t)data[4] << 8) | data[5] );
    gyro_temp[0] = (int16_t) (((int16_t)data[6] << 8) | data[7] );
    gyro_temp[1] = (int16_t) (((int16_t)data[8] << 8) | data[9] );
    gyro_temp[2] = (int16_t) (((int16_t)data[10] << 8) | data[11] );

    accel_bias[0] += (int32_t) accel_temp[0]; // 合并为 32 位数据
    accel_bias[1] += (int32_t) accel_temp[1];
    accel_bias[2] += (int32_t) accel_temp[2];
    gyro_bias[0] += (int32_t) gyro_temp[0];
    gyro_bias[1] += (int32_t) gyro_temp[1];
    gyro_bias[2] += (int32_t) gyro_temp[2];

}

    accel_bias[0] /= (int32_t) packet_count; // 计算总的平均偏移量
    accel_bias[1] /= (int32_t) packet_count;
    accel_bias[2] /= (int32_t) packet_count;
    gyro_bias[0] /= (int32_t) packet_count;
    gyro_bias[1] /= (int32_t) packet_count;
    gyro_bias[2] /= (int32_t) packet_count;

    if(accel_bias[2] > 0L) {accel_bias[2] -= (int32_t) accelsensitivity;} // 将重力移除, 才能判断姿态角度
    else {accel_bias[2] += (int32_t) accelsensitivity;}

// 将陀螺仪偏移量写入陀螺仪偏移寄存器

    data[0] = (-gyro_bias[0]/4 >> 8) & 0xFF; // 分 4 份来适配 32.9 LSB /deg/s 的记录度数

```

```

data[1] = (-gyro_bias[0]/4)      & 0xFF; // 如果读取到偏移值, 那么4等分计算平均值
data[2] = (-gyro_bias[1]/4 >> 8) & 0xFF;
data[3] = (-gyro_bias[1]/4)      & 0xFF;
data[4] = (-gyro_bias[2]/4 >> 8) & 0xFF;
data[5] = (-gyro_bias[2]/4)      & 0xFF;

// 将偏移值写入寄存器

writeByte(MPU9250_ADDRESS, XG_OFFSET_H, data[0]);
writeByte(MPU9250_ADDRESS, XG_OFFSET_L, data[1]);
writeByte(MPU9250_ADDRESS, YG_OFFSET_H, data[2]);
writeByte(MPU9250_ADDRESS, YG_OFFSET_L, data[3]);
writeByte(MPU9250_ADDRESS, ZG_OFFSET_H, data[4]);
writeByte(MPU9250_ADDRESS, ZG_OFFSET_L, data[5]);

//输出陀螺仪的偏移量

dest1[0] = (float) gyro_bias[0]/(float) gyrosensitivity;
dest1[1] = (float) gyro_bias[1]/(float) gyrosensitivity;
dest1[2] = (float) gyro_bias[2]/(float) gyrosensitivity;

int32_t accel_bias_reg[3] = {0, 0, 0}; // 记录出厂时加速度偏移量

readBytes(MPU9250_ADDRESS, XA_OFFSET_H, 2, &data[0]); // 读取工厂加速度计偏移量的数值
accel_bias_reg[0] = (int32_t) (((int16_t)data[0] << 8) | data[1]);

readBytes(MPU9250_ADDRESS, YA_OFFSET_H, 2, &data[0]);
accel_bias_reg[1] = (int32_t) (((int16_t)data[0] << 8) | data[1]);

readBytes(MPU9250_ADDRESS, ZA_OFFSET_H, 2, &data[0]);
accel_bias_reg[2] = (int32_t) (((int16_t)data[0] << 8) | data[1]);

uint32_t mask = 1uL; // 标志数组最低位为温度范围值
uint8_t mask_bit[3] = {0, 0, 0}; //定义标志数组

```

```

for(ii = 0; ii < 3; ii++) {

    if((accel_bias_reg[ii] & mask)) mask_bit[ii] = 0x01; // 如果设置温度位, 在 mask 位记录下来

}

// 计算整个偏移量

accel_bias_reg[0] -= (accel_bias[0]/8); //计算整个加速度的平均值 2048 LSB/g (16 g 全范围取值)

accel_bias_reg[1] -= (accel_bias[1]/8);

accel_bias_reg[2] -= (accel_bias[2]/8);


data[0] = (accel_bias_reg[0] >> 8) & 0xFF;

data[1] = (accel_bias_reg[0])      & 0xFF;

data[1] = data[1] | mask_bit[0]; //将温度补偿数值写入加速度漂移量中

data[2] = (accel_bias_reg[1] >> 8) & 0xFF;

data[3] = (accel_bias_reg[1])      & 0xFF;

data[3] = data[3] | mask_bit[1]; // 记录下温飘值以补偿加速度计

data[4] = (accel_bias_reg[2] >> 8) & 0xFF;

data[5] = (accel_bias_reg[2])      & 0xFF;

data[5] = data[5] | mask_bit[2];

// 其实以上将温漂拿来补偿加速度计的方法在 MPU9250 中是没有效果的

// 将加速度的偏移量写入寄存器

writeByte(MPU9250_ADDRESS, XA_OFFSET_H, data[0]);

writeByte(MPU9250_ADDRESS, XA_OFFSET_L, data[1]);

writeByte(MPU9250_ADDRESS, YA_OFFSET_H, data[2]);

writeByte(MPU9250_ADDRESS, YA_OFFSET_L, data[3]);

writeByte(MPU9250_ADDRESS, ZA_OFFSET_H, data[4]);

writeByte(MPU9250_ADDRESS, ZA_OFFSET_L, data[5]);

// 输出加速度计的偏移值

dest2[0] = (float)accel_bias[0]/(float)accelsensitivity;

dest2[1] = (float)accel_bias[1]/(float)accelsensitivity;

```

```
dest2[2] = (float)accel_bias[2]/(float)accelsensitivity;  
}
```

A. 14 关于复数计算

复数加法:

$$(a_1+b_1i)+(a_2+b_2i)=(a_1+a_2)+(b_1+b_2)i$$

复数减法:

$$(a_1+b_1i)-(a_2+b_2i)=(a_1-a_2)+(b_1-b_2)i$$

复数的系数缩放:

$$\lambda(a_1+b_1i)=\lambda a_1+\lambda b_1i$$

复数的积:

$$z_1=a_1+b_1i$$

$$z_2=a_2+b_2i$$

$$z_1z_2=(a_1+b_1i)(a_2+b_2i)$$

$$=a_1a_2+a_1b_2i+b_1a_2i+b_1b_2i^2$$

$$=(a_1a_2-b_1b_2)+(a_1b_2+b_1a_2)i$$

复数的平方:

$$z=a+bi$$

$$z^2=(a+bi)(a+bi)$$

$$=(a^2-b^2)+2abi$$

共轭复数:

复数的共轭就是指把复数的虚数部分变成负的。共轭复数的符号是 z^- 或 z^* 。

$$z=a+bi$$

$$z^*=a-bi$$

复数和它的共轭复数的乘积是：

$$zz^*=(a+bi)(a-bi)=a^2-abi+abi+b^2=a^2+b^2$$

复数的绝对值：

我们使用共轭复数来计算复数的绝对值：

$$z=a+bi$$

$$|z| = \sqrt{zz^*} = \sqrt{(a+bi)(a-bi)} = \sqrt{a^2+b^2}$$

两复数的商：

$$z_1=a_1+b_1i$$

$$z_2=a_2+b_2i$$

$$\begin{aligned}\frac{z_1}{z_2} &= \frac{a_1 + b_1i}{a_2 + b_2i} = \frac{(a_1 + b_1i)(a_2 - b_2i)}{(a_2 + b_2i)(a_2 - b_2i)} \\ &= \frac{a_1a_2 - a_1b_2i + b_1a_2i - b_1b_2i^2}{a_2^2 + b_2^2} \\ &= \frac{a_1a_2 + b_1b_2}{a_2^2 + b_2^2} + \frac{b_1a_2 - a_1b_2}{a_2^2 + b_2^2}i\end{aligned}$$

A. 15 四元数的计算公式

四元数的加减

和复数类似，四元数也可以被加减：

$$q_a=[s_a,a]$$

$$q_b=[s_b,b]$$

$$q_a+q_b=[s_a+s_b,a+b]$$

$$q_a-q_b=[s_a-s_b,a-b]$$

四元数的积

我们也可以表示四元数的乘积：

$$\begin{aligned}q_a q_b &= [s_a, a][s_b, b] \\&= (s_a + x_a i + y_a j + z_a k)(s_b + x_b i + y_b j + z_b k) \\&= (s_a s_b - x_a x_b - y_a y_b - z_a z_b) + (s_a x_b + s_b x_a + y_a z_b - y_b z_a) i \\&\quad + (s_a y_b + s_b y_a + z_a x_b - z_b x_a) j + (s_a z_b + s_b z_a + x_a y_b - x_b y_a) k\end{aligned}$$

可以看到，四元数的乘积依然还是一个四元数。如果我们把虚数 i、j、k 替换成有序对：

$$i=[0,i] \quad j=[0,j] \quad k=[0,k]$$

以及还有 $[1,0] = 1$ ，将它们代入前面的表达式，就得到了：

$$\begin{aligned}q_a q_b &= (s_a s_b - x_a x_b - y_a y_b - z_a z_b)[1,0] + (s_a x_b + s_b x_a + y_a z_b - y_b z_a)[0,i] \\&\quad + (s_a y_b + s_b y_a + z_a x_b - z_b x_a)[0,j] + (s_a z_b + s_b z_a + x_a y_b - x_b y_a)[0,k]\end{aligned}$$

再把这个表达式扩展成多个有序对的和：

$$\begin{aligned}q_a q_b &= [s_a s_b - x_a x_b - y_a y_b - z_a z_b, 0] + [0, (s_a x_b + s_b x_a + y_a z_b - y_b z_a) i] \\&\quad + [0, (s_a y_b + s_b y_a + z_a x_b - z_b x_a) j] + [0, (s_a z_b + s_b z_a + x_a y_b - x_b y_a) k]\end{aligned}$$

如果把后 3 个四元数相加，并提取公共部分，就可以把等式改写成：

$$\begin{aligned}q_a q_b &= [s_a s_b - x_a x_b - y_a y_b - z_a z_b, 0] + [0, s_a(x_b i + y_b j + z_b k) + s_b(x_a i + y_a j + z_a k) \\&\quad + (y_a z_b - y_b z_a) i + (z_a x_b - z_b x_a) j + (x_a y_b - x_b y_a) k]\end{aligned}$$

这个等式是 2 个有序对的和。第 1 个有序对是一个实四元数，第 2 个是一个纯四元数。这两个四元数也可以合并成一个：

$$\begin{aligned}q_a q_b &= [(s_a s_b - x_a x_b - y_a y_b - z_a z_b) \\&\quad + s_a(x_b i + y_b j + z_b k) + s_b(x_a i + y_a j + z_a k) \\&\quad + (y_a z_b - y_b z_a) i + (z_a x_b - z_b x_a) j + (x_a y_b - x_b y_a) k]\end{aligned}$$

如果把下面的表达式代入上面的等式：

$$a=x_a i+y_a j+z_a k$$

$$b=x_b i+y_b j+z_b k$$

$$a \cdot b=x_a x_b i^2+y_a y_b j^2+z_a z_b k^2$$

$$a \times b=(y_a z_b-y_b z_a) i+(z_a x_b-z_b x_a) j+(x_a y_b-x_b y_a) k$$

注意，第三条和第四条并不是四元数的点积和叉积，而是向量的点积和叉积。

我们就得到了：

$$q_a q_b=[s_a s_b-a \cdot b, s_a b+s_b a+a \times b]$$

这就是四元数乘积的一般式。

实四元数

一个实四元数是一个虚部向量为零向量的四元数：

$$q=[s, 0]$$

两个实四元数的乘积是另一个实四元数：

$$q_a=[s_a, 0]$$

$$q_b=[s_b, 0]$$

$$q_a q_b=[s_a, 0][s_b, 0]=[s_a s_b, 0]$$

这和 2 个虚部为 0 的复数的乘积几乎一样：

$$z_1=a_1+0i$$

$$z_2=a_2+0i$$

$$z_1 z_2=(a_1+0i)(a_2+0i)=a_1 a_2$$

四元数的系数缩放

我们也可以用系数（实数）去乘四元数：

$$q=[s, v]$$

$$\lambda q=\lambda [s, v]=[\lambda s, \lambda v]$$

可以用实四元数与普通四元数的乘积来确认这个等式是否正确：

$$q=[s, v]$$

$$\lambda=[\lambda, 0]$$

$$\lambda q=[\lambda, 0][s, v]=[\lambda s, \lambda v]$$

纯四元数

和实四元数相似，Hamilton 也定义了纯四元数。纯四元数是 $s=0$ 的四元数：

$$q=[0, v]$$

也可以写成下面的形式：

$$q=xi+yk+zk$$

然后是 2 个纯四元数的乘积：

$$q_a=[0, a]$$

$$q_b=[0, b]$$

$$q_a q_b=[0, a][0, b]=[-a \cdot b, a \times b]$$

四元数的加法形式

我们可以把四元数写成实四元数和纯四元数的和：

$$q=[s, v]$$

$$=[s, 0]+[0, v]$$

单位四元数

给定任意的向量 v ，我们可以把这个向量写成一个系数和一个单位方向向量的乘积：

$$v=vv^{\wedge} \quad v=|v|, |v^{\wedge}|=1$$

将这个定义和纯四元数的定义结合，就得到了：

$$q=[0, v]$$

$$=[0, vv^{\wedge}]$$

$$=v[0, v^{\wedge}]$$

然后，我们可以定义单位四元数了，它是一个 $s=0$ 、 vv 为单位向量的四元数：

$$q^{\wedge}=[0, v^{\wedge}]$$

四元数的二元形式

我们现在可以把单位四元数的定义和四元数的加法形式结合到一起，就创造了一种新的四元数的表示法，这种表示法和复数的表示法形似：

$$q=[s, v]$$

$$=[s, 0]+[0, v]$$

$$=[s, 0]+v[0, v^{\wedge}]$$

$$=s+vq^{\wedge}$$

这就给了我们一种和复数非常相似的四元数表示法：

$$z=a+bi$$

$$q=s+vq^{\wedge}$$

共轭四元数

共轭四元数的计算，就是将四元数的虚向量取反：

$$q=[s, v]$$

$$q^*=[s, -v]$$

四元数和它的共轭四元数的乘积：

$$qq^*=[s, v][s, -v]$$

$$=[s^2-v \cdot (-v), -sv+sv+v \times (-v)]$$

$$=[s^2+v \cdot v, 0]$$

$$=[s^2+v^2, 0]$$

四元数范数

回忆下复数范数的定义：

$$|z|=a^2+b^2 \text{-----} \sqrt{|z|=a^2+b^2}$$

$$zz^*=|z|^2$$

类似的，四元数的范数可以这样定义：

$$q=[s, v]$$

$$|q|=s^2+v^2 \text{-----} \sqrt{|q|=s^2+v^2}$$

这也让我们可以这样表达四元数范数：

$$qq^*=|q|^2$$

A. 16 欧拉角、余弦向量、四元数之间的转换

设四元数为 Q，那么：

$$Q = q_0 + q_1i + q_2j + q_3k$$

然后现在给出四元数转换成方向余弦矩阵的公式：

$$\begin{bmatrix} x_b \\ y_b \\ z_b \end{bmatrix} = \begin{bmatrix} q_0^2+q_1^2-q_2^2-q_3^2 & 2(q_1q_2-q_0q_3) & 2(q_1q_3+q_0q_2) \\ 2(q_1q_2+q_0q_3) & q_0^2-q_1^2+q_2^2-q_3^2 & 2(q_2q_3-q_0q_1) \\ 2(q_1q_3-q_0q_2) & 2(q_2q_3+q_0q_1) & q_0^2-q_1^2-q_2^2+q_3^2 \end{bmatrix} \begin{bmatrix} x_n \\ y_n \\ z_n \end{bmatrix},$$

当然，用欧拉角也可以表示余弦矩阵：

$$\begin{bmatrix} \cos\gamma\cos\Psi + \sin\gamma\sin\Psi\sin\theta & -\cos\gamma\sin\Psi + \sin\gamma\cos\Psi\sin\theta & -\sin\gamma\cos\theta \\ \sin\Psi\cos\theta & \cos\Psi\cos\theta & \sin\theta \\ \sin\gamma\cos\Psi - \cos\gamma\sin\Psi\sin\theta & -\sin\gamma\sin\Psi - \cos\gamma\cos\Psi\sin\theta & \cos\gamma\cos\theta \end{bmatrix}$$

用欧拉角表示余弦矩阵：

$$C_n^b = (C_b^n)^T = \begin{bmatrix} T_{11} & T_{21} & T_{31} \\ T_{12} & T_{22} & T_{32} \\ T_{13} & T_{23} & T_{33} \end{bmatrix}$$

记住这个顺序，然后我们可得：

$$\begin{cases} \theta = \arcsin(T_{32}) \\ \gamma_{\pm} = \arctan\left(-\frac{T_{31}}{T_{33}}\right) \\ \Psi_{\pm} = \arctan\left(\frac{T_{12}}{T_{22}}\right) \end{cases}$$

上面的 T 的数值直接换成 q 即可。然后，我们需要再加上个真值表来判断姿态的方向。如下所示：

表 9.2.1 航向角 Ψ 真值表

T_{22}	T_{12}	Ψ
$\rightarrow 0$	+	90°
$\rightarrow 0$	-	-90°
+	+	$\Psi_{\pm}$
+	-	$\Psi_{\pm}$
-	+	$\Psi_{\pm} + 180^\circ$
-	-	$\Psi_{\pm} - 180^\circ$

表 9.2.2 横滚角 γ 真值表

$\gamma_{\pm}$	T_{33}	γ
+	+	$\gamma_{\pm}$
-		
+	-	$\gamma_{\pm} - 180^\circ$
-	-	$\gamma_{\pm} + 180^\circ$

我们甚至还可以用方向余弦表示四元数：

$$a = \frac{1}{2}(1 + c_{11} + c_{22} + c_{33})^{1/2}$$

$$b = \frac{1}{4a}(c_{32} - c_{23})$$

$$c = \frac{1}{4a}(c_{13} - c_{31})$$

$$d = \frac{1}{4a}(c_{21} - c_{12})$$

这个适用于小角度位移。

用欧拉角直接表示四元数：

$$\begin{aligned}
 a &= \cos \frac{\phi}{2} \cos \frac{\theta}{2} \cos \frac{\psi}{2} + \sin \frac{\phi}{2} \sin \frac{\theta}{2} \sin \frac{\psi}{2} \\
 b &= \sin \frac{\phi}{2} \cos \frac{\theta}{2} \cos \frac{\psi}{2} - \cos \frac{\phi}{2} \sin \frac{\theta}{2} \sin \frac{\psi}{2} \\
 c &= \cos \frac{\phi}{2} \sin \frac{\theta}{2} \cos \frac{\psi}{2} + \sin \frac{\phi}{2} \cos \frac{\theta}{2} \sin \frac{\psi}{2} \\
 d &= \cos \frac{\phi}{2} \cos \frac{\theta}{2} \sin \frac{\psi}{2} + \sin \frac{\phi}{2} \sin \frac{\theta}{2} \cos \frac{\psi}{2}
 \end{aligned}$$

A. 17 卡尔曼滤波五个公式

下面我们来看一下卡尔曼滤波的五个公式的内容（适用于建模）：

以下建模推导内容来源于：
http://blog.csdn.net/qq_29540745/article/details/52027466

首先，我们先要引入一个离散控制过程的系统：

$$X(k) = A X(k-1) + B U(k) + W(k)$$

再加上系统的测量值：

$$Z(k) = H X(k) + V(k)$$

其中， $X(k)$ 是 k 时刻的系统状态， $U(k)$ 是 k 时刻对系统的控制量。 A 和 B 是系统参数，对于多模型系统，他们为矩阵。 $Z(k)$ 是 k 时刻的测量值， H 是测量系统的参数，对于多测量系统， H 为矩阵。 $W(k)$ 和 $V(k)$ 分别表示过程和测量的噪声。他们被假设成高斯白噪声 (White Gaussian Noise)，他们的 covariance 分别是 Q ， R （这里我们假设他们不随系统状态变化而变化）。

满足上面的条件（线性随机微分系统，过程和测量都是高斯白噪声），卡尔曼滤波器是最优的信息处理器。

下面来预测下一状态的系统。假设现在的系统状态是 k ，根据系统的模型，可以基于系统的上一状态而预测出现在状态：

$$X(k|k-1) = A X(k-1|k-1) + B U(k) \quad \text{.....} \quad (1)$$

式(1)中, $X(k|k-1)$ 是利用上一状态预测的结果, $X(k-1|k-1)$ 是上一状态最优的结果, $U(k)$ 为现在状态的控制量, 如果没有控制量, 它可以为 0。

到现在为止, 我们的系统结果已经更新了, 可是, 对应于 $X(k|k-1)$ 的 covariance 还没更新。我们用 P 表示 covariance:

$$P(k|k-1) = A P(k-1|k-1) A' + Q \quad \text{.....} \quad (2)$$

式(2)中, $P(k|k-1)$ 是 $X(k|k-1)$ 对应的 covariance, $P(k-1|k-1)$ 是 $X(k-1|k-1)$ 对应的 covariance, A' 表示 A 的转置矩阵, Q 是系统过程的 covariance。式子 1, 2 就是卡尔曼滤波器 5 个公式当中的前两个, 也就是对系统的预测。

现在我们有现在状态的预测结果, 然后我们再收集现在状态的测量值。结合预测值和测量值, 我们可以得到现在状态 (k) 的最优化估算值 $X(k|k)$:

$$X(k|k) = X(k|k-1) + K_g(k) (Z(k) - H X(k|k-1)) \quad \text{.....} \quad (3)$$

其中 K_g 为卡尔曼增益 (Kalman Gain):

$$K_g(k) = P(k|k-1) H' / (H P(k|k-1) H' + R) \quad \text{.....} \quad (4)$$

到现在为止, 我们已经得到了 k 状态下最优的估算值 $X(k|k)$ 。但是为了要另卡尔曼滤波器不断的运行下去直到系统过程结束, 我们还要更新 k 状态下 $X(k|k)$ 的 covariance:

$$P(k|k) = (I - K_g(k) H) P(k|k-1) \quad \text{.....} \quad (5)$$

其中 I 为 1 的矩阵, 对于单模型单测量, $I=1$ 。当系统进入 $k+1$ 状态时, $P(k|k)$ 就是式子(2)的 $P(k-1|k-1)$ 。这样, 算法就可以自回归的运算下去。

卡尔曼滤波器的原理基本描述了, 式子 1, 2, 3, 4 和 5 就是 5 个基本公式。计算机建模就可以利用这 5 各公式来预判出结果。

A. 18 IMU 姿态融合算法

IMU 的计算方案是不经过磁力计, 而是直接使用加速度计得到的数据对得到

的姿态做补偿：

1. 先将测量得到的加速度转换为机体坐标系的数值
2. 计算出误差的偏差
3. PID 累计误差的比例并带入陀螺仪计算得到的机体坐标系
4. 实时转换成四元数的数据。

下面我们就来看一看 IMU 的代码是怎么实现的吧：

```
void IMUUpdate(float gx, float gy, float gz, float ax, float ay, float az) {  
  
    float norm;  
  
    float vx, vy, vz;  
  
    float ex, ey, ez;  
  
    // 测量正常化  
  
    norm = sqrt(ax*ax + ay*ay + az*az);  
  
    ax = ax / norm;  
  
    ay = ay / norm;  
  
    az = az / norm;  
  
    // 估计方向的重力  
  
    vx = 2*(q1*q3 - q0*q2);  
  
    vy = 2*(q0*q1 + q2*q3);  
  
    vz = q0*q0 - q1*q1 - q2*q2 + q3*q3;  
  
    // 错误的领域和方向传感器测量参考方向之间的交叉乘积的总和  
  
    ex = (ay*vz - az*vy);  
  
    ey = (az*vx - ax*vz);  
  
    ez = (ax*vy - ay*vx);  
  
    // 积分误差比例积分增益  
  
    exInt = exInt + ex*Ki;  
  
    eyInt = eyInt + ey*Ki;  
  
    ezInt = ezInt + ez*Ki;  
}
```

```

// 调整后的陀螺仪测量

gx = gx + Kp*ex + exInt;

gy = gy + Kp*ey + eyInt;

gz = gz + Kp*ez + ezInt;

// 整合四元数率和正常化

q0 = q0 + (-q1*gx - q2*gy - q3*gz)*halfT;

q1 = q1 + (q0*gx + q2*gz - q3*gy)*halfT;

q2 = q2 + (q0*gy - q1*gz + q3*gx)*halfT;

q3 = q3 + (q0*gz + q1*gy - q2*gx)*halfT;

// 正常化四元

norm = sqrt(q0*q0 + q1*q1 + q2*q2 + q3*q3);

q0 = q0 / norm;

q1 = q1 / norm;

q2 = q2 / norm;

q3 = q3 / norm;

}

```


附录 B MPU9250-DMP

MPU 系列传感器都自带有 DMP，它可以方便内部开发人员直接获取传感器计算后的数据。DMP 是芯片内部自带一种功能，通过获取 DMP 数据你不需要再去做滤波做算法，就可以直接获得数值。MPU6050 的 DMP 已共享在了资料盘中，这里讲一下 MPU9250 的 DMP 使用和操作。注意，MPU9250 的方案不适用于 MPU6050。此处例程采用的是 I2C 通讯方式。只需 MCU 接入传感器的 SCL SDA VCC GND 即可，如图 B-1 所示。

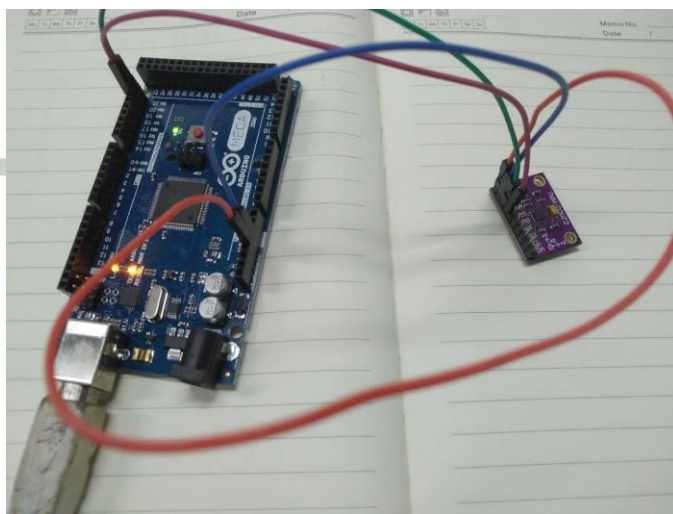


图 B-1 arduino 通过 I2C 连接 MPU9250

先进入 MPU 的配置功能：

```
//配置模式选择

//ways = 1 选择普通配置

//ways = 2 选择 DMP 配置

void MPU9250::mpuconfig(int ways)//ways=2 时进入 DMP 配置模式

{

    if(ways==1)

    {

        I2CWriteByte(MPU9250_ADDRESS, 0x6B, 0x80); //重置寄存器

        delay(100);
```

```

I2CWriteByte(MPU9250_ADDRESS, 0x1C, ACC_FULL_SCALE_2_G); //设置重力计单位值

I2CWriteByte(MPU9250_ADDRESS, 0x1B, GYRO_FULL_SCALE_250_DPS); //设置陀螺仪单位值

}

if(ways==2)

{

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_PWR_MGMT_1, 0x80); //pw1

    delay(100);

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_PWR_MGMT_1, 0x00); //pw1

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_PWR_MGMT_2, 0x00); //pw2

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_SMPLRT_DIV, 0x04); //1000/50 = 20hz

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_CONFIG, 0x06); //1KHZ

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_GYRO_CONFIG, 0x18); //+- 2000

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_ACCEL_CONFIG, 0x10); //+- 4g

    //设置 MPU9250 中断, 如不需要就无需设置

    // MPU9250_INT_init();

    /*****DMP 的配置(开启 DMP , 设置 DMP FIFO)*****/

    loadfirmware();

    loadcfgupd();

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_DMP_CFG_1, 0x03);

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_DMP_CFG_2, 0x00);

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_USER_CTRL, 0xcc);

    /*****/

}

}

```

那么, 其中 loadfirmware 和 loadcfgupd 是分别将 2 组数据写入指定寄存器。

```

void MPU9250::loadfirmware(void)

{

    unsigned int datanum=0;

```

```

unsigned char bank;

unsigned lastbank=1;

for (datanum=0;datanum<1929;datanum++)
{
    bank=datanum/256;

    if(lastbank!=bank)
    {
        I2CWriteByte(MPU9250_ADDRESS, 0x6d, bank);

        I2CWriteByte(MPU9250_ADDRESS, 0x6e, 0);

    }

    I2CWriteByte(MPU9250_ADDRESS, 0x6f, dmpmemorydata[datanum]);

    lastbank=bank;

};
}

void MPU9250::loadcfgupd(void)
{
    unsigned char line;

    unsigned char bank;

    unsigned char datacounts=0;

    unsigned char bytes2write;

    unsigned char offset;

    unsigned char writingcounts;

    for (line=0;line<37;line++)
    {
        bank=dmpcfgupddata[datacounts];

        datacounts++;

        offset=dmpcfgupddata[datacounts];

        datacounts++;

        bytes2write=dmpcfgupddata[datacounts];

```

```

I2CWriteByte(MPU9250_ADDRESS, 0x6d, bank);

I2CWriteByte(MPU9250_ADDRESS, 0x6e, offset);

for (writingcounts=0;writingcounts<bytes2write;writingcounts++)

{

    datacounts++;

I2CWriteByte(MPU9250_ADDRESS, 0x6f, dmpcfgupddata[datacounts]);

}

    datacounts++;

}

}

```

这两组数组如图 B-2, B-3 所示:

```

const unsigned char dmpmemorydata[1929]={
// bank 0, 256 bytes
0xFB, 0x00, 0x00, 0x3E, 0x00, 0x0B, 0x00, 0x36, 0x00, 0x01, 0x00, 0x02, 0x00, 0x03, 0x00, 0x00,
0x00, 0x65, 0x00, 0x54, 0xFF, 0xEF, 0x00, 0x00, 0xFA, 0x80, 0x00, 0x0B, 0x12, 0x82, 0x00, 0x01,
0x00, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x28, 0x00, 0x00, 0xFF, 0xFF, 0x45, 0x81, 0xFF, 0xFF, 0xFA, 0x72, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x03, 0xE8, 0x00, 0x00, 0x00, 0x01, 0x00, 0x01, 0x7F, 0xFF, 0xFF, 0xFE, 0x80, 0x01,
0x00, 0x1B, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x3E, 0x03, 0x30, 0x40, 0x00, 0x00, 0x00, 0x02, 0xCA, 0xE3, 0x09, 0x3E, 0x80, 0x00, 0x00,
0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x60, 0x00, 0x00, 0x00,
0x41, 0xFF, 0x00, 0x00, 0x00, 0x00, 0x0B, 0x2A, 0x00, 0x00, 0x16, 0x55, 0x00, 0x00, 0x21, 0x82,
0xFD, 0x87, 0x26, 0x50, 0xFD, 0x80, 0x00, 0x00, 0x00, 0x1F, 0x00, 0x00, 0x00, 0x05, 0x80, 0x00,
0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x02, 0x00, 0x00, 0x00, 0x03, 0x00, 0x00,
0x40, 0x00, 0x00, 0x00, 0x00, 0x04, 0x6F, 0x00, 0x02, 0x65, 0x32, 0x00, 0x00, 0x5E, 0xC0,
0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0xFB, 0x8C, 0x6F, 0x5D, 0xFD, 0x5D, 0x08, 0xD9, 0x00, 0x7C, 0x73, 0x3B, 0x00, 0x6C, 0x12, 0xCC,
0x32, 0x00, 0x13, 0x9D, 0x32, 0x00, 0xD0, 0xD6, 0x32, 0x00, 0x08, 0x00, 0x40, 0x00, 0x01, 0xF4,
0xFF, 0xE6, 0x80, 0x79, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0xD0, 0xD6, 0x00, 0x00, 0x27, 0x10,

```

图 B-2 dmp 内存数据配置部分数组

```

const unsigned char dmpcfgupddata[239] = {
// dmp config
// BANK   OFFSET  LENGTH  [DATA]
0x03, 0x7B, 0x03, 0x4C, 0xCD, 0x6C,
0x03, 0xAB, 0x03, 0x36, 0x56, 0x76,
0x00, 0x68, 0x04, 0x02, 0xCB, 0x47, 0xA2,
0x02, 0x18, 0x04, 0x00, 0x05, 0x8B, 0xC1,
0x01, 0x0C, 0x04, 0x00, 0x00, 0x00, 0x00,
0x03, 0x7F, 0x06, 0x0C, 0xC9, 0x2C, 0x97, 0x97, 0x97,
0x03, 0x89, 0x03, 0x26, 0x46, 0x66,
0x00, 0x6C, 0x02, 0x20, 0x00,
0x02, 0x40, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x44, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x48, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x4C, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x50, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x54, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x58, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0x5C, 0x04, 0x00, 0x00, 0x00, 0x00,
0x02, 0xBC, 0x04, 0x00, 0x00, 0x00, 0x00,
0x01, 0xEC, 0x04, 0x00, 0x00, 0x40, 0x00,
0x03, 0x7F, 0x06, 0x0C, 0xC9, 0x2C, 0x97, 0x97, 0x97,
0x04, 0x02, 0x03, 0xD0, 0x35, 0x5D,
0x04, 0x09, 0x04, 0x87, 0x2D, 0x35, 0x3D,
0x00, 0xA3, 0x01, 0x00,
0x00, 0x00, 0x01, 0x01,

```

图 B-3 DMP 配置数据数组

经过配置完成后我们就可以进行数据读取了。

```
uint16_t fifo_count = 0;

while(1)//使用循环为了等待 DMP 转换完成
{
    fifo_count = mpu9250.getFIFOCount(); //读取 FIFO 计数
    //Serial.println(fifo_count); //输出读取的 fifo 计数值

    if(fifo_count >= 0x0200) //如果 FIFO 值>0x0200, 此时 DMP 的结果错误, 直接复位 FIFO
    {
        mpu9250.resetFIFO();
    }
    else
    {
        if(fifo_count >= 0x2a) //如果 FIFO 值 > 0x2a, 此时 DMP 转换完成并且数值正常
        {
            mpu9250.ReadDmp(); //首先要读取 DMP FIFO, 读取之后才能进行计算姿态的操作
            mpu9250.resetFIFO();

            mpu9250.getyawpitchroll(); //计算并且获取 yaw、pitch、roll, 结果保存在 yprf[3]数组中
            break; //转换并且计算完成之后退出循环*/
        }
    }
}

Serial.print("yaw="); Serial.print(mpu9250.yprf[0]); Serial.print(" ");
Serial.print("pitch="); Serial.print(mpu9250.yprf[1]); Serial.print(" ");
Serial.print("roll="); Serial.println(mpu9250.yprf[2]);

delay(100);
```

通过以上可以得到间隔 100 毫秒输出一次的欧拉角数据, 如图 B-4 所示:



图 B-4 得到的欧拉角数值

其中读取 dmp 数据函数为 42 个，读取后将其中 16 个转化为四元数，即可得到 q_0 ， q_1 ， q_2 和 q_3 ，此时即可使用之前讲过的四元数与欧拉角的转换公式将其转化为欧拉角并储存在 $yprf$ 数组中。

```
void MPU9250::ReadDmp(void)//读取 DMP 的值，保存在 dmpdatas 数组中
{
    I2CRead(MPU9250_ADDRESS, 0x74, 42, dmpdatas);//读取 DMP 中 42 个数据

    I2CWriteByte(MPU9250_ADDRESS, MPU9250_RA_USER_CTRL, 0xcc);//FIFO_RST | DMP_RST | DMP_EN | FIFO_EN
}

void MPU9250::getyawpitchroll(void)//计算欧拉角
{
    getquaternion();//计算四元数

    yprf[0]=-atan2(2.0*(q[0]*q[3] + q[1]*q[2]),
                  1 - 2.0*(q[2]*q[2] + q[3]*q[3]))*57.3; //yaw

    yprf[1]=-asin(2.0*(q[0]*q[2] - q[3]*q[1]))*57.3;//pitch

    yprf[2] = atan2(2.0*(q[0]*q[1] + q[2]*q[3]),
                   1 - 2.0*(q[1]*q[1] + q[2]*q[2]))*57.3; //roll
}

void MPU9250::getquaternion(void)//提取四元数
```

```

{
    q[0] = getdmp1ong(0)/1073741824.0;
    q[1] = getdmp1ong(4)/1073741824.0;
    q[2] = getdmp1ong(8)/1073741824.0;
    q[3] = getdmp1ong(12)/1073741824.0;
}

long MPU9250::getdmp1ong(unsigned char address)//dmp 数据合并
{
    long dmptempl;
    dmptempl = dmpdatas[address];
    dmptempl <<= 8;
    dmptempl |= dmpdatas[address+1];
    dmptempl <<= 8;
    dmptempl |= dmpdatas[address+2];
    dmptempl <<= 8;
    dmptempl |= dmpdatas[address+3];
    return dmptempl;
}

```

配置 DMP 模式后，芯片将自动计算数值并保存在 dmp 数组中，我们只需要等待 fifo 数据量，并读取数值即可。本程序的全部资料都在资料中提供了下载，感兴趣的读者可以下载详细研究。

附录 C 寄存器

本附录讲解 MPU9250 内部寄存器的使用和功能， MPU9250 的寄存器表：

地址		寄存器名称	是否读写	位 7	位 6	位 5	位 4	位 3	位 2	位 1	位 0
16	10										
00	0	SELF_TEST_X_GYRO	读写	Xg_st_data[7:0]							
01	1	SELF_TEST_Y_GYRO	读写	Yg_st_data[7:0]							
02	2	SELF_TEST_Z_GYRO	读写	Zg_st_data[7:0]							
0D	13	SELF_TEST_X_ACCEL	读写	XA_ST_DATA[7:0]							
0E	14	SELF_TEST_Y_ACCEL	读写	YA_ST_DATA[7:0]							
0F	15	SELF_TEST_Z_ACCEL	读写	ZA_ST_DATA[7:0]							
13	19	XG_OFFSET_H	读写	X_OFF_USR[15:8]							
14	20	XG_OFFSET_L	读写	X_OFF_USR[7:0]							
15	21	YG_OFFSET_H	读写	Y_OFF_USR[15:8]							

16	22	YG_OFF SET_L	读写	Y_OFF_USR[7:0]							
17	23	ZG_OFF SET_H	读写	Z_OFF_USR[15:8]							
18	24	ZG_OFF SET_L	读写	Z_OFF_USR[7:0]							
19	25	SMPLRT _DIV	读写	SMPLRT_DIV[7:0]							
1A	26	CONFIG	读写	-	FIFO_MO DE	EXT_SYNC_SET[2:0]		DLPF_CFG[2:0]			
1B	27	GYRO_C ONFIG	读写	XGYR O_Ct en	YGYRO_C ten	ZGYRO_ Cten	GYRO_FS_SEL[1:0]	-	FCHOICE_B[1:0]		
1C	28	ACCEL_ CONFIG	读写	Ax_s t_en	Ay_st_e n	Az_st_ en	ACCEL_FS_SEL[1: 0]	-			
1D	29	ACCEL_ CONFIG 2	读写	-			ACCEL_FCHOICE_B		A_DLPF_CFG		
1E	30	LP_ACC EL_ODR	读写	-			Lposc_clkse1[3:0]				
1F	31	WOM_TH R	读写	WOM_Threshold[7:0]							
23	35	FIFO_E N	读写	TEMP _FIF O_EN	GYRO_XO _UT	GYRO_Y O_UT	GYRO_ZO _UT	ACCEL	SLV2	SLV1	SLV0
24	36	I2C_MS T_CTRL	读写	MULT _MST _EN	WAIT_FO R_ES	SLV3_F IFO_EN	I2C_MST _P_NSR	I2C_MST_CLK[3:0]			
25	37	I2C_SL VO_ADD	读写	I2C_ SLV0	I2C_ID_0[6:0]						

		R		_RNW				
26	38	I2C_SLV0_REG	读写	I2C_ID_0[7:0]				
27	39	I2C_SLV0_CTRL	读写	I2C_SLV0_EN	I2C_SLV0_BYTE_SW	I2C_SLV0_REG_DIS	I2C_SLV0_GRP	I2C_SLV0_LEN[3:0]
28	40	I2C_SLV1_ADDR	读写	I2C_SLV1_RNW	I2C_ID_1[6:0]			
29	41	I2C_SLV1_REG	读写	I2C_SLV1_REG[7:0]				
2A	42	I2C_SLV1_CTRL	读写	I2C_SLV1_EN	I2C_SLV1_BYTE_SW	I2C_SLV1_REG_DIS	I2C_SLV1_GRP	I2C_SLV1_LEN[3:0]
2B	43	I2C_SLV2_ADDR	读写	I2C_SLV2_RNW	I2C_ID_2[6:0]			
2C	44	I2C_SLV2_REG	读写	I2C_SLV2_REG[7:0]				
2D	45	I2C_SLV2_CTRL	读写	I2C_SLV2_EN	I2C_SLV2_BYTE_SW	I2C_SLV2_REG_DIS	I2C_SLV2_GRP	I2C_SLV2_LEN[3:0]
2E	46	I2C_SLV3_ADDR	读写	I2C_SLV3_RNW	I2C_ID_3[6:0]			
2F	47	I2C_SLV3_REG	读写	I2C_SLV3_REG[7:0]				
30	48	I2C_SLV3_CTRL	读写	I2C_SLV3_EN	I2C_SLV3_BYTE_SW	I2C_SLV3_REG_DIS	I2C_SLV3_GRP	I2C_SLV3_LEN[3:0]

31	49	I2C_SLV4_ADD_R	读写	I2C_SLV4_RNW	I2C_ID_4[6:0]						
32	50	I2C_SLV4_REG	读写	I2C_SLV4_REG[7:0]							
33	51	I2C_SLV4_D0	读写	I2C_SLV4_D0[7:0]							
34	52	I2C_SLV4_CTRL	读写	I2C_SLV4_EN	I2C_DONE_INT_EN	I2C_SLV4_REG_DIS	I2C_MST_DLY[4:0]				
35	53	I2C_SLV4_D1	读	I2C_SLV4_DIV[7:0]							
36	54	I2C_MST_STATUS	读	PASS_THROUGH	I2C_SLV4_DONE	I2C_LOST_ARB	I2C_SLV4_NACK	I2C_SLV3_NACK	I2C_SLV2_NACK	I2C_SLV1_NACK	I2C_SLV0_NACK
37	55	INT_PIN_CFG	读写	ACTL	OPEN	LATCH_INT_EN	INT_ANYRD_2CLEAR	ACTL_FSYNC	FSYNC_INT_MODE_EN	BYPASS_EN	—
38	56	INT_ENABLE	读写	—	WOM_EN	—	FIFO_OFLOW_EN	FSYNC_INT_EN	—	—	RAW_RDY_EN
3A	58	INT_STATUS	读	—	WOM_INT	—	FIFO_OFLOW_INT	FSYNC_INT_INT	—	—	RAW_RDY_INT
3B	59	ACCEL_XOUT_H	读	ACCEL_XOUT_H[15:8]							
3C	60	ACCEL_XOUT_L	读	ACCEL_XOUT_L[7:0]							
3D	61	ACCEL_YOUT_H	读	ACCEL_YOUT_H[15:8]							
3E	62	ACCEL_	读	ACCEL_YOUT_L[7:0]							

		YOUT_L		
3F	63	ACCEL_ZOUT_H	读	ACCEL_ZOUT_H[15:8]
40	64	ACCEL_ZOUT_L	读	ACCEL_ZOUT_L[7:0]
41	65	TEMP_OUT_H	读	TEMP_OUT_H[15:8]
42	66	TEMP_OUT_L	读	TEMP_OUT_L[7:0]
43	67	GYRO_XOUT_H	读	GYRO_XOUT_H[15:8]
44	68	GYRO_XOUT_L	读	GYRO_XOUT_L[7:0]
45	69	GYRO_YOUT_H	读	GYRO_YOUT_H[15:8]
46	70	GYRO_YOUT_L	读	GYRO_YOUT_L[7:0]
47	71	GYRO_ZOUT_H	读	GYRO_ZOUT_H[15:8]
48	72	GYRO_ZOUT_L	读	GYRO_ZOUT_L[7:0]
49	73	EXT_SENS_DATA_00	读	EXT_SENS_DATA_00[7:0]
4A	74	EXT_SENS_DATA_01	读	EXT_SENS_DATA_01[7:0]
4B	75	EXT_SENS_DATA_02	读	EXT_SENS_DATA_02[7:0]

		A_02		
4C	76	EXT_SE NS_DAT A_03	读	EXT_SENS_DATA_03[7:0]
4D	77	EXT_SE NS_DAT A_04	读	EXT_SENS_DATA_04[7:0]
4E	78	EXT_SE NS_DAT A_05	读	EXT_SENS_DATA_05[7:0]
4F	79	EXT_SE NS_DAT A_06	读	EXT_SENS_DATA_06[7:0]
50	80	EXT_SE NS_DAT A_07	读	EXT_SENS_DATA_07[7:0]
51	81	EXT_SE NS_DAT A_08	读	EXT_SENS_DATA_08[7:0]
52	82	EXT_SE NS_DAT A_09	读	EXT_SENS_DATA_09[7:0]
53	83	EXT_SE NS_DAT A_10	读	EXT_SENS_DATA_10[7:0]
54	84	EXT_SE NS_DAT A_11	读	EXT_SENS_DATA_11[7:0]
55	85	EXT_SE NS_DAT A_12	读	EXT_SENS_DATA_12[7:0]

56	86	EXT_SENS_DATA_13	读	EXT_SENS_DATA_13[7:0]
57	87	EXT_SENS_DATA_14	读	EXT_SENS_DATA_14[7:0]
58	88	EXT_SENS_DATA_15	读	EXT_SENS_DATA_15[7:0]
59	89	EXT_SENS_DATA_16	读	EXT_SENS_DATA_16[7:0]
5A	90	EXT_SENS_DATA_17	读	EXT_SENS_DATA_17[7:0]
5B	91	EXT_SENS_DATA_18	读	EXT_SENS_DATA_18[7:0]
5C	92	EXT_SENS_DATA_19	读	EXT_SENS_DATA_19[7:0]
5D	93	EXT_SENS_DATA_20	读	EXT_SENS_DATA_20[7:0]
5E	94	EXT_SENS_DATA_21	读	EXT_SENS_DATA_21[7:0]
5F	95	EXT_SENS_DATA_22	读	EXT_SENS_DATA_22[7:0]
60	96	EXT_SENS_DATA_23	读	EXT_SENS_DATA_23[7:0]

		A_23									
63	99	I2C_SLV0_DO	读写	I2C_SLV0_DO[7:0]							
64	100	I2C_SLV1_DO	读写	I2C_SLV1_DO[7:0]							
65	101	I2C_SLV2_DO	读写	I2C_SLV2_DO[7:0]							
66	102	I2C_SLV3_DO	读写	I2C_SLV3_DO[7:0]							
67	103	I2C_MST_DELAY_CTRL	读写	DELAY_ES_SHADOW	-	-	I2C_SLV4_DLY_EN	I2C_SLV3_DLY_EN	I2C_SLV2_DLY_EN	I2C_SLV1_DLY_EN	I2C_SLV0_DLY_EN
68	104	SIGNAL_PATH_RESET	读写	-	-	-	-	-	GYRO_RST	ACCEL_RST	TEMP_RST
69	105	MOT_DETECT_CTRL	读写	ACCEL_INT_EN	ACCEL_INT_MODE	-		-		-	
6A	106	USER_CTRL	读写	-	FIFO_EN	I2C_MST_EN	I2C_IF_DIS	-	FIFO_RST	I2C_MST_RST	SIG_COND_RST
6B	107	PWR_MGMT_1	读写	H_RESET	SLEEP	CYCLE	GYRO_STANBY	PD_PTA_T	CLKSEL[2:0]		
6C	108	PWR_MGMT_2	读写	-		DIS_XA	DIS_YA	DIS_ZA	DIS_XG	DIS_YG	DIS_ZG
72	114	FIFO_COUNTH	读写	-			FIFO_CNT[12:8]				
73	115	FIFO_COUNTL	读写	FIFO_CNT[7:0]							

74	11 6	FIFO_R _W	读 写	D[7:0]	
75	11 7	WHO_AM _I	读	WHOAMI[7:0]	
77	11 9	XA_OFF SET_H	读 写	XA_OFFS[14:7]	
78	12 0	XA_OFF SET_L	读 写	XA_OFFS[6:0]	—
7A	12 2	YA_OFF SET_H	读 写	YA_OFFS[14:7]	
7B	12 3	YA_OFF SET_L	读 写	YA_OFFS[6:0]	—
7D	12 5	ZA_OFF SET_H	读 写	ZA_OFFS[14:7]	
7E	12 6	ZA_OFF SET_L	读 写	ZA_OFFS[6:0]	—

上表中_H和_L分别表示寄存器数据中的高字节和低字节。

寄存器

下面来看 MPU9250 中的全部寄存器的功能及使用方法：

寄存器 0-2——陀螺仪自检寄存器

使用方式：读/写

复位值：0x00（当向该寄存器写入该值的时候，寄存器将会重置，数据回归0x00）

寄存器名	寄存器位	功能
SELF_TEST_X_GY	XG_ST_DATA[7:	这个寄存器的值表示生产时的自测输出

RO	0]	值。用户可以用来检测检测数据是否正确。应用方法参见第 4 章结尾处
SELF_TEST_Y_GYRO	YG_ST_DATA[7:0]	
SELF_TEST_Z_GYRO	ZG_ST_DATA[7:0]	

寄存器 13~15——加速度自检寄存器

使用方法：读/写

复位值：0x00

寄存器	寄存器位	功能
SELF_TEST_X_ACCEL	XA_ST_DATA[7:0]	这个寄存器的值表示生产时的自测输出值。同上
SELF_TEST_Y_ACCEL	YA_ST_DATA[7:0]	
SELF_TEST_Z_ACCEL	ZA_ST_DATA[7:0]	

寄存器 19~24——陀螺仪偏移寄存器

使用方法：读/写

复位值：0x00

寄存器位	寄存器名	功能
[7:0]	X_OFFSET_USR[15:8]	USR 寄存器的高低字节（14h） 精度偏移量 $\text{OffsetLSB} = X_OFFSET_USR * 4 / 2^{\text{FS_SEL}}$ 角度 $\text{OffsetDPS} = X_OFFSET_USR * 4 / 2^{\text{FS_SEL}} /$

		Gyro_Sensitivity 计算条件 FS_SEL = 0 $Gyro_Sensitivity = 2^{16} \text{ LSB} / 500\text{dps}$ 最大值 999.969dps 最小值 -1000dps 精度 0.0305dps
[7:0]	X_OFFS_USR[7:0]	USR 寄存器内部高低字节 (13h)
[7:0]	Y_OFFS_USR[15:8]	USR 寄存器的高低字节 (16h) 精度偏移量 $Offset_{LSB} = Y_OFFS_USR * 4 / 2^{FS_SEL}$ 角度 $Offset_{DPS} = Y_OFFS_USR * 4 / 2^{FS_SEL} / Gyro_Sensitivity$ 计算条件 FS_SEL = 0 $Gyro_Sensitivity = 2^{16} \text{ LSB} / 500\text{dps}$ 最大值 999.969dps 最小值 -1000dps 精度 0.0305dps
[7:0]	Y_OFFS_USR[7:0]	USR 寄存器内部高低字节 (15h)
[7:0]	Z_OFFS_USR[15:8]	USR 寄存器的高低字节 (16h) 精度偏移量 $Offset_{LSB} = Y_OFFS_USR * 4 / 2^{FS_SEL}$ 角度 $Offset_{DPS} = Y_OFFS_USR * 4 / 2^{FS_SEL} / Gyro_Sensitivity$ 计算条件 FS_SEL = 0 $Gyro_Sensitivity = 2^{16} \text{ LSB} / 500\text{dps}$ 最大值 999.969dps 最小值 -1000dps 精度 0.0305dps

[7:0]	Z_OFFSET_USR[7:0]	USR 寄存器的高低字节 (17h)
-------	-------------------	--------------------

这个寄存器中的数值就是陀螺仪的 DC 偏移量，但陀螺仪的数据不需要再做减法了，因为我们从 67-72 号寄存器读出的数据是已经消除偏移量的数值了。但是我们还是可以读取和利用偏移量传感器，具体方案参见第四章结尾处。

寄存器 25——分频采样寄存器

使用方法：读/写

复位值：0x00

寄存器位	寄存器名	功能
[7:0]	SMPLRT_DIV[7:0]	控制内部输出的采样率（详见配置寄存器）的频率 FIFO 采样率。说明：当配置 Fchoice = 2' b11 (fchoice_b 寄存器的值为 2' b00) 并且 (0 < dlpf_cfg < 7) 时，此寄存器生效。 数据更新频率： SAMPLE_RATE= Internal_Sample_Rate / (1 + SMPLRT_DIV)

这个寄存器就是为采样频率分频用的，比如将 1MHz 的采样率 2 分频，则可以得到 5KHz。采样速率并不是越快越好，采样速度快了反而出错率会变高。按照上面的速率采样，而 SMPLRT_DIV 只用于 1KHz 的内部采样。

寄存器 26——配置寄存器

寄存器位	寄存器名	功能
[7]	—	保留位
[6]	FIFO_MODE	当设置为 1 时，FIFO 缓冲区满后将不再写入数值。 当设置为 0 时，FIFO 缓冲区满后新数据会不停

		更新以替换老数据。																		
[5:3]	EXT_SYNC_SET[2:0]	使能 FSYNC 引脚以采样数据。 <table><tr><td>EXT_SYNC_SET</td><td>FSYNC 位地址</td></tr><tr><td>0</td><td>功能关闭</td></tr><tr><td>1</td><td>TEMP_OUT_L[0]</td></tr><tr><td>2</td><td>GYRO_XOUT_L[0]</td></tr><tr><td>3</td><td>GYRO_YOUT_L[0]</td></tr><tr><td>4</td><td>GYRO_ZOUT_L[0]</td></tr><tr><td>5</td><td>ACCEL_XOUT_L[0]</td></tr><tr><td>6</td><td>ACCEL_YOUT_L[0]</td></tr><tr><td>7</td><td>ACCEL_ZOUT_L[0]</td></tr></table> FSYNC 将会捕捉到一个短的信号并产生一次锁存值，同一个信号只能产生一次锁存值。条件是必须外接第三方驱动设备，而且 fsync 通信速率要低于我们的正常设备的采样率。	EXT_SYNC_SET	FSYNC 位地址	0	功能关闭	1	TEMP_OUT_L[0]	2	GYRO_XOUT_L[0]	3	GYRO_YOUT_L[0]	4	GYRO_ZOUT_L[0]	5	ACCEL_XOUT_L[0]	6	ACCEL_YOUT_L[0]	7	ACCEL_ZOUT_L[0]
EXT_SYNC_SET	FSYNC 位地址																			
0	功能关闭																			
1	TEMP_OUT_L[0]																			
2	GYRO_XOUT_L[0]																			
3	GYRO_YOUT_L[0]																			
4	GYRO_ZOUT_L[0]																			
5	ACCEL_XOUT_L[0]																			
6	ACCEL_YOUT_L[0]																			
7	ACCEL_ZOUT_L[0]																			
[2: 0]	DLPF_CFG[2:0]	若要使用 DLPF，fchoice[1:0] 必须设置成 2’ b11，fchoice_b[1:0]设置成 2’ b00。 详见下面表																		

DLPF 功能是由 DLPF_CFG 决定的，当 FCHOICE_B[1:0]=2b' 00，陀螺仪和温度传感器的值会按照如下的参数配置。注意。下表中 FCHOICE 的值和 FCHOICE_B 是反的。（例如：FCHOICE=2b' 00 和 FCHOICE_B=2b' 11 是一样的）

FCHOICE		DLPF_CF G	陀螺仪			温度	
<1>	<0>		带宽 (Hz)	延时 (ms)	Fs (kHz)	带 宽 (Hz)	延 时 (ms)
x	0	x	8800	0.064	32	4000	0.04
0	1	x	3600	0.11	32	4000	0.04
1	1	0	250	0.97	8	4000	0.04
1	1	1	184	2.9	1	188	1.9
1	1	2	92	3.9	1	98	2.8
1	1	3	41	5.9	1	42	4.8
1	1	4	20	9.9	1	20	8.3
1	1	5	10	17.85	1	10	13.4
1	1	6	5	33.48	1	5	18.6
1	1	7	3600	0.17	8	4000	0.04

不知道大家还记不记得第二章曾讲过 MPU9250 可以和其他传感器相互连接的功能——读取其他传感器的数据并储存起来。这个寄存器就是用来设置缓冲区和采样速率的。FSYNC 连接单片机的外部中断引脚，给单片机输出的上升沿的外部中断信号来告诉单片机进行读取操作，可以节约单片机的读取时间。

寄存器 27——陀螺仪设置

使用方法：读/写

初始化值：0x00

位	位名	位功能
[7]	XGYRO_Cten	x 轴陀螺仪自检

[6]	YGYRO_Cten	y 轴陀螺仪自检
[5]	ZGYRO_Cten	z 轴陀螺仪自检
[4:3]	GYRO_FS_SEL[1:0]	陀螺仪全范围选择： 00 = +250dps 01 =+500dps 10 =+1000dps 11 =+2000dps
[2]	—	预留位
[1:0]	Fchoice_b[1:0]	和上一个表格所表示相反。 Fchoice_b 的上表的 Fchoice 相反。

这个寄存器非常重要，主要设置陀螺仪的采样范围，决定了工作范围和数据的大小。

寄存器 28——加速度配置

使用方法：读/写

初始值：0x00

位	位名	位功能
[7]	ax_st_en	加速度 x 轴自检
[6]	ay_st_en	加速度 y 轴自检
[5]	az_st_en	加速度 z 轴自检
[4:3]	ACCEL_FS_SEL[1:0]	加速度全范围选择： ±2g（00），±4g（01），±8g（10），±16g（11）
[2:0]	—	预留

和上个寄存器同样重要，设置加速度计的检测范围值。

寄存器 29——加速度配置 2

使用方法：读/写

复位值：0x00

位	位名	位功能
[7:6]	预留	
[5:4]	预留	
[3]	accel_fchoice_b	和下表的 accel_fchoice 相反的
[2:0]	A_DLPFCFG	加速度低通滤波，设置如下表

加速度传输速率和带宽（普通模式）

ACCEL_FCHOICE	A_DLPF_CFG	输出			
		带宽 (Hz)	延时 (ms)	信 噪 比 (ug/rHz)	速率 (kHz)
0	X	1.13K	0.75	250	4
1	0	460	1.94	250	1
1	1	184	5.80	250	1
1	2	92	7.80	250	1
1	3	41	11.80	250	1
1	4	20	19.80	250	1
1	5	10	35.70	250	1
1	6	5	66.96	250	1
1	7	460	1.94	250	1

DLPF 的输出速率和 $1/(1+SMPLRT_DIV)$ 的值成正比。（SMPLRT_DIV 是

一个 8 位的整数)

然后 ODRs 就是一个加速度计在正常模式下的速率的子集 (Hz) :

3.91, 7.81, 15.63, 31.25, 62.50, 125, 250, 500, 1K

下表展现的是当加速度计工作在低功耗模式下的带宽速率和相关配置。

加速度速率和带宽 (低功耗模式)

ACCEL_FCHOICE	ODR (Hz)	输出	
		带宽 (Hz)	延时 (ms)
0	0.24	1.1k	1
0	0.49	1.1k	1
0	0.98	1.1k	1
0	1.95	1.1k	1
0	3.91	1.1k	1
0	7.81	1.1k	1
0	15.63	1.1k	1
0	31.25	1.1k	1
0	62.50	1.1k	1
0	125	1.1k	1
0	250	1.1k	1
0	500	1.1k	1

诚如您所见, 一些 ODRs 在加速度在正常模式下工作的时候也可以像在低功耗模式下一样配置。具体的配置方法, 参见 2.10 中运动唤醒配置方案。

寄存器 30——加速度低功耗 ODR 控制位

使用方法：读/写

复位值：0x00

位	位名	功能																												
[7:4]	预留																													
[3:0]	Lpose_clkse1[3:0]	设置加速度计的唤醒采样率-低功耗加速度输出速率 <table><tr><th>Lpose_clkse1</th><th>Output Frequency (Hz)</th></tr><tr><td>0</td><td>0.24</td></tr><tr><td>1</td><td>0.49</td></tr><tr><td>2</td><td>0.98</td></tr><tr><td>3</td><td>1.95</td></tr><tr><td>4</td><td>3.91</td></tr><tr><td>5</td><td>7.81</td></tr><tr><td>6</td><td>15.63</td></tr><tr><td>7</td><td>31.25</td></tr><tr><td>8</td><td>62.50</td></tr><tr><td>9</td><td>125</td></tr><tr><td>10</td><td>250</td></tr><tr><td>11</td><td>500</td></tr><tr><td>12-15</td><td>预留</td></tr></table>	Lpose_clkse1	Output Frequency (Hz)	0	0.24	1	0.49	2	0.98	3	1.95	4	3.91	5	7.81	6	15.63	7	31.25	8	62.50	9	125	10	250	11	500	12-15	预留
Lpose_clkse1	Output Frequency (Hz)																													
0	0.24																													
1	0.49																													
2	0.98																													
3	1.95																													
4	3.91																													
5	7.81																													
6	15.63																													
7	31.25																													
8	62.50																													
9	125																													
10	250																													
11	500																													
12-15	预留																													

寄存器 31——运动唤醒

使用方法：读/写

复位值：0x00

位	位名	功能
[7:0]	WOM_Threshold	此位设置了加速度运动唤醒值的大小。LSB=4mg 范围是 0mg 到 1020mg

这个功能挺重要的，而且现在运用的比较多。比如智能手环，用过智能手环的同学应该知道，当你把手环放在一个平面上，并保持长时间不动，手环检测到长时间数值没有变化会自动进入休眠。同理，智能手机放在桌子上长时间不动有些品牌的也会触发这个功能。当我们此时拿起手环或者手机后，手环会开始运转，手机也会自动亮屏。这就是归功于运动唤醒的功能。这个是如何配置的呢？需要下面 8 步：

1) 确保加速度计正常工作：

在 PWR_MGMT_1 (0x6B) 寄存器中设置 CYCLE=0 SLEEP=0 STANDBY=0

在 PWR_MGMT_2 (0x6C) 把 DIS_XA, DIS_YA, DIS_ZA 设置为 0，
DIS_XG, DIS_YG, DIS_ZG 设置为 1

2) 将加速度计的 LPF 设置成 184Hz 带宽：

在 ACCEL_CONFIG 2 (0x1D) 中设置 ACCEL_FCHOICE_B1=1
A_DLPFCFG[2:]=1 (b001)

3) 使能运动中断：

直接将 INT_ENABLE (0x38) 设置成 0x40，以使能运动中断信号。

4) 使能智能加速度硬件检测功能：

在 MOT_DETECT_CTRL (0x69) 寄存器中，将 ACCEL_INTEL_EN 设置为 1
ACCEL_INTEL_MODE 设置为 1

5) 设置运动阈值:

在 WOM_THR(0x1F) 寄存器中, 设置 WOM_Threshold[7:0] 在 1~255 之间 (0~1020mg)

6) 设置唤醒频率:

在 LP_ACCEL_ODR(0x1E) 寄存器中, 设置 Lposc_clkse1[3:0]=0.24Hz~500Hz

7) 开启循环模式 (加速度低功耗模式):

把 PWR_MGMT_1(0x6B) 的 CYCLE 设置为 1

8) 运动中断配置完成。

寄存器 35——FIFO 使能

使用方法: 读/写

复位值: 0x00

位	位名	功能
[7]	TEMP_OUT	1-写 TEMP_OUT_H 和 TEMP_OUT_L 值到 FIFO 的采样区 使能时, 缓冲数据都会输出 0-功能关闭
[6]	GYRO_XOUT	1-写 GYRO_XOUT_H 和 GYRO_XOUT_L 到 FIFO 采样区 使能时, 缓冲数据都会输出 0-功能关闭
[5]	GYRO_YOUT	1-写 GYRO_YOUT_H 和 GYRO_YOUT_L 到 FIFO 采样区 使能时, 缓冲数据都会输出 0-功能关闭
[4]	GYRO_ZOUT	1-写 GYRO_ZOUT_H 和 GYRO_ZOUT_L 到 FIFO 采样

		区 使能时，缓冲数据都会输出 0-功能关闭
[3]	ACCEL	1-写加速度的 XYZ 轴的值到 FIFO 中 使能时，缓冲数据都会输出 0-功能关闭
[2]	SLV_2	1-写 EXT_SENS_DATA 寄存器和 SLV_2 的值到 FIFO 使能时，缓冲数据都会输出 0-功能关闭
[1]	SLV_1	1-写 EXT_SENS_DATA 寄存器和 SLV_1 的值到 FIFO 使能时，缓冲数据都会输出 0-功能关闭
[0]	SLV_0	1-写 EXT_SENS_DATA 寄存器和 SLV_0 的值到 FIFO 使能时，缓冲数据都会输出 0-功能关闭 说明： I2C_SLV3_CTRL 寄存器使能 SLV_3

这个寄存器是控制读取数据到缓冲区的功能的开关寄存器。

寄存器 36——I2C 主机控制模式

操作方式：读/写

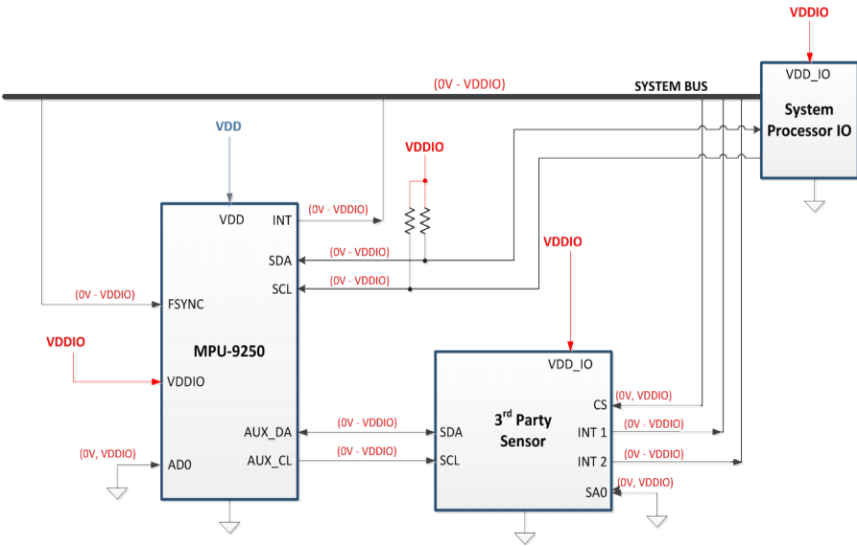
复位值：0x00

位	位名	功能
[7]	MULT_MST_EN	使能主机模式。当关闭时，I2C_MST_IF 也可以关闭，同时数据仲裁也可以关闭。

[6]	WAIT_FOR_ES	延时中断准备数据直到其他传感器数据加载。 如果 I2C_MST_IF 未开启，中断将会继续。																																				
[5]	SLV_3_FIFO_EN	1--- 写 EXT_SENS_DATA 的值和 SLV_3(I2C_SLV0_CTRL 和 I2C_SLV1_CTRL 和 I2C_SLV2_CTR)到 FIFO，在相同的采样率下。 0---关闭此功能。																																				
[4]	I2C_MST_P_NSR	此位控制 I2C 主机模式从一个从机到另一个从机读值。当为 0 时，再读值时会从新开始。当 设置为 1 时，再读值时会停止。																																				
[3: 0]	I2C_MST_CLK[3: 0]	I2C_MST_CLK 是一个 4 位无符号整型数据来分频 MPU9250 内部的 8MHz 的振荡频率。I2C 的主机时钟速率配置如下表： <table border="1"> <thead> <tr> <th>I2C_MST_CLK</th><th>I2C MASTER CLOCK SPEED (kHz)</th><th>8MHz clock divider</th></tr> </thead> <tbody> <tr><td>0</td><td>348</td><td>23</td></tr> <tr><td>1</td><td>333</td><td>24</td></tr> <tr><td>2</td><td>320</td><td>25</td></tr> <tr><td>3</td><td>308</td><td>26</td></tr> <tr><td>4</td><td>296</td><td>27</td></tr> <tr><td>5</td><td>286</td><td>28</td></tr> <tr><td>6</td><td>276</td><td>29</td></tr> <tr><td>7</td><td>267</td><td>30</td></tr> <tr><td>8</td><td>258</td><td>31</td></tr> <tr><td>9</td><td>500</td><td>16</td></tr> <tr><td>10</td><td>471</td><td>17</td></tr> </tbody> </table>	I2C_MST_CLK	I2C MASTER CLOCK SPEED (kHz)	8MHz clock divider	0	348	23	1	333	24	2	320	25	3	308	26	4	296	27	5	286	28	6	276	29	7	267	30	8	258	31	9	500	16	10	471	17
I2C_MST_CLK	I2C MASTER CLOCK SPEED (kHz)	8MHz clock divider																																				
0	348	23																																				
1	333	24																																				
2	320	25																																				
3	308	26																																				
4	296	27																																				
5	286	28																																				
6	276	29																																				
7	267	30																																				
8	258	31																																				
9	500	16																																				
10	471	17																																				

		11	444	18
		12	421	19
		13	400	20
		14	381	21
		15	364	22

这个寄存器主要是设置当 MPU9250 作为主机通过 I2C 连接其他传感器时，读取时的一些配置设置。



说明：更多关于 EXT_SENS_DATA 寄存器信息请参考 73 到 96 寄存器。

图 3-1 传感器与 MCU 的参考连接图

上图 3-1 就是 MPU9250 连接第三方传感器的典型连接图。

寄存器 37——I2C_SLV0_ADDR

操作方式：读/写

复位值：0x00

位	位名	功能
---	----	----

[7]	I2C_SLV0_RNW	1-读取值 0-写入值
[6:0]	I2C_ID_0[6:0]	I2C 从设备 0 地址的物理地址

这个寄存器是控制 MPU9250 控制第三方辅助传感器的寄存器，I2C 的物理地址就是指的 0 号辅助传感器的 I2C 地址。比如第三方传感器是一个气压计，地址是 0x01，如果要读取，此处配置为 0x81，如果要往其中写入，则配置这个值为 0x01。MPU9250 传感器可以连接 5 个 I2C 设备。

寄存器 38——I2C_SLV0_REG

操作方式：读/写

复位值：0x00

位	位名	功能
[7:0]	I2C_SLV0_REG[7:0]	I2C 的 0 号从设备要开始读取数据的寄存器地址

这个寄存器就是第三方辅助传感器内部要读取数据的寄存器地址存放的地方，比如第三方是个气压计，我们要从 0x55 开始读取气压值，那么此处配置为 0x55。

寄存器 39——I2C_SLV0_CTRL

操作方式：读/写

复位值：0x00

位	位名	功能
[7]	I2C_SLV0_EN	1-使能读取 0 号从设备的数据的第一个数据，并储存在 EXT_SEN_DATA_00 寄存器中

		0-关闭这个功能
[6]	I2C_SLV0_BYTE_SW	1-自动读取并替换高低位字节数据 比如： 如果0号第三方传感器读取数据地址数据是(I2C_SLV0_REG)0x1，数据长度为(I2C_SLV0_LEN)0x4； 1.0x1 读取的数据将会存储在EXT_SENS_DATA_00； 2.0x2 地址读取的数据将会被储存在EXT_SENS_DATA_02,0x3 地址读取的数据将会被储存在EXT_SENS_DATA_01 3.0x4 地址读取的数据被储存在EXT_SENS_DATA_03 0-不发出读取命令不会自动替换高低字节数据。
[5]	I2C_SLV0_REG_DIS	配置后，不会再写入寄存器数值，只会读数据或写数据。
[4]	I2C_SLV0_GRP	将外部读取的数据作为一个数组，自动转换高低字节。 1--将地址0和1视为一组 0--将地址1和2视为一组
[3:0]	I2C_SLV_LEN[3:0]	从0号辅助传感器中读取字节的数量

这个寄存器是用来设置辅助传感器的数据配置和数据读取的。具体功能都在上面的功能介绍里面了。

由于MPU9250芯片可以连接5个I2C从设备，所以接下来的寄存器都是一样的功能。

寄存器 40 到 42——I2C 1 号从设备

功能和 0 号一样，唯一不同的是存储的位置是 EXT_SENS_DATA_03，EXT_SENS_DATA_04，EXT_SENS_DATA_05 和 EXT_SENS_DATA_06。

寄存器 43 到 45——I2C 2 号从设备

功能同上。

寄存器 46 到 48——I2C 3 号从设备

功能同上。

寄存器 49 到 53——I2C 4 号从设备

ADDR, REG 和 DO 上面的功能相同，这里就不再赘述。我们来看看不同的寄存器

寄存器 51 号——I2C_SLV4_DO

操作方式：读/写

复位值：0x00

位	位名	功能
[7:0]	I2C_SLV4_DO[7:0]	如果使能，数据将会写入辅助传感器

寄存器 52 号——I2C_SLV4_CTRL

操作方式：读/写

复位值：0x00

位	位名	功能
[7]	I2C_SLV4_EN	1—使能数据传输功能，如果发送读命令，将数据储存在 I2C_SLV4_DI 中。如果发送写命令，将数据储存在 I2C_SLV4_DO 中。

		当传输完成后，该位将清 0. 请确保先写数据到 I2C_SLV4_D0 中。 0--关闭此功能
[6]	SLV4_DONE_INT_EN	1--当 4 号从地址传感器发生数据交换时，产生一个中断。 2--不会产生中断。
[5]	I2C_SLV4_REG_DIS	设置后，不会再写入寄存器数值，只会读数据或写数据。
[4:0]	I2C_MST_DLY	当使能 I2C_MST_DELAY_CTRL 后，从设备将会以 (1+I2C_MST_DLY) 采样率对寄存器内的数据进行采样。

这个 4 号从设备的寄存器功能在后面和之前的有点儿不一样，大家要注意一下这个地方。

寄存器 53 号——I2C_SLV4_DI

操作方式：读

复位值：0x00

位	位名	功能
[7:0]	I2C_SLV4_DI[7:0]	读取 4 号辅助设备数据

寄存器 54——I2C 主设备状态

操作方式：读/写

复位值：0x00

位	位名	功能
[7]	PASS_THROUGH	FSYNC 外部中断的状态位。--发送外部中断信号给 MPU9250 时使用。若使能 INT_PIN_CFG

		的 FSYNC_INT_EN 位，FSYNC 会产生一个上升沿中断。在读取后此寄存器状态清零
[6]	I2C_SLV4_DONE	INT_ENABLE 的 I2C_MST_INT_EN 位使能且 I2C_SLV4_CTRL 的 SLV_DONE_INT_EN 位使能。当 4 号传感器读取完成后会产生一个中断
[5]	I2C_LOST_ARB	当 INT_ENABLE 的 I2C_MST_INT_EN 位使能后，从机释放 I2C 总线仲裁信号的时候，将会产生一个外部中断信号
[4]	I2C_SLV4_NACK	当 INT_ENABLE 中的 I2C_MST_INT_EN 位使能后，从设备 4 数据丢包时会产生一个中断
[3]	I2C_SLV3_NACK	当 INT_ENABLE 中的 I2C_MST_INT_EN 位使能后，从设备 3 数据丢包时会产生一个中断
[2]	I2C_SLV2_NACK	当 INT_ENABLE 中的 I2C_MST_INT_EN 位使能后，从设备 2 数据丢包时会产生一个中断
[1]	I2C_SLV1_NACK	当 INT_ENABLE 中的 I2C_MST_INT_EN 位使能后，从设备 1 数据丢包时会产生一个中断
[0]	I2C_SLV0_NACK	当 INT_ENABLE 中的 I2C_MST_INT_EN 位使能后，从设备 0 数据丢包时会产生一个中断

这个寄存器全部都是从主设备进行 I2C 数据传输时，其中设备的传输状态，主要是检测错误信号。

寄存器 55——INT 引脚/电路迂回使能配置寄存器

操作方式：读/写

复位值：0x00

位	位名	功能
[7]	ACTL	1-触发 INT 的逻辑电平为低电平 0-触发 INT 的逻辑电平为高电平
[6]	OPEN	1--INT 脚设置为开漏脚

		0--INT 脚设置为推挽输出
[5]	LATCH_INT_EN	1--INT 脚电平不变直到非中断状态 0--INT 脚电平表明中断脉冲宽度为 50us
[4]	INT_ANYRD_2CLEAR	1--当有任何读取动作时处于非中断状态 0--当读取 INT_STATUS 寄存器时处于非中断状态
[3]	ACTL_FSYNC	1--激活 FSYNC 的中断电平为低电平 0--激活 FSYNC 的中断电平位高电平
[2]	FSYNC_INT_MODE_EN	1--开启 FSYNC 的中断功能，ACTL_FSYNC 位激活电平的转换将产生中断。请在 I2C MASTER 状态寄存器的 PASS_THROUGH 位读取中断状态。 0--关闭 FSYNC 引脚的中断功能。
[1]	BYPSS_EN	启动后，i2c 主机功能关闭时，接口的 ES_CL 和 ES_DA 将会进入电路迂回模式。此时引脚会产生高电平。
[0]	保留	

外部中断的配置寄存器，需要连接外部辅助设备的时候才会用到。配合之前的 I2C 寄存器使用。

寄存器 56——中断使能

操作方式：读/写

复位值：0x00

位	位名	功能
[7]	RESERVED	
[6]	WOM_EN	1--运动中断唤醒使能 0--关闭
[5]	RESERVED	

[4]	FIFO_OVERFLOW_EN	1--使能 FIFO 溢出中断 0--关闭
[3]	FSYNC_INT_EN	1--使能 FSYNC 中断使能 0--关闭
[2]	RESERVED	
[1]	RESERVED	
[0]	RAW_RDY_EN	1--使能寄存器数据溢出中断（请设置 I2C_MST_CTRL 和 WAIT_FOR_ES 的[6]） 0--关闭

本寄存器配置中断使能传输中的中断使能功能。

寄存器 58——中断状态

操作方式：读/写

复位值：0x00

位	位名	功能
[7]	Reserved	
[6]	WOM_INT	1--运动唤醒
[5]	Reserved	
[4]	FIFIO_OVERFLOW_INT	1--FIFO 数据溢出中断。最老的数据会被丢弃
[3]	FSYNC_INT	1--FSYNC 中断发生
[2]	Reserved	
[1]	Reserved	
[0]	RAW_DATA_RDY_INT	1--传感器完成数据更新可以被读取状态。 （I2C_MST_CTRL 和 WAIT_FOR_ES [6] 设置为使能）

单片机可以通过读取这些数据的值来判断当前的数据读取状态,不过这个地方的数据还是和 I2C 从主机配合使用。

寄存器 59 到 64——加速度测量

名称: ACCEL_XOUT_H

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	加速度 x 轴高字节

名称: ACCEL_XOUT_L

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	加速度 x 轴低字节

名称: ACCEL_YOUT_H

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	加速度 y 轴高字节

名称: ACCEL_YOUT_L

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	加速度 y 轴低字节

名称: ACCEL_ZOUT_H

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	加速度 z 轴高字节

名称: ACCEL_ZOUT_L

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	加速度 z 轴低字节

寄存器 65 到 66——温度测量

名称: TEMP_OUT_H

测量顺序: 同步

复位值：0x00（如果传感器不可用）

位	位名	功能
[7:0]	D[7:0]	传感器温度值高字节

名称：TEMP_OUT_L

测量顺序：同步

复位值：0x00（如果传感器不可用）

位	位名	功能
[7:0]	D[7:0]	$\text{TEMP_degC} = ((\text{TEMP_OUT} - \text{RoomTemp_Offset}) / \text{Temp_Sensitivity}) + 21\text{degC}$ 温度值输出低字节： Temp_degC 为摄氏度，TEMP_OUT 是传感器测得的实际温度。

寄存器 67 到 72——陀螺仪测量

名称：GYRO_XOUT_H

测量顺序：同步

复位值：0x00（如果传感器不可用）

位	位名	功能
[7:0]	D[7:0]	陀螺仪 x 轴高字节

名称: GYRO_XOUT_L

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	陀螺仪 x 轴低字节 $\text{GYRO_XOUT} = \text{Gyro_Sensitivity} * \text{X_angular_rate}$ 此时 FS_SEL=0 条件 Gyro_Sensitivity=131LSB/(° /s)

名称: GYRO_YOUT_H

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	陀螺仪 y 轴高字节

名称: GYRO_YOUT_L

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	陀螺仪 y 轴低字节 $\text{GYRO_YOUT} = \text{Gyro_Sensitivity} * \text{Y_angular_rate}$

		此时 FS_SEL=0 条件 Gyro_Sensitivity=131LSB/(° /s)
--	--	--

名称: GYRO_ZOUT_H

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	陀螺仪 z 轴高字节

名称: GYRO_ZOUT_L

测量顺序: 同步

复位值: 0x00 (如果传感器不可用)

位	位名	功能
[7:0]	D[7:0]	陀螺仪 z 轴低字节 $\text{GYRO_ZOUT} = \text{Gyro_Sensitivity} * \text{Z_angular_rate}$ 此时 FS_SEL=0 条件 Gyro_Sensitivity=131LSB/(° /s)

以上的寄存器是存放加速度计, 温度计和陀螺仪测量数据的寄存器。非常重要。

寄存器 73 到 96——外部数据储存寄存器

名称: ACCEL_ZOUT_H

测量顺序: 同步

复位值: 0x00

位	位名	功能
[7:0]	D[7:0]	这些寄存器就是前面 SLV0-4 所保存的数据值

寄存器 99-102——I2C 从设备数据输出

这几个寄存器就是 SLV0-3 的 D0，配置的数据是要写入从设备的数据值。

寄存器 103——I2C 主机延时控制

名称：I2C_MST_DELAY_CTRL

功能：读/写

复位值：0x00

位	位名	功能
[7]	DELAY_ES_SHADOW	延时直到所有的传感器数据被接收
[6:5]	RESERVED	
[4]	I2C_SLV4_DLY_EN	使能后，SLV4 的数据将被按照 (1+I2C_MST_DLY) 和 SMPLRT_DIV 和 DLPF_CFG 设置的速率被读取
[3]	I2C_SLV3_DLY_EN	使能后，SLV3 的数据将被按照 (1+I2C_MST_DLY) 和 SMPLRT_DIV 和 DLPF_CFG 设置的速率被读取
[2]	I2C_SLV2_DLY_EN	使能后，SLV2 的数据将被按照 (1+I2C_MST_DLY) 和 SMPLRT_DIV 和 DLPF_CFG 设置的速率被读取
[1]	I2C_SLV1_DLY_EN	使能后，SLV1 的数据将被按照 (1+I2C_MST_DLY) 和 SMPLRT_DIV 和 DLPF_CFG 设置的速率被读取
[0]	I2C_SLV0_DLY_EN	使能后，SLV0 的数据将被按照 (1+I2C_MST_DLY) 和 SMPLRT_DIV 和 DLPF_CFG 设置的速率被读取

该寄存器是使能读取和接收时间段的重要寄存器。使能后可以对采集速率和时间等进行配置。

寄存器 104——信号重置

名称: GYRO_ZOUT_L

响应顺序: 同步

复位值: 0x00

位	位名	功能
[7:3]	预留	
[2]	GYRO_RST	重置陀螺仪说明: 寄存器未清空, 使用 SIG_COND_RST
[1]	ACCEL_RST	重置加速度说明: 寄存器未清空, 使用 SIG_COND_RST
[0]	TEMP_RST	重置温度计说明: 寄存器未清空, 使用 SIG_COND_RST

重置三个值的数据。

寄存器 105——加速度中断控制

名称: ACCEL_INTEL_CTRL

响应顺序: 同步

复位值: 0x00

位	位名	功能
[7]	ACCEL_INTEL_EN	运动唤醒逻辑检测使能位
[6]	ACCEL_INTEL_MODE	1--将当前数据与之前的数据进行对比 0--不使用
[5:0]	Reserved	

配合之前的运动唤醒功能使用。

寄存器 106——用户控制

名称: GYRO_ZOUT_L

响应顺序: 同步

复位值: 0x00

位	位名	功能
[7]	Reserved	
[6]	FIFO_EN	1-使能 FIFO 操作模式 0-关闭 FIFO 串行模式 在 FIFO_EN 寄存器中关闭 FIFO 的 dma 写入功能。
[5]	I2C_MST_EN	1-使能 I2C 主模式;ES_DA 和 ES_SCL 和 SDA/SDI 以及 SCL/SCLK 隔离 0-关闭 I2C 主机模式, 引脚 ES_DA 和 ES_SCL 被 SDA/SDI 和 SCL/SCLK 的逻辑电平驱动
[4]	I2C_IF_DIS	1-重置 I2C 从机模式并进入 SPI 模式 此位将会在一个时钟周期后自动清零
[3]	Reserved	
[2]	FIFO_RST	1-重置 FIFO 模式, 异步重置 一个时钟周期后此位也会自动清理
[1]	I2C_MST_RST	1-重置 I2C 主机模式, 异步重置, 一个时钟周期后此位清零 说明: 此位只能在 I2C 主模式时设置, 当设置时, 从机将会被挂载。
[0]	SIG_COND_RST	1-重置所有数据。一个时钟周期后此位将清零。

寄存器 107——电源管理 1

名称: PWR_MGMT_1

功能: 读/写

复位值: (被 PU_SLEEP_MODE 位决定, 见下表)

位	位名	功能														
[7]	H_RESET	1-重置所有寄存器和已经配置数据，重置后此位将自动清 0														
[6]	SLEEP	设置时，芯片进入休眠模式（加载 OTP 后，PU_SLEEP_MODE 位将会被写在此位）														
[5]	CYCLE	设置后，当 SLEEP 和 STANDBY 没有设置时，芯片将进入休眠和读数的循环状态，此时速率由 LP_ACCEL_ODR 寄存器决定。 说明：当加速度计关闭时，芯片还是会被中断唤醒，但是芯片此时不读值。														
[4]	GYRO_STANDBY	当被设置时，陀螺仪驱动电路使能，但传感器不工作。这是专门用在低功耗模式快速使能陀螺仪用的。														
[3]	PD_PTAT	PTAT 和 PTAT ADC 掉电														
[2:0]	CLKSEL[2:0]	<table><tr><th>代码</th><th>时钟源</th></tr><tr><td>0</td><td>内部 20MHz 振荡</td></tr><tr><td>1</td><td>自动选择最佳时钟源-否则使用外部振荡</td></tr><tr><td>2</td><td>自动选择最佳时钟源-否则使用外部振荡</td></tr><tr><td>3</td><td>自动选择最佳时钟源-否则使用外部振荡</td></tr><tr><td>4</td><td>自动选择最佳时钟源-否则使用外部振荡</td></tr><tr><td>5</td><td>自动选择最佳时钟源-否则使用外部</td></tr></table>	代码	时钟源	0	内部 20MHz 振荡	1	自动选择最佳时钟源-否则使用外部振荡	2	自动选择最佳时钟源-否则使用外部振荡	3	自动选择最佳时钟源-否则使用外部振荡	4	自动选择最佳时钟源-否则使用外部振荡	5	自动选择最佳时钟源-否则使用外部
代码	时钟源															
0	内部 20MHz 振荡															
1	自动选择最佳时钟源-否则使用外部振荡															
2	自动选择最佳时钟源-否则使用外部振荡															
3	自动选择最佳时钟源-否则使用外部振荡															
4	自动选择最佳时钟源-否则使用外部振荡															
5	自动选择最佳时钟源-否则使用外部															

		振荡
		6 内部 20MHz 振荡
		7 停止时钟源并保持
		OTP 加载完成后，PU_SLEEP_MODE 状态将会被写入 CLKSEL[0]

寄存器 108——电源管理 2

名称：PWR_MGMT_2

功能：读/写

复位值：0x00

位	位名	功能
[7:6]	保留	
[5]	DISABLE_XA	1-x 轴加速度 关闭 0-x 轴加速度 开启
[4]	DISABLE_YA	1-y 轴加速度 关闭 0-y 轴加速度 开启
[3]	DISABLE_ZA	1-z 轴加速度 关闭 0-z 轴加速度 开启
[2]	DISABLE_XG	1-x 轴陀螺仪 关闭 0-x 轴陀螺仪 开启
[1]	DISABLE_YG	1-y 轴陀螺仪 关闭 0-y 轴陀螺仪 开启
[0]	DISABLE_ZG	1-z 轴陀螺仪 关闭 0-z 轴陀螺仪 开启

非常重要的寄存器，要进入 MPU-9250 的低功耗加速度计模式：

- (i) 设置 CYCLE 位为 1
- (ii) 设置 SLEEP 位为 0
- (iii) 设置 TEMP_DIS 位为 1
- (iv) 设置 DIS_XG, DIS_YG, DIS_ZG 位为 1

第 (i) 和 (iii) 提到的位可以去管理器 1 寄存器里面寻找（寄存器 107）

在这种模式下，只有 I2C 在工作中，其他全部设备都将关闭，等待着加速度数据的变化将他们唤醒。

寄存器 114 和 115——FIFO 计数寄存器

名称：FIFO_COUNTH

地址：114

操作方法：只读

复位值：0x00

位	位名	功能
[7:5]	预留	
[4:0]	FIFO_CNT[12:8]	高位数据表明的是 FIFO 中写入字节的数量 此位和 115 号寄存器是链接在一起的

名称：FIFIO_COUNTL

地址：115

操作方式：只读

重置值：0x00

位	位名	功能
[7:0]	FIFO_CNT[7:0]	低位和 114 连接在一起的，表明 FIFO 写入字节的数量。必须先读取完 FIFO_COUNTH 才能读取该位

寄存器 116——FIFO 读写

名称：FIFO_R_W

操作方式：读/写

复位值：0x00

位	位名	功能
[7:0]	D[7:0]	FIFO 的读/写命令

说明：

此寄存器被用来读写 FIFO 的缓存数据。

寄存器 117——WHO AM I

名字：WHOAMI

操作方式：只读

复位值：0x68

位	位名	功能
[7:0]	WHOAMI	寄存器指示当前被接入的设备地址

本寄存用来核实是否正确接入设备，返回值应该是 0x71。

寄存器 119, 120, 122, 123, 125, 126——加速度偏移寄存器

MPU9250 模式：

名称：XA_OFFS_H

地址：119

操作方式：读/写

重置值：0x00

位	位名	功能
[7:0]	XA_OFFS[14:7]	加速度偏移量 x 轴高位 +/- 16g 全范围模式偏移 15 位 0.98-mg 步进

名称：XA_OFFS_L

地址：120

操作方式：读/写

复位值：0x00

位	位名	功能
[7:1]	XA_OFFS[6:0]	上一位的低字节部分
[0]	预留	

名称：YA_OFFS_H

地址：122

操作方式：读/写

重置值：0x00

位	位名	功能
[7:0]	YA_OFFS[14:7]	加速度偏移量 y 轴高位 +/- 16g 全范围模式偏移 15 位 0.98-mg 步进

名称：YA_OFFS_L

地址：123

操作方式：读/写

复位值：0x00

位	位名	功能
[7:1]	YA_OFFS[6:0]	上一位的低字节部分
[0]	预留	

名称：ZA_OFFS_H

地址：125

操作方式：读/写

重置值：0x00

位	位名	功能
---	----	----

[7:0]	XA_OFFS[14:7]	加速度偏移量 z 轴高位 +/- 16g 全范围模式偏移 15 位 0.98-mg 步进
-------	---------------	---

名称：ZA_OFFS_L

地址：126

操作方式：读/写

复位值：0x00

位	位名	功能
[7:1]	XA_OFFS[6:0]	上一位的低字节部分
[0]	预留	

以上几位的数值其实已经出厂设置过了，不需要更改和读取。

3.3 磁力计寄存器

MPU-9250 中的磁力计（AK8963）寄存器清单。这里的地址注意和之前的不同。

名称	地址	读/写	描述	位宽	解释
WIA	00H	读	设备 ID	8	
INFO	01H	读	数据	8	
ST1	02H	读	状态 1	8	数据状态
HXL	03H	读	测量数据	8	x 轴数据
HXH	04H			8	

HYL	05H			8	y 轴数据
HYH	06H			8	
HZL	07H			8	z 轴数据
HZH	08H			8	
ST2	09H	读	状态 2	8	数据状态
CNTL	0AH	读/写	控制	8	
RSV	0BH	读/写	保留	8	不可进入
ASTC	0CH	读/写	自检	8	
TS1	0DH	读/写	测试 1	8	不可进入
TS2	0EH	读/写	测试 2	8	不可进入
I2CDIS	0FH	读/写	I2C 关闭	8	
ASAX	10H	读	x 轴灵敏度调节	8	
ASAY	11H	读	y 轴灵敏度调节	8	
ASAZ	12H	读	z 轴灵敏度调节	8	

表 2 寄存器表

3.3.1 磁力计寄存器详细说明

地址	寄存器名	位 7	位 6	位 5	位 4	位 3	位 2	位 1	位 0
只读寄存器									
00H	WIA	0	1	0	0	1	0	0	0

01H	INFO	INFO 7	INFO 6	INFO 5	INFO4	INFO3	INFO2	INFO1	INFO0
02H	ST1	0	0	0	0	0	0	DOR	DRDY
03H	HXL	HX7	HX6	HX5	HX4	HX3	HX2	HX1	HX0
04H	HXH	HX15	HX14	HX13	HX12	HX11	HX10	HX9	HX8
05H	HYL	HYZ7	HY6	HY5	HY4	HY3	HY2	HY1	HY0
06H	HYH	HY15	HY14	HY13	HY12	HY11	HY10	HY9	HY8
07H	HZL	HZ7	HZ6	HZ5	HZ4	HZ3	HZ2	HZ1	HZ0
08H	HZH	HZ15	HZ14	HZ13	HZ12	HZ11	HZ10	HZ9	HZ8
09H	ST2	0	0	0	BITM	HOFL	0	0	0
读写寄存器									
0AH	CNTL1	0	0	0	0	MODE3	MODE2	MODE1	MODE0
0BH	CNTL2	0	0	0	0	0	0	0	SRST
0CH	ASTC	—	SELF	—	—	—	—	—	—
0DH	TS1	—	—	—	—	—	—	—	—
0EH	TS2	—	—	—	—	—	—	—	—
0FH	I2CDIS	I2CDI S7	I2CD IS6	I2CD IS5	I2CD IS4	I2CDI S3	I2CDI S2	I2CDI S1	I2CDI S0
只读寄存器									
10H	ASAX	COEFX 7	COEF X6	COEF X5	COEF X4	COEFX 3	COEFX 2	COEFX 1	COEFX 0

11H	ASAY	COEFY 7	COEF Y6	COEF Y5	COEF Y4	COEFY 3	COEFY 2	COEFY 1	COEFY 0
12H	ASAZ	COEFZ 7	COEF Z6	COEF Z5	COEF Z4	COEFZ 3	COEFZ 2	COEFZ 1	COEFZ 0

说明：当 VDD 上电以后，AK893 的寄存器和功能就初始化了。TS1 和 TS2 是测试寄存器的存储器，请不要使用。RSV 是预留寄存器，也请勿使用。

这部分讲解 MPU-9250 的磁力计寄存器。

3.3.2 WIA: 设备 ID

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
00H	WIA	0	1	0	0	1	0	0	0

磁力计设备 ID—48H，一个字节而且值不变。

3.3.3 INFO: 信息

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
01H	INFO	INFO 7	INFO6	INFO 5	INFO 4	INFO3	INFO2	INFO1	INFO0

INFO[7:0]：磁力计信息

一般不用

3.3.4 ST1：状态 1

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
02H	ST1	0	0	0	0	0	0	0	DRDY
重置		0	0	0	0	0	0	0	0

DRDY：准备数据

“0”：正常

“1”：数据准备

测试模式下 DRDY 位会变 1，当在状态 2 或测量数据（HXL 到 HZH）被读取后会变 0。

DOR：数据溢出

“0”：正常

“1”：数据溢出

DOR 位变“1”当数据跳过测试模式，当在状态 2 或测量数据时变为 0（HXL~HZH）。

3.3.5 HXL 到 HZH:测量数据

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
03H	HXL	HX7	HX6	HX5	HX4	HX3	HX2	HX1	HX0
04H	HXH	HX15	HX14	HX13	HX12	HX11	HX10	HX9	HX8

05H	HYL	HY7	HY6	HY5	HY4	HY3	HY2	HY1	HY0
06H	HYH	HY15	HY14	HY13	HY12	HY11	HY10	HY9	HY8
07H	HZL	HZ7	HZ6	HZ5	HZ4	HZ3	HZ2	HZ1	HZ0
08H	HZH	HZ15	HZ14	HZ13	HZ12	HZ11	HZ10	HZ9	HZ8
重置		0	0	0	0	0	0	0	0

磁力计的三轴测量数据：

HXL[7:0]:X-轴数据低 8 位

HXH[15:8]:x 轴数据高 8 位

HYL[7:0]:y 轴数据低 8 位

HYH[15:8]:y 轴数据高 8 位

HZL[7:0]:z 轴数据低 8 位

HZH[15:8]:z 轴数据高 8 位

测量数据被存在 2 个寄存器中，每个轴的 16 位测量范围在-32760~32760

数据测量（每个轴）[15:0]			磁流密度[uT]
2 个寄存器	十六进制	十位	
0111 1111 1111 1000	7FF8	32760	4912(最大)
0000 0000 0000 0001	0001	1	0.15
0000 0000 0000 0000	0000	0	0
1111 1111 1111	FFFF	-1	-0.15

1111			
1000 0000 0000 1000	8008	-32760	-4912（最小）

表 4 测量数据版式

3.3.6 ST2：状态 2

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
09H	ST2	0	0	0	BITM	HOFL	0	0	0
重置		0	0	0	0	0	0	0	0

HOFL: 磁力计溢出

“0”：正常

“1”：磁力计发生溢出

在单独测量模式，连续测量模式，外部触发测量模式和自测模式下，磁力计在数据未满的情况下也可能发生溢出。这种模式下，测量数据错误 HOFL 位置“1”。当下次测量开始时，此位将置“0”。

BITM 位：输出位设置（镜像）

“0”：14 位输出

“1”：16 位输出

CNTL1 寄存器位的镜像。

ST2 寄存器做位数据读取的最后一个寄存器，请确保在每次读取新一组数据的时候，ST2 始终是最后读取的寄存器。

3.3.7 CNTL1: 控制 1

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
0AH	CNTL1	0	0	0	BIT	MODE3	MODE2	MODE1	MODE0
重置		0	0	0	0	0	0	0	0

MODE[3:0]:操作模式设置

“0000” :掉电模式

“0001” :单独测试

“0010” :连续测量模式 1

“0110” :连续测量模式 2

“0100” :外部触发测量模式

“1000” :自检模式

“1111” :烧写 ROM 模式

此外其他的指令都是禁止设置的。

BIT: 输出位设置

“0” :14 位输出

“1” :16 位输出

当设置完成后, 磁力计 (AK8963) 就退出设置模式。

当 CNTL 寄存器进入写入模式时, 02H 到 09H 已经被加载完成。

3.3.8 CNTL2: 控制 2

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
只读寄存器									
0BH	CNTL2	0	0	0	0	0	0	0	SRST
重置		0	0	0	0	0	0	0	0

SRST: 软复位

“0” : 正常

“1” : 复位

当设置 1 时，所有的寄存器将初始化。之后，SRST 位将自动归 0。

3.3.9 ASTC: 自测控制

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
读写寄存器									
0CH	ASTC	–	SELF	–	–	–	–	–	–
重置		0	0	0	0	0	0	0	0

SELF: 自测控制

“0” : 正常

“1” : 产生测试磁场

千万不要将除 SELF 位之外的其他位上写 1，若写 1 后，将无法正常测量数据。

3.3.10 TS1 TS2: 测试 1、2

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
读写寄存器									
0DH	TS1	—	—	—	—	—	—	—	—
0EH	TS2	—	—	—	—	—	—	—	—
重置		0	0	0	0	0	0	0	0

TS1 和 TS2 寄存器为测试用寄存器，请勿使用。

3.3.11 I2CDIS: I2C 关闭

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
读写寄存器									
0FH	I2CDIS	I2CDIS7	I2CDIS6	I2CDIS5	I2CDIS4	I2CDIS3	I2CDIS2	I2CDIS1	I2CDIS0
重置		0	0	0	0	0	0	0	0

此寄存器关闭 I2C 总线接口。I2C 接口默认开启。如果要关闭 I2C 接口请配置 I2CDIS 寄存器为” 00011011” 到 I2CDIS 寄存器。此时 I2C 接口关闭。

一旦 I2C 接口关闭，将无法继续写入 I2CDIS 寄存器数据。若想重新启动 I2C 总线，需要重置磁力计或输入开始条件 8 次。

3.3.12 ASAX ASAY ASAZ: 灵敏值调节

地址	寄存器名	D7	D6	D5	D4	D3	D2	D1	D0
----	------	----	----	----	----	----	----	----	----

只读寄存器									
10H	ASAX	COEF X7	COEF X6	COEFX 5	COEFX 4	COEFX 3	COEFX 2	COEFX 1	COEFX 0
11H	ASAY	COEF Y7	COEF Y6	COEFY 5	COEFY 4	COEFY 3	COEFY 2	COEFY 1	COEFY 0
12H	ASAZ	COEF Z7	COEF Z6	COEFZ 5	COEFZ 4	COEFZ 3	COEFZ 2	COEFZ 1	COEFZ 0
重置		—	—	—	—	—	—	—	—

每个轴的灵敏度数据调节，配置将存入 ROM

ASAX[7:0]:磁力计 x 轴调节值

ASAY[7:0]:磁力计 y 轴调节值

ASAZ[7:0]:磁力计 z 轴调节值

灵敏度调节

灵敏度的调节方程式：

$$H_{adj} = H \times \left(\frac{(ASA - 128) \times 0.5}{128} + 1 \right),$$

H 为寄存器测量数据，ASA 为灵敏度调节值，H_{adj} 为调节测量数据。

MPU9250 与 MPU6050 的区别就是多了一个磁力计，所以这两个传感器很多寄存器地址都差不多。有很多同学会发现在网上找的 MPU6050 的库拿来可以直接用在 MPU9250 的里面。但是有的时候程序又会出错，那就是寄存器不完全一样（不兼容）惹的祸。不过我专门写了 MPU9250 的库函数，开源给大家使用。如果需要的话可以在附录里找到下载地址。